

1. Overview

Agentic AI is the use of large language (and other multi-modal) models not just to generate text, but to act as reasoning, goal-driven agents that can plan, call tools, and adapt their actions to deliver outcomes.

The JVM is a compelling platform for this because its strong type safety provides guardrails for integrating LLM-driven behaviors with real systems. Because so many production applications already run on the JVM it is the natural place to embed AI.

While Agentic AI has been hyped, much of it has lived in academic demos with little practical value; by integrating directly into legacy and enterprise JVM applications, we can unlock AI capabilities without rewriting core systems or tearing down a factory to install new machinery.

1.1. Glossary

Before we begin, in this glossary we'll explain some terms that may be new if you're taking your first steps as an applied AI software developer. It is assumed that you already know what a large language model (LLM) is from an end-user's point of view.

You may skim or skip this section if you're already a seasoned agentic AI engineer.

Agent

An Agent in the Embabel framework is a self-contained component that bundles together domain logic, AI capabilities, and tool usage to achieve a specific goal on behalf of the user.

Inside, it exposes multiple @Action methods, each representing discrete steps the agent can take. Actions depend on typically structured (sometimes natural language) input. The input is used to perform tasks on behalf of the user - executing domain code, calling AI models or even calling other agents as a sub-process.

When an AI model is called it may be given access to tools that expand its capabilities in order to achieve a goal. The output is a new type, representing a transformation of the input, however during execution one or more side-effects can occur. An example of side effects might be new records stored in a database, orders placed on an e-commerce site and so on.

Tools

Tools extend the raw capabilities of an LLM by letting it interact with the outside world. On its own, a language model can only generate responses from its training data and context window, which risks producing inaccurate or "hallucinated" answers.

While tool usage is inspired by a technique known as **ReAct** (Reason + Act), which itself builds on **Chain of Thought** reasoning, most recent LLMs allow specifying tools specifically instead of relying on prompt engineering techniques.

When tools are present, the LLM interprets the user request, plans steps, and then delegates certain tasks to tools in a loop. This lets the model alternate between reasoning ("what needs to be done?") and acting ("which tool can do it?").

Benefits of tools include:

- The ability to answer questions or perform tasks beyond what the LLM was trained on, by delegating to domain-specific or external systems.
 - Producing useful **side effects**, such as creating database records, generating visualizations, booking flights, or invoking any process the system designer provides.
- In short, tools are one way to bridge the gap between **text prediction** and **real-world action**, turning an LLM into a practical agent capable of both reasoning and execution. In Embabel many tools are bound domain objects.

MCP

Model Context Protocol ([MCP](#)) is a standardized way of hosting and sharing tools. Unlike plain tools, which are usually wired directly into one agent or app, an MCP Server makes tools discoverable and reusable across models and runtimes they can be registered system-wide or at runtime, and invoked through a common protocol. Embabel can both consume and publish such tools for systems integration.

Domain Integrated Context Engineering (DICE)

Enhances context engineering by grounding both LLM inputs and outputs in typed domain objects. Instead of untyped prompts, context is structured with business-aware models that provide precision, testability, and seamless integration with existing systems. DICE transforms context into a re-usable, inspectable, and reliably manipulable artifact.

1.2. Why do we need an Agent Framework?

Aren't LLMs smart enough to solve our problems directly? Aren't MCP tools all we need to allow them to solve complex problems?

LLMs seem to get more capable by the day and MCPs can give LLMs access to a lot of empowering tools, making them even more capable.

But there are still many reasons that a higher level orchestration technology is needed, especially for business applications. Here are some of the most important:

- **Explainability:** Why were choices made in solving a problem?
- **Discoverability:** How do we find the right tools at each point, and ensure that models aren't confused in choosing between them?
- **Ability to mix models**, so that we are not reliant only on the largest models but can use local, cheaper, private models for many tasks
- **Ability to inject guardrails** at any point in a flow
- **Ability to manage flow execution** and introduce greater resilience
- **Composability of flows at scale.** We'll soon be seeing not just agents running on one system, but federations of agents.
- **Safer integration with sensitive existing systems** such as databases, where it is dangerous to allow even the best LLM write access.

Agent frameworks break complex tasks into smaller, manageable components, offering greater control and predictability.

Agent frameworks offer "code agency" as well as "LLM agency." This division is well described in this [paper from NVIDIA Research](#).

Further reading:

- [Embabel: A new Agent Platform For the JVM](#)
- [The Embabel Vision](#)

1.3. Embabel Differentiators

So how does Embabel differ from other agent frameworks? We like to believe the Embabel agent framework is to be the best fit for developing agentic AI in the enterprise.

1.3.1. Sophisticated Planning

Goes beyond a finite state machine or sequential execution with nesting by introducing a true planning step, using a non-LLM AI algorithm. This enables the system to perform tasks it wasn't programmed to do by combining known steps in a novel order, as well as make decisions about parallelization and other runtime behavior.

1.3.2. Superior Extensibility and Reuse

Because of dynamic planning, adding more domain objects, actions, goals and conditions can extend the capability of the system, *without editing FSM definitions* or existing code.

1.3.3. Strong Typing and Object Orientation

Actions, goals and conditions are informed by a domain model, which can include behavior. Everything is strongly typed and prompts and manually authored code interact cleanly. No more magic maps. Enjoy full refactoring support.

1.3.4. Platform Abstraction

Clean separation between programming model and platform internals allows running locally while potentially offering higher QoS in production without changing application code.

1.3.5. LLM Mixing

It is easy to build applications that mix LLMs, ensuring the most cost-effective yet capable solution. This enables the system to leverage the strengths of different models for different tasks. In particular, it facilitates the use of local models for point tasks. This can be important for cost and privacy.

1.3.6. Spring and JVM Integration

Built on Spring and the JVM, making it easy to access existing enterprise functionality and capabilities. For example:

- Spring can inject and manage agents, including using Spring AOP to decorate functions.
- Robust persistence and transaction management solutions are available.

1.3.7. Designed for Testability

Both unit testing and agent end-to-end testing are easy from the ground up.

1.4. Core Concepts

Agent frameworks break up tasks into separate smaller interactions, making LLM use more predictable and focused.

Embabel models agentic flows in terms of:

- **Actions:** Steps an agent takes. These are the building blocks of agent behavior.
- **Goals:** What an agent is trying to achieve.
- **Conditions:** Conditions to do evaluations while planning. Conditions are reassessed after each action is executed.
- **Domain Model:** Objects underpinning the flow and informing Actions, Goals and Conditions.

This enables Embabel to create a **plan**: A sequence of actions to achieve a goal. Plans are dynamically formulated by the system, not the programmer. The system replans after the completion of each action, allowing it to adapt to new information as well as observe the effects of the previous action. This is effectively an [OODA loop](#).

Application developers don't usually have to deal with conditions and planning directly, as most conditions result from data flow defined in code, allowing the system to infer pre and post conditions to (re-)evaluate the plan.

1.4.1. Complete Example

Let's look at a complete example that demonstrates how Embabel infers conditions from input/output types and manages data flow between actions. This example comes from the [Embabel Agent Examples](#) repository:

JavaKotlin

```
@Agent(description="Find news based on a person's star sign")public class StarNewsFinder {private final HoroscopeService horoscopeService;private final int storyCount;public StarNewsFinder(HoroscopeService horoscopeService, @Value("${star-news-finder.story.count:5}") int storyCount) {this.horoscopeService = horoscopeService; this.storyCount = storyCount;}@Actionpublic StarPerson extractStarPerson(UserInput userInput, OperationContext context) {return context.ai().withLlm(OpenAiModels.GPT_41).createObject("""Create a person from this user input, extracting their name and star sign:
```

```
%s""").formatted(userInput.getContent(), StarPerson.class);}@Actionpublic Horoscope retrieveHoroscope(StarPerson starPerson) {// Uses regular injected Spring service - not LLMreturn new Horoscope(horoscopeService.dailyHoroscope(starPerson.sign()));}@Actionpublic RelevantNewsStories findNewsStories(StarPerson person, Horoscope horoscope, OperationContext context) {var prompt = """%s is an astrology believer with the sign %s. Their horoscope for today is: %s Given this, use web tools to find %d relevant news stories.
```

```
""").formatted(person.name(), person.sign(), horoscope.summary(), storyCount);return context.ai().withDefaultLlm().withToolGroup(CoreToolGroups.WEB).createObject(prompt, RelevantNewsStories.class);}@AchievesGoal(description="Write an amusing writeup based on horoscope and news")@Actionpublic Writeup writeup(StarPerson person, RelevantNewsStories stories, Horoscope horoscope, OperationContext context) {var llm = LlmOptions.fromCriteria(ModelSelectionCriteria.getAuto()).withTemperature(0.9);var storiesFormatted = stories.items().stream().map(s -> "- " + s.url() + ": " + s.summary()).collect(Collectors.joining("\n"));var prompt = """Write something amusing for %s based on their horoscope and news stories. Format as Markdown with links. <horoscope>%s</horoscope> <news_stories>%s</news_stories>
```

```
""").formatted(person.name(), horoscope.summary(), storiesFormatted);return context.ai().withLlm(llm).createObject(prompt, Writeup.class);}}
```

Agent Declaration: The @Agent annotation defines this as an agent capable of a multi-step flow.

Spring Integration: Regular Spring dependency injection - the agent uses both LLM services and traditional business services.

Service Injection: HoroscopeService is injected like any Spring bean - agents can mix AI and non-AI operations seamlessly.

Action Definition: @Action marks methods as steps the agent can take. Each action represents a capability.

Input Condition Inference: The method signature extractStarPerson(UserInput userInput, ...) tells Embabel:

- **Precondition:** "A UserInput object must be available"
- **Required Data:** The agent needs user input to proceed
- **Capability:** This action can extract structured data from unstructured input

Output Condition Creation: Returning StarPerson creates:

- **Postcondition:** "A StarPerson object is now available in the world state"
- **Data Availability:** This output becomes input for subsequent actions
- **Type Safety:** The domain model enforces structure

Non-LLM Action: Not all actions use LLMs - this demonstrates hybrid AI/traditional programming.

Data Flow Chain: The method signature retrieveHoroscope(StarPerson starPerson) creates:

- **Precondition:** "A StarPerson object must exist" (from previous action)
- **Dependency:** This action can only execute after extractStarPerson completes
- **Service Integration:** Uses the injected horoscopeService rather than an LLM

Regular Service Call: This action calls a traditional Spring service - demonstrating how agents blend AI and conventional operations.

Another Action: This action uses tools specified at the PromptRunner level.

Multi-Input Dependencies: This method requires both StarPerson and Horoscope - showing complex data flow orchestration.

Tool-Enabled LLM: withToolGroup(CoreToolGroups.WEB) adds web search tools to this LLM call, allowing it to search for current news stories.

Goal Achievement: @AchievesGoal marks this as a terminal action that completes the agent’s objective.

Complex Input Requirements: The final action requires three different data types, showing sophisticated orchestration.

Creative Configuration: High temperature (0.9) optimizes for creative, entertaining output - appropriate for amusing writeups.

Structured Prompt with Data: The prompt includes both the horoscope summary and formatted news stories using XML-style tags. This ensures the LLM has all the context it needs from earlier actions.

Final Output: Returns Writeup, completing the agent’s goal with personalized content.

State is managed by the framework, through the process blackboard.

1.4.2. The Inferred Execution Plan for the Example

Based on the type signatures alone, Embabel automatically infers this execution plan for the example agent above:

Goal: Produce a Writeup (final return type of @AchievesGoal action)

The initial plan:

- To emit Writeup → need writeup() action
- writeup() requires StarPerson, RelevantNewsStories, and Horoscope
- To get StarPerson → need extractStarPerson() action
- To get Horoscope → need retrieveHoroscope() action (requires StarPerson)
- To get RelevantNewsStories → need findNewsStories() action (requires StarPerson and Horoscope)
- extractStarPerson() requires UserInput → must be provided by user

Execution sequence:

UserInput → extractStarPerson() → StarPerson → retrieveHoroscope() → Horoscope → findNewsStories() → RelevantNewsStories → writeup() → Writeup and achieves goal.

1.4.3. Key Benefits of Type-Driven Flow

Automatic Orchestration: No manual workflow definition needed - the agent figures out the sequence from type dependencies. This is particularly beneficial if things go wrong, as the planner can re-evaluate the situation and may be able to find an alternative path to the goal.

Dynamic Replanning: After each action, the agent reassesses what’s possible based on available data objects.

Type Safety: Compile-time guarantees that data flows correctly between actions. No magic string keys.

Flexible Execution: If multiple actions could produce the required input type, the agent chooses based on context and efficiency. (Actions can have cost and value.)

This demonstrates how Embabel transforms simple method signatures into sophisticated multi-step agent behavior, with the complex orchestration handled automatically by the framework.

2. Getting Started

2.1. Quickstart

There are two GitHub template repos you can use to create your own project:

- Java template - github.com/embabel/java-agent-template
- Kotlin template - github.com/embabel/kotlin-agent-template

Or you can use our [project creator](#) to create a custom project:

```
uvx --fromgit+https://github.com/embabel/project-creator.git project-creator
```

The uvx command can be installed from the [astral-uv](#) package. It is a Python package and project manager used to run the Embabel [project creator](#) scripts.

Now you have the code you need to run Embabel with LLMs from Open AI or Anthropic by using the included Maven profiles. Skip ahead to [Environment Setup](#) for API-key configuration, or detailed instructions on how to use other LLM providers.

2.2. Getting the Binaries

The easiest way to get started with Embabel Agent is to add the Spring Boot starter dependency to your project. Embabel release binaries are published to Maven Central.

2.2.1. Build Configuration

Add the appropriate Embabel Agent Spring Boot starter to your build file depending on your choice of application type:

Shell Starter

Starts the application in console mode with an interactive shell powered by Embabel.

Maven (pom.xml)Gradle Kotlin DSL (build.gradle.kts)Gradle Groovy DSL (build.gradle)

```
<dependency><groupId>com.embabel.agent</groupId><artifactId>embabel-agent-starter-shell</artifactId><version>${embabel-agent.version}</version></dependency>
```

Features:

- ☒ Interactive command-line interface
- ☒ Agent discovery and registration
- ☒ Human-in-the-loop capabilities
- ☒ Progress tracking and logging
- ☒ Development-friendly error handling

MCP Server Starter

Starts the application with HTTP listener where agents are autodiscovered and registered as MCP servers, available for integration via SSE, Streamable-HTTP or Stateless Streamable-HTTP protocols.

Maven (pom.xml)Gradle Kotlin DSL (build.gradle.kts)Gradle Groovy DSL (build.gradle)

```
<dependency><groupId>com.embabel.agent</groupId><artifactId>embabel-agent-starter-mcpserver</artifactId><version>${embabel-agent.version}</version></dependency>
```

Features:

- ☒ MCP protocol server implementation
- ☒ Tool registration and discovery
- ☒ JSON-RPC communication via SSE (Server-Sent Events), Streamable-HTTP or Stateless Streamable-HTTP
- ☒ Integration with MCP-compatible clients

- ☒ Security and sandboxing

Basic Agent Platform Starter

Initializes Embabel Agent Platform in the Spring Container. Platform beans are available via Spring Dependency Injection mechanism. Application startup mode (web, console, microservice, etc.) is determined by the Application Designer.

Maven (pom.xml)Gradle Kotlin DSL (build.gradle.kts)Gradle Groovy DSL (build.gradle)

```
<dependency><groupId>com.embabel.agent</groupId><artifactId>embabel-agent-starter</artifactId><version>${embabel-agent.version}</version></dependency>
```

Features:

- ☒ Application decides on startup mode (console, web application, etc)
- ☒ Agent discovery and registration
- ☒ Agent Platform beans available via Dependency Injection mechanism
- ☒ Progress tracking and logging
- ☒ Development-friendly error handling

Embabel Snapshots

If you want to use Embabel snapshots, you'll need to add the Embabel repository to your build.

Maven (pom.xml)Gradle Kotlin DSL (build.gradle.kts)Gradle Groovy DSL (build.gradle)

```
<repositories><repository><id>embabel-releases</id><url>https://repo.embabel.com/artifactory/libs-release</url><releases><enabled>true</enabled></releases><snapshots><enabled>false</enabled></snapshots></repository><repository><id>embabel-snapshots</id><url>https://repo.embabel.com/artifactory/libs-snapshots</url><releases><enabled>false</enabled></releases><snapshots><enabled>true</enabled></snapshots></repository><repository><id>spring-milestones</id><url>https://repo.spring.io/milestone</url><snapshots><enabled>false</enabled></snapshots></repository></repositories>
```

2.2.2. Environment Setup

Before running your application, you'll need to set up your environment with API keys for the LLM providers you plan to use.

Example .env file:

```
OPENAI_API_KEY=your_openai_api_key_here
ANTHROPIC_API_KEY=your_anthropic_api_key_here
GEMINI_API_KEY=your_gemini_api_key_here
MISTRAL_API_KEY=your_mistral_api_key_here
```

If you added the binaries directly to your projecdt or want to use other LLM providers than Open AI and Anthropic you will also need to add some dependencies specific to those vendors. Just follow the instructions below for your vendor(s) of choice.

OpenAI Compatible (GPT-4, GPT-5, etc.)

- Required:
- OPENAI_API_KEY: API key for OpenAI or compatible services (e.g., Azure OpenAI, etc.)
- Optional:
- OPENAI_BASE_URL: base URL of the OpenAI deployment (for Azure AI use {resource-name}.openai.azure.com/openai)
- OPENAI_COMPLETIONS_PATH: custom path for completions endpoint (default: /v1/completions)
- OPENAI_EMBEDDINGS_PATH: custom path for embeddings endpoint (default: /v1/embeddings)

Alternatively, configure via application.yml:

```
embabel:agent:platform:models:openai:api-key:${OPENAI_API_KEY:sk-dev-key}base-url:${OPENAI_BASE_URL:}
```

API key with optional default for local development

Optional base URL override

If you are not using the Embabel template projects you also need to add the embabel-agent-starter-openai starter.

Add this to your build system as follows:

Maven (pom.xml)Gradle Kotlin DSL (build.gradle.kts)Gradle Groovy DSL (build.gradle)

```
<dependency><groupId>com.embabel.agent</groupId><artifactId>embabel-agent-starter-openai</artifactId><version>${embabel-agent.version}</version></dependency>
```

OpenAi Custom

- Required:
- OPENAI_CUSTOM_API_KEY: API key for the OpenAI-compatible service
- Optional:
- OPENAI_CUSTOM_BASE_URL: base URL for the OpenAI-compatible API
- OPENAI_CUSTOM_MODELS: comma-separated list of custom model names to register (useful for OpenAI-compatible providers like Groq, Together AI, etc.)

When using OPENAI_CUSTOM_MODELS, set EMBABEL_MODELS_DEFAULT_LLM to specify which model to use as the default.

Example for using Groq:

```
export OPENAI_CUSTOM_BASE_URL="https://api.groq.com/openai/v1"export OPENAI_CUSTOM_API_KEY="your-groq-api-key"export OPENAI_CUSTOM_MODELS="llama-3.3-70b-versatile,mixtral-8x7b-32768"export EMBABEL_MODELS_DEFAULT_LLM="llama-3.3-70b-versatile"
```

Alternatively, configure via application.yml:

```
embabel:agent:platform:models:openai:custom:api-key:${OPENAI_CUSTOM_API_KEY:your-dev-key}base-url:https://api.groq.com/openai/v1models:llama-3.3-70b-versatile,mixtral-8x7b-32768
```

You also need to add the embabel-agent-starter-openai-custom starter.

Add this to your build system as follows:

Maven (pom.xml)Gradle Kotlin DSL (build.gradle.kts)Gradle Groovy DSL (build.gradle)

```
<dependency><groupId>com.embabel.agent</groupId><artifactId>embabel-agent-starter-openai-custom</artifactId><version>${embabel-agent.version}</version></dependency>
```

Anthropic (Claude 3.x, etc.)

- Required:
- ANTHROPIC_API_KEY: API key for Anthropic services
- Optional:
- ANTHROPIC_BASE_URL: base URL for Anthropic API

Alternatively, configure via application.yml:

```
embabel:agent:platform:models:anthropic:api-key:${ANTHROPIC_API_KEY:sk-ant-dev-key}base-url:${ANTHROPIC_BASE_URL:}
```

If you are not using the Embabel template projects you also need to add the embabel-agent-starter-anthropic starter.

Add this to your build system as follows:

Maven (pom.xml)Gradle Kotlin DSL (build.gradle.kts)Gradle Groovy DSL (build.gradle)

```
<dependency><groupId>com.embabel.agent</groupId><artifactId>embabel-agent-starter-anthropic</artifactId><version>${embabel-agent.version}</version></dependency>
```

DeepSeek

- Required:
- DEEPSEEK_API_KEY: API key for DeepSeek services
- Optional:
- DEEPSEEK_BASE_URL: base URL for DeepSeek API (default: api.deepseek.com)

Alternatively, configure via application.yml:

```
embabel:agent:platform:models:deepseek:api-key:${DEEPSEEK_API_KEY:sk-dev-key}base-url:${DEEPSEEK_BASE_URL:}
```

You also need to add the embabel-agent-starter-deepseek starter.

Add this to your build system as follows:

Maven (pom.xml)Gradle Kotlin DSL (build.gradle.kts)Gradle Groovy DSL (build.gradle)

```
<dependency><groupId>com.embabel.agent</groupId><artifactId>embabel-agent-starter-deepseek</artifactId><version>${embabel-agent.version}</version></dependency>
```

Google Gemini (OpenAI Compatible)

Uses the OpenAI-compatible endpoint for Gemini models.

- Required:
- GEMINI_API_KEY: API key for Google Gemini services
- Optional:
- GEMINI_BASE_URL: base URL for Gemini API (default: generativelanguage.googleapis.com/v1beta/openai)

Alternatively, configure via application.yml:

```
embabel:agent:platform:models:gemini:api-key:${GEMINI_API_KEY:your-dev-key}base-url:${GEMINI_BASE_URL:}
```

You also need to add the embabel-agent-starter-gemini starter.

Add this to your build system as follows:

Maven (pom.xml)Gradle Kotlin DSL (build.gradle.kts)Gradle Groovy DSL (build.gradle)

```
<dependency><groupId>com.embabel.agent</groupId><artifactId>embabel-agent-starter-gemini</artifactId><version>${embabel-agent.version}</version></dependency>
```

Google GenAI (Native)

Uses the native Google GenAI SDK for direct access to Gemini models with full feature support including thinking mode.

- Required (API Key authentication):
- GOOGLE_API_KEY: API key for Google AI Studio
- Required (Vertex AI authentication - alternative to API key):
- GOOGLE_PROJECT_ID: Google Cloud project ID
- GOOGLE_LOCATION: Google Cloud region (e.g., us-central1)

Use API key authentication for Google AI Studio, or Vertex AI authentication for Google Cloud deployments. Vertex AI authentication requires Application Default Credentials (ADC) to be configured.

Gemini 3 models are only available in the global location on Vertex AI. To use Gemini 3 with Vertex AI, you must set GOOGLE_LOCATION=global.

To add Google GenAI support to your project add the embabel-agent-starter-google-genai starter.

Add this to your build system as follows:

Maven (pom.xml)Gradle Kotlin DSL (build.gradle.kts)Gradle Groovy DSL (build.gradle)

```
<dependency><groupId>com.embabel.agent</groupId><artifactId>embabel-agent-starter-google-genai</artifactId><version>${embabel-agent.version}</version></dependency>
```

Available LLM models include:

- gemini-3.1-pro-preview - Latest Gemini 3.1 Pro preview with advanced reasoning
- gemini-2.5-pro - High-performance model with thinking support
- gemini-2.5-flash - Best price-performance model
- gemini-2.5-flash-lite - Cost-effective high-throughput model
- gemini-2.0-flash - Fast and efficient
- gemini-2.0-flash-lite - Lightweight version

Available embedding models include:

- gemini-embedding-001 - Recommended embedding model (3072 dimensions)

Example configuration in application.yml:

```
embabel:models:default-llm:gemini-2.5-flashdefault-embedding-model:gemini-embedding-001llms:fast:gemini-2.5-flashbest:gemini-2.5-proreasoning:gemini-3.1-pro-previewembedding-services:default:gemini-embedding-001agent:platform:models:googlegenai:api-key:${GOOGLE_API_KEY}# Or use Vertex AI authentication:# project-id: ${GOOGLE_PROJECT_ID}# location: ${GOOGLE_LOCATION}max-attempts:10backoff-millis:5000
```

Set a Google GenAI model as the default LLM

Set a Google GenAI embedding model as the default embedding model

Google GenAI specific configuration

API key can be set here or via environment variable GOOGLE_API_KEY

Mistral AI

- Required:
- MISTRAL_API_KEY: API key for Mistral AI services
- Optional:
- MISTRAL_BASE_URL: base URL for Mistral AI API (default: api.mistral.ai)

Alternatively, configure via application.yml:

```
embabel:agent:platform:models:mistralai:api-key:${MISTRAL_API_KEY:your-dev-key}base-url:${MISTRAL_BASE_URL:}
```

You also need to add the embabel-agent-starter-mistral-ai starter.

Add this to your build system as follows:

Maven (pom.xml)Gradle Kotlin DSL (build.gradle.kts)Gradle Groovy DSL (build.gradle)

```
<dependency><groupId>com.embabel.agent</groupId><artifactId>embabel-agent-starter-mistral-ai</artifactId><version>${embabel-agent.version}</version></dependency>
```

LM Studio

LM Studio is a desktop application that lets you easily discover, download, and run powerful LLMs on your own computer (Windows, Mac, Linux) for free, enabling offline use, local document Q&A, and even hosting an OpenAI-compatible API server for your projects, making advanced AI accessible without relying on cloud services. It supports formats like GGUF and offers privacy and control over your models.

The LM Studio **Local Server** allows you to run an LLM API server on localhost.

To add LM Studio support, add the embabel-agent-starter-lmstudio starter.

Add this to your build system as follows:

Maven (pom.xml)Gradle Kotlin DSL (build.gradle.kts)Gradle Groovy DSL (build.gradle)

```
<dependency><groupId>com.embabel.agent</groupId><artifactId>embabel-agent-starter-lmstudio</artifactId><version>${embabel-agent.version}</version></dependency>
```

Configure an LLM API on LM Studio by following [these instructions](#). Specify the example configuration in application.yml. Below I have an open-ai llm and embedding model downloaded on my LLM studio and exposed via the LLM Server.

```
embabel:agent:platform:autonomy:agent-confidence-cut-off:0.8goal-confidence-cut-off:0.8models:lmstudio:base-url:http://127.0.0.1:1234models:default-llm:openai/gpt-oss-20bdefault-embedding-model:text-embedding-nomic-embed-text-v1.5Ollama
```

[Ollama](#) is an open source application that lets you easily [discover](#), download, and run powerful LLMs on your own computer (Windows, Mac, Linux) for free, enabling offline use, local document Q&A, and even hosting an API server for your projects, making advanced AI accessible without relying on cloud services.

The Ollama application allows you to run an LLM API server on localhost. Exposing both its own Ollama API and an Open-AI-compatible API.

Get Ollama running locally by following [these instructions](#).

To use the Ollama API with your agent, add the embabel-agent-starter-ollama starter.

Add this to your build system as follows:

Maven (pom.xml)Gradle Kotlin DSL (build.gradle.kts)Gradle Groovy DSL (build.gradle)

```
<dependency><groupId>com.embabel.agent</groupId><artifactId>embabel-agent-starter-ollama</artifactId><version>${embabel-agent.version}</version></dependency>
```

Specify the example configuration in application.yml. Embabel uses Spring AI configuration to configure the Ollama integration.

```
spring:ai:ollama:# Not needed when using the default portbase-url:http://localhost:11434embabel:models:defaultLlm:ministral-3:8bdefault-embedding-model:qwen3-embedding
```

To instead use the Open-AI-compatible API with your agent, add the embabel-agent-starter-openai-custom starter.

Add this to your build system as follows:

Maven (pom.xml)Gradle Kotlin DSL (build.gradle.kts)Gradle Groovy DSL (build.gradle)

```
<dependency><groupId>com.embabel.agent</groupId><artifactId>embabel-agent-starter-openai-custom</artifactId><version>${embabel-agent.version}</version></dependency>
```

Specify the example configuration in application.yml. Embabel uses Spring AI configuration to configure the Ollama integration.

```
embabel:agent:platform:models:openai:custom:api-key:not-set# Property needed but value not used by Ollamabase-url:http://localhost:11434models:ministral-3:8b,qwen3-embeddingmodels:defaultLlm:ministral-3:8bdefault-embedding-model:qwen3-embedding
```

2.3. Getting Embabel Running

2.3.1. Running the Examples

The quickest way to get started with Embabel is to run the examples:

```
# Clone and run examplesgit clone https://github.com/embabel/embabel-agent-examplescd embabel-agent-examples/scripts/java./shell.sh
```

Choose the java or kotlin scripts directory depending on your preference.

2.3.2. Prerequisites

- Java 21+
- API Key from OpenAI, Anthropic, or Google
- Maven 3.9+ (optional)

Set your API keys:

```
export OPENAI_API_KEY="your_openai_key"export ANTHROPIC_API_KEY="your_anthropic_key"export GOOGLE_API_KEY="your_google_api_key"
```

For Google GenAI, you can use either GOOGLE_API_KEY (Google AI Studio) or Vertex AI authentication with GOOGLE_PROJECT_ID and GOOGLE_LOCATION.

2.3.3. Using the Shell

Spring Shell is an easy way to interact with the Embabel agent framework, especially during development.

Type help to see available commands. Use execute or x to run an agent:

```
execute "Lynda is a Scorpio, find news for her" -p -r
```

This will look for an agent, choose the star finder agent and run the flow. -p will log prompts -r will log LLM responses. Omit these for less verbose logging.

Options:

- -p logs prompts
- -r logs LLM responses

Use the chat command to enter an interactive chat with the agent. It will attempt to run the most appropriate agent for each command.

Spring Shell supports history. Type !! to repeat the last command. This will survive restarts, so is handy when iterating on an agent.

2.3.4. Example Commands

Try these commands in the shell:

```
# Simple horoscope agent
```

```
execute "My name is Sarah and I'm a Leo"
```

```
# Research with web tools (requires Docker Desktop with MCP extension)
```

```
execute "research the recent australian federal election. what is the position of the Greens party?"
```

```
# Fact checking
```

```
x "fact check the following: holden cars are still made in australia"
```

2.3.5. Implementing Your Own Shell Commands

Particularly during development, you may want to implement your own shell commands to try agents or flows. Simply write a Spring Shell component and Spring will inject it and register it automatically.

For example, you can inject the AgentPlatform and use it to invoke agents directly, as in this code from the examples repository:

JavaKotlin

```
@ShellComponentpublicrecordSupportAgentShellCommands(AgentPlatformagentPlatform){@ShellMethod("Get bank support for a customer query")publicStringbankSupport(@ShellOption(value="id",help="customer id",defaultValue="123")Longid,@ShellOption(value="query",help="customer query",defaultValue="What's my balance, including pending amounts?")Stringquery){varsupportInput=newSupportInput(id,query);System.out.println("Support input: "+supportInput);varinvocation=AgentInvocation.builder(agentPlatform).options(ProcessOptions.builder().verbosity(v->v.showPrompts(true))).build().build(SupportOutput.class);varresult=invocation.invoke(supportInput);returnresult.toString();}}
```

2.4. Adding a Little AI to Your Application

Before we get into the magic of full-blown Embabel agents, let's see how easy it is to add a little AI to your application using the Embabel framework. Sometimes this is all you need.

The simplest way to use Embabel is to inject an OperationContext and use its AI capabilities directly. This approach is consistent with standard Spring dependency injection patterns.

JavaKotlin


```
packagecom.embabel.example.injection;importcom.embabel.agent.api.common.OperationContext;importcom.embabel.common.ai.model.LlmOptions;importorg.springframework.stereotype.Component;/**
 * Demonstrate the simplest use of Embabel's AI capabilities,
 * injecting an AI helper into a Spring component.
 * The jokes will be terrible, but don't blame Embabel, blame the LLM.
 */
```

```
@ComponentpublicrecordInjectedComponent(Aiai){publicrecordJoke(Stringleadup,Stringpunchline){}publicStringtellJokeAbout(Stringtopic){returnai.withDefaultLlm().generateText("Tell me a joke about "+topic);}publicJokecreateJokeObjectAbout(Stringtopic1,Stringtopic2,Stringvoice){returnai.withLlm(LlmOptions.withDefaultLlm().withTemperature(.8)).createObject("""
    Tell me a joke about %s and %s.
    The voice of the joke should be %s.
    The joke should have a leadup and a punchline.
    """.formatted(topic1,topic2,voice),Joke.class);}}
```

This example demonstrates several key aspects of Embabel’s design philosophy:

- **Standard Spring Integration:** The Ai object is injected like any other Spring dependency using constructor injection
- **Simple API:** Access AI capabilities through the Ai interface directly or OperationContext.ai(), which can also be injected in the same way
- **Flexible Configuration:** Configure LLM options like temperature on a per-call basis
- **Type Safety:** Generate structured objects directly with createObject() method
- **Consistent Patterns:** Works exactly like you’d expect any Spring component to work

The Ai type provides access to all of Embabel’s AI capabilities without requiring a full agent setup, making it perfect for adding AI features to existing applications incrementally.

The Ai and OperationContext` APIs are used throughout Embabel applications, as a convenient gateway to key AI and other functionality.

2.5. Writing Your First Agent

The easiest way to create your first agent is to use the [Java or Kotlin template repositories](#).

2.5.1. Example: WriteAndReviewAgent

The template includes a WriteAndReviewAgent that demonstrates key concepts:

```
JavaKotlin
@Agent(description="Agent that writes and reviews stories")publicclassWriteAndReviewAgent{@ActionpublicStorywriteStory(UserInputuserInput,OperationContextcontext){returncontext.ai().withAutoLlm().createObject("""
    You are a creative writer who aims to delight and surprise.
    Write a story about %s
    """.formatted(userInput.getContent()),Story.class);}@AchievesGoal(description="Review a story")@ActionpublicReviewedStoryreviewStory(Storystory,OperationContextcontext){returncontext.ai().withLlmByRole("reviewer").createObject("""
    You are a meticulous editor.
    Carefully review this story:
    %s
    """.formatted(story.text),ReviewedStory.class);}}
```

2.5.2. Key Concepts Demonstrated

Multiple LLMs with Different Configurations:

- Writer LLM uses high temperature (0.8) for creativity
- Reviewer LLM uses low temperature (0.2) for analytical review
- Different personas guide the model behavior

Actions and Goals:

- @Action methods are the steps the agent can take
- @AchievesGoal marks the final action that completes the agent’s work

Domain Objects:

- Story and ReviewedStory are strongly-typed domain objects
- Help structure the interaction between actions

2.5.3. Running Your Agent

Set your API keys and run the shell:

```
export OPENAI_API_KEY="your_key_here" ./scripts/shell.sh
```

In the shell, try:

```
x "Tell me a story about a robot learning to paint"
```

The agent will:

1. Generate a creative story using the writer LLM
2. Review and improve it using the reviewer LLM
3. Return the final reviewed story

2.5.4. Next Steps

- Explore the [examples repository](#) for more complex agents
- Read the [Reference Documentation](#) for detailed API information
- Try building your own domain-specific agents

2.6. Embabel Modules

Embabel spans multiple modules, in this and other repositories in the embabel organization.

The status of these modules varies. There are three statuses:

- **Stable:** these modules are considered production ready. We strive to avoid breaking changes.
- **Incubating:** these modules are under active development and may have breaking changes in minor releases. However, they are considered generally usable and can be expected to graduate to stable. Use with caution.
- **Experimental:** these modules are early stage and may have breaking changes in any release. They are not recommended for production use. These modules may be removed without replacement and there is no guarantee of them graduating to a more stable status.

Of course, contributions are welcome to all modules!

2.6.1. Module Directory

The following are modules intended for direct use (versus supporting infrastructure).

Core Modules

Name	Location	Purpose	Notes	Status
embabel-agent-api	This repo	Core API	Main programming interface for building agents	Stable

embabel-agent-domain	This repo	Domain types and entities	Shared domain model	Incubating
----------------------	-----------	---------------------------	---------------------	------------

Feature Modules

Name	Location	Purpose	Notes	Status
embabel-agent-a2a	This repo	Agent-to-Agent protocol support	Google A2A protocol implementation	Incubating
embabel-agent-code	This repo	Coding domain library	Code analysis and generation utilities	Stable
embabel-agent-discord	This repo	Discord bot integration	Build agents as Discord bots	Experimental
embabel-agent-eval	This repo	Agent evaluation framework	Assess agent performance on tasks	Experimental
embabel-agent-mcpserver	This repo	MCP server support	Export agents as MCP servers	Stable
embabel-agent-openai	This repo	OpenAI-specific utilities	Structured outputs, response format	Stable
embabel-agent-remote	This repo	Remote action support	Execute actions on remote systems, enabling dynamic registration to extend the capabilities of an Embabel server	Experimental
embabel-agent-shell	This repo	Command-line interface	Interactive shell for agent development	Stable
embabel-agent-skills	This repo	Support for emerging Agent Skills standard	Composable agent skills	Experimental
embabel-agent-spec	This repo	Serializable action and goal definitions	Enables agents to be defined in YML or otherwise persisted in a serialized format	Experimental

RAG and Context Engineering Modules

Name	Location	Purpose	Notes	Status
embabel-agent-rag-core	This repo	Core RAG abstractions	Base interfaces for RAG, encompassing programming model (ToolishRag), storage abstractions (SearchOperations) and document model.	Stable
embabel-agent-rag-lucene	This repo	Lucene RAG store	Local storage with Apache Lucene supporting vector and text search	Stable
embabel-agent-rag-tika	This repo	Apache Tika integration	Document parsing (Markdown, PDF, Word, etc.)	Incubating
embabel-agent-rag-neo-drivine	embabel/embabel-agent-rag-neo-drivine	Neo4j graph RAG	RAG store for Neo4j graph database	Incubating
embabel-rag-pgvector	embabel/embabel-rag-pgvector	PostgreSQL pgvector RAG	RAG store for PostgreSQL with pgvector extension supporting hybrid search (vector, full-text, fuzzy)	Incubating
dice	embabel/dice	Support for Domain Oriented Context Engineering	Sophisticated pipeline for context engineering and integration with enterprise data. Incorporates proposition extraction and projection into knowledge graphs, memory and experimental representations.	Incubating

Spring Boot Starters

Name	Location	Purpose	Notes	Status
embabel-agent-starter	This repo	Base starter	Core dependencies only (no LLM provider)	Stable
embabel-agent-starter-anthropic	This repo	Anthropic starter	Quick start with Claude	Stable
embabel-agent-starter-openai	This repo	OpenAI starter	Quick start with GPT	Stable
embabel-agent-starter-ollama	This repo	Ollama starter	Quick start with local Ollama	Stable
embabel-agent-starter-shell	This repo	Shell starter	Add interactive shell for development	Stable
embabel-agent-starter-a2a	This repo	A2A starter	Add A2A server support	Incubating
embabel-agent-starter-mcpserver	This repo	MCP server starter	Add MCP server support	Stable
embabel-agent-starter-bedrock	This repo	Bedrock starter	Quick start with AWS Bedrock	Stable
embabel-agent-starter-deepseek	This repo	DeepSeek starter	Quick start with DeepSeek	Stable
embabel-agent-starter-gemini	This repo	Gemini starter	Quick start with Vertex AI	Stable
embabel-agent-starter-google-genai	This repo	Google GenAI starter	Quick start with AI Studio	Incubating
embabel-agent-starter-lmstudio	This repo	LM Studio starter	Quick start with LM Studio	Incubating
embabel-agent-starter-mistral-ai	This repo	Mistral AI starter	Quick start with Mistral	Stable
embabel-agent-starter-dockermodels	This repo	Docker Models starter	Quick start with Docker Desktop AI	Stable
embabel-agent-starter-openai-custom	This repo	Custom OpenAI starter	Quick start with OpenRouter, etc.	Stable

Test Support

Name	Location	Purpose	Notes	Status
embabel-agent-test	This repo	Test utilities	JUnit extensions, test DSL	Incubating

Example Repositories

Name	Location	Purpose	Notes	Status
embabel-agent-examples	embabel/embabel-agent-examples	Example agents	Sample implementations and tutorials	Stable

java-agent-template	embabel/java-agent-template	Java project template	Starter template for Java agents	Stable
---------------------	-----------------------------	-----------------------	----------------------------------	--------

2.6.2. Experimental APIs

While the status of modules may change over time, any module may contain clearly identified experimental functionality. This enables us to innovate in the open without excessive build complexity. Please try and provide feedback on this functionality, but don’t rely on it and be aware that it may change without notice. Any type or method annotated with the @ApiStatus.Experimental annotation is not guaranteed to be stable.

3. Reference

3.1. Invoking an Agent

Agents can be invoked programmatically or via user input. See [Invoking Embabel Agents](#) for details on programmatic invocation. Programmatic invocation typically involves structured types other than user input. In the case of user input, an LLM will choose the appropriate agent via the Autonomy class. Behavior varies depending on configuration:

- In closed mode, the LLM will select the agent based on the user input and the available agents in the system.
- In open mode, the LLM will select the goal based on the user input and then assemble an agent that can achieve that goal from the present world state.

3.2. Agent Process Flow

When an agent is invoked, Embabel creates an AgentProcess with a unique identifier that manages the complete execution lifecycle.

3.2.1. AgentProcess Lifecycle

An AgentProcess maintains state throughout its execution and can transition between various states:

Process States:

- NOT_STARTED: The process has not started yet
- RUNNING: The process is executing without any known problems
- COMPLETED: The process has completed successfully
- FAILED: The process has failed and cannot continue
- TERMINATED: The process was killed by an early termination policy
- KILLED: The process was killed by the user or platform
- STUCK: The process cannot formulate a plan to progress (may be temporary)
- WAITING: The process is waiting for user input or external event
- PAUSED: The process has paused due to scheduling policy

Process Execution Methods:

- tick(): Perform the next single step and return when an action completes
- run(): Execute the process as far as possible until completion, failure, or a waiting state

These methods are not directly called by user code, but are managed by the framework to control execution flow.

Each AgentProcess maintains:

- **Unique ID:** Persistent identifier for tracking and reference
- **History:** Record of all executed actions with timing information
- **Goal:** The objective the process is trying to achieve
- **Failure Info:** Details about any failure that occurred
- **Parent ID:** Reference to parent process for nested executions

3.2.2. Planning

Planning occurs after each action execution using Goal-Oriented Action Planning (GOAP). The planning process:

1. **Analyze Current State:** Examine the current blackboard contents and world state
2. **Identify Available Actions:** Find all actions that can be executed based on their preconditions
3. **Search for Action Sequences:** Use A* algorithm to find optimal paths to achieve the goal
4. **Select Optimal Plan:** Choose the best action sequence based on cost and success probability
5. **Execute Next Action:** Run the first action in the plan and replan

This creates a dynamic **OODA loop** (Observe-Orient-Decide-Act): - **Observe:** Check current blackboard state and action results
- **Orient:** Understand what has changed since the last planning cycle - **Decide:** Formulate or update the plan based on new information
- **Act:** Execute the next planned action

The replanning approach allows agents to:

- Adapt to unexpected action results
- Handle dynamic environments where conditions change
- Recover from partial failures
- Take advantage of new opportunities that arise

3.2.3. Blackboard

The Blackboard serves as the shared memory system that maintains state throughout the agent process execution. It implements the [Blackboard architectural pattern](#), a knowledge-based system approach.

Most of the time, user code doesn’t need to interact with the blackboard directly, as it is managed by the framework. For example, action inputs come from the blackboard, and action outputs are automatically added to the blackboard, and conditions are evaluated based on its contents.

Key Characteristics:

- **Central Repository:** Stores all domain objects, intermediate results, and process state
- **Type-Based Access:** Objects are indexed and retrieved by their types
- **Ordered Storage:** Objects maintain the order they were added, with latest being default
- **Immutable Objects:** Once added, objects cannot be modified (new versions can be added)
- **Condition Tracking:** Maintains boolean conditions used by the planning system

Core Operations:

```
JavaKotlin
// Add objects to blackboard blackboard.add(person);blackboard.set("result",analysis);// Retrieve objects by
typePersonperson=blackboard.last(Person.class);List<Person>allPersons=blackboard.all(Person.class);// Check
conditionsblackboard.setCondition("userVerified",true);booleanverified=blackboard.getCondition("userVerified");// Hide an
objectblackboard.hide(somethingWeDontWantToPlanOnLater);
```

Adding Objects: Objects are added to the blackboard automatically when returned from action methods, so you don’t typically need to call this API. They can also be added manually using the += operator (Kotlin only) or add/set method with an optional key.

Conditions: Conditions are normally calculated in @Condition methods, so you don’t usually need to check or set them via the API.

Hiding Objects: Prevents an object from being considered in future planning cycles. For example, the object might be a command that we have handled. It will remain in the blackboard history but will not be available to planning or via the Blackboard API.

Data Flow:

- 1. **Input Processing:** Initial user input is added to the blackboard
- 2. **Action Execution:** Each action reads inputs from blackboard and adds results
- 3. **State Evolution:** Blackboard accumulates objects representing the evolving state
- 4. **Planning Input:** Current blackboard state informs the next planning cycle
- 5. **Result Extraction:** Final results are retrieved from blackboard upon completion

The blackboard enables:

- **Loose Coupling:** Actions don't need direct references to each other
- **Flexible Data Flow:** Actions can consume any available data of the right type
- **State Persistence:** Complete execution history is maintained
- **Debugging Support:** Full visibility into state evolution for troubleshooting

3.2.4. Binding

By default, items in the blackboard are matched by type. When there are multiple candidates of the same type, the most recently added one is provided. It is also possible to assign a specific name to blackboard items.

An example of explicit binding in an action method:

```
JavaKotlin
@ActionpublicPersonextractPerson(UserInputuserInput,OperationContextcontext){PersonImplmaybeAPerson=context.promptRunner().withLlm(LlmOptions.fromModel(OpenAiModels.GPT_41)).createObjectIfPossible("""
    Create a person from this user input, extracting their name:
```

```
%s""".formatted(userInput.getContent()),PersonImpl.class);if(maybeAPerson!=null){context.bind("user",maybeAPerson);}returnmaybeAPerson;}
```

Explicit binding to the blackboard. Not usually necessary as action method return values are automatically bound.

The following example requires a Thing named thingOne to be present in the blackboard:

```
JavaKotlin
@ActionpublicWhateverdoWithThing(@RequireNameMatchThingthingOne){
```

The @RequireNameMatch annotation on the parameter specifies that the parameter should be matched by both type and name. Multiple parameters can be so annotated.

The following example uses @Action.outputBinding to cause a thingOne to be bound in the blackboard, satisfying the previous example:

```
JavaKotlin
@Action(outputBinding="thingOne")publicThingbindThing1(){...
```

When routing flows by type, the name is not important, but for reference the default name is 'it'.

3.2.5. Context

Embabel offers a way to store longer term state: the com.embabel.agent.core.Context. While a blackboard is tied to a specific agent process, a context can persist across multiple processes.

Contexts are identified by a unique contextId string. When starting an agent process, you can specify a contextId in the ProcessOptions. This will populate that process's blackboard with any data stored in the specified context.

Context persistence is dependent on the implementation of com.embabel.agent.spi.ContextRepository. The default implementation works only in memory, so does not survive server restarts.

3.3. Goals, Actions and Conditions

3.4. Domain Objects

Domain objects in Embabel are not just strongly-typed data structures - they are real objects with behavior that can be selectively exposed to LLMs and used in agent actions.

3.4.1. Objects with Behavior

Unlike simple structs or DTOs, Embabel domain objects can encapsulate business logic and expose it to LLMs through the @Tool annotation. For example:

```
JavaKotlin
@EntitypublicclassCustomer{privateStringname;privateLoyaltyLevelloyaltyLevel;privateList<Order>orders;@Tool(description="Calculate the customer's loyalty discount percentage")publicBigDecimalgetLoyaltyDiscount(){returnloyaltyLevel.calculateDiscount(orders.size());}@Tool(description="Check if customer is eligible for premium service")publicbooleanisPremiumEligible(){returnorders.stream().mapToDouble(Order::getTotal).sum()>1000.0;}publicvoidupdateLoyaltyLevel(){// Internal business logic}}
```

The @Tool annotation exposes this method to LLMs when the object is added via PromptRunner.withToolObject().

Unannotated methods such as updateLoyaltyLevel are never exposed to LLMs, regardless of their visibility level. This ensures that tool exposure is safe, explicit and controlled.

3.4.2. Selective Tool Exposure

The @Tool annotation allows you to selectively expose domain object methods to LLMs. For example:

- **Business Logic:** Expose methods that provide *safely invocable* business value to the LLM
- **Calculated Properties:** Methods that compute derived values. This can help LLMs with calculations they might otherwise get wrong.
- **Business Rules:** Methods that implement domain-specific rules

Always keep internal implementation details hidden, and think carefully before exposing methods that mutate state or have side effects.

3.4.3. Use of Domain Objects in Actions

Domain objects can be used naturally in action methods, combining LLM interactions with traditional object-oriented programming. The availability of the domain object instances also drives Embabel planning.

```
JavaKotlin
@ActionpublicRecommendationgenerateRecommendation(Customercustomer,OperationContextcontext){varprompt=String.format("Generate a personalized recommendation for %s based on their profile",customer.getName());returncontext.ai().withToolObject(customer).withDefaultLlm().createObject(prompt,Recommendation.class);}
```

The Customer domain object is provided as a tool object, allowing the LLM to call its @Tool methods. The LLM has access to customer.getLoyaltyDiscount() and customer.isPremiumEligible().

Domain object methods, even if annotated, will not be exposed to LLMs unless explicitly added via withToolObject().

3.4.4. Domain Understanding is Critical

As outlined in [Context Engineering Needs Domain Understanding](#), Rod Johnson's blog introducing DICE (Domain-Integrated Context Engineering), domain understanding is fundamental to effective context engineering. Domain objects serve as the bridge between:

- **Business Domain:** Real-world entities and their relationships
- **Agent Behavior:** How LLMs understand and interact with the domain
- **Code Actions:** Traditional programming logic that operates on domain objects

3.4.5. Benefits

- **Rich Context:** LLMs receive both data structure and behavioral context
- **Encapsulation:** Business logic stays within domain objects where it belongs
- **Reusability:** Domain objects can be used across multiple agents
- **Testability:** Domain logic can be unit tested independently
- **Evolution:** Adding new tools to domain objects extends agent capabilities

This approach ensures that agents work with meaningful business entities rather than generic data structures, leading to more natural and effective AI interactions.

3.5. Configuration

3.5.1. Enabling Embabel

Annotate your Spring Boot application class to get agentic behavior.

Example:

```
JavaKotlin
@SpringBootApplicationpublicclassMyAgentApplication{publicstaticvoidmain(String[]args){SpringApplication.run(MyAgentApplication.class,args);}}
```

This is a normal Spring Boot application class. You can add other Spring Boot annotations as needed.

You also need to add the [dependency and configuration for your LLM provider\(s\) of choice](#).

3.5.2. Configuration Properties

The following table lists all available configuration properties in Embabel Agent Platform. Properties are organized by their configuration prefix and include default values where applicable. They can be set via application.properties, application.yml, profile-specific configuration files or environment variables, as per standard Spring configuration practices.

Setting default LLM and roles

From ConfigurableModelProviderProperties - configuration for default LLMs and role-based model selection.

Property	Type	Default	Description
embabel.models.default-llm	String	gpt-4.1-mini	Default LLM name. It’s good practice to override this in configuration.
embabel.models.default-embedding-model	String	null	Default embedding model name. Need not be set, in which case it defaults to null.
embabel.models.llms	Map<String, String>	{}	Map of role to LLM name. Each entry will require an LLM to be registered with the same name. May not include the default LLM.
embabel.models.embedding-services	Map<String, String>	{}	Map of role to embedding service name. Does not need to include the default embedding service. You can create as many roles as you wish.

Role-based model selection allows you to assign specific LLMs or embedding services to different roles within your application. For example:

```
embabel:models:default-llm:gpt-4o-minidefault-embedding-model:text-embedding-3-smallllms:cheapest:gpt-4o-minibest:gpt-4oreasoning:o1-previewembedding-services:fast:text-embedding-3-smallaccurate:text-embedding-3-large
```

It’s good practice to decouple your code from specific models in this way.

Platform Configuration

From AgentPlatformProperties - unified configuration for all agent platform properties.

Property	Type	Default	Description
embabel.agent.platform.name	String	embabel-default	Core platform identity name
embabel.agent.platform.description	String	Embabel Default Agent Platform	Platform description

Logging Personality

Configuration for agent logging output style and theming.

Property	Type	Default	Description
embabel.agent.logging.personality	String	(none)	Themed logging messages to add personality to agent output

Value	Description
starwars	Star Wars themed logging messages
severance	Severance themed logging messages. Praise Kier
colossus	Colossus: The Forbin Project themed messages
hitchhiker	Hitchhiker’s Guide to the Galaxy themed messages
montypython	Monty Python themed logging messages

Table 1. Available Personality Values

Example Configuration

```
embabel:agent:logging:personality:hitchhiker
```

Agent Scanning

From AgentPlatformProperties.ScanningConfig - configures scanning of the classpath for agents.

Property	Type	Default	Description
embabel.agent.platform.scanning.annotation	Boolean	true	Whether to auto register beans with @Agent and @Agentic annotation
embabel.agent.platform.scanning.bean	Boolean	false	Whether to auto register as agents Spring beans of type Agent

Ranking Configuration

From AgentPlatformProperties.RankingConfig - configures ranking of agents and goals based on user input when the platform should choose the agent or goal.

Property	Type	Default	Description
embabel.agent.platform.ranking.llm	String	null	Name of the LLM to use for ranking, or null to use auto selection

embabel.agent.platform.ranking.max-attempts	Int	5	Maximum number of attempts to retry ranking
embabel.agent.platform.ranking.backoff-millis	Long	100	Initial backoff time in milliseconds
embabel.agent.platform.ranking.backoff-multiplier	Double	5.0	Multiplier for backoff time
embabel.agent.platform.ranking.backoff-max-interval	Long	180000	Maximum backoff time in milliseconds

LLM Operations

From AgentPlatformProperties.LlmOperationsConfig - configuration for LLM operations including prompts and data binding.

Property	Type	Default	Description
embabel.agent.platform.llm-operations.prompts.maybe-prompt-template	String	maybe_prompt_contribution	Template for "maybe" prompt, enabling failure result when LLM lacks information
embabel.agent.platform.llm-operations.prompts.generate-examples-by-default	Boolean	true	Whether to generate examples by default
embabel.agent.platform.llm-operations.data-binding.max-attempts	Int	10	Maximum retry attempts for data binding
embabel.agent.platform.llm-operations.data-binding.fixed-backoff-millis	Long	30	Fixed backoff time in milliseconds between retries

Process ID Generation

From AgentPlatformProperties.ProcessIdGenerationConfig - configuration for process ID generation.

Property	Type	Default	Description
embabel.agent.platform.process-id-generation.include-version	Boolean	false	Whether to include version in process ID generation
embabel.agent.platform.process-id-generation.include-agent-name	Boolean	false	Whether to include agent name in process ID generation

Autonomy Configuration

From AgentPlatformProperties.AutonomyConfig - configures thresholds for agent and goal selection. Certainty below thresholds will result in failure to choose an agent or goal.

Property	Type	Default	Description
embabel.agent.platform.autonomy.agent-confidence-cut-off	Double	0.6	Confidence threshold for agent operations
embabel.agent.platform.autonomy.goal-confidence-cut-off	Double	0.6	Confidence threshold for goal achievement

Model Provider Configuration

From AgentPlatformProperties.ModelsConfig - model provider integration configurations.

Anthropic

Property	Type	Default	Description
embabel.agent.platform.models.anthropic.max-attempts	Int	10	Maximum retry attempts
embabel.agent.platform.models.anthropic.backoff-millis	Long	5000	Initial backoff time in milliseconds
embabel.agent.platform.models.anthropic.backoff-multiplier	Double	5.0	Backoff multiplier
embabel.agent.platform.models.anthropic.backoff-max-interval	Long	180000	Maximum backoff interval in milliseconds

OpenAI

Property	Type	Default	Description
embabel.agent.platform.models.openai.max-attempts	Int	10	Maximum retry attempts
embabel.agent.platform.models.openai.backoff-millis	Long	5000	Initial backoff time in milliseconds
embabel.agent.platform.models.openai.backoff-multiplier	Double	5.0	Backoff multiplier
embabel.agent.platform.models.openai.backoff-max-interval	Long	180000	Maximum backoff interval in milliseconds

Google GenAI (Native)

Uses the native Google GenAI SDK (spring-ai-google-genai) for direct access to Gemini models with full feature support.

Property	Type	Default	Description
embabel.agent.platform.models.googlegenai.max-attempts	Int	10	Maximum retry attempts
embabel.agent.platform.models.googlegenai.backoff-millis	Long	5000	Initial backoff time in milliseconds
embabel.agent.platform.models.googlegenai.backoff-multiplier	Double	5.0	Backoff multiplier
embabel.agent.platform.models.googlegenai.backoff-max-interval	Long	180000	Maximum backoff interval in milliseconds

Google GenAI models (both LLM and embedding) are configured via the embabel-agent-starter-google-genai starter dependency. The following embedding models are available:

Model Name	Model ID	Dimensions	Price (per 1M tokens)
gemini_embedding_001	gemini-embedding-001	3072	\$0.15

The following environment variables control authentication:

Environment Variable	Description
GOOGLE_API_KEY	API key for Google AI Studio authentication
GOOGLE_PROJECT_ID	Google Cloud project ID (for Vertex AI authentication)
GOOGLE_LOCATION	Google Cloud region, e.g., us-central1 (for Vertex AI authentication)

Either GOOGLE_API_KEY or both GOOGLE_PROJECT_ID and GOOGLE_LOCATION must be set. Gemini 3 models are only available in the global location on Vertex AI. To use Gemini 3 with Vertex AI, you must set GOOGLE_LOCATION=global.

To add new Google GenAI embedding models, edit the configuration file:
embabel-agent-autoconfigure/models/embabel-agent-google-genai-autoconfigure/
src/main/resources/models/google-genai-models.yml
embedding_models:-name:"gemini_embedding_001"model_id:"gemini-embedding-001"display_name:"Gemini Embedding
001"dimensions:3072pricing_model:usd_per1m_tokens:0.15

HTTP Client Configuration

From NettyClientFactoryProperties - configuration for the HTTP client used by model providers (OpenAI, Anthropic, etc.) when making API calls.

Embabel uses Reactor Netty as the HTTP client for improved performance and non-blocking I/O. This is particularly important for LLM API calls which can have long response times.

Dependency Requirement

To use the Netty client, you must manually add the following autoconfiguration dependency to your project:

<dependency><groupId>com.embabel.agent</groupId><artifactId>embabel-agent-netty-client-
autoconfigure</artifactId></dependency>

For Gradle:

implementation'com.embabel.agent:embabel-agent-netty-client-autoconfigure'

Property	Type	Default	Description
embabel.agent.platform.http-client.connect-timeout	Duration	25s	Connection timeout for establishing HTTP connections to model providers
embabel.agent.platform.http-client.read-timeout	Duration	1m	Read timeout (response timeout) for receiving responses from model providers. Increase this value for models that generate long responses or when using extended thinking features.

Example Configuration

embabel:agent:platform:http-client:connect-timeout:10sread-timeout:10m

For models with extended thinking enabled (like Claude with thinking mode), consider increasing read-timeout to 10m or higher to accommodate longer processing times.

When to Adjust Timeouts

- Long-running LLM calls:** If you experience timeout errors during complex reasoning tasks, increase read-timeout
- Slow network environments:** Increase connect-timeout if connection establishment is failing
- Streaming responses:** The read-timeout applies to the initial response; streaming content has its own handling

The HTTP client configuration applies to all model providers that use Spring’s RestClient and WebClient, including OpenAI, Anthropic, and OpenAI-compatible endpoints.

Server-Sent Events

From AgentPlatformProperties.SseConfig - server-sent events configuration.

Property	Type	Default	Description
embabel.agent.platform.sse.max-buffer-size	Int	100	Maximum buffer size for SSE
embabel.agent.platform.sse.max-process-buffers	Int	1000	Maximum number of process buffers

Test Configuration

From AgentPlatformProperties.TestConfig - test configuration.

Property	Type	Default	Description
embabel.agent.platform.test.mock-mode	Boolean	true	Whether to enable mock mode for testing

Process Repository Configuration

From ProcessRepositoryProperties - configuration for the agent process repository.

Property	Type	Default	Description
embabel.agent.platform.process-repository.window-size	Int	1000	Maximum number of agent processes to keep in memory when using default InMemoryAgentProcessRepository. When exceeded, oldest processes are evicted.

Standalone LLM Configuration

LLM Operations Prompts

From LlmOperationsPromptsProperties - properties for ChatClientLlmOperations operations.

Property	Type	Default	Description
embabel.llm-operations.prompts.maybe-prompt-template	String	maybe_prompt_contribution	Template to use for the "maybe" prompt, which can enable a failure result if the LLM does not have enough information to create the desired output structure
embabel.llm-operations.prompts.generate-examples-by-default	Boolean	true	Whether to generate examples by default
embabel.llm-operations.prompts.default-timeout	Duration	60s	Default timeout for operations

LLM Data Binding

From LlmDataBindingProperties - data binding properties with retry configuration for LLM operations.

Property	Type	Default	Description
embabel.llm-operations.data-binding.max-attempts	Int	10	Maximum retry attempts for data binding
embabel.llm-operations.data-binding.fixed-backoff-millis	Long	30	Fixed backoff time in milliseconds between retries

Additional Model Providers

AWS Bedrock

From BedrockProperties - AWS Bedrock model configuration properties.

Property	Type	Default	Description
embabel.models.bedrock.models	List	[]	List of Bedrock models to configure
embabel.models.bedrock.models[].name	String	""	Model name
embabel.models.bedrock.models[].knowledge-cutoff	String	""	Knowledge cutoff date

embabel.models.bedrock.models[].input-price	Double	0.0	Input token price
embabel.models.bedrock.models[].output-price	Double	0.0	Output token price

Docker Local Models

From DockerProperties - configuration for Docker local models (OpenAI-compatible).

Property	Type	Default	Description
embabel.docker.models.base-url	String	localhost:12434/engines	Base URL for Docker model endpoint
embabel.docker.models.max-attempts	Int	10	Maximum retry attempts
embabel.docker.models.backoff-millis	Long	2000	Initial backoff time in milliseconds
embabel.docker.models.backoff-multiplier	Double	5.0	Backoff multiplier
embabel.docker.models.backoff-max-interval	Long	180000	Maximum backoff interval in milliseconds

3.6. Annotation model

Embabel provides a Spring-style annotation model to define agents, actions, goals, and conditions. This is the recommended model to use in Java, and remains compelling in Kotlin.

3.6.1. The @Agent annotation

This annotation is used on a class to define an agent. It is a Spring stereotype annotation, so it triggers Spring component scanning. Your agent class will automatically be registered as a Spring bean. It will also be registered with the agent framework, so it can be used in agent processes.

You must provide the description parameter, which is a human-readable description of the agent. This is particularly important as it may be used by the LLM in agent selection.

3.6.2. The @EmbabelComponent annotation

This annotation is used on a class to indicate that this class exposes actions, goals and conditions that may be used by agents, but is not an agent in itself. It is a Spring stereotype annotation, so it triggers Spring component scanning. Your Embabel component class will automatically be registered as a Spring bean. It will also be registered with the agent framework, so its actions, goals and conditions can be used in agent processes.

Embabel Components are most useful in combination with the [Utility AI planner](#) that selects the most valuable next action among all available actions.

3.6.3. The @Action annotation

The @Action annotation is used to mark methods that perform actions within an agent.

Action metadata can be specified on the annotation, including:

- description: A human-readable description of the action.
- pre: A list of preconditions *additional to the input types* that must be satisfied before the action can be executed.
- post: A list of postconditions *additional to the output type(s)* that may be satisfied after the action is executed.
- canRerun: A boolean indicating whether the action can be rerun if it has already been executed. Defaults to false.
- readOnly: A boolean indicating whether the action has no external side effects. Read-only actions only analyze data and produce derived objects without modifying external systems (APIs, databases, files, etc.). This is useful for learning/catchup modes where you want to ingest and understand data without triggering mutations. Defaults to false.
- clearBlackboard: A boolean indicating whether to clear the blackboard after this action completes. When true, all objects on the blackboard are removed except the action’s output. This is useful for resetting context in multi-step workflows. It can also make persistence of flows more efficient by dispensing with objects that are no longer needed. Defaults to false.
- cost:Relative cost of the action from 0-1. Defaults to 0.0.
- value: Relative value of performing the action from 0-1. Defaults to 0.0.

Clearing the Blackboard

The clearBlackboard attribute is useful in two scenarios:

- Multi-step workflows** where you want to reset the processing context
- Looping states** where an action returns to a previously-visited state type

When an action with clearBlackboard = true completes, all objects on the blackboard are removed except the action’s output. This prevents accumulated intermediate data from affecting subsequent processing and enables loops.

Looping States

The most common use case for clearBlackboard is enabling loops in state-based workflows:

JavaKotlin

```
@StaterecordProcessingState(Stringdata,intiteration){@Action(clearBlackboard=true)LoopOutcomeprocess(){if(iteration>=3){returnnewDoneState(data);}returnnewProcessingState(data+"+",iteration+1);}}
```

clearBlackboard = true enables returning to the same state type

Without clearing, returning ProcessingState would be blocked since the type already exists

See [Using States](#) for more details on looping state patterns.

Resetting Context

You can also use clearBlackboard to reset context in multi-step workflows:

JavaKotlin

```
@Agent(description="Multi-step document processing")publicclassDocumentProcessor{@Action(clearBlackboard=true)publicProcessedDocumentpreprocess(RawDocumentdoc){returnnewProcessedDocument(doc.getContent().trim());}@AchievesGoal(description="Produce final output")@ActionpublicFinalOutputtransform(ProcessedDocumentdoc){returnnewFinalOutput(doc.getContent().toUpperCase());}}
```

After preprocess completes, the blackboard is cleared and only ProcessedDocument remains. The original RawDocument is removed.

The transform action receives only the ProcessedDocument, not any earlier inputs.

Avoid using clearBlackboard on goal-achieving actions (those with @AchievesGoal). Clearing the blackboard removes hasRun tracking conditions, which may interfere with goal satisfaction. Use clearBlackboard on intermediate actions instead.

Dynamic Cost Computation with @Cost

While the cost and value fields on @Action allow specifying static values, you can compute these dynamically at planning time using the @Cost annotation. This is useful when the cost of an action depends on the current state of the blackboard.

The @Cost annotation marks a method that returns a cost value (a double between 0.0 and 1.0). You then reference this method from the @Action annotation using costMethod or valueMethod.

JavaKotlin


```
@Agent(description="Processor with dynamic cost")publicclassDataProcessor{@Cost(name="processingCost")publicdoublecomputeProcessingCost(@NullableLargeDataSetdata){if(data!=null&&data.size()>1000){return0.9;// High cost for large datasets}return0.1;// Low cost for small or missing datasets}@Action(costMethod="processingCost")publicProcessedDataprocess(RawDatainput){returnnewProcessedData(input.transform());}}
```

The @Cost annotation marks a method for dynamic cost computation. The name parameter identifies this cost method.

Domain object parameters in @Cost methods must be nullable. If the object isn’t on the blackboard, null is passed.

The costMethod field references the @Cost method by name.

Key differences from @Condition methods:

- All domain object parameters in @Cost methods must be nullable (use @Nullable in Java or ? in Kotlin)
- When a domain object is not available on the blackboard, null is passed instead of causing the method to fail
- The method must return a double between 0.0 and 1.0
- The Blackboard can be passed as a parameter for direct access to all available objects

You can also compute dynamic value using valueMethod:

JavaKotlin

```
@Agent(description="Agent with dynamic value computation")publicclassPrioritizedAgent{@Cost(name="urgencyValue")publicdoublecomputeUrgency(@NullableTasktask){if(task==null){return0.5;}if(task.getPriority()==Priority.HIGH){return1.0;}if(task.getPriority()==Priority.MEDIUM){return0.6;}return0.2;}}@AchievesGoal(description="Process high-priority tasks")@Action(valueMethod="urgencyValue")publicResultprocessTask(Tasktask){returnnewResult(String.format("Processed: %s",task.getName()));}}
```

The @Cost method is called during planning, before the action executes. It allows the planner to make informed decisions about which actions to prefer based on runtime state.

Dynamic cost is especially useful with **Utility planning** (PlannerType.UTILITY), where cost/value tradeoffs are a core concept. The utility planner evaluates actions based on their net value (value minus cost), making dynamic cost computation essential for sophisticated decision-making.

3.6.4. The @Condition annotation

The @Condition annotation is used to mark methods that evaluate conditions. They can take an OperationContext parameter to access the blackboard and other infrastructure. If they take domain object parameters, the condition will automatically be false until suitable instances are available.

Condition methods should not have side effects—for example, on the blackboard. This is important because they may be called multiple times.

Dynamic Conditions with SpEL

In addition to using @Condition methods, you can specify dynamic preconditions directly on @Action annotations using Spring Expression Language (SpEL). These expressions are evaluated against the blackboard, allowing you to create conditions based on runtime state without writing separate condition methods.

The expression language is pluggable, but currently SpEL is the only supported implementation. See the [Spring Expression Language \(SpEL\) documentation](#) for full syntax details.

SpEL conditions are specified in the pre array with a spel: prefix:

JavaKotlin

```
@Action(pre={"spel:assessment.urgency > 0.5"})publicvoidhandleUrgentIssue(Issueissue,IssueAssessmentassessment){// This action only runs when urgency exceeds 0.5}
```

The spel: prefix indicates this is a SpEL expression evaluated against the blackboard.

Expression Syntax

SpEL expressions reference blackboard objects by their binding names (typically the camelCase form of the class name). The expression must evaluate to a boolean.

JavaKotlin

```
@Agent(description="Issue triage agent")publicclassIssueTriageAgent{@Action(pre={"spel:issueAssessment.urgency > 0.0"})publicvoidescalateUrgentIssue(GHIssueissue,IssueAssessmentissueAssessment){logger.info("Escalating urgent issue #{",issue.getNumber());}@Action(pre={"spel:ghIssue instanceof T(org.kohsuke.github.GHPullRequest) && ghIssue.changedFiles > 10"})publicvoidreviewLargePullRequest(GHPullRequestissue,PullRequestAssessmentassessment){logger.info("Large PR detected: #{ with } files changed",issue.getNumber(),issue.getChangedFiles());}}
```

Simple property comparison: action fires only when urgency property exceeds 0.0.

Type check with property access: action fires only for pull requests with more than 10 changed files. The T() operator references a Java type for instanceof checks.

Collection Filtering

SpEL’s collection selection syntax (#{}) is useful for checking conditions on collections stored in the blackboard:

JavaKotlin

```
@Action(pre={"spel:newEntity.newEntities.[#this instanceof T(com.example.domain.Issue) "+"&& !(#this instanceof T(com.example.domain.PullRequest))].size() > 0"})publicIssueAssessmentreactToNewIssue(GHIssueghIssue,NewEntity<?>newEntity,Aiai){// Fires only when newEntities contains Issues that aren't PullRequestsreturnai.withLlm("claude-sonnet-4").creating(IssueAssessment.class).fromTemplate("issue_triage",Map.of("issue",ghIssue));}@Action(pre={"spel:newEntity.newEntities.[#this instanceof T(com.example.domain.PullRequest)].size() > 0"})publicPullRequestAssessmentreactToNewPullRequest(GHPullRequestpr,NewEntity<?>newEntity,Aiai){// Fires only when newEntities contains PullRequestsreturnai.withLlm("claude-sonnet-4").creating(PullRequestAssessment.class).fromTemplate("pr_triage",Map.of("pr",pr));}
```

The #{ operator filters the collection. #this refers to each element. This expression checks that at least one element is an Issue but not a PullRequest.

Simpler filter checking for PullRequest instances.

Common SpEL Patterns

Pattern	Description
spel:obj.property > value	Simple property comparison
spel:obj instanceof T(com.example.Type)	Type checking using fully qualified class name
spel:collection.size() > 0	Check collection is not empty
spel:collection.[condition].size() > 0	Check that filtered collection has elements

spel:obj.property != null	Null checking
spel:condition1 && condition2	Combining conditions with AND
spel:condition1 condition2	Combining conditions with OR

Use SpEL conditions for simple property checks and type discrimination. For complex logic or conditions that need to be reused across multiple actions, prefer @Condition methods. For reactive scenarios where you simply want an action to fire when a specific type is added to the blackboard, consider using the [trigger field](#) instead—it’s simpler than writing a SpEL expression.

Blackboard binding names are derived from the class name in camelCase by default. You can specify explicit binding names using outputBinding on actions or by adding objects to the blackboard with specific names.

Both Action and Condition methods may be inherited from superclasses. That is, annotated methods on superclasses will be treated as actions on a subclass instance.
Give your Action and Condition methods unique names, so the planner can distinguish between them.

3.6.5. Parameters

@Action methods must have at least one parameter. @Condition methods must have zero or more parameters, but otherwise follow the same rules as @Action methods regarding parameters. Ordering of parameters is not important.

Parameters fall in two categories:

- *Domain objects.* These are the normal inputs for action methods. They are backed by the blackboard and will be used as inputs to the action method. A nullable domain object parameter will be populated if it is non-null on the blackboard. This enables nice-to-have parameters that are not required for the action to run. In Kotlin, use a nullable parameter with ?: in Java, mark the parameter with the org.springframework.lang.Nullable or another Nullable annotation.
- *Infrastructure parameters,* such as the OperationContext, ProcessContext, and Ai may be used in action or condition methods.

Domain objects drive planning, specifying the preconditions to an action.

The ActionContext or ExecutingOperationContext subtype can be used in action methods. It adds asSubProcess methods that can be used to run other agents in subprocesses. This is an important element of composition.

Use the least specific type possible for parameters. Use OperationContext unless you are creating a subprocess.

Custom Parameters

Besides two default parameter categories described above, you can provide your own parameters by implementing the ActionMethodArgumentResolver interface. The two main methods of this interface are:

- supportsParameter, which indicates what kind of parameters are supported, and
 - resolveArgument, which resolves the argument into an object used to invoke the action method.
- Note the similarity with Spring MVC, where you can provide custom parameters by implementing a HandlerMethodArgumentResolver.

All default parameters are provided by ActionMethodArgumentResolver implementations.

To register your custom argument resolver, provide it to the DefaultActionMethodManager component in your Spring configuration. Typically, you will register (some of) the defaults as well your custom resolver, in order to support the default parameters. Make sure to register the BlackboardArgumentResolver as last resolver, to ensure that others take precedence.

The @Provided Annotation

The @Provided annotation marks an action method parameter as being provided by the platform context (such as Spring’s ApplicationContext) rather than resolved from the blackboard.

This is particularly useful for:

- **Accessing the enclosing component** from within @State classes (which must be static or top-level)
- **Injecting services** that aren’t domain objects but are needed for processing
- **Accessing configuration** or other platform-managed beans

```
JavaKotlin
@EmbabelComponentpublicclassReservationFlow{privatefinalBookingServicebookingService;privatefinalNotificationService
otificationService;publicReservationFlow(BookingServicebookingService,NotificationServicenotificationService){this.bookingServ
ice=bookingService;this.notificationService=notificationService;}@ActionpublicCollectDetailsstart(UserRequestrequest){returnne
wCollectDetails(request.customerId());}@StatepublicrecordCollectDetails(StringcustomerId){@ActionpublicConfirmReservatio
nconfirm(ReservationDetailsdetails,@ProvidedReservationFlowflow){varbooking=flow.bookingService.reserve(details);flow.noti
ficationService.sendConfirmation(booking);returnnewConfirmReservation(booking);}}@StatepublicrecordConfirmReservation(B
ookingbooking){@AchievesGoal(description="Reservation completed")@ActionpublicBookingResultcomplete(){returnnewBookingResult(booking);}}}
```

ReservationDetails is a domain object resolved from the blackboard.

ReservationFlow is injected via @Provided from the Spring context - this gives access to the services in the enclosing component.

How It Works

When Spring is available, the SpringContextProvider resolves @Provided parameters by looking up beans from the ApplicationContext. The parameter type must match a bean in the context.

```
JavaKotlin
@StatepublicrecordProcessingState(Stringdata){@ActionpublicNextStateprocess(@ProvidedMyServiceuserService,@ProvidedA
ppConfigconfig){varresult=userService.process(data,config.getSetting());returnnewNextState(result);}}
```

Any Spring bean can be injected using @Provided.

Multiple @Provided parameters can be used in a single method.

When to Use @Provided

Use @Provided when you need access to:

- The enclosing @EmbabelComponent or @Agent class from a @State action
- Services that are infrastructure concerns, not domain objects
- Configuration or environment values

Do **not** use @Provided for:

- Domain objects that should drive planning (use regular parameters instead)
- Objects that need to be tracked on the blackboard

Since @State classes must be static nested classes or top-level classes, @Provided is the recommended way to access the enclosing component’s services. This keeps state classes serializable while still providing access to dependencies.

@Provided parameters are resolved before blackboard parameters. If a type could come from either source, @Provided takes precedence.

3.6.6. Binding by name

The @RequireNameMatch annotation can be used to [bind parameters by name](#).

3.6.7. Reactive triggers with trigger

The trigger field on the @Action annotation enables reactive behavior where an action only fires when a specific type is the *most recently added* value to the blackboard. This is useful in event-driven scenarios where you want to react to a particular event even when multiple parameters of various types are available.

For example, in a chat system you might want an action to fire only when a new user message arrives, not when other context is updated:

JavaKotlin

```
@Agent(description="Chat message handler")publicclassChatAgent{@AchievesGoal(description="Respond to user message")@Action(trigger=UserMessage.class)publicResponsehandleMessage(UserMessagemessage,Conversationconversation){returnnewResponse("Received: "+message.content());}}
```

The trigger field means this action only fires when UserMessage is the last result added to the blackboard.

Conversation must also be available, but doesn't need to be the triggering event.

Without trigger, an action fires as soon as all its parameters are available on the blackboard. With trigger, the specified type must additionally be the most recent value added.

This is particularly useful when:

- You have multiple actions that could handle different event types
- You want to distinguish between "data available" and "event just occurred"
- You're building event-driven or reactive workflows

JavaKotlin

```
@Agent(description="Multi-event processor")publicclassEventProcessor{@Action(trigger=EventA.class)publicResulthandleEventA(EventAeventA,EventBeventB){returnnewResult("Triggered by A");}@AchievesGoal(description="Handle event B")@Action(trigger=EventB.class)publicResulthandleEventB(EventAeventA,EventBeventB){returnnewResult("Triggered by B");}}
```

handleEventA fires when EventA is added (and EventB is available).

handleEventB fires when EventB is added (and EventA is available).

The trigger field checks that the specified type matches the lastResult() on the blackboard. The last result is the most recent object added via any binding operation.

3.6.8. Handling of return types

Action methods normally return a single domain object.

Nullable return types are allowed. Returning null will trigger replanning. There may or not be an alternative path from that point, but it won't be what the planner was previously trying to achieve.

There is a special case where the return type can essentially be a union type, where the action method can return one ore more of several types. This is achieved by a return type implementing the SomeOf tag interface. Implementations of this interface can have multiple nullable fields. Any non-null values will be bound to the blackboard, and the postconditions of the action will include all possible fields of the return type.

For example:

JavaKotlin

```
// Must implement the SomeOf interfacepublicrecordFrogOrDog(@NullableFrogfrog,@NullableDogdog)implementsSomeOf{}@Agent(description="Illustrates use of the SomeOf interface")publicclassReturnsFrogOrDog{@ActionpublicFrogOrDogfrogOrDog(){returnnewFrogOrDog(newFrog("Kermit"),null);}// This works because the frog field of the return type was set@AchievesGoal(description="Create a prince from a frog")@ActionpublicPersonWithReverseTooltoPerson(Frogfrog){returnnewPersonWithReverseTool(frog.name());}}
```

This enables routing scenarios in an elegant manner.

Multiple fields of the SomeOf instance may be non-null and this is not an error. It may enable the most appropriate routing.

Routing can also be achieved via subtypes, as in the following example:

JavaKotlin

```
@ActionpublicIntentclassifyIntent(UserInputuserInput){returnswitch(userInput.content()){case"billing"->newBillingIntent();case"sales"->newSalesIntent();case"service"->newServiceIntent();default->{logger.warn("Unknown intent: ",userInput);yieldnull;}}}@ActionpublicIntentClassificationSuccessbillingAction(BillingIntentintent){returnnewIntentClassificationSuccess("billing");}@ActionpublicIntentClassificationSuccesssalesAction(SalesIntentintent){returnnewIntentClassificationSuccess("sales");};// ...
```

Classification action returns supertype Intent. Real classification would likely use an LLM.

billingAction and other action methods takes a subtype of Intent, so will only be invoked if the classification action returned that subtype.

3.6.9. Action method implementation

Embabel makes it easy to seamlessly integrate LLM invocation and application code, using common types. An @Action method is a normal method, and can use any libraries or frameworks you like.

The only special thing about it is its ability to use the OperationContext parameter to access the blackboard and invoke LLMs.

3.6.10. The @AchievesGoal annotation

The @AchievesGoal annotation can be added to an @Action method to indicate that the completion of the action achieves a specific goal.

3.6.11. Implementing the StuckHandler interface

If an annotated agent class implements the StuckHandler interface, it can handle situations where an action is stuck itself. For example, it can add data to the blackboard.

Example:

JavaKotlin

```
@Agent(description="self unsticking agent")publicclassSelfUnstickingAgentimplementsStuckHandler{privatebooleancalled=false;// The agent will get stuck as there's no dog to convert to a frog@Action@AchievesGoal(description="the big goal in the sky")publicFrogtoFrog(Dogdog){returnnewFrog(dog.name());}// This method will be called when the agent is stuck@OverridepublicStuckHandlerResulthandleStuck(AgentProcessagentProcess){called=true;agentProcess.addObject(newDog("Duke"));returnnewStuckHandlerResult("Unsticking myself",this,StuckHandlingResultCode.REPLAN,agentProcess);}}
```

3.6.12. Advanced Usage: Nested processes

An @Action method can invoke another agent process. This is often done to use a stereotyped process that is composed using the DSL. Use the ActionContext.asSubProcess method to create a sub-process from the action context.

For example:

JavaKotlin

```
@ActionpublicScoredResult<Report,SimpleFeedback>report(ReportRequestreportRequest,ActionContextcontext){returncontext.asSubProcess{// Will create an agent sub process with strong typingEvaluatorOptimizer.generateUntilAcceptable(5,ctx-
```



```
>ctx.promptRunner().withToolGroup(CoreToolGroups.WEB).create(String.format("""
    Given the topic, generate a detailed report in %d words.
# Topic %s
# Feedback %s
""",reportRequest.words(),reportRequest.topic(),ctx.getInput()!=null?ctx.getInput():"No feedback provided")),ctx-
>ctx.promptRunner().withToolGroup(CoreToolGroups.WEB).create(String.format("""
    Given the topic and word count, evaluate the report and provide feedback
    Feedback must be a score between 0 and 1, where 1 is perfect.
# Report %s
# Report request: %s
    Word count: %d
""",ctx.getInput().report(),reportRequest.topic(),reportRequest.words())));}
```

3.6.13. Running Subagents with RunSubagent

The RunSubagent utility provides a convenient way to run a nested agent from within an @Action method without needing direct access to ActionContext. This is particularly useful when you want to delegate work to another @Agent-annotated class or an Agent instance.

Running an @Agent-annotated Instance

Use RunSubagent.fromAnnotatedInstance() when you have an instance of a class annotated with @Agent: The annotated instance can be Spring-injected into your agent class. Since @Agent is a Spring stereotype annotation, you can inject one agent into another and run it as a subagent. This enables clean separation of concerns while maintaining testability.

```
JavaKotlin
@Agent(description="Outer agent that delegates to an injected
subagent")publicclassOuterAgent{privatefinalInnerSubAgentinnerSubAgent;publicOuterAgent(InnerSubAgentinnerSubAgent){this.innerSubAgent=innerSubAgent;}@ActionpublicTaskOutputstart(UserInputinput){returnRunSubagent.fromAnnotatedInstance(innerSubAgent,TaskOutput.class);}@Action@AchievesGoal(description="Processing
complete")publicTaskOutputdone(TaskOutputoutput){returnoutput;}}@Agent(description="Inner subagent that processes
input")publicclassInnerSubAgent{@ActionpublicIntermediatestepOne(UserInputinput){returnnewIntermediate(input.getContext());}@Action@AchievesGoal(description="Subagent
complete")publicTaskOutputstepTwo(Intermediatedata){returnnewTaskOutput(data.value().toUpperCase());}}
```

Spring injects the InnerSubAgent bean via constructor injection.

The injected instance is passed to RunSubagent.fromAnnotatedInstance().

In Kotlin, you can use the reified version for a more concise syntax:

```
JavaKotlin
@Agent(description="Outer agent via explicit type
parameter")publicclassOuterAgentExplicit{@ActionpublicTaskOutputstart(UserInputinput){returnRunSubagent.fromAnnotate
dInstance(newInnerSubAgent(),TaskOutput.class);}@Action@AchievesGoal(description="Processing
complete")publicTaskOutputdone(TaskOutputoutput){returnoutput;}}
```

Running an Agent Instance

Use RunSubagent.instance() when you already have an Agent object (for example, one created programmatically or via AgentMetadataReader):

```
JavaKotlin
@Agent(description="Outer agent with Agent
instance")publicclassOuterAgentWithAgentInstance{@ActionpublicTaskOutputstart(UserInputinput){Agentagent=(Agent)new
AgentMetadataReader().createAgentMetadata(newInnerSubAgent());returnRunSubagent.instance(agent,TaskOutput.class);}@A
ction@AchievesGoal(description="Processing complete")publicTaskOutputdone(TaskOutputoutput){returnoutput;}}
```

In Kotlin with reified types:

```
JavaKotlin
@Agent(description="Outer agent via explicit agent
instance")publicclassOuterAgentExplicitInstance{@ActionpublicTaskOutputstart(UserInputinput){Agentagent=(Agent)newAg
entMetadataReader().createAgentMetadata(newInnerSubAgent());returnRunSubagent.instance(agent,TaskOutput.class);}@Acti
on@AchievesGoal(description="Processing complete")publicTaskOutputdone(TaskOutputoutput){returnoutput;}}
```

How It Works

RunSubagent methods throw a SubagentExecutionRequest exception that is caught by the framework. The framework then executes the subagent as a subprocess within the current agent process, sharing the same blackboard context. The result of the subagent’s goal-achieving action is returned to the calling action.

This approach has several advantages:

- **Cleaner syntax:** No need to pass ActionContext to the action method
- **Type safety:** The return type is enforced at compile time
- **Composition:** Easily compose complex workflows from simpler agents
- **Reusability:** The same subagent can be used in multiple contexts

Comparison with ActionContext.asSubProcess

Both RunSubagent and ActionContext.asSubProcess achieve the same result, but differ in style:

Approach	When to use	Example
RunSubagent.fromAnnotatedInstance()	When you have an @Agent-annotated instance and don’t need ActionContext	RunSubagent.fromAnnotatedInstance(new SubAgent(), Result.class)
RunSubagent.instance()	When you have an Agent object	RunSubagent.instance(agent, Result.class)
ActionContext.asSubProcess()	When you need access to ActionContext for other operations	context.asSubProcess(Result.class, agent)

Use RunSubagent when your action method only needs to delegate to a subagent. Use ActionContext.asSubProcess() when you need additional context operations.

3.7. DSL

You can also create agents using a DSL in Kotlin or Java.

This is useful for workflows where you want an atomic action that is complete in itself but may contain multiple steps or actions.

3.7.1. Standard Workflows

There are a number of standard workflows, constructed by builders, that meet common requirements. These can be used to create agents that will be exposed as Spring beans, or within @Action methods within other agents. All are type safe. As far as possible, they use consistent APIs.

- SimpleAgentBuilder: The simplest agent, with no preconditions or postconditions.
- ScatterGatherBuilder: Fork join pattern for parallel processing.
- ConsensusBuilder: A pattern for reaching consensus among multiple sources. Specialization of ScatterGather.
- RepeatUntil: Repeats a step until a condition is met.
- RepeatUntilAcceptable: Repeats a step while a condition is met, with a separate evaluator providing feedback.

Creating a simple agent:

JavaKotlin

```
varagent=SimpleAgentBuilder.returning(Joke.class).running(tac->tac.ai().withDefaultLlm().createObject("Tell me a joke",Joke.class)).buildAgent("joker","This is guaranteed to return a dreadful joke");
```

Specify the return type.

specify the action to run. Takes a SupplierActionContext<RESULT> OperationContext parameter allowing access to the current AgentProcess.

Build an agent with the given name and description.

A more complex example:

JavaKotlin

```
@ActionFactChecksrunAndConsolidateFactChecks(DistinctFactualAssertionsdistinctFactualAssertions,ActionContextcontext){varllmFactChecks=properties.models().stream().flatMap(model->factCheckWithSingleLlm(model,distinctFactualAssertions,context)).toList();returnScatterGatherBuilder.returning(FactChecks.class).fromElements(FactCheck.class).generatedBy(llmFactChecks).consolidatedBy(this::reconcileFactChecks).asSubProcess(context);}
```

Start building a scatter gather agent.

Specify the return type of the overall agent.

Specify the type of elements to be gathered.

Specify the list of functions to run in parallel, each generating an element, here of type FactCheck.

Specify a function to consolidate the results. In this case it will take a list of FactCheck and return a FactCheck and return a FactChecks object.

Build and run the agent as a subprocess of the current process. This is an alternative to buildAgent shown in the SimpleAgentBuilder example. The API is consistent.

If you wish to experiment, the [embabel-agent-examples](#) repository includes the [fact checker](#) shown above.

3.7.2. Registering Agent beans

Whereas the @Agent annotation causes a class to be picked up immediately by Spring, with the DSL you’ll need an extra step to register an agent with Spring. As shown in the example below, any @Bean of Agent type results auto registration, just like declaring a class annotated with @Agent does.

JavaKotlin

```
@ConfigurationpublicclassFactCheckerAgentConfiguration{@BeanpublicAgentfactChecker(FactCheckerPropertiesfactCheckerProperties){returnfactCheckerAgent(List.of(LlmOptions.create(AnthropicModels.CLAUDE_35_HAIKU).withTemperature(0.3),LlmOptions.create(AnthropicModels.CLAUDE_35_HAIKU).withTemperature(0.0)),factCheckerProperties);}}
```

3.8. Core Types

3.8.1. LlmOptions

The LlmOptions class specifies which LLM to use and its hyperparameters. It’s defined in the [embabel-common](#) project and provides a fluent API for LLM configuration:

JavaKotlin

```
// Create LlmOptions with model and temperaturevaroptions=LlmOptions.withModel(OpenAiModels.GPT_4O_MINI).withTemperature(0.8);// Use different hyperparameters for different tasksvaranalyticalOptions=LlmOptions.withModel(OpenAiModels.GPT_4O_MINI).withTemperature(0.2).withTopP(0.9);
```

Important Methods:

- withModel(String): Specify the model name
- withRole(String): Specify the model role. The role must be one defined in configuration via embabel.models.llms.<role>=<model-name>
- withTemperature(Double): Set creativity/randomness (0.0-1.0)
- withTopP(Double): Set nucleus sampling parameter
- withTopK(Integer): Set top-K sampling parameter
- withPersona(String): Add a system message persona

LlmOptions is deserializable, so you can set properties of type LlmOptions in application.yml and other application configuration files. This is a powerful way of externalizing not only models, but hyperparameters.

3.8.2. PromptRunner

All LLM calls in user applications should be made via the PromptRunner interface. Once created, a PromptRunner can run multiple prompts with the same LLM, hyperparameters, tool groups and PromptContributors.

Getting a PromptRunner

You obtain a PromptRunner from an OperationContext using the fluent API:

JavaKotlin

```
@ActionpublicStorycreateStory(UserInputinput,OperationContextcontext){// Get PromptRunner with default LLMvarrunner=context.ai().withDefaultLlm();// Get PromptRunner with specific LLMoptionsvarcustomRunner=context.ai().withLlm(LlmOptions.withModel(OpenAiModels.GPT_4O_MINI).withTemperature(0.8));returncustomRunner.createObject("Write a story about: "+input.getContent(),Story.class);}
```

PromptRunner Methods

Core Object Creation:

- createObject(String, Class<T>): Create a typed object from a prompt, otherwise throw an exception. An exception triggers retry. If retry fails repeatedly, re-planning occurs.
- createObjectIfPossible(String, Class<T>): Try to create an object, return null on failure. This can cause replanning.
- generateText(String): Generate simple text response

Normally you want to use one of the createObject methods to ensure the response is typed correctly.

Tool and Context Management:

- withToolGroup(String): Add [tool groups](#) for LLM access

- withToolObject(Object): Add domain objects with [@Tool](#) methods
- withPromptContributor(PromptContributor): Add [context](#) contributors
- withImage(AgentImage): Add an image to the prompt for vision-capable LLMs
- withImages(AgentImage...): Add multiple images to the prompt

LLM Configuration:

- withLlm(LlmOptions): Use specific LLM configuration
- withGenerateExamples(Boolean): Control example generation

Returning a Specific Type

- creating(Class<T>): Go into the Creating fluent API for returning a particular type.

For example:

```
JavaKotlin
varstory=context.ai().withDefaultLlm().withToolGroup(CoreToolGroups.WEB).creating(Story.class).fromPrompt("Create a story about: "+input.getContent());
```

The main reason to do this is to add strongly typed examples for [few-shot prompting](#). For example:

```
JavaKotlin
varstory=context.ai().withDefaultLlm().withToolGroup(CoreToolGroups.WEB).creating(Story.class).withExample("A children's story",newStory("Once upon a time...")).fromPrompt("Create a story about: "+input.getContent());
```

Example: The example will be included in the prompt in JSON format to guide the LLM.

Working with Images:

```
JavaKotlin
varimage=AgentImage.fromFile(imageFile);varanswer=context.ai().withLlm(AnthropicModels.CLAUDE_35_HAIKU).withImage(image).generateText("What is in this image?");
```

Vision-capable model required: Use Claude 3.x, GPT-4 Vision, or other multimodal LLMs

Add image: Images are sent with the text prompt to the LLM. Can be used multiple times for multiple images.

Advanced Features:

- rendering(String): Use [linja](#) templates for prompts (returns Rendering interface)
- withTool(Subagent.ofClass(MyAgent.class).consuming(MyInput.class)): Enable handoffs to other agents (see [Subagent: Agent Handoffs as Tools](#))
- evaluateCondition(String, String): Evaluate boolean condition

Validation

Embabel supports [JSR-380](#) bean validation annotations on domain objects. When creating objects via PromptRunner.createObject or createObjectIfPossible, validation is automatically performed after deserialization. If validation fails, Embabel transparently retries the LLM call to obtain a valid object, describing the validation errors to the LLM to help it correct its response.

If validation fails a second time, InvalidLlmReturnTypeException is thrown. This will trigger replanning if not caught. You can also choose to catch it within the action method making the LLM call, and take appropriate action in your own code.

Simple example of annotation use:

```
JavaKotlin
publicclassUser{@NotNull(message="Name cannot be null")privateStringname;@AssertTrue(message="Working must be true")privatebooleanworking;@Size(min=10,max=200,message="About Me must be between 10 and 200 characters")privateStringaboutMe;@Min(value=18,message="Age should not be less than 18")@Max(value=150,message="Age should not be greater than 150")privateintage;@Email(message="Email should be valid")privateStringemail;// standard setters and getters}
```

You can also use custom annotations with validators that will be injected by Spring. For example:

```
JavaKotlin
@Target({ElementType.FIELD,ElementType.PARAMETER})@Retention(RetentionPolicy.RUNTIME)@Constraint(validatedBy=PalindromeValidator.class)public@interfaceMustBePalindrome{Stringmessage()default"Must be a palindrome";Class<?>[]groups()default{};Class<?extendsPayload>[]payload()default{};}publicclassPalindromic{@MustBePalindromeprivateStringeats;publicPalindromic(Stringeats){this.eats=eats;}publicStringgetEats(){returneats;}@ComponentpublicclassPalindromeValidatorimplementsConstraintValidator<MustBePalindrome,String>{privatefinalAi ai;publicPalindromeValidator(Ai ai){this.ai=ai;}@OverridepublicbooleanisValid(Stringfield,ConstraintValidatorContextcontext){if(field==null){returnfalse;}returnfield.equals(newStringBuilder(field).reverse().toString());}}
```

Define the custom annotation

Apply the annotation to a field

Implement the validator as a Spring component. Note the @Component annotation.

Spring will inject the validator with dependencies, such as the Ai instance in this case

Thus we have standard JSR-280 validation with full Spring dependency injection support.

3.8.3. AgentImage

Represents an image for use with vision-capable LLMs.

Factory Methods:

- AgentImage.fromFile(File): Load from file (auto-detects MIME type from common extensions)
- AgentImage.fromPath(Path): Load from path (auto-detects MIME type)
- AgentImage.create(String, byte[]): Create with explicit MIME type and byte array
- AgentImage.fromBytes(String, byte[]): Create from filename and bytes (auto-detects MIME type)

For uncommon image formats or if auto-detection fails, use AgentImage.create() with an explicit MIME type.

3.9. Tools

Tools can be passed to LLMs to allow them to perform actions. Tools can either be outside the JVM process, as with MCP, or inside the JVM process, as with domain objects exposing @LlmTool methods.

Embabel allows you to provide tools to LLMs in two ways:

- Via the PromptRunner by providing one or more in process **tool instances**. A tool instance is an object with methods annotated with Embabel @LlmTool or Spring AI @Tool.
- At action or PromptRunner level, from a **tool group**.

LlmReference implementations also expose tools, but this is handled internally by the framework.

3.9.1. In Process Tools: Implementing Tool Instances

Implement one or more methods annotated with @LlmTool on a class. You do not need to annotate the class itself. Each annotated method represents a distinct tool that will be exposed to the LLM.

A simple example of a tool method:

```
JavaKotlin
```



```
public class MathTools {
    @LlmTool(description = "add two numbers")
    public double add(double a, double b) {
        return a + b;
    }
    // Other tools
}
```

Classes implementing tools can be stateful. They are often domain objects. Tools on mapped entities are especially useful, as they can encapsulate state that is never exposed to the LLM. See [Domain Tools: Direct Access, Zero Ceremony](#) for a discussion of tool use patterns.

The `@Tool` annotation comes from [Spring AI](#).

Tool methods can have any visibility, and can be static or instance scope. They are allowed on inner classes.

You can define any number of arguments for the method (including no argument) with most types (primitives, POJOs, enums, lists, arrays, maps, and so on). Similarly, the method can return most types, including void. If the method returns a value, the return type must be a serializable type, as the result will be serialized and sent back to the model.

The following types are not currently supported as parameters or return types for methods used as tools:

- *Optional*
- *Asynchronous types (e.g. CompletableFuture, Future)*
- *Reactive types (e.g. Flow, Mono, Flux)*
- *Functional types (e.g. Function, Supplier, Consumer).*

— *Spring AI* Tool Calling

You can obtain the current `AgentProcess` in a Tool method implementation via `AgentProcess.get()`. This enables tools to bind to the `AgentProcess`, making objects available to other actions. For example:

```
Java Kotlin
@LlmTool(description = "My Tool")
public String bindCustomer(Long id) {
    var customer = customerRepository.findById(id);
    var agentProcess = AgentProcess.get();
    if (agentProcess != null) {
        agentProcess.addObject(customer);
        return "Customer bound to blackboard";
    }
    return "No agent process: Unable to bind customer";
}
```

3.9.2. Tool Groups

Embabel introduces the concept of a **tool group**. This is a level of indirection between user intent and tool selection. For example, we don't ask for Brave or Google web search: we ask for "web" tools, which may be resolved differently in different environments.

Tools use should be focused. Thus tool groups are not specified at agent level, but on individual actions.

Tool groups are often backed by [MCP](#).

Configuring Tool Groups in configuration files

If you have configured MCP servers in your application configuration, you can selectively expose tools from those servers to agents by configuring tool groups. The easiest way to do this is in your `application.yml` or `application.properties` file. Select tools by name.

For example:

```
embabel:agent:platform:tools:includes:weather:description:Get weather for location
provider:Dockertools:-weather
```

Configuring Tool Groups in Spring @Configuration

You can also use Spring's `@Configuration` and `@Bean` annotations to expose ToolGroups to the agent platform with greater control. The framework provides a default `ToolGroupsConfiguration` that demonstrates how to inject MCP servers and selectively expose MCP tools:

```
Java Kotlin
@Configuration
public class ToolGroupsConfiguration {
    private final List<McpSyncClient> mcpSyncClients;

    public ToolGroupsConfiguration(List<McpSyncClient> mcpSyncClients) {
        this.mcpSyncClients = mcpSyncClients;
    }

    @Bean
    public MathTools mathToolGroup() {
        return new MathTools();
    }

    @Bean
    public ToolGroup mcpWebToolsGroup() {
        return new McpToolGroup(
            CoreToolGroups.WEB_DESCRIPTION,
            "docker-web",
            "Docker",
            Set.of(ToolGroupPermission.INTERNET_ACCESS),
            mcpSyncClients,
            callback -> {
                // Only expose specific web tools, exclude rate-limited ones
                String name = callback.getToolDefinition().name();
                return (name.contains("brave") || name.contains("fetch")) && !name.contains("brave_local_search");
            }
        );
    }
}
```

This method creates a Spring bean of type `ToolGroup`. This will automatically be picked up by the agent platform, allowing the tool group to be requested by name (role).

Key Configuration Patterns

MCP Client Injection: The configuration class receives a `List<McpSyncClient>` via constructor injection. Spring automatically provides all available MCP clients that have been configured in the application.

Selective Tool Exposure: Each `McpToolGroup` uses a filter lambda to control which tools from the MCP servers are exposed to agents. This allows fine-grained control over tool availability and prevents unwanted or problematic tools from being used.

Tool Group Metadata: Tool groups include descriptive metadata like name, provider, and description to help agents understand their capabilities. The permissions property declares what access the tool group requires (e.g., `INTERNET_ACCESS`).

Creating Custom Tool Group Configurations

Applications can implement their own `@Configuration` classes to expose custom tool groups, which can be backed by any service or resource, not just MCP.

```
Java Kotlin
@Configuration
public class MyToolGroupsConfiguration {
    @Bean
    public ToolGroup databaseToolsGroup(DataSource dataSource) {
        return new DatabaseToolGroup(dataSource);
    }

    @Bean
    public ToolGroup emailToolsGroup(EmailService emailService) {
        return new EmailToolGroup(emailService);
    }
}
```

This approach leverages Spring's dependency injection to provide tool groups with the services and resources they need, while maintaining clean separation of concerns between tool configuration and agent logic.

Using Tools in Action Methods

Tools are specified on the `PromptRunner` when making LLM calls. This gives you fine-grained control over which tools are available for each specific prompt.

Here's an example from the `StarNewsFinder` agent that demonstrates web tool usage within an action:

```
Java Kotlin
@Action
public RelevantNewsStories findNewsStories(StarPerson person, Horoscope horoscope, OperationContext context) {
    var prompt = """
        %s is an astrology believer with the sign %s.
        Their horoscope for today is:
        <horoscope>%s</horoscope>
        Given this, use web tools and generate search queries
        to find %d relevant news stories summarize them in a few sentences.
        Include the URL for each story.
        Do not look for another horoscope reading or return results directly about astrology;
        find stories relevant to the reading above.
        """
        .formatted(person.name(), person.sign(), horoscope.summary(), storyCount);
    // Tools are specified on the PromptRunner
    return context.ai().withDefaultLlm().withToolGroup(CoreToolGroups.WEB)
        // Add web search tools
        .createObject(prompt, RelevantNewsStories.class);
}
```

Key Tool Usage Patterns

PromptRunner Tool Methods: Tools are added to the `PromptRunner` fluent API using methods like `withToolGroup()`, `withTools()`, and `withToolObject()`.

Multiple Tool Groups: Actions can add multiple tool groups by chaining withToolGroup() calls when they need different types of capabilities.

Tool-Aware Prompts: Prompts should explicitly instruct the LLM to use the available tools. For example, "use web tools and generate search queries" clearly directs the LLM to utilize the web search capabilities.

Additional PromptRunner Examples

```
JavaKotlin
// Add tool groups to a specific promptcontext.ai().withAutoLlm().withToolGroup(CoreToolGroups.WEB).create("Given the topic,
generate a detailed report using web research.\n\n"+ "# Topic\n"+reportRequest.getTopic()); // Add multiple tool
groupscontext.ai().withDefaultLlm().withToolGroup(CoreToolGroups.WEB).withToolGroup(CoreToolGroups.MATH).createObject("
Calculate stock performance with web data",StockReport.class);
```

Adding Tool Objects with @LlmTool Methods:

You can also provide domain objects with @LlmTool methods directly to specific prompts:

```
JavaKotlin
context.ai().withDefaultLlm().withToolObject(jokerTool).createObject("Create a UserInput object for fun",UserInput.class); // Add
tool object with filtering and custom naming
strategycontext.ai().withDefaultLlm().withToolObject(newToolObject(calculatorService).withNamingStrategy(name-
>"calc_"+name).withFilter(methodName->methodName.startsWith("compute"))).createObject("Perform calculations",Result.class);
```

Available PromptRunner Tool Methods:

- withToolGroup(String): Add a single tool group by name
- withToolGroup(ToolGroup): Add a specific ToolGroup instance
- withToolGroups(Set<String>): Add multiple tool groups
- withTools(vararg String): Convenient method to add multiple tool groups
- withToolObject(Any): Add domain object with @LlmTool or @Tool methods
- withToolObject(ToolObject): Add ToolObject with custom configuration
- withTool(Tool): Add a framework-agnostic Tool instance
- withTools(List<Tool>): Add multiple framework-agnostic Tool instances

3.9.3. Framework-Agnostic Tool Interface

In addition to Spring AI’s @Tool annotation, Embabel provides its own framework-agnostic Tool interface in the com.embabel.agent.api.tool package. This allows you to create tools that are not tied to any specific LLM framework, making your code more portable and testable.

The Tool interface includes nested types to avoid naming conflicts with framework-specific types:

- Tool.Definition - Describes the tool (name, description, input schema)
- Tool.InputSchema - Defines the parameters the tool accepts
- Tool.Parameter - A single parameter with name, type, and description
- Tool.Result - The result returned by a tool (text, artifact, or error)
- Tool.Handler - Functional interface for implementing tool logic

Creating Tools Programmatically

You can create tools using the Tool.create() factory methods:

```
JavaKotlin
// Simple tool with no parametersToolgreetTool=Tool.create("greet","Greets the user",(input)->Tool.Result.text("Hello!")); // Tool
with parameters (using factory methods)TooladdTool=Tool.create("add","Adds two numbers
together",Tool.InputSchema.of(Tool.Parameter.integer("a","First number"),Tool.Parameter.integer("b","Second number")),
(input)->{// Parse input JSON and compute resultreturnTool.Result.text("42");}); // Tool with metadata (e.g., return directly without LLM
processing)TooldirectTool=Tool.create("lookup","Looks up data directly",Tool.Metadata.create(true), // returnDirect = true(input)-
>Tool.Result.text("Direct result"));
```

The Tool.Parameter class provides factory methods for common parameter types:

- Tool.Parameter.string(name, description) - String parameter
- Tool.Parameter.string(name, description, required) - String with optional flag
- Tool.Parameter.string(name, description, required, enumValues) - String with allowed values
- Tool.Parameter.integer(name, description) - Integer parameter
- Tool.Parameter.double(name, description) - Floating-point parameter

All factory methods default to required = true.

Creating Strongly Typed Tools

For tools with complex input and output structures, use Tool.fromFunction() to work with domain objects directly. The input schema is generated automatically from the input type, and JSON marshaling is handled for you.

```
JavaKotlin
// Define input and output typesrecordAddRequest(inta,intb){}recordAddResult(intsum){} // Create typed tool - schema is generated
from AddRequestTooladdTool=Tool.fromFunction("add","Adds two numbers together",AddRequest.class,AddResult.class,input-
>newAddResult(input.a()+input.b())); // Call the tool - input is deserialized, output is serializedTool.Resultresult=addTool.call("{\n\"a\":
5, \n\"b\": 3}"); // Result contains: {\"sum\":8} // String output is returned directly (not double-
serialized)ToolgreetTool=Tool.fromFunction("greet","Greets someone",GreetRequest.class,String.class,input->"Hello
"+input.name()+"!"); // With custom metadataTooldirectTool=Tool.fromFunction("lookup","Looks up data
directly",LookupRequest.class,LookupResult.class,Tool.Metadata.create(true), // returnDirect = trueinput-
>newLookupResult(findData(input)));
```

You can also instantiate TypedTool directly:

```
valtool=TypedTool(name="add",description="Add two
numbers",inputType=AddRequest::class.java,outputType=AddResult::class.java,){input->AddResult(input.a+input.b)}
```

Key features of typed tools:

- **Automatic schema generation:** The input schema is derived from the input type’s structure
- **JSON marshaling:** Input JSON is deserialized to the input type, and output is serialized from the output type
- **String pass-through:** If the output type is String, it’s returned directly without JSON serialization
- **Result pass-through:** If the function returns a Tool.Result, it’s used as-is
- **Exception handling:** Exceptions thrown by the function are converted to Tool.Result.Error
- **Control flow signals:** Exceptions implementing ToolControlFlowSignal (like ReplanRequestedException) propagate through

Creating Tools from Annotated Methods

Embabel provides @LlmTool and @LlmTool.Param annotations for creating tools from annotated methods. This approach is similar to Spring AI’s @Tool but uses Embabel’s framework-agnostic abstractions.

```
JavaKotlin
publicclassMathService{@LlmTool(description="Adds two numbers together")publicintadd(@LlmTool.Param(description="First
number")inta,@LlmTool.Param(description="Second number")intb){returna+b;}@LlmTool(description="Multiplies two
numbers")publicintmultiply(@LlmTool.Param(description="First number")inta,@LlmTool.Param(description="Second
number")intb){returna*b;}} // Create tools from all annotated methods on an
```



```
instanceList<Tool>mathTools=Tool.fromInstance(newMathService()); // Or safely create tools (returns empty list if no annotations found)List<Tool>tools=Tool.safelyFromInstance(someObject);
```

The @LlmTool annotation supports:

- name: Tool name (defaults to method name if empty)
- description: Description of what the tool does (required)
- returnDirect: Whether to return the result directly without further LLM processing

The @LlmTool.Param annotation supports:

- description: Description of the parameter (helps the LLM understand what to provide)
- required: Whether the parameter is required (defaults to true)

Adding Framework-Agnostic Tools via PromptRunner

Use withTool() or withTools() to add framework-agnostic tools to a PromptRunner:

```
JavaKotlin// Add a single toolToolcalculatorTool=Tool.create("calculate","Performs calculations",(input)->Tool.Result.text("Result: 42"));context.ai().withDefaultLlm().withTool(calculatorTool).createObject("Calculate 6 * 7",MathResult.class); // Add tools from annotated methodsList<Tool>mathTools=Tool.fromInstance(newMathService());context.ai().withDefaultLlm().withTools(mathTools).createObject("Add 5 and 3",MathResult.class); // Combine with other toolcontext.ai().withDefaultLlm().withToolGroup(CoreToolGroups.WEB) // Tool group.withToolObject(domainObject) // Spring AI @Tool methods.withTools(mathTools) // Framework-agnostic tools.createObject("Research and calculate",Report.class);
```

Tool Results

Tools return Tool.Result which can be one of three types:

```
JavaKotlin// Text result (most common)Tool.Result.text("The answer is 42");// Result with an artifact (e.g., generated file, image)Tool.Result.withArtifact("Generated report",reportBytes); // Error resultTool.Result.error("Failed to process request",exception);
```

Modifying Tool Descriptions

Tools provide withDescription() and withNote() methods to create copies with modified descriptions. This is useful when you need to customize a tool’s description for a specific context without modifying the original tool.

withDescription(newDescription)

Creates a new tool with a completely replaced description:

```
JavaKotlin// Replace the entire descriptionToolcustomTool=originalTool.withDescription("Custom description for this context"); // The original tool is unchangedSystem.out.println(originalTool.getDefinition().getDescription()); // original descriptionSystem.out.println(customTool.getDefinition().getDescription()); // Custom description for this context
```

withNote(note)

Creates a new tool with an appended note to the existing description:

```
JavaKotlin// Add a note to the existing descriptionToolannotatedTool=originalTool.withNote("Use this when querying large datasets"); // Result: "Original description. Use this when querying large datasets"System.out.println(annotatedTool.getDefinition().getDescription());
```

Both methods preserve all other tool properties (name, input schema, metadata, functionality):

```
JavaKotlinTooloriginal=Tool.create("calculator","Performs calculations",Tool.InputSchema.of(Tool.Parameter.integer("x","Number")),input->Tool.Result.text("42")); // Create a customized versionToolcustomized=original.withDescription("Specialized math tool").withNote("Optimized for financial calculations"); // Name and functionality unchangedassertcustomized.getDefinition().getName().equals("calculator");assertcustomized.call("{}").text().equals("42");
```

When to Use Each Approach

Approach	Use When
Spring AI @Tool	You’re comfortable with Spring AI and want IDE support for tool annotations on domain objects
Tool.create() / Tool.of()	You need programmatic tool creation with simple inputs, want framework independence, or are creating tools dynamically
Tool.fromFunction()	You need programmatic tool creation with complex typed inputs and outputs, automatic JSON marshaling, and schema generation
@LlmTool / @LlmTool.Param	You prefer annotation-based tools but want Embabel’s framework-agnostic abstractions
Tool Groups	You need to organize related tools, use MCP servers, or control tool availability at deployment time

3.9.4. Tool Decoration: Extending Tool Behavior

Embabel uses a powerful decoration pattern to extend tool behavior without modifying the underlying tool or complicating the PromptRunner. A decorated tool wraps another tool, intercepting calls to add functionality like artifact capture, event publishing, or blackboard integration.

This pattern is fundamental to Embabel’s architecture:

- Subagents** use decoration to wrap agent execution as a tool
- Asset tracking** uses decoration to capture tool outputs for chatbot interfaces
- Blackboard publishing** uses decoration to make tool results available to other actions
- Event streaming** uses decoration to publish tool calls to external systems
- Internal platform features like observability and exception handling also use decoration

The DelegatingTool Interface

All tool decorators implement DelegatingTool:

```
JavaKotlinpublicinterfaceDelegatingToolextendsTool{ToolgetDelegate();}
```

This allows decorators to be unwrapped when needed, and enables chaining multiple decorators.

ArtifactSinkingTool: Capturing Tool Outputs

ArtifactSinkingTool captures artifacts from Tool.Result.WithArtifact results and sends them to a sink. This is the foundation for making structured tool outputs available elsewhere.

```
JavaKotlin// Capture all artifacts and publish to blackboardToolwrapped=Tool.publishToBlackboard(myTool); // Capture specific typesToolwrapped=Tool.publishToBlackboard(myTool,SearchResult.class); // With filtering and transformationToolwrapped=Tool.publishToBlackboard(myTool,SearchResult.class,result->result.getScore()>0.5, // filterresult->result.getDocument() // transform); // Capture to a custom sinkToolwrapped=Tool.sinkArtifacts(myTool,SearchResult.class,mySink);
```

Built-in Sinks

Embabel provides several ArtifactSink implementations:

Sink	Purpose
BlackboardSink	Publishes to the current AgentProcess blackboard, making artifacts available to other actions
ListSink	Collects artifacts into a list, useful for aggregating results
CompositeSink	Delegates to multiple sinks, enabling multi-destination publishing

Creating Custom Sinks

Implement ArtifactSink to create custom destinations:

```
JavaKotlin
// Publish to an event streamArtifactSink=artifact->{eventPublisher.publish(newToolArtifactEvent(artifact));};// Use with any toolToolwrapped=Tool.sinkArtifacts(myTool,MyType.class,eventSink);
```

How Decoration Enables Extension

The decoration pattern lets Embabel add sophisticated behavior while keeping PromptRunner simple. When you use Subagent.ofClass(MyAgent.class) (Java) or Subagent.ofClass(MyAgent::class.java) (Kotlin), Embabel creates a tool that:

- 1. Wraps agent execution in a Tool.call() method
- 2. Shares the parent blackboard with the child process
- 3. Captures the agent’s result as a tool artifact

Similarly, when you configure asset tracking in a chatbot, Embabel wraps tools with AssetAddingTool to capture outputs as viewable assets.

This approach has key advantages:

- **Composable:** Multiple decorators can be chained
- **Transparent:** The underlying tool doesn’t know it’s wrapped
- **Extensible:** New behaviors can be added without framework changes
- **Type-safe:** Generic decorators like ArtifactSinkingTool<T> preserve type information

3.9.5. Subagent: Agent Handoffs as Tools

A Subagent is a specialized Tool that delegates to another Embabel agent. When the LLM invokes this tool, it runs the specified agent as a subprocess, sharing the parent process’s blackboard context. This enables composition of agents and "handoff" patterns where one agent delegates specialized tasks to another.

Creating Subagents

Subagent uses a fluent builder pattern. First select how to reference the agent, then specify the input type using consuming():

```
JavaKotlin
// From an @Agent annotated class (most common)Subagent.ofClass(ConcertAssembler.class).consuming(ConcertPlan.class)// By agent name (resolved at runtime from platform)Subagent.byName("ConcertAssembler").consuming(ConcertPlan.class)// From an already-resolved Agent instanceSubagent.ofInstance(resolvedAgent).consuming(ConcertPlan.class)// From an instance of an @Agent annotated class (e.g., a Spring bean)Subagent.ofAnnotatedInstance(myAgentBean).consuming(ConcertPlan.class)
```

The consuming() method specifies the input type that the LLM will provide when invoking this tool. This type is used to generate the JSON schema that guides the LLM’s tool invocation.

Using Subagents with PromptRunner

Use withTool() to add a Subagent to your prompt:

```
JavaKotlin
@ActionpublicConcertassembleConcert(ConcertPlanplan,OperationContextcontext){returncontext.ai().withDefaultLlm().withTool(Subagent.ofClass(PerformanceFinder.class).consuming(WorksToFind.class)).creating(Concert.class).fromPrompt("Assemble a concert based on: "+plan);}
```

The LLM can now invoke PerformanceFinder as a tool, providing WorksToFind input to delegate the performance search task.

Subagent with Asset Tracking

For chat applications that track assets, wrap the Subagent with AssetAddingTool to automatically track returned artifacts:

```
JavaKotlin
@ActionpublicConcertassembleConcert(ConcertPlanplan,OperationContextcontext){varsubagent=Subagent.ofClass(PerformanceFinder.class).consuming(WorksToFind.class);vartrackedSubagent=assetTracker.addReturnedAssets(subagent);returncontext.ai().withDefaultLlm().withTool(trackedSubagent).creating(Concert.class).fromPrompt("Assemble a concert based on: "+plan);};// With filtering - only track certain assetsvartrackedSubagent=assetTracker.addReturnedAssets(subagent,asset->asset instanceof Performance);// Only track Performance assets);}
```

Wrap with addReturnedAssets() to track artifacts returned by the subagent.

Input Type and JSON Schema

The input type you specify with consuming() determines the JSON schema that the LLM sees when invoking the tool.

For example:

```
JavaKotlin
// The input typepublicrecordWorksToFind(List<String>composers,Stringera,intmaxResults){};// Create the subagent with explicit input typeSubagent.ofClass(PerformanceFinder.class).consuming(WorksToFind.class)
```

The Subagent tool will:

- Use "PerformanceFinder" as the tool name (from @Agent annotation)
- Use "Finds performances" as the tool description (from @Agent annotation)
- Generate a JSON schema from WorksToFind

From the LLM’s perspective, a Subagent is just another tool. The calling LLM sees the JSON schema for WorksToFind and can populate it directly:

```
{
  "composers": ["Mozart", "Beethoven"],
  "era": "Classical",
  "maxResults": 5
}
```

When the tool is invoked, Subagent deserializes this JSON into a WorksToFind object and passes it to the target agent. The input type should match the first non-injected parameter of the agent’s entry-point action.

Blackboard Sharing

When a Subagent runs, it receives a **spawned blackboard** from the parent process. This means:

- The subagent can read objects from the parent’s blackboard
- Objects added by the subagent are available to the parent after the subagent completes
- The subagent operates in its own process context but shares state appropriately

When to Use Subagent

Scenario	Recommendation
----------	----------------

Complex specialized task that has its own multi-action workflow	Use Subagent - the target agent can plan and execute multiple steps
Simple tool call with deterministic logic	Use a regular @LlmTool method instead
LLM-orchestrated mini-workflow with sub-tools	Consider AgenticTool which operates at the tool level
Need the full power of GOAP planning for the subtask	Subagent is ideal - the target agent uses its own planner

3.9.6. Agentic Tools

An **agentic tool** is a tool that uses an LLM to orchestrate other tools. Unlike a regular tool which executes deterministic logic, an agentic tool delegates to an LLM that decides which sub-tools to call based on a prompt.

This pattern is useful for encapsulating a mini-orchestration as a single tool that can be used in larger workflows. Embabel provides three agentic tool implementations, each offering different levels of control over tool availability:

Choosing an Agentic Tool

Tool Type	Tool Availability	Best For	Example Use Case
SimpleAgenticTool	All tools available immediately	Simple orchestration, exploration tasks	Math calculator with add/multiply/divide tools
PlaybookTool	Progressive unlock via conditions (prerequisites, artifacts, blackboard)	Structured workflows, guided processes	Research workflow: search → analyze → summarize
StateMachineTool	State-based availability using enum states	Formal state machines, multi-phase processes	Order processing: DRAFT → CONFIRMED → SHIPPED → DELIVERED

All three implement the AgenticTool interface and share a common fluent API with with* methods.

The AgenticTool interface defines:

```
JavaKotlin
publicinterfaceAgenticTool<THISextendsAgenticTool<THIS>>extendsTool{LlmOptionsgetLlm();// LLM
configurationintgetMaxIterations();// Max tool loop iterations (default: 20)THISwithLlm(LlmOptionsllm);THISwithSystemPrompt(Stringprompt);THISwithSystemPrompt(AgenticSystemPromptCreat
orcreator);// Dynamic
promptTHISwithMaxIterations(intmaxIterations);THISwithParameter(Tool.Parameterparameter);THISwithToolObject(Objecttool
lObject);}
```

The AgenticSystemPromptCreator functional interface receives both the ExecutingOperationContext (for access to blackboard, process options, etc.) and the input string passed to the tool:

```
JavaKotlin
tool.withSystemPrompt((ctx,input)->"Context: "+ctx.getProcessContext().getProcessOptions().getContextId()+" Task: "+input);
```

For complex workflows with defined outputs, branching logic, loops, or state management, use Embabel’s [GOAP planner](#), [Utility AI](#), or [@State workflows](#) instead. These provide deterministic, typesafe planning that is far more powerful and predictable than LLM-driven orchestration.

SimpleAgenticTool: Flat Tool Orchestration

SimpleAgenticTool makes all sub-tools available immediately. The LLM decides freely which tools to use based on the prompt.

```
JavaKotlin
importcom.embabel.agent.api.tool.agentic.simple.SimpleAgenticTool;// Create the agentic
toolSimpleAgenticToolmathOrchestrator=newSimpleAgenticTool("math-orchestrator","Orchestrates math
operations").withTools(addTool,multiplyTool,divideTool).withParameter(Tool.Parameter.string("expression","Math expression to
evaluate")).withLlm(LlmOptions.withModel("gpt-4"));// Use it like any other
toolcontext.ai().withDefaultLlm().withTool(mathOrchestrator).generateText("What is 5 + 3 * 2?");
```

PlaybookTool: Conditional Tool Unlocking

PlaybookTool allows tools to be progressively unlocked based on conditions:

- **Prerequisites:** unlock after other tools have been called
- **Artifacts:** unlock when certain artifact types are produced
- **Blackboard:** unlock based on process state
- **Custom predicates:** unlock based on arbitrary conditions

```
JavaKotlin
importcom.embabel.agent.api.tool.agentic.playbook.PlaybookTool;// Tools unlock
progressivelyPlaybookToolresearcher=newPlaybookTool("researcher","Research and analyze
topics").withTools(searchTool,fetchTool)// Always available.withTool(analyzeTool).unlockedBy(searchTool)// Unlocks after
search.withTool(summarizeTool).unlockedBy(analyzeTool)// Unlocks after
analyze.withParameter(Tool.Parameter.string("topic","Research topic"));// Multiple prerequisites
(AND).withTool(reportTool).unlockedByAll(searchTool,analyzeTool)// Any prerequisite
(OR).withTool(processTool).unlockedByAny(searchTool,fetchTool)// Unlock when artifact type
produced.withTool(formatTool).unlockedByArtifact(Document.class)// Unlock based on blackboard
state.withTool(actionTool).unlockedByBlackboard(UserProfile.class)// Custom predicate.withTool(finalizeTool).unlockedWhen(ctx-
>ctx.getIterationCount()>=3)
```

When a locked tool is called before its conditions are met, the LLM receives an informative message guiding it to use prerequisite tools first.

StateMachineTool: State-Based Availability

StateMachineTool uses explicit states defined by an enum. Tools are registered with specific states where they’re available, and can trigger transitions to other states.

```
JavaKotlin
importcom.embabel.agent.api.tool.agentic.state.StateMachineTool;enumOrderState{DRAFT,CONFIRMED,SHIPPED,DELIVERED}State
MachineTool<OrderState>orderProcessor=newStateMachineTool<>("orderProcessor","Process
orders",OrderState.class).withInitialState(OrderState.DRAFT).inState(OrderState.DRAFT).withTool(addItemTool).withTool(confir
mTool).transitionsTo(OrderState.CONFIRMED).inState(OrderState.CONFIRMED).withTool(shipTool).transitionsTo(OrderState.SHI
PPED).inState(OrderState.SHIPPED).withTool(deliverTool).transitionsTo(OrderState.DELIVERED).inState(OrderState.DELIVERED).
withTool(reviewTool).build().withGlobalTools(statusTool,helpTool)// Available in all
states.withParameter(Tool.Parameter.string("orderId","Order to process"));
```

The startingIn(state) method allows starting in a different state at runtime:

```
JavaKotlin
// Resume an order that's already confirmedToolresumedProcessor=orderProcessor.startingIn(OrderState.CONFIRMED);
```

Domain Tools: Tools from Retrieved Objects

All three agentic tools support **domain tools** - @LlmTool methods on domain objects that become available when a single instance is retrieved.

JavaKotlin


```
// Domain class with @LlmTool
methodspublicclassUser{privatefinalStringid;privatefinalStringname;publicUser(Stringid,Stringname){this.id=id;this.name=name;}@LlmTool(description="Get user's profile information")publicStringgetProfile(){return"Profile for "+name;}@LlmTool(description="Update user's settings")publicStringupdateSettings(Stringsettings){return"Settings updated for "+name;}}// Register domain tools - they become available when a single User is
retrievedPlaybookTooluserManager=newPlaybookTool("userManager","Manage
users").withTools(searchUserTool,getUserTool).withToolChainingFrom(User.class);// User methods available after getUserTool
returns a single User
```

Domain tools are "declared" to the LLM immediately but return an error until an instance is bound. When a tool returns a **single** artifact (not a collection) of a registered type, that instance is bound and its @LlmTool methods become executable.

Creating Agentic Tools

Create agentic tools using the constructor and fluent with* methods:

```
JavaKotlin
// Create sub-toolsTooladdTool=Tool.create("add","Adds two numbers",input->{// Parse JSON input and compute
resultreturnTool.Result.text("5");});ToolmultiplyTool=Tool.create("multiply","Multiplies two numbers",input-
>{returnTool.Result.text("6");});// Create the agentic toolSimpleAgenticToolmathOrchestrator=newSimpleAgenticTool("math-
orchestrator","Orchestrates math operations").withTools(addTool,multiplyTool).withLlm(LlmOptions.withModel("gpt-
4")).withSystemPrompt("Use the available tools to solve the given math problem");// Use it like any other
toolcontext.ai().withDefaultLlm().withTool(mathOrchestrator).generateText("What is 5 + 3 * 2?");
```

The withSystemPrompt call is optional. By default, agentic tools generate a system prompt from the tool's description: "You are an intelligent agent that can use tools to help you complete tasks. Use the provided tools to perform the following task: {description}". Only call withSystemPrompt if you need custom orchestration instructions.

Defining Input Parameters

You must define input parameters for your agentic tool so the LLM knows what arguments to pass when calling it. Without parameters, the LLM won't know what input format to use.

Use the withParameter method with Tool.Parameter factory methods for concise parameter definitions:

```
JavaKotlin
// Research tool that requires a topic parameterSimpleAgenticToolresearcher=newSimpleAgenticTool("researcher","Research a
topic thoroughly").withParameter(Tool.Parameter.string("topic","The topic to
research")).withToolObjects(newSearchTools(),newSummarizerTools());// Calculator with multiple
parametersSimpleAgenticToolcalculator=newSimpleAgenticTool("smart-calculator","Perform complex
calculations").withParameter(Tool.Parameter.string("expression","Mathematical expression to
evaluate")).withParameter(Tool.Parameter.integer("precision","Decimal places for result",false))//
optional.withToolObject(newMathTools());
```

Available parameter factory methods:

- Tool.Parameter.string(name, description, required?) - String parameter
- Tool.Parameter.integer(name, description, required?) - Integer parameter
- Tool.Parameter.double(name, description, required?) - Floating-point parameter

All factory methods default to required = true. Set required = false for optional parameters.

Creating Agentic Tools from Annotated Objects

Use withToolObject or withToolObjects to add tools from objects with @LlmTool-annotated methods:

```
JavaKotlin
// Tool classes with @LlmTool methodspublicclassSearchTools{@LlmTool(description="Search the
web")publicStringsearch(Stringquery){return"Results for: "+query;}}publicclassCalculatorTools{@LlmTool(description="Add
two numbers")publicintadd(inta,intb){returna+b;}@LlmTool(description="Multiply two
numbers")publicintmultiply(inta,intb){returna*b;}}// Create agentic tool with tools from multiple objects// Uses default system
prompt based on descriptionSimpleAgenticToolassistant=newSimpleAgenticTool("assistant","Multi-capability
assistant").withToolObjects(newSearchTools(),newCalculatorTools());// With LLM options and custom system
promptSimpleAgenticToolsmartAssistant=newSimpleAgenticTool("smart-assistant","Smart
assistant").withToolObjects(newSearchTools(),newCalculatorTools()).withLlm(LlmOptions.withModel("gpt-
4")).withSystemPrompt("Use tools intelligently");
```

Objects without @LlmTool methods are silently ignored, allowing you to mix objects safely.

Agentic Tools with Spring Dependency Injection

Agentic tools can encapsulate stateful services via dependency injection:

```
JavaKotlin
@ComponentpublicclassResearchOrchestrator{privatefinalWebSearchServicewebSearchService;privatefinalSummarizerServi
cesummarizerService;publicResearchOrchestrator(WebSearchServicewebSearchService,SummarizerServicesummarizerService){
this.webSearchService=webSearchService;this.summarizerService=summarizerService;}@LlmTool(description="Search the web for
information")publicList<SearchResult>search(Stringquery){returnwebSearchService.search(query);}@LlmTool(description="Su
mmarize text content")publicStringssummarize(Stringcontent){returnsummarizerService.summarize(content);}}// In your
configuration@ConfigurationpublicclassToolConfiguration{@BeanpublicSimpleAgenticToolresearchTool(ResearchOrchestrat
ororchestrator){returnnewSimpleAgenticTool("research-assistant","Research topics using web search and
summarization").withToolObject(orchestrator).withLlm(newLlmOptions().withRole("smart"));// Uses default system prompt based
on description}}
```

How Agentic Tools Execute

When an agentic tool's call() method is invoked:

1. The tool retrieves the current AgentProcess context
2. It configures a PromptRunner with the specified LlmOptions
3. It adds all sub-tools to the prompt runner
4. It executes the prompt with the input, allowing the LLM to orchestrate the sub-tools
5. The final LLM response is returned as the tool result

This means agentic tools create a nested LLM interaction: the outer LLM decides to call the agentic tool, then the inner LLM orchestrates the sub-tools.

Modifying Agentic Tools

Use the with* methods to create modified copies:

```
JavaKotlin
SimpleAgenticToolbase=newSimpleAgenticTool("base","Base orchestrator").withTools(tool1).withSystemPrompt("Original
prompt");// Create copies with modificationsSimpleAgenticToolwithNewLlm=base.withLlm(newLlmOptions().withModel("gpt-
4"));SimpleAgenticToolwithMoreTools=base.withTools(tool2,tool3);SimpleAgenticToolwithNewPrompt=base.withSystemPrompt(
"Updated prompt");// Add input
parametersSimpleAgenticToolwithParams=base.withParameter(Tool.Parameter.string("query","Search query"));// Add tools from
an object with @LlmTool methodsSimpleAgenticToolwithAnnotatedTools=base.withToolObject(calculatorService);// Add tools from
multiple objectsSimpleAgenticToolwithMultipleObjects=base.withToolObjects(searchService,calculatorService);// Dynamic system
```



```
prompt based on execution context and inputSimpleAgenticToolwithDynamicPrompt=base.withSystemPrompt((ctx,input)->{StringcontextId=ctx.getProcessContext().getProcessOptions().getContextId().getId();return"Process requests for context "+contextId+" . Task: "+input;});
```

The available modification methods are:

- withParameter(Tool.Parameter): Add an input parameter (use Tool.Parameter.string(), .integer(), .double())
- withLlm(LlmOptions): Set LLM configuration
- withTools(vararg Tool): Add additional Tool instances
- withToolObject(Any): Add tools from an object with @LlmTool methods
- withToolObjects(vararg Any): Add tools from multiple annotated objects
- withSystemPrompt(String): Set a fixed system prompt
- withSystemPrompt((ExecutingOperationContext, String) → String): Set a dynamic prompt based on execution context and input
- withCaptureNestedArtifacts(Boolean): Control whether artifacts from nested agentic tool calls are captured (default: false)
- withToolChainingFrom(Class<T>): Register a class whose @LlmTool methods become available when an artifact of that type is returned
- withToolChainingFrom(Class<T>, DomainToolPredicate<T>): Register with a predicate to filter which instances contribute tools
- withToolChainingFromAny(): Auto-discover tools from any returned artifact with @LlmTool methods

Controlling Artifact Capture in Nested Agentic Tools

When an agentic tool orchestrates other tools, those sub-tools may return artifacts (via Tool.Result.WithArtifact). By default, artifacts from nested agentic tool calls are **not** captured—only the final result from the outermost agentic tool is returned.

This prevents intermediate artifacts from bubbling up when you only care about the final result. For example, if an outer assembleConcert tool calls an inner findPerformances tool, you typically want only the final Concert artifact, not all the intermediate Performance artifacts.

Use withCaptureNestedArtifacts(true) if you need to capture artifacts from nested agentic tools:

```
JavaKotlin
// Default: nested artifacts are NOT capturedSimpleAgenticToolconcertAssembler=newSimpleAgenticTool("assembleConcert","Assemble a concert program").withTools(findPerformancesTool,createConcertTool);// Only the Concert artifact from createConcert is returned// Opt-in: capture all nested artifactsSimpleAgenticToolfullCapture=concertAssembler.withCaptureNestedArtifacts(true);// Both Performance artifacts from findPerformances AND Concert from createConcert are captured
```

This setting only affects artifacts from nested agentic tool calls. Artifacts from regular (non-agentic) tools are always captured.

Tool Chaining

When working with objects returned by tools, you often want to expose @LlmTool methods on those objects as additional tools—but only after the object has been retrieved. The withToolChainingFrom() method enables this pattern.

Tool chaining increases determinism. Once a tool returns a specific object, the LLM gains access to that object’s business methods—navigating a data structure through well-defined operations rather than unstructured reasoning. This keeps the LLM on a guided path through your domain logic.

Tool chaining is available on both AgenticTool and PromptRunner, via the shared ToolChaining interface. This means you can use tool chaining not only in agentic tool loops, but also in simple createObject and generateText calls through PromptRunner. This is significant because it enables any action to dynamically discover and use tools from returned artifacts without requiring a full agentic tool setup. When you register a class, placeholder tools are created for each @LlmTool method on that class. Initially, these tools return "not available yet" messages. When a tool returns an artifact matching the registered type, the placeholder tools become active and delegate to the bound instance.

Last Wins Semantics: When multiple artifacts of the same type are returned, only the most recent one’s tools are active. This ensures the LLM always works with the "current" instance.

```
JavaKotlin
// Domain class with tool methodspublicclassUser{privatefinalStringid;privateStringemail;@LlmTool("Update the user's email address")publicStringupdateEmail(StringnewEmail){this.email=newEmail;return"Email updated to "+newEmail;}}// Create agentic tool with tool chainingSimpleAgenticTooluserManager=newSimpleAgenticTool("userManager","Manage user accounts").withTools(searchUserTool,getUserTool)// Tools to find/retrieve users.withToolChainingFrom(User.class);// User methods become tools when retrieved// Flow:// 1. LLM calls searchUserTool to find users// 2. LLM calls getUserTool which returns a User artifact// 3. updateEmail() becomes available as a tool bound to that User// 4. LLM calls updateEmail("new@example.com")
```

Predicate-Based Filtering

You can control which instances contribute tools using a predicate. The predicate receives the artifact and the current AgentProcess, allowing filtering based on object state or process context.

```
JavaKotlin
// Only expose tools for admin usersSimpleAgenticTooladminManager=newSimpleAgenticTool("adminManager","Manage admin users").withTools(searchUserTool,getUserTool).withToolChainingFrom(User.class,(user,agentProcess)->user.getRole().equals("admin"));// Regular users won't have their tools exposed// Only when an admin User is retrieved will updateEmail() become available
```

Auto-Discovery Mode

For maximum flexibility, use withToolChainingFromAny() to automatically discover and expose tools from any returned artifact that has @LlmTool methods. Unlike registered sources, auto-discovery replaces ALL previous bindings when a new artifact is discovered—ensuring only one "current" object’s tools are active at a time.

```
JavaKotlin
// Auto-discover tools from any returned objectSimpleAgenticToolexplorer=newSimpleAgenticTool("explorer","Explore and manipulate objects").withTools(searchTool,getTool).withToolChainingFromAny();// Tools from any returned object are exposed// Flow:// 1. LLM calls getTool which returns a User -> User tools are available// 2. LLM calls another getTool which returns an Order -> Order tools replace User tools// 3. Only the most recent object's tools are active
```

This pattern is useful when:

- Objects have operations:** The object itself knows how to perform actions (e.g., user.updateEmail(), order.cancel())
- Context-dependent tools:** Operations only make sense after retrieving a specific instance
- Clean API design:** Tools are defined on the class rather than as separate tool classes
- Exploratory workflows:** The LLM dynamically works with whatever object is "current"

All agentic tool types support tool chaining:

- SimpleAgenticTool: Chained tools are available as soon as an artifact is returned
- PlaybookTool: Chained tools are available immediately (not subject to unlock conditions)
- StateMachineTool: Chained tools are available globally (not state-bound)

Tool Chaining on PromptRunner

Tool chaining is not limited to agentic tools. Because both AgenticTool and PromptRunner implement the ToolChaining interface, you can use withToolChainingFrom() and withToolChainingFromAny() directly on a PromptRunner obtained from an action’s OperationContext.

This is important because it enables dynamic tool discovery within simple createObject and generateText calls—without requiring a full SimpleAgenticTool wrapper.

```
JavaKotlin
```

```
// In an @Action method:PromptRunnerai=context.ai().withToolChainingFrom(User.class)// Chained tools from
User.withTools(searchUserTool,getUserTool);// When getUserTool returns a User artifact, User's @LlmTool methods// automatically
become available for the LLM to callStringresult=ai.generateText("Find user Alice and update her email to alice@new.com");
```

Filtering Artifacts for Asset Tracking

When using tools with an AssetTracker (common in chat applications), you can filter which artifacts become tracked assets. The addReturnedAssets and addAnyReturnedAssets methods accept a Predicate<Asset> filter that works with both Java and Kotlin:

```
JavaKotlin
// Track only assets that pass the filterToolwrapped=assetTracker.addReturnedAssets(concertTool,asset->{// Only track concerts with
at least 3 worksreturnassetinstanceofConcertconcert&&concert.getWorks().size()>=3;});// Apply the same filter to multiple
toolsList<Tool>wrappedTools=assetTracker.addAnyReturnedAssets(List.of(tool1,tool2,tool3),asset-
>asset.getId().startsWith("important-"));
```

The filter is applied after type matching, so you can use type-specific criteria to decide which artifacts are worth tracking.

Migration from Other Frameworks

If you're coming from frameworks like LangChain or Google ADK, Embabel's agentic tools provide a familiar pattern similar to their "supervisor" architectures:

Framework	Pattern	Embabel Equivalent
LangChain/LangGraph	Supervisor agent with worker agents	SimpleAgenticTool with sub-tools
Google ADK	Coordinator with sub_agents / AgentTool	SimpleAgenticTool with sub-tools

The key differences:

- Tool-centric:** Embabel's agentic tools operate at the tool level, not the agent level. They're lightweight and can be mixed freely with regular tools.
- Simpler model:** No graph-based workflows or explicit Sequential/Parallel/Loop patterns—just LLM-driven orchestration.
- Composable:** An agentic tool is still "just a tool" that can be used anywhere tools are accepted.

However, for anything beyond simple orchestration, Embabel offers far more powerful alternatives:

Scenario	Use This Instead
Business processes with defined outputs	GOAP planner - deterministic, goal-oriented planning with preconditions and effects
Exploration and event-driven systems	Utility AI - selects highest-value action at each step
Branching, looping, or stateful workflows	@State workflows - typesafe state machines with GOAP planning within each state

These provide **deterministic, typesafe planning** that is far more predictable and powerful than supervisor-style LLM orchestration.

Use SimpleAgenticTool for simple cases, PlaybookTool for structured workflows, or StateMachineTool for formal state machines.

Graduate to GOAP, Utility, or @State for production workflows where predictability matters.

For supervisor-style orchestration with typed outputs and full blackboard state management, see [SupervisorInvocation](#). It operates at a higher level than agentic tools, orchestrating @Action methods rather than Tool instances, and produces typed goal objects with currying support.

3.9.7. Progressive Tools

Great fleas have little fleas upon their backs to bite 'em,
And little fleas have lesser fleas, and so ad infinitum.
And the great fleas themselves, in turn, have greater fleas to go on;
While these again have greater still, and greater still, and so on.

— Augustus De Morgan

Progressive tools enable dynamic tool disclosure—presenting a simplified interface initially, then revealing more granular tools based on context or when the LLM expresses intent.

The Progressive Tool Hierarchy

Embabel provides a hierarchy of progressive tool interfaces:

- ProgressiveTool:** The base interface for tools that can reveal inner tools based on context. Its innerTools(process: AgentProcess) method returns tools that may vary depending on the current agent process state.
- UnfoldingTool:** A ProgressiveTool with a fixed set of inner tools. When invoked, it "unfolds" to reveal its contents—like opening a folded map to see the details inside. This is the most commonly used progressive tool type.

An UnfoldingTool presents a high-level description to the LLM and, when invoked, exposes its inner tools. This pattern is useful for **progressive tool disclosure**—reducing initial complexity while allowing access to detailed functionality on demand.

When to Use UnfoldingTool

UnfoldingTool is useful when:

- You have many related tools that might overwhelm the LLM with choices
- You want to group tools by category (e.g., "database operations", "file operations")
- You want the LLM to express intent before revealing detailed options
- You need to reduce token usage for tool descriptions

Creating a Simple UnfoldingTool

The simplest form exposes all inner tools when invoked:

```
JavaKotlin
importcom.embabel.agent.api.tool.progressive.UnfoldingTool;importcom.embabel.agent.api.tool.Tool;// Create inner
toolsToolqueryTool=Tool.create("query_table","Execute a SQL query",Tool.InputSchema.of(Tool.Parameter.string("sql","The SQL
query to execute")),input->Tool.Result.text("{\"rows\": 5}"));ToolinsertTool=Tool.create("insert_record","Insert a new
record",Tool.InputSchema.of(Tool.Parameter.string("table","Table name")),input->Tool.Result.text("{\"id\":
123}"));TooldeleteTool=Tool.create("delete_record","Delete a record",Tool.InputSchema.of(Tool.Parameter.integer("id","Record ID
to delete")),input->Tool.Result.text("{\"deleted\": true}"));// Create the UnfoldingTool
facadevardatabaseTool=UnfoldingTool.of("database_operations","Use this tool to work with the database. Invoke to see specific
operations.",List.of(queryTool,insertTool,deleteTool));
```

Fluent Builder API

UnfoldingTool supports a fluent builder pattern for combining tools from multiple sources. Use withTools() to add individual tools or withToolObject() to add tools from @LlmTool annotated objects:

```
JavaKotlin
importcom.embabel.agent.api.tool.progressive.UnfoldingTool;// Start with base tools and add
morevarcombinedTools=UnfoldingTool.of("workspace","Workspace operations. Invoke to see available
tools.",List.of(baseTool)).withTools(searchTool,filterTool)// Add individual tools.withToolObject(newDatabaseOperations())// Add
from @LlmTool class.withToolObject(newFileOperations());// Chain multiple sources
```

This is useful when:

- Combining existing tools:** Merge tools from different sources into one progressive facade
- Adding ad-hoc tools:** Start with annotated tool classes and add programmatic tools

- **Context-specific grouping:** Build different tool combinations for different invocation contexts

The builder preserves all properties (removeOnInvoke, childToolUsageNotes) from the original UnfoldingTool.

Category-Based Tool Selection

Use byCategory to expose different tools based on the category the LLM selects:

```
JavaKotlin
import com.embabel.agent.api.tool.progressive.UnfoldingTool;
import java.util.Map;
// Define tools by category
Map<String, List<Tool>> toolsByCategory = Map.of("read", List.of(readFileTool, listDirectoryTool, searchFilesTool), "write", List.of(writeFileTool, deleteFileTool, moveFileTool));
// Create category-based UnfoldingTool
var fileTool = UnfoldingTool.byCategory("file_operations", "File operations. Pass category: 'read' for reading files, 'write' for modifying files.", toolsByCategory);
// The tool's schema automatically includes the category as an enum parameter
// When invoked with {"category": "read"}, only read tools are exposed
// When invoked with {"category": "write"}, only write tools are exposed
```

Custom Selection Logic

For more complex selection logic, use selectable:

```
JavaKotlin
import com.embabel.agent.api.tool.progressive.UnfoldingTool;
import com.fasterxml.jackson.databind.ObjectMapper;
List<Tool> allTools = List.of(basicTool, advancedTool, adminTool);
var permissionBasedTool = UnfoldingTool.selectable("api_operations", "API operations. Pass 'accessLevel': 'basic', 'advanced', or 'admin'." , allTools, Tool.InputSchema.of(Tool.Parameter.string("accessLevel", "Access level for operations"), true, List.of("basic", "advanced", "admin"))), true, // removeOnInvoke
input->{// Custom selection logic
try {
ObjectMapper mapper = new ObjectMapper();
Map<String, Object> params = mapper.readValue(input, Map.class);
String level = (String) params.get("accessLevel");
return switch (level) {
case "basic" -> List.of(basicTool);
case "advanced" -> List.of(basicTool, advancedTool);
case "admin" -> allTools;
default -> List.of(basicTool);
};
} catch (Exception e) {
return List.of(basicTool);
};
};
```

Keeping the Facade Available

By default, a UnfoldingTool is removed after invocation (replaced by its inner tools). Set removeOnInvoke = false to keep the facade available for re-invocation with different arguments:

```
JavaKotlin
// Facade stays available after invocation
var persistentTool = UnfoldingTool.of("operations", "Operations. Invoke multiple times with different needs.", allTools, false // removeOnInvoke = false);
```

Enabling UnfoldingTool in the Tool Loop

UnfoldingTool is **enabled by default** when using Embabel’s tool loop. The ToolInjectionStrategy.DEFAULT includes UnfoldingToolInjectionStrategy, so no additional configuration is needed. If you need to combine with custom strategies, use ChainedToolInjectionStrategy:

```
JavaKotlin
import com.embabel.agent.spi.loop.ChainedToolInjectionStrategy;
// Combine UnfoldingTool support with custom strategies
ChainedToolInjectionStrategy combined = ChainedToolInjectionStrategy.withUnfolding(customStrategy1, customStrategy2);
```

How UnfoldingTool Works

1. **Initial state:** The LLM sees only the facade tool (e.g., "database_operations")
2. **LLM invokes:** The LLM calls the facade with optional arguments
3. **Strategy evaluates:** UnfoldingToolInjectionStrategy detects the invocation
4. **Tools replaced:** The facade is removed (if removeOnInvoke is true) and inner tools are added
5. **Continue:** The LLM now sees and can use the specific inner tools

This flow reduces the initial tool set complexity while allowing the LLM to access detailed tools when it needs them.

Context Preservation and Usage Notes

When a UnfoldingTool is expanded, its child tools replace the facade. Without context preservation, the LLM would lose important information about *why* these tools are grouped together. For example, a "spotify_search" tool containing vector_search, text_search, and regex_search would expand to just three generic search tools - the LLM wouldn’t know these are specifically for searching Spotify music data.

Embabel solves this by automatically injecting a **context tool** alongside the child tools. This context tool:

- Preserves the parent’s description ("Search Spotify for music data")
- Lists the available child tools
- Includes optional usage notes (via childToolUsageNotes)

The childToolUsageNotes parameter provides guidance on when and how to use the child tools. This guidance appears **once** in the context tool rather than being duplicated in each child tool’s description:

```
JavaKotlin
var spotifySearch = UnfoldingTool.of("spotify_search", "Search Spotify for music data including artists, albums, and tracks.", List.of(vectorSearchTool, textSearchTool, regexSearchTool), true, // removeOnInvoke
"Try vector search first for semantic queries like 'upbeat jazz'. "+"Use text search for exact artist or album names. "+"Use regex search for pattern matching on metadata.");
```

After the LLM invokes spotify_search, it will see:

- vector_search - the actual search tool
- text_search - the actual search tool
- regex_search - the actual search tool
- spotify_search_context - context tool with description and usage notes

The context tool’s description includes the original purpose and available tools. When called, it returns full details about each child tool plus the usage notes - providing a single reference point without polluting individual tool descriptions.

Annotation-Based UnfoldingTool

For a more declarative approach, use the @UnfoldingTools class annotation combined with @LlmTool method annotations:

```
JavaKotlin
import com.embabel.agent.api.annotation.UnfoldingTools;
import com.embabel.agent.api.annotation.LlmTool;
@UnfoldingTools(name="database_operations", description="Database operations. Invoke to see specific tools.")
public class DatabaseTools {
    @LlmTool(description="Execute a SQL query")
    public QueryResult query(String sql) { // implementation }
    @LlmTool(description="Insert a record")
    public InsertResult insert(String table, Map<String, Object> data) { // implementation }
    @LlmTool(description="Delete a record")
    public void delete(Long id) { // implementation }
}
// Create the UnfoldingTool from the annotated class
var tool = UnfoldingTool.fromInstance(new DatabaseTools());
```

You can also specify childToolUsageNotes in the annotation to provide guidance on using the child tools:

```
JavaKotlin
@UnfoldingTools(name="music_search", description="Search music database for artists, albums, and tracks", childToolUsageNotes="Try vector search first for semantic queries. "+"Use text search for exact artist names.")
public class MusicSearchTools {
    @LlmTool(description="Semantic search using embeddings")
    public List<Track> vectorSearch(String query) { // implementation }
    @LlmTool(description="Exact match text search")
    public List<Track> textSearch(String query) { // implementation }
}
```

Category-Based Selection with Annotations

Add category to @LlmTool annotations to automatically create a category-based UnfoldingTool:

JavaKotlin

```
@UnfoldingTools(name="file_operations",description="File operations. Pass category: 'read' or 'write'.")publicclassFileTools{@LlmTool(description="Read file contents",category="read")publicStringreadFile(Stringpath){returnFiles.readString(Path.of(path));}@LlmTool(description="List directory contents",category="read")publicList<String>listDir(Stringpath){returnFiles.list(Path.of(path)).map(Path::toString).toList();}@LlmTool(description="Write file contents",category="write")publicvoidwriteFile(Stringpath,Stringcontent){Files.writeString(Path.of(path),content);}@LlmTool(description="Delete a file",category="write")publicvoiddeleteFile(Stringpath){Files.delete(Path.of(path));}}// Automatically creates category-based selectionvartool=UnfoldingTool.fromInstance(newFileTools());// When invoked with {"category": "read"}, only read tools are exposed// When invoked with {"category": "write"}, only write tools are exposed
```

@UnfoldingTools Annotation Attributes

Attribute	Type	Default	Description
name	String	Required	Name of the facade tool the LLM will see
description	String	Required	Description explaining the tool category
removeOnInvoke	boolean	true	Whether to remove the facade after invocation
categoryParameter	String	"category"	Name of the parameter for category selection

@LlmTool Category Attribute

The category attribute on @LlmTool is used when the containing class has @UnfoldingTools:

- Tools with the same category are grouped together
- Tools without a category are added to all category groups plus an "all" category
- If no tools have categories, a simple (non-category-based) UnfoldingTool is created

Real-World Example: Spotify Integration

Here’s a real-world example from the Impromptu chatbot that uses @UnfoldingTools to progressively disclose Spotify functionality:

JavaKotlin

```
@UnfoldingTools(name="spotify",description="Access Spotify music features. Invoke this tool to enable Spotify "+"operations like playing music, searching tracks, managing playlists, "+"and controlling playback.")publicrecordSpotifyTools(ImpromptuUseruser,SpotifyServicesspotifyService){@LlmTool(description="Check if user has linked their Spotify account")publicStringcheckSpotifyStatus(){/* ... */}@LlmTool(description="Get the user's Spotify playlists")publicStringgetPlaylists(){/* ... */}@LlmTool(description="Search for tracks on Spotify by song name, artist, or both")publicStringsearchTracks(Stringquery){/* ... */}@LlmTool(description="Play a track on Spotify by searching for it")publicStringplayTrack(Stringquery){/* ... */}@LlmTool(description="Pause the current Spotify playback")publicStringpausePlayback(){/* ... */}// ... more tools}
```

With this setup:

- The LLM initially sees a single spotify tool
- When the user says "play some jazz", the LLM invokes spotify
- The spotify facade is replaced with all the inner tools (getPlaylists, searchTracks, playTrack, etc.)
- The LLM can then call searchTracks or playTrack to fulfill the request

Auto-Detection with Tool.fromInstance()

When you use Tool.fromInstance() on a class annotated with @UnfoldingTools, it automatically creates an UnfoldingTool:

JavaKotlin

```
// Auto-detects @UnfoldingTools and creates an UnfoldingToolList<Tool>tools=Tool.fromInstance(newSpotifyTools(user,service));// Returns a single UnfoldingTool, not individual tools
```

This works seamlessly with withToolObject() on PromptRunner:

JavaKotlin

```
context.ai().withToolObject(newSpotifyTools(user,spotifyService)).respond("Play some classical music");// The SpotifyTools are automatically exposed as a single UnfoldingTool facade
```

Wrapping Tool Objects with fromToolObject()

UnfoldingTool.fromInstance() requires the class to be annotated with @UnfoldingTools. This doesn’t work for objects like interface implementations with @LlmTool default methods that you cannot or should not annotate with @UnfoldingTools.

Use fromToolObject() to wrap **any** object with @LlmTool methods into an UnfoldingTool, providing name and description explicitly:

JavaKotlin

```
importcom.embabel.agent.api.tool.progressive.UnfoldingTool;// FileWriteTools is an interface with @LlmTool default methods—// it cannot be annotated with @UnfoldingToolsFileWriteToolsfileTools=newFileWriteToolsImpl(workspace);vartool=UnfoldingTool.fromToolObject(fileTools,"file_write_tools","Tools for writing and managing files. Invoke to see specific operations.");
```

All standard options are available:

JavaKotlin

```
vartool=UnfoldingTool.fromToolObject(fileTools,"file_write_tools","Tools for writing and managing files.",false,// removeOnInvoke"Use writeFile for new files, appendFile for existing ones."// childToolUsageNotes);
```

Use fromToolObject() when the tool class is an interface, a third-party class, or any class where adding @UnfoldingTools is impractical.

Use fromInstance() when you control the class and can add the @UnfoldingTools annotation.

Nested UnfoldingTools

UnfoldingTools can be nested for multi-level progressive disclosure. This enables organizing large tool collections into logical hierarchies where the LLM navigates by invoking facade tools.

Programmatic Nesting

Use UnfoldingTool.of() to create nested hierarchies programmatically:

JavaKotlin

```
// Inner UnfoldingTool for user managementvaruserManagement=UnfoldingTool.of("user_management","User management operations",List.of(createUserTool,deleteUserTool,updateUserTool));// Inner UnfoldingTool for system configvarsystemConfig=UnfoldingTool.of("system_config","System configuration operations",List.of(updateConfigTool,backupTool,restoreTool));// Outer UnfoldingTool containing bothvaradminTool=UnfoldingTool.of("admin_operations","Administrative operations. Invoke to see categories.",List.of(userManagement,systemConfig));// Flow:// 1. LLM sees: admin_operations// 2. LLM invokes: admin_operations -> sees: user_management, system_config// 3. LLM invokes: user_management -> sees: createUser, deleteUser, updateUser
```

Annotation-Based Nesting with Inner Classes

You can also create nested hierarchies using @UnfoldingTools annotations on inner classes. When UnfoldingTool.fromInstance() is called, it automatically discovers and includes any nested inner classes that are also annotated with @UnfoldingTools:

JavaKotlin

```
@UnfoldingTools(name="admin_operations",description="Administrative operations. Invoke to access specific
areas.")publicclassAdminTools{@LlmTool(description="Get system status")publicStringgetStatus(){return"System is healthy";}//
Nested inner class - automatically discovered and included as a nested
UnfoldingTool@UnfoldingTools(name="user_management",description="User management operations. Invoke to see specific
tools.")publicstaticclassUserManagement{@LlmTool(description="Create a new
user")publicStringcreateUser(Stringusername){return"Created user: "+username;}@LlmTool(description="Delete a
user")publicStringdeleteUser(Stringusername){return"Deleted user: "+username;}// Can nest even
deeper@UnfoldingTools(name="user_permissions",description="User permission
operations")publicstaticclassPermissions{@LlmTool(description="Grant permission to
user")publicStringgrant(Stringuser,Stringpermission){return"Granted";}@LlmTool(description="Revoke permission from
user")publicStringrevoke(Stringuser,Stringpermission){return"Revoked";}}@UnfoldingTools(name="system_config",description
="System configuration. Invoke to see config tools.")publicstaticclassSystemConfig{@LlmTool(description="Update
configuration")publicStringupdateConfig(Stringkey,Stringvalue){return"Updated";}@LlmTool(description="Backup
configuration")publicStringbackup(){return"Backed up";}}// Create the full nested hierarchy
automaticallyvaradminTool=UnfoldingTool.fromInstance(newAdminTools());// Flow:// 1. LLM sees: admin_operations// 2. LLM
invokes: admin_operations -> sees: getStatus, user_management, system_config// 3. LLM invokes: user_management -> sees: createUser,
deleteUser, user_permissions// 4. LLM invokes: user_permissions -> sees: grant, revoke
```

This approach provides several benefits:

- **Encapsulation:** All related tools are organized in a single class hierarchy
- **Automatic discovery:** No manual wiring - inner classes with @UnfoldingTools are automatically included
- **Arbitrary depth:** Nest as many levels as needed to organize your tools logically
- **Mixed content:** Each level can have both direct @LlmTool methods and nested @UnfoldingTools classes

Dynamically Configured Inner Tools

A powerful pattern with UnfoldingTool.selectable() is creating inner tools that are **configured** based on the parameters passed when invoking the facade. The selector function can create new tool instances with captured state, connection strings, or other configuration:

```
JavaKotlin
// UnfoldingTool that configures database tools based on connection
parametervardatabaseTool=UnfoldingTool.selectable("database","Database operations. Pass 'connection' to configure
tools.",Collections.emptyList(),// Tools created dynamicallyTool.InputSchema.of(Tool.Parameter.string("connection","Database
connection string")),true,// removeOnInvokeinput->{// Parse connection from
inputObjectMappermapper=newObjectMapper();Map<String,Object>params=mapper.readValue(input,Map.class);Stringconnecti
on=(String)params.getOrDefault("connection","localhost");// Create tools configured with the connection
stringreturnList.of(Tool.create("query","Query database at "+connection,queryInput->{// Tool has captured the connection
stringreturnTool.Result.text("Queried "+connection+": "+queryInput);}),Tool.create("insert","Insert into database at
"+connection,insertInput->{returnTool.Result.text("Inserted into "+connection);}));});// When LLM invokes with {"connection": "prod-
db.example.com"}// The injected tools are configured to use that specific connection
```

This pattern is useful for:

- **Multi-tenant systems:** Configure tools with tenant-specific credentials or endpoints
- **Environment selection:** Let the LLM choose between dev/staging/prod environments
- **Stateful operations:** Create tools that share state (like a shopping cart’s item list)
- **Dynamic service discovery:** Configure tools based on runtime service locations

Example: Stateful Shopping Cart Tools

```
JavaKotlin
varcartTool=UnfoldingTool.selectable("shopping_cart","Shopping cart. Pass 'cart_id' to select which cart to operate
on.",Collections.emptyList(),Tool.InputSchema.of(Tool.Parameter.string("cart_id","Shopping cart ID")),true,input->{// Each
invocation creates a fresh set of tools with shared stateStringcartId=parseCartId(input);List<String>cartItems=newArrayList<>();//
Shared statereturnList.of(Tool.create("add_item","Add item to cart "+cartId,Tool.InputSchema.of(Tool.Parameter.string("item","Item
name")),itemInput->{Stringitem=parseItem(itemInput);cartItems.add(item);// Captured statereturnTool.Result.text("Added
"+item+". Total: "+cartItems.size());}),Tool.create("view_cart","View cart "+cartId+" contents",viewInput-
>{returnTool.Result.text("Cart "+cartId+": " +String.join(" ",cartItems);})),Tool.create("checkout","Checkout cart
"+cartId,checkoutInput->{Stringtotal=calculateTotal(cartItems);cartItems.clear();returnTool.Result.text("Checked out "+cartId+" for
"+total);}));});
```

Comparison with Other Approaches

Other agent frameworks address large tool collections with different approaches, each with trade-offs:

- **Anthropic’s Tool Search Tool:** Uses a defer_loading: true flag to prevent tools from being loaded upfront. Tools are discovered via a separate "Tool Search Tool" that searches tool metadata. This requires maintaining searchable tool descriptions and adds latency for each discovery step.
- **LangGraph Dynamic Tool Calling:** Uses vector stores and semantic search to select relevant tools based on the user’s query. This requires embedding infrastructure, vector database setup, and careful tuning of similarity thresholds.
- **Google ADK AgentTool:** Uses sub-agents that recursively delegate to other agents, each potentially having their own tool sets. Tool discovery is implicit through the agent hierarchy.
- **LangChain4j ToolProvider:** Provides a ToolProvider interface for dynamic tool selection, but it works *before* the LLM call by analyzing the incoming user message. For example, "if the message contains 'booking', include booking tools." This is pre-filtering based on message content, not progressive disclosure through tool invocation. LangChain4j’s documentation also suggests embedding-based classification, RAG over tool descriptions, or two-pass LLM selection—all requiring additional infrastructure or extra LLM calls.

UnfoldingTool takes a fundamentally different approach: **invoke to reveal**. Instead of searching through tool metadata, the LLM simply invokes a facade tool to unlock the tools it contains.

Beyond Search: Dynamic Tool Configuration

Crucially, UnfoldingTool goes far beyond what any search-based approach can offer. Search can only **find** pre-existing tools—it cannot create new ones or modify their behavior. With UnfoldingTool.selectable(), the selector function can:

- **Create entirely new tool instances** with different implementations based on runtime parameters
- **Capture configuration** (connection strings, credentials, endpoints) into the tool’s behavior
- **Share mutable state** between the tools created in a single invocation
- **Customize tool descriptions** to reflect the specific context of use

For example, when an LLM invokes a "database" UnfoldingTool with {"connection": "prod-db.example.com"}, the returned tools don’t just have different descriptions—they have **different behavior** that operates on that specific database. This is fundamentally impossible with search-based discovery, which can only return references to pre-defined tools.

This provides several advantages:

Aspect	Other Approaches	UnfoldingTool
--------	------------------	---------------

Infrastructure	Requires vector stores, embeddings, search indices, or pre-filtering logic	No additional infrastructure required
Selection Timing	Before LLM call (pre-filtering based on message analysis)	After LLM decides to invoke a facade (LLM-driven discovery)
Latency	Search/embedding adds latency; two-pass selection doubles LLM calls	Instant unlock on invocation
Scalability	Search quality degrades with very large tool sets; requires careful tuning	Scales to any number of tools via nesting without degradation
Determinism	Search results can vary based on embedding similarity	Deterministic: invoking a facade always reveals the same tools
Cost	Embedding generation, vector search, or extra LLM calls incur compute costs	No additional compute beyond the tool call itself
Dynamic Behavior	Can only return references to pre-existing tools	Can create new tool instances with runtime-configured behavior

The hierarchical nesting capability of `UnfoldingTool` means you can organize thousands of tools into a logical tree structure. The LLM navigates this tree by making simple invocations, with no search overhead at any level. For example, a top-level "admin_operations" facade might reveal 5 category facades, each revealing 20 specific tools—giving access to 100 tools with at most 2 invocations.

UnfoldingTool vs LlmReference

Both `UnfoldingTool` and `LlmReference` expose tools to the LLM, but they serve different purposes:

Use `UnfoldingTool` when:

- You have a single top-level capability that the LLM can invoke as one tool
- The prompt contribution is short and can fit in the tool description
- Example: A "database" tool that reveals query/insert/delete tools on invocation

Use `LlmReference` when:

- The prompt contribution is long or of general significance (appears in system prompt)
- You have a bunch of related tools, not just one top-level tool
- You need `notes()` for detailed usage instructions separate from the tool descriptions
- The reference contributes context beyond just tool availability

Implementing both:

Classes like `Memory` and `ToolishRag` implement both `Tool` and `LlmReference`, giving maximum flexibility:

Java

```
// Use as LlmReference (adds to system prompt + tools)ai.withReference(memory).respond(...); // Use as Tool directly (just the tool)ai.withTool(memory).respond(...);
```

Kotlin

```
// Use as LlmReference (adds to system prompt + tools)ai.withReference(memory).respond(...); // Use as Tool directly (just the tool)ai.withTool(memory).respond(...)
```

When used as an `LlmReference`, the `tools()` method exposes the inner tools directly. When used as a `Tool`, the implementation wraps them in an `UnfoldingTool` facade.

3.9.8. Process Introspection Tools

`Embabel` provides built-in `UnfoldingTool` implementations for introspecting the current agent process and its blackboard. These tools enable agentic workflows where the LLM can monitor its own progress, check resource usage, and access data from previous steps.

AgentProcessTools: Runtime Awareness

`AgentProcessTools` provides tools for the LLM to understand its current execution context. This is useful when you want an agent to be aware of its own operational status - for example, to check how much budget remains before undertaking an expensive operation, or to review what actions have been taken so far.

When to use `AgentProcessTools`:

- Budget-aware agents:** Check remaining cost or token budget before expensive operations
- Long-running workflows:** Monitor elapsed time and action history
- Debugging and logging:** Understand what models and tools have been used
- Self-reflection:** Agents that need to reason about their own behavior

Sub-tools exposed:

Tool Name	Purpose
process_status	Current process ID, status, running time, and goal information
process_budget	Budget limits (cost, tokens, actions) and remaining capacity
process_cost	Total cost, LLM invocation count, and detailed token usage
process_history	List of actions taken so far with execution times
process_tools_stats	Tool usage statistics (call counts per tool)
process_models	LLM models that have been invoked

Java**Kotlin**

```
import com.embabel.agent.tools.process.AgentProcessTools;import com.embabel.agent.api.tool.progressive.UnfoldingTool; // Create the tool - typically added to an agentic toolvar processTools = new AgentProcessTools().create(); // Add to SimpleAgenticToolvar assistant = new SimpleAgenticTool("assistant", "...").withTools(processTools);
```

These tools require an active `AgentProcess` context. If called outside of an agent execution, they return an error message indicating no process is available.

BlackboardTools: Accessing Workflow Data

`BlackboardTools` provides tools for the LLM to access objects in the current process's blackboard. The blackboard is `Embabel`'s shared context mechanism - it holds artifacts from previous actions, tool outputs (when using `ArtifactSink`), and any other objects bound to the process.

When to use `BlackboardTools`:

- Multi-step workflows:** Access results from earlier actions without re-execution
- Tool output access:** When tools use `ArtifactSink` to publish structured data, `BlackboardTools` lets the LLM retrieve it
- Context awareness:** Let the LLM explore what data is available in the current context
- Debugging:** Inspect blackboard contents during development

Sub-tools exposed:

Tool Name	Purpose
-----------	---------

blackboard_list	List all objects in the blackboard with their types and indices
blackboard_get	Get an object by its binding name (e.g., "user", "searchResults")
blackboard_last	Get the most recent object of a given type (matches simple name or FQN)
blackboard_describe	Get a detailed description/formatting of an object by binding name
blackboard_count	Count the number of objects of a given type in the blackboard

```
JavaKotlin
import com.embabel.agent.tools.blackboard.BlackboardTools;
import com.embabel.agent.api.tool.progressive.UnfoldingTool;
// Create with default formatting
var blackboardTools = new BlackboardTools().create();
// Or with custom formatting for blackboard entries
var blackboardTools = new BlackboardTools().create(myCustomFormatter);
// Add to SimpleAgenticTool
var assistant = new SimpleAgenticTool("assistant", "...").withTools(blackboardTools);
```

Formatting blackboard entries:

By default, BlackboardTools uses DefaultBlackboardEntryFormatter which:

- Uses infoString() for objects implementing HasInfoString
- Uses content property for objects implementing HasContent
- Falls back to toString() for other objects

You can provide a custom BlackboardEntryFormatter to control how objects are presented to the LLM.

Type matching:

The blackboard_last and blackboard_count tools match types by:

- **Simple name:** "Person" matches any class named Person
- **Fully qualified name:** "com.example.Person" matches that specific class

This flexibility lets the LLM query by whatever name is most convenient.

Combining Process Introspection Tools

For agents that need full situational awareness, combine both tools:

```
JavaKotlin
SimpleAgenticToolawarenessAgent = new SimpleAgenticTool("aware_assistant", "An assistant that can check its own status and access previous results").withTools(new AgentProcessTools().create(), new BlackboardTools().create());
```

3.9.9. McpToolFactory: MCP Tool Integration

McpToolFactory is an interface that provides a convenient way to integrate [Model Context Protocol \(MCP\)](#) tools into your application. It creates Embabel Tool instances from MCP servers, with support for filtering tools and wrapping them in UnfoldingTool facades.

SpringAiMcpToolFactory is the Spring AI-based implementation.

Creating McpToolFactory

SpringAiMcpToolFactory requires a list of McpSyncClient instances, which are typically provided by Spring’s MCP auto-configuration:

```
JavaKotlin
import com.embabel.agent.tools.mcp.McpToolFactory;
import com.embabel.agent.spi.support.springai.SpringAiMcpToolFactory;
import io.modelcontextprotocol.client.McpSyncClient;
@Configuration
public class ToolConfiguration {
    @Bean
    public McpToolFactory mcpToolFactory(List<McpSyncClient> clients) {
        return new SpringAiMcpToolFactory(clients);
    }
}
```

MCP clients are configured in application.yml. See [MCP Integration](#) for configuration details.

Getting Individual MCP Tools

Use toolByName to retrieve a single MCP tool by its exact name:

```
JavaKotlin
// Returns null if not found
Tool braveSearch = mcpToolFactory.toolByName("brave_web_search");
if (braveSearch != null) {
    ai.withTool(braveSearch).generateText("Search for recent news about AI");
}
// Throws IllegalArgumentException if not found (with helpful error message)
Tool requiredTool = mcpToolFactory.requiredToolByName("brave_web_search");
```

Creating UnfoldingTool Facades from MCP

McpToolFactory can wrap groups of MCP tools in an UnfoldingTool facade for progressive disclosure. This is useful when you have many MCP tools but want to present them as logical categories.

By Exact Tool Names:

```
JavaKotlin
// Create a UnfoldingTool with specific tool names
var wikipediaTool = mcpToolFactory.unfoldingByName("wikipedia", "Search and find content from Wikipedia");
Set.of("search_wikipedia", "get_article", "get_related_topics", "get_summary");
```

By Regex Patterns:

```
JavaKotlin
import java.util.regex.Pattern;
// Match tools by regex patterns
var dbTool = mcpToolFactory.unfoldingMatching("database_operations", "Database operations. Invoke to access database tools.");
List.of(Pattern.compile("^db_.*"), Pattern.compile(".*query.*"));
```

With Custom Filter:

```
JavaKotlin
// Custom filter predicate
var webTool = mcpToolFactory.unfolding("web_operations", "Web operations. Invoke to access web tools.");
callback->callback.getToolDefinition().name().startsWith("web_");
```

Controlling Facade Removal

By default, UnfoldingTool facades created by McpToolFactory are removed after invocation (replaced by inner tools).

Use removeOnInvoke = false to keep the facade:

```
JavaKotlin
// Keep facade even after invocation
var persistentTool = mcpToolFactory.unfoldingByName("wikipedia", "Search Wikipedia");
Set.of("search_wikipedia", "get_article"), false // removeOnInvoke = false;
```

Real-World Example: Chatbot with MCP Tools

Here’s a real-world example from a production chatbot that uses McpToolFactory to integrate MCP tools with graceful degradation:

```
JavaKotlin
@Configuration
public class ChatConfiguration {
    @Bean
    public McpToolFactory mcpToolFactory(List<McpSyncClient> clients) {
        return new SpringAiMcpToolFactory(clients);
    }
    @Bean
    public CommonTools commonTools(McpToolFactory mcpToolFactory) {
        var deferMessage = "Use this tool only after trying local sources";
        var tools = new LinkedList<>();
        // Single MCP tool - gracefully handle missing
        var braveSearch = mcpToolFactory.toolByName("brave_web_search");
        if (braveSearch != null) {
            tools.add(braveSearch.withNote(deferMessage));
        }
        // UnfoldingTool grouping related Wikipedia MCP
        var wikipediaTool = mcpToolFactory.unfoldingByName("wikipedia", "Search and find content from Wikipedia:");
        "+deferMessage, Set.of("search_wikipedia", "get_article", "get_related_topics", "get_summary");
        if (!wikipediaTool.getInnerTools().isEmpty()) {
            tools.add(wikipediaTool);
        }
        return new CommonTools(tools);
    }
}
```

This pattern:

- **Gracefully degrades** when MCP tools aren’t available (e.g., in test environments)
- **Groups related tools** behind a descriptive facade using UnfoldingTool

- **Adds usage hints** with `withNote()` to guide the LLM on when to use external tools
- **Checks for empty results** before adding tools to avoid empty facades

McpToolFactory Method Summary

Method	Description
<code>toolByName(String)</code>	Get a single MCP tool by exact name. Returns null if not found.
<code>requiredToolByName(String)</code>	Get a single MCP tool by exact name. Throws <code>IllegalArgumentException</code> if not found, with a helpful error message listing available tools.
<code>unfoldingByName(name, description, toolNames, removeOnInvoke?)</code>	Create an <code>UnfoldingTool</code> containing tools with exact matching names.
<code>unfoldingMatching(name, description, patterns, removeOnInvoke?)</code>	Create an <code>UnfoldingTool</code> containing tools matching any of the regex patterns.
<code>unfolding(name, description, filter, removeOnInvoke?)</code>	Create an <code>UnfoldingTool</code> with a custom filter predicate.

3.10. Structured Prompt Elements

Embabel provides a number of ways to structure and manage prompt content.

Prompt contributors are a fundamental way to structure and inject content into LLM prompts. You don’t need to use them—you can simply build your prompts as strings—but they can be useful to achieve consistency and reuse across multiple actions or even across multiple agents using the same domain objects.

Prompt contributors implement the `PromptContributor` interface and provide text that gets included in the final prompt sent to the LLM. By default the text will be included in the system prompt message.

3.10.1. The PromptContributor Interface and LlmReference Subinterface

All prompt contributors implement the `PromptContributor` interface with a `contribution()` method that returns a string to be included in the prompt.

Add `PromptContributor` instances to a `PromptRunner` using the `withPromptContributor()` method.

A subinterface of `PromptContributor` is `LlmReference`.

An `LlmReference` is a prompt contributor that can also provide tools via annotated `@Tool` methods. So that tool naming can be disambiguated, an `LlmReference` must also include name and description metadata.

Add `LlmReference` instances to a `PromptRunner` using the `withReference()` method.

Use `LlmReference` if:

- You want to provide both prompt content and tools from the same object
- You want to provide specific instructions on how to use these tools, beyond the individual tool descriptions
- Your data may be best exposed as either prompt content or tools, depending on the context. For example, if you have a list of 10 items you might just put in the prompt and say "Here are all the items: ...". If you have a list of 10,000 objects, you would include advice to use the tools to query them.

An `LlmReference` is somewhat similar to a Claude Skill.

`LlmReference` instances can be created programmatically or defined in YML using `LlmReferenceProvider` implementations. For example, you could define a `references.yml` file in this format:

```
-fqn:com.embabel.coding.tools.git.GitHubRepositoryurl:https://github.com/embabel/embabel-agent-examples.gitdescription:Embabel
examples Repository-fqn:com.embabel.coding.tools.git.GitHubRepositoryurl:https://github.com/embabel/java-agent-
template.gitdescription:>Java agent template Repository: Simplest Java example with MavenCan be used as a GitHub template-
fqn:com.embabel.coding.tools.git.GitHubRepositoryurl:https://github.com/embabel/embabel-agent.gitdescription:>Embabel agent
framework implementation repo: Look to check code under embabel-agent-api-
fqn:com.embabel.coding.tools.api.ApiReferenceProvidername:embabel-agentdescription:Embabel Agent APIacceptedPackages:-
com.embabel.agent-com.embabel.common
```

The `fqn` property specifies the fully qualified class name of the `LlmReferenceProvider` implementation. This enables you to define your own `LlmReferenceProvider` implementations. Out of the box, Embabel provides:

- `com.embabel.agent.api.reference.LiteralText`: Text in notes
- `com.embabel.agent.api.reference.SpringResource`: Contents of the given Spring resource path
- `com.embabel.agent.api.reference.WebPage`: Content of the given web page, if it can be fetched
- `com.embabel.coding.tools.git.GitHubRepository`: GitHub repositories (embabel-agent-code module)
- `com.embabel.coding.tools.api.ApiReferenceProvider`: API from classpath (embabel-agent-code module)

You can parse your YML files into `List<LlmReference>` using the `LlmReferenceProviders.fromYml` method.

The resource argument is a Spring resource specification.

Thus `LlmReferenceProviders.fromYml("references.yml")` will load `references.yml` under `src/main/resources/`

3.10.2. Built-in Convenience Classes

Embabel provides several convenience classes that implement `PromptContributor` for common use cases. These are optional utilities - you can always implement the interface directly for custom needs.

Persona

The `Persona` class provides a structured way to define an AI agent’s personality and behavior:

Java
Kotlin

```
varpersona=Persona.create("Alex the Analyst","A detail-oriented data analyst with expertise in financial markets","Professional yet
approachable, uses clear explanations","Help users understand complex financial data through clear analysis");
```

This generates a prompt contribution like:

You are Alex the Analyst.

Your persona: A detail-oriented data analyst with expertise in financial markets.

Your objective is Help users understand complex financial data through clear analysis.

Your voice: Professional yet approachable, uses clear explanations.

RoleGoalBackstory

The `RoleGoalBackstory` class follows the Crew AI pattern and is included for users migrating from that framework:

Java
Kotlin

```
varagent=RoleGoalBackstory.withRole("Senior Software Engineer").andGoal("Write clean, maintainable code").andBackstory("10+
years experience in enterprise software development");
```

This generates:

Role: Senior Software Engineer

Goal: Write clean, maintainable code

Backstory: 10+ years experience in enterprise software development

3.10.3. Custom PromptContributor Implementations

You can create custom prompt contributors by implementing the interface directly. This gives you complete control over the prompt content:

JavaKotlin

```
public class CustomSystemPrompt implements PromptContributor {
    private final String systemName;
    public CustomSystemPrompt(String systemName) {
        this.systemName = systemName;
    }
    @Override
    public String contribution() {
        return "System: " + systemName + " - Current time: " + LocalDateTime.now();
    }
}

public class ConditionalPrompt implements PromptContributor {
    private final Supplier<Boolean> condition;
    private final String trueContent;
    private final String falseContent;
    public ConditionalPrompt(Supplier<Boolean> condition, String trueContent, String falseContent) {
        this.condition = condition;
        this.trueContent = trueContent;
        this.falseContent = falseContent;
    }
    @Override
    public String contribution() {
        return condition.get() ? trueContent : falseContent;
    }
}
```

3.10.4. Examples from embabel-agent-examples

The [embabel-agent-examples](#) repository demonstrates various agent development patterns and Spring Boot integration approaches for building AI agents with Embabel.

3.10.5. Best Practices

- Keep prompt contributors focused and single-purpose
- Use the convenience classes (Persona, RoleGoalBackstory) when they fit your needs
- Implement custom PromptContributor classes for domain-specific requirements
- Consider using dynamic contributors for context-dependent content
- Test your prompt contributions to ensure they produce the desired LLM behavior

3.11. Templates

Embabel supports Jinja templates for generating prompts. You do this via the PromptRunner.rendering(String) method. This method takes a Spring resource path to a Jinja template. The default location is under classpath:/prompts/ and the .jinja extension is added automatically.

You can also specify a full resource path with [Spring resource conventions](#).

Once you have specified the template, you can create objects using a model map.

An example:

JavaKotlin

```
DistinctFactualAssertions distinctFactualAssertions = context.ai().withLlm(properties.deduplicationLlm()) // Jinjava template from classpath at prompts/factchecker/consolidate_assertions.jinja.rendering("factchecker/consolidate_assertions").createObject(DistinctFactualAssertions.class, Map.of("assertions", allAssertions, "reasoningWordCount", properties.reasoningWordCount()));
```

3.11.1. Custom Template Renderer

By default, rendering() uses the platform’s TemplateRenderer (a Jinja-based renderer that loads templates from the classpath). You can override this on a per-rendering basis using withTemplateRenderer(), which lets you supply a custom TemplateRenderer implementation.

This is useful when you need to load templates from a different source—for example, pulling templates from a user-specific directory on the file system, or from a database, enabling per-tenant or per-user prompt customization.

JavaKotlin

```
TemplateRenderer perUserRenderer = createRendererForUser(userId);
String result = context.ai().rendering("user-greeting").withTemplateRenderer(perUserRenderer).generateText(Map.of("userName", userName));
```

Don’t rush to externalize prompts. In modern languages with multi-line strings, it’s often easier to keep prompts in the codebase. Externalizing them can sacrifice type safety and lead to complexity and maintenance challenges.

3.12. RAG (Retrieval-Augmented Generation)

Retrieval-Augmented Generation (RAG) is a technique that enhances LLM responses by retrieving relevant information from a knowledge base before generating answers. This grounds LLM outputs in specific, verifiable sources rather than relying solely on training data.

For more background on RAG concepts, see:

- [Wikipedia: Retrieval-Augmented Generation](#)
- [AWS: What is RAG?](#)

Embabel Agent provides RAG support through the LlmReference interface, which allows you to attach references (including RAG stores) to LLM calls. The key classes are ToolishRag for exposing search operations as LLM tools, and SearchOperations for the underlying search functionality.

3.12.1. Agentic RAG Architecture

Unlike traditional RAG implementations that perform a single retrieval step, Embabel Agent’s RAG is **entirely agentic and tool-based**. The LLM has full control over the retrieval process:

- **Autonomous Search:** The LLM decides when to search, what queries to use, and how many results to retrieve
- **Iterative Refinement:** The LLM can perform multiple searches with different queries until it finds relevant information
- **Cross-Reference Discovery:** The LLM can follow references, expand chunks to see surrounding context, and zoom out to parent sections
- **HyDE Support:** The LLM can generate hypothetical documents (HyDE queries) to improve semantic search results

This agentic approach produces better results than single-shot RAG because the LLM can:

1. Start with a broad search and narrow down
2. Try different phrasings if initial queries return poor results
3. Expand promising results to get more context
4. Combine information from multiple chunks

3.12.2. Facade Pattern for Safe Tool Exposure

Embabel Agent uses a facade pattern to expose RAG capabilities safely and consistently across different store implementations.

The ToolishRag class acts as a facade that:

1. **Inspects Store Capabilities:** Examines which SearchOperations subinterfaces the store implements
2. **Exposes Appropriate Tools:** Only creates tool wrappers for supported operations
3. **Provides Consistent Interface:** All tools use the same parameter patterns regardless of underlying store

JavaKotlin

```
@Override
public List<Tool> tools() {
    List<Object> toolObjects = new ArrayList<>();
    if (searchOperations instanceof VectorSearch) {
        toolObjects.add(new VectorSearchTools((VectorSearch) searchOperations));
    }
    if (searchOperations instanceof TextSearch) {
        toolObjects.add(new TextSearchTools((TextSearch) searchOperations));
    }
    if (searchOperations instanceof ResultExpander) {
        toolObjects.add(new ResultExpanderTools((ResultExpander) searchOperations));
    }
    if (searchOperations instanceof RegexSearchOperations) {
        toolObjects.add(new RegexSearchTools((RegexSearchOperations) searchOperations));
    }
    return toolObjects.stream().flatMap(obj -> Tool.fromInstance(obj).stream()).toList();
}
```

This means:

- A Lucene store exposes vector search, text search, regex search, AND result expansion tools
- A Spring AI VectorStore adapter exposes only vector search tools
- A basic text-only store exposes only text search tools
- A directory-based text search exposes text search and regex search

The LLM sees only the tools that actually work with the configured store, preventing runtime errors from unsupported operations.

3.12.3. Getting Started

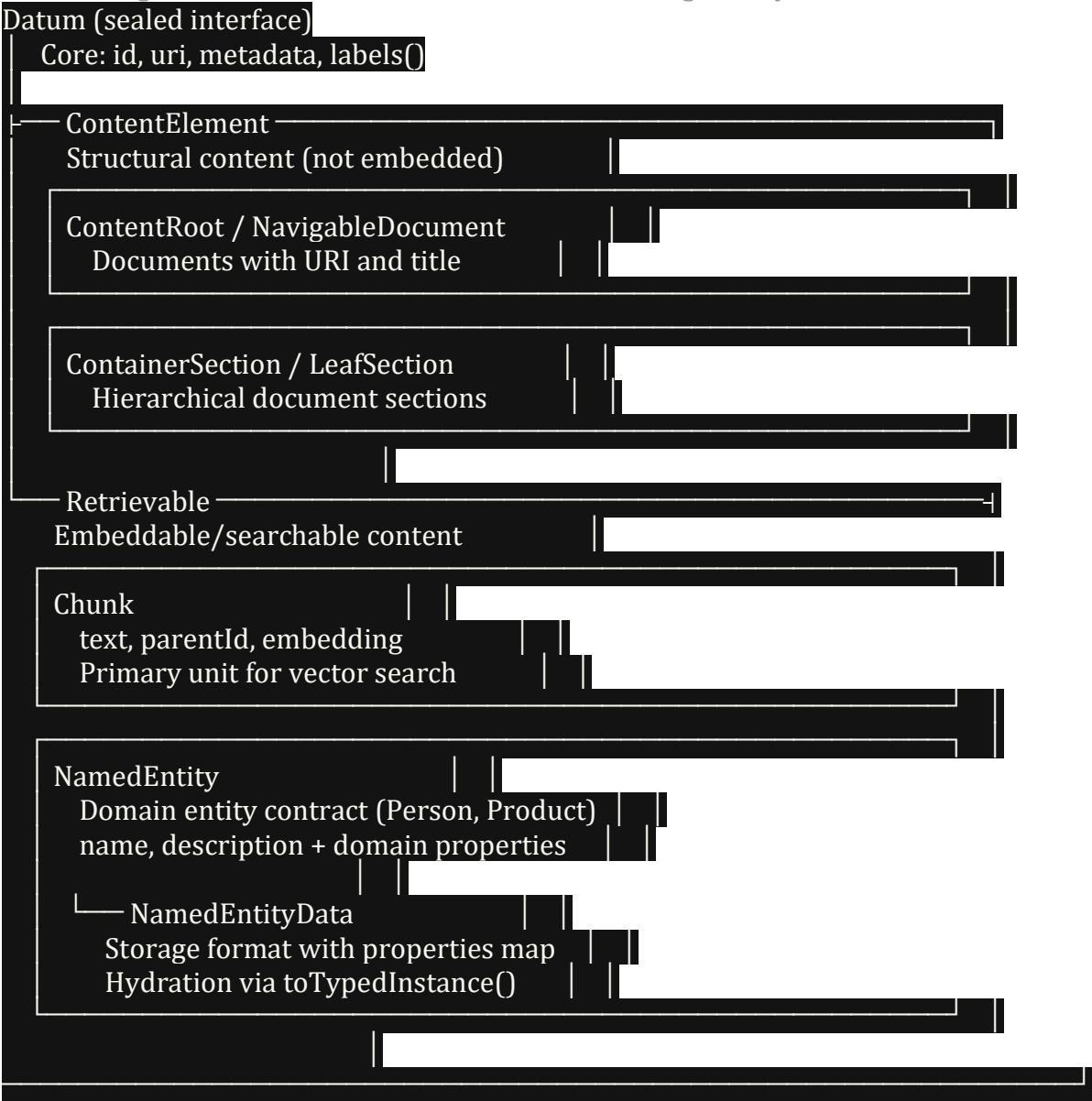
To use RAG in your Embabel Agent application, add the rag-core module and a store implementation to your pom.xml:

```
<dependency><groupId>com.embabel.agent</groupId><artifactId>embabel-agent-rag-lucene</artifactId><version>${embabel-agent.version}</version></dependency><dependency><groupId>com.embabel.agent</groupId><artifactId>embabel-agent-rag-tika</artifactId><version>${embabel-agent.version}</version></dependency>
```

The embabel-agent-rag-lucene module provides Lucene-based vector and text search. The embabel-agent-rag-tika module provides Apache Tika integration for parsing various document formats.

3.12.4. Our Model

Embabel Agent uses a hierarchical content model that goes beyond traditional flat chunk storage:



Key Design Points:

- Datum is the root sealed interface for all data objects
- ContentElement branch contains structural content (documents, sections) that is NOT embedded
- Retrievable branch contains searchable content with embeddings (chunks, entities)
- NamedEntity is the domain contract for typed entities
- NamedEntityData is the storage format with generic properties map and hydration support

Content Elements

The ContentElement interface is the supertype for all content in the RAG system. Key subtypes include:

- **ContentRoot / NavigableDocument:** The root of a document hierarchy, with a required URI and title
- **Section:** A hierarchical division of content with a title
- **ContainerSection:** A section containing other sections
- **LeafSection:** A section containing actual text content
- **Chunk:** Traditional RAG text chunks, created by splitting LeafSection content

Chunks

Chunk is the primary unit for vector search. Each chunk:

- Contains a text field with the content
- Has a parentId linking to its source section
- Includes metadata with information about its origin (root document, container section, leaf section)
- Can compute its pathFromRoot through the document hierarchy

This hierarchical model enables advanced RAG capabilities like "zoom out" to parent sections or expansion to adjacent chunks.

3.12.5. SearchOperations

SearchOperations is the tag interface for search functionality. Concrete implementations implement one or more subinterfaces based on their capabilities. This design allows stores to implement only what’s natural and efficient for them—a vector database need not pretend to support full-text search, and a text search engine need not fake vector similarity.

VectorSearch

Classic semantic vector search:

```
JavaKotlin
publicinterfaceVectorSearchextendsSearchOperations{<TextendsRetrievable>List<SimilarityResult<T>>vectorSearch(TextSimilaritySearchRequestrequest,Class<T>clazz);}
```

TextSearch

Full-text search using Lucene query syntax:

```
JavaKotlin
publicinterfaceTextSearchextendsSearchOperations{<TextendsRetrievable>List<SimilarityResult<T>>textSearch(TextSimilaritySearchRequestrequest,Class<T>clazz);}
```

Supported query syntax includes:

- +term - term must appear
- -term - term must not appear
- "phrase" - exact phrase match
- term* - prefix wildcard
- term~ - fuzzy match

ResultExpander

Expand search results to surrounding context:

JavaKotlin

```
public interface ResultExpander extends SearchOperations {
    List<ContentElement> expandResult(String id, Method method, int elementsToAdd);
}
```

Expansion methods:

- SEQUENCE - expand to previous and next chunks
- ZOOM_OUT - expand to enclosing section

RegexSearchOperations

Pattern-based search across content:

JavaKotlin

```
public interface RegexSearchOperation extends SearchOperations {
    <TextendsRetrievable> List<SimilarityResult<T>> regexSearch(
        Pattern regex, int topK, Class<T> clazz);
}
```

Useful for finding specific patterns like error codes, identifiers, or structured content that doesn't match well with semantic or keyword search.

CoreSearchOperations

A convenience interface combining the most common search capabilities:

JavaKotlin

```
public interface CoreSearchOperation extends VectorSearch, TextSearch {}
```

Stores that support both vector and text search can implement this single interface for convenience.

3.12.6. ToolishRag

ToolishRag is an LlmReference that exposes SearchOperations as LLM tools. This gives the LLM fine-grained control over RAG searches.

Configuration

Create a ToolishRag by wrapping your SearchOperations:

JavaKotlin

```
public ChatActions(SearchOperations searchOperations) {
    this.toolishRag = new ToolishRag("sources", "Sources for answering user questions", searchOperations);
}
```

Using with LLM Calls

Attach ToolishRag to an LLM call using .withReference():

JavaKotlin

```
@Action(canRerun = true, trigger = UserMessage.class) void respond(
    Conversation conversation, ActionContext context) {
    var assistantMessage = context.ai().withLlm(properties.chatLlm()).withReference(toolishRag).rendering("ragbot").respondWithSystemPrompt(
        conversation, Map.of("properties", properties));
    context.sendMessage(conversation.addMessage(assistantMessage));
}
```

Based on the capabilities of the underlying SearchOperations, ToolishRag exposes:

- **VectorSearchTools:** vectorSearch(query, topK, threshold) - semantic similarity search
- **TextSearchTools:** textSearch(query, topK, threshold) - BM25 full-text search with Lucene syntax
- **RegexSearchTools:** regexSearch(regex, topK) - pattern-based search using regular expressions
- **ResultExpanderTools:** broadenChunk(chunkId, chunksToAdd) - expand to adjacent chunks, zoomOut(id) - expand to parent section

The LLM autonomously decides when to use these tools based on user queries.

Eager Search

By default, ToolishRag is entirely agentic—the LLM decides when to search and what queries to use. However, when the topic of the conversation is already known, you can **preload** relevant results before the LLM starts, giving it a head start and reducing the number of tool calls needed.

ToolishRag implements the EagerSearch interface, which provides withEagerSearchAbout():

JavaKotlin

```
// Preload results about the user's topic before the LLM starts
ToolishRageagerRag = toolishRag.withEagerSearchAbout("Kotlin coroutines", 10);
context.ai().withReference(eagerRag).respondWithSystemPrompt(
    conversation, Map.of());
```

The preloaded results are included in the prompt as hints. The LLM still has access to all the usual search tools and can perform additional searches as needed.

For more control over the search parameters, pass a TextSimilaritySearchRequest directly:

JavaKotlin

```
var request = new TextSimilaritySearchRequest("Kotlin coroutines", 0.7, 10);
ToolishRageagerRag = toolishRag.withEagerSearchAbout(request);
```

Combining eager search with agentic tools is the sweet spot: preloaded results give the LLM an immediate head start (no round-trip needed), while the tools remain available for follow-up searches if the preloaded results aren't sufficient. You get the latency benefit of traditional RAG with the quality benefit of agentic RAG.

Eager search requires VectorSearch support in the underlying SearchOperations. If the store does not support vector search, withEagerSearchAbout() throws UnsupportedOperationException eagerly at configuration time.

The EagerSearch Interface

EagerSearch<T> is a general-purpose interface in the com.embabel.agent.api.reference package for any LlmReference that can preload context via similarity search. ToolishRag is one implementation, but other reference types can implement EagerSearch to provide the same consistent pattern for preloading relevant context before an LLM call.

ToolishRag lifecycle

It is safe to create a ToolishRag instance and reuse across many LLM calls. However, instances are not expensive to create, so you can create a new instance per LLM call. You might choose to do this if you provide a ResultListener that will collect queries and results for logging or analysis: for example, to track which queries were most useful for answering user questions and the complexity in terms of number of searches performed. This can be useful for implementing a learning feedback loop, for example to discern which queries performed badly, indicating that content such as documentation needs to be enhanced.

Result Filtering

In multi-tenant applications or scenarios where searches should be scoped to specific data subsets, ToolishRag supports **result filtering**. Filters are applied transparently to all searches—the LLM does not see or control them, ensuring security and data isolation.

Embabel Agent provides two types of filters:

- **Metadata Filters:** Filter on the metadata map of Datum objects (chunks, sections, etc.)
- **Property Filters:** Filter on object properties of typed entities (e.g., fields of NamedEntityData or custom entity classes)

Both use the same PropertyFilter type but are applied at different levels.

Motivation

Consider a document management system where:

- Each document belongs to an owner (user or organization)
- Some documents are shared reference data accessible to all users
- The LLM should only search documents the current user is authorized to access

Without filtering, you would need separate RAG stores per user or risk data leakage. With filtering, a single ToolishRag instance can be scoped per-request to the current user's data.

Filter API

Embabel Agent provides two filter interfaces for RAG searches:

- **PropertyFilter**: Filters on map-based properties (metadata, entity properties)
- **EntityFilter**: Extends PropertyFilter to add entity-specific filtering, particularly label-based filtering

PropertyFilter

The PropertyFilter sealed class hierarchy provides type-safe filter expressions for map-based properties:

Filter Type	Description	Example
Eq	Equals	PropertyFilter.eq("owner", "alice")
Ne	Not equals	PropertyFilter.ne("status", "deleted")
Gt, Gte	Greater than (or equal)	PropertyFilter.gte("score", 0.8)
Lt, Lte	Less than (or equal)	PropertyFilter.lt("priority", 5)
In	Value in list	PropertyFilter.in("category", "tech", "science")
Nin	Value not in list	PropertyFilter.nin("status", "deleted", "archived")
Contains	String contains substring	PropertyFilter.contains("tags", "important")
And	Logical AND	PropertyFilter.and(filter1, filter2)
Or	Logical OR	PropertyFilter.or(filter1, filter2)
Not	Logical NOT	PropertyFilter.not(filter)

EntityFilter

EntityFilter extends PropertyFilter to add entity-specific filtering. Currently, it adds label-based filtering via HasAnyLabel:

Filter Type	Description	Example
HasAnyLabel	Matches entities with any of the specified labels	EntityFilter.hasAnyLabel("Person", "Organization")

HasAnyLabel is particularly useful for:

- **Type-safe entity searches**: Filter results to only include specific entity types
- **Multi-type queries**: Search across multiple entity types in one query

JavaKotlin

```
import com.embabel.agent.rag.filter.EntityFilter; import com.embabel.agent.rag.filter.PropertyFilter; // Filter by single label
EntityFilter personFilter = EntityFilter.hasAnyLabel("Person"); // Filter by multiple labels (OR semantics - entity must have ANY of these labels)
EntityFilter entityFilter = EntityFilter.hasAnyLabel("Person", "Organization"); // Combine HasAnyLabel with property filters using fluent API
PropertyFilter simpleCombo = EntityFilter.hasAnyLabel("Person").and(PropertyFilter.eq("status", "active")); // Multiple conditions
PropertyFilter complexFilter = EntityFilter.hasAnyLabel("Person").and(PropertyFilter.eq("status", "active")).and(PropertyFilter.gte("score", 0.8)); // OR combinations
PropertyFilter orFilter = EntityFilter.hasAnyLabel("Person").or(PropertyFilter.eq("fallback", true)); // With negation
PropertyFilter notDeleted = EntityFilter.hasAnyLabel("Person").and(PropertyFilter.not(PropertyFilter.eq("status", "deleted"))); // Complex grouping
PropertyFilter accessFilter = PropertyFilter.or(PropertyFilter.and(EntityFilter.hasAnyLabel("Person", "Employee"), PropertyFilter.eq("active", true)), PropertyFilter.eq("role", "admin"));
```

Since EntityFilter extends PropertyFilter, all filter types share the same and, or, not operators and can be freely combined. EntityFilter.HasAnyLabel is typically handled via in-memory filtering as most vector stores don’t have native label support. When using Neo4j backends, labels can be translated to native Cypher label predicates for optimal performance.

Limitation: Nested Properties Not Supported

Filters currently operate on top-level properties only. Nested property paths like "address.city" or "metadata.source" are **not** supported. The filter key must match a direct key in the metadata map or a top-level property on the entity object.

For example:

- PropertyFilter.eq("owner", "alice") - **Supported**: filters on top-level owner property
- PropertyFilter.eq("address.city", "London") - **Not supported**: nested path will not match

Kotlin Operator Syntax

Kotlin users can use operator and infix functions for a more natural DSL syntax:

JavaKotlin

```
import com.embabel.agent.rag.filter.PropertyFilter; // Simple filter with not operator
PropertyFilter notDeleted = PropertyFilter.not(PropertyFilter.eq("status", "deleted")); // Combine with 'and' and 'or'
PropertyFilter userAccess = PropertyFilter.and(PropertyFilter.eq("owner", userId), PropertyFilter.gte("confidenceScore", 0.7)); // Complex expressions with grouping
PropertyFilter accessFilter = PropertyFilter.or(PropertyFilter.and(PropertyFilter.eq("owner", userId), PropertyFilter.ne("status", "deleted")), PropertyFilter.eq("role", "admin"));
```

Metadata vs Entity Filters

ToolishRag accepts two separate filter parameters:

- **metadataFilter**: A PropertyFilter that filters on the metadata map of Datum objects. Metadata is typically ingestion-time information like source URI, ingestion date, owner ID, etc.
- **entityFilter**: An EntityFilter that filters on entity properties and labels. For NamedEntityData, this filters on the properties map and labels(). For typed entities, reflection is used to access top-level fields.

JavaKotlin

```
// Filter on metadata (e.g., which user owns the document)
PropertyFilter metadataFilter = PropertyFilter.eq("ownerId", currentUserId); // Filter on entity labels and properties
EntityFilter entityFilter = EntityFilter.hasAnyLabel("Person"); // Apply both filters
ToolishRags scopedRag = toolishRag.withMetadataFilter(metadataFilter).withEntityFilter(entityFilter);
```

In most cases, you’ll use metadata filters for access control and entity filters for type-based and business logic filtering.

Neo4j Cypher Filtering

When using Neo4j via the Drivine module, metadata filters are automatically converted to Cypher WHERE clauses using CypherFilterConverter:

JavaKotlin

```
// The filter is converted to Cypher WHERE clause automatically
PropertyFilter filter = PropertyFilter.and(PropertyFilter.eq("owner", "alice"), PropertyFilter.gte("confidenceScore", 0.7)); // In DrivineNamedEntityDataRepository:
List<SimilarityResult<T>> results = repository.vectorSearch(request, filter); // Generates: WHERE (e.owner = $_filter_0) AND (e.confidenceScore >= $_filter_1) AND ...
```

The converter produces parameterized queries for safety and handles all filter types including nested logical expressions.

For both DrivineStore (chunks) and DrivineNamedEntityDataRepository (named entities), **both** metadata and property filters are translated to native Cypher WHERE clauses. This is because Neo4j stores all data as node properties - metadata is simply the set of properties that aren't core fields like id, text, parentId, etc. This provides optimal performance by filtering at the database level rather than in-memory.

Basic Usage

Apply a metadata filter to scope all searches to a specific owner:

JavaKotlin

```
// Create a filter for the current userPropertyFilterownerFilter=PropertyFilter.eq("ownerId",currentUserId);// Apply to ToolishRag - all searches will be filteredToolishRagscopedRag=toolishRag.withMetadataFilter(ownerFilter);// Use in LLM call - LLM cannot see or bypass the filtercontext.ai().withReference(scopedRag).respondWithSystemPrompt(conversation,Map.of());
```

Complex Filters

Combine filters for more sophisticated access control:

JavaKotlin

```
// User can access their own documents OR documents in their departmentsPropertyFilteraccessFilter=PropertyFilter.or(PropertyFilter.eq("ownerId",currentUserId),PropertyFilter.in("departmentId",userDepartmentIds));ToolishRagscopedRag=toolishRag.withMetadataFilter(accessFilter);// Organization-scoped with status restrictionPropertyFilterorgFilter=PropertyFilter.and(PropertyFilter.eq("orgId",currentOrgId),PropertyFilter.ne("status","deleted"),PropertyFilter.gte("confidenceScore",0.7));ToolishRagscopedRag2=toolishRag.withMetadataFilter(orgFilter);
```

Per-Request Scoping Pattern

A common pattern is to create a scoped ToolishRag per request in a web application:

JavaKotlin

```
@Action(trigger=UserMessage.class)voidrespond(Conversationconversation,ActionContextcontext){// Get current user from security contextStringuserId=SecurityContextHolder.getContext().getAuthentication().getName();// Create user-scoped RAG for this requestToolishRaguserScopedRag=toolishRag.withMetadataFilter(PropertyFilter.eq("ownerId",userId));context.ai().withReference(userScopedRag).rendering("assistant").respondWithSystemPrompt(conversation,Map.of());}
```

Backend Implementation

Filters are applied at different levels depending on the backend:

- Spring AI VectorStore:** Metadata filters are translated to Filter.Expression for native filtering; entity filters (including HasAnyLabel) are applied in-memory
- Neo4j (Drivine):** Both metadata and entity filters (including HasAnyLabel) are translated to native Cypher WHERE clauses and label predicates (optimal performance)
- Lucene:** Both filter types are applied as post-filters with inflated topK to compensate for filtered-out results
- Custom stores:** Can implement FilteringVectorSearch / FilteringTextSearch for native translation, or fall back to in-memory filtering

The InMemoryPropertyFilter utility class provides fallback filtering for any store implementation:

JavaKotlin

```
// In your SearchOperations implementationList<SimilarityResult<T>>results=performSearch(request);returnInMemoryPropertyFilter.filterResults(results,metadataFilter,entityFilter);
```

For EntityFilter.HasAnyLabel, the in-memory filter checks if the entity has any of the specified labels via NamedEntityData.labels().

This ensures filtering works across all backends, with native optimization for metadata filters where available.

3.12.7. Ingestion

Document Parsing with Tika

Embabel Agent uses Apache Tika for document parsing. TikaHierarchicalContentReader reads various formats (Markdown, HTML, PDF, Word, etc.) and extracts a hierarchical structure:

JavaKotlin

```
@ShellMethod("Ingest URL or file path")Stringingest(@ShellOption(defaultValue="/data/document.md")Stringlocation){varuri=location.startsWith("http://")||location.startsWith("https://")?location.Path.of(location).toAbsolutePath().toUri().toString():varingested=NeverRefreshExistingDocumentContentPolicy.INSTANCE.ingestUriIfNeeded(luceneSearchOperations,newTikaHierarchicalContentReader(),uri);returningested!=null?"Ingested document with ID: "+ingested:"Document already exists, no ingestion performed.";}
```

Chunking Configuration

Content is split into chunks with configurable parameters:

```
ragbot:chunker-config:max-chunk-size:800overlap-size:100
```

Configuration options:

- maxChunkSize - Maximum characters per chunk (default: 1500)
- overlapSize - Character overlap between consecutive chunks (default: 200)
- includeSectionTitleInChunk - Include section title in chunk text (default: true)

Chunk Transformation

When chunks are created from documents, they often lack the context needed for effective retrieval. A chunk containing "This approach improves performance by 40%" is not useful unless the reader knows what "this approach" refers to. The ChunkTransformer interface allows you to enrich chunks with additional context before they are indexed.

The urtext Field

Every Chunk has two text fields:

- text - The indexed content, which may be transformed with additional context
- urtext - The original, unmodified chunk text

The urtext field preserves the original content for accurate citations. When displaying search results to users, use urtext to show exactly what appeared in the source document, while using the enriched text for vector embeddings and search.

AddTitlesChunkTransformer

The recommended default transformer is AddTitlesChunkTransformer, which prepends document and section titles to each chunk:

JavaKotlin

```
@BeanChunkTransformerchunkTransformer(){returnAddTitlesChunkTransformer.INSTANCE;}
```

This transforms a chunk like:

This approach improves performance by 40% compared to the baseline.

Into:

Title: Performance Optimization Guide

URI: <https://docs.example.com/performance>

Section: Caching Strategies

This approach improves performance by 40% compared to the baseline.

Now the chunk carries its context, improving both retrieval accuracy and LLM understanding.

Custom Transformers

You can create custom transformers by implementing ChunkTransformer or extending AbstractChunkTransformer:

JavaKotlin

```
publicclassMetadataEnrichingTransformerextendsAbstractChunkTransformer{@OverridepublicMap<String,Object>additionalMetadata(Chunkchunk,ChunkTransformationContextcontext){returnMap.of("documentType",context.getDocument().getMetad
```

```
ata().get("type"),"lastModified",Instant.now().toString());}@OverridepublicStringnewText(Chunkchunk,ChunkTransformationContextcontext){// Optionally modify the textreturnchunk.getText();}}
```

The ChunkTransformationContext provides access to:

- section - The Section containing this chunk
- document - The ContentRoot (may be null for orphan sections)

Chaining Transformers

Use ChainedChunkTransformer to apply multiple transformations in sequence:

JavaKotlin

```
@BeanChunkTransformerchunkTransformer(){returnnewChainedChunkTransformer(List.of(AddTitlesChunkTransformer.INSTANCE,newMetadataEnrichingTransformer(),newCustomCleanupTransformer()));}
```

Transformers are applied in order, with each receiving the output of the previous transformer.

Configuring the Store

Pass your ChunkTransformer to the store implementation:

JavaKotlin

```
@BeanDrivineStoredrivineStore(PersistenceManagerpersistenceManager,EmbeddingServiceembeddingService,ChunkTransformerchunkTransformer,MyPropertiesproperties){returnnewDrivineStore(persistenceManager,properties.neoRag(),properties.chunkerConfig(),chunkTransformer,embeddingService,platformTransactionManager,newDrivineCypherSearch(persistenceManager));}
```

Inject the ChunkTransformer bean

Pass it to the store constructor

For most use cases, AddTitlesChunkTransformer is all you need. It adds essential context that significantly improves retrieval quality without adding complexity.

Using Docling for Markdown Conversion

While we believe that you should write your Gen AI **applications** in Java or Kotlin, ingestion is more in the realm of data science, and Python is indisputably strong in this area.

For complex documents like PDFs, consider using [Docling](#) to convert to Markdown first:

```
docling https://example.com/document.pdf --frompdf --tomd --output./data
```

Markdown is easier to parse hierarchically and produces better chunks than raw PDF extraction.

3.12.8. Supported Stores

Embabel Agent provides several RAG store implementations:

Lucene (embabel-agent-rag-lucene)

Full-featured store with vector search, text search, and result expansion. Supports both in-memory and file-based persistence:

JavaKotlin

```
@BeanLuceneSearchOperationsluceneSearchOperations(ModelProvidermodelProvider,RagbotPropertiesproperties){varembelddingService=modelProvider.getEmbeddingService(DefaultModelSelectionCriteria.INSTANCE);returnLuceneSearchOperations.withName("docs").withEmbeddingService(embeddingService).withChunkerConfig(properties.chunkerConfig()).withIndexPath(Paths.get("./.lucene-index"))// File persistence.buildAndLoadChunks();}
```

Omit .withIndexPath() for in-memory only storage.

Neo4j

Graph database store for RAG (available in separate modules embabel-agent-rag-neo-drivine and embabel-agent-rag-neo-ogm). Ideal when you need graph relationships between content elements.

PostgreSQL pgvector (embabel-rag-pgvector)

PostgreSQL-based RAG store using the pgvector extension (available in the separate embabel/embabel-rag-pgvector repository). Supports hybrid search combining vector similarity, full-text search via tsvector/tsquery, and fuzzy matching via pg_trgm. Ideal when you already use PostgreSQL and want a familiar, battle-tested database for RAG.

Spring AI VectorStore (SpringVectorStoreVectorSearch)

Adapter that wraps any Spring AI VectorStore, enabling use of any vector database Spring AI supports:

JavaKotlin

```
publicclassSpringVectorStoreVectorSearchimplementsVectorSearch{privatefinalVectorStorevectorStore;publicSpringVectorStoreVectorSearch(VectorStorevectorStore){this.vectorStore=vectorStore;}@Overridepublic<TextendsRetrievable>List<SimilarityResult<T>>vectorSearch(TextSimilaritySearchRequestrequest,Class<T>clazz){SearchRequestsearchRequest=SearchRequest.builder().query(request.getQuery()).similarityThreshold(request.getSimilarityThreshold()).topK(request.getTopK()).build();List<Document>results=vectorStore.similaritySearch(searchRequest);// ... convert results}}
```

This allows integration with Pinecone, Weaviate, Milvus, Chroma, and other stores via Spring AI.

3.12.9. Implementing Your Own RAG Store

To implement a custom RAG store, implement only the SearchOperations subinterfaces that are natural and efficient for your store.

This is a key design principle: **stores should only implement what they can do well**.

For example:

- A **vector database** like Pinecone might implement only VectorSearch since that's its strength
- A **full-text search engine** might implement TextSearch and RegexSearchOperations
- A **hierarchical document store** might add ResultExpander for context expansion
- A **full-featured store** like Lucene can implement all interfaces

The ToolishRag facade automatically exposes only the tools that your store supports. This means you don't need to provide stub implementations or throw "not supported" exceptions—simply don't implement interfaces that don't fit your store's capabilities.

JavaKotlin

```
// A store that only supports vectorsearchpublicclassMyVectorOnlyStoreimplementsVectorSearch{@Overridepublic<TextendsRetrievable>List<SimilarityResult<T>>vectorSearch(TextSimilaritySearchRequestrequest,Class<T>clazz){// Implement vector similarity search}}// A store that supports both vector and textsearchpublicclassMyFullTextStoreimplementsVectorSearch,TextSearch{@Overridepublic<TextendsRetrievable>List<SimilarityResult<T>>vectorSearch(TextSimilaritySearchRequestrequest,Class<T>clazz){// Implement vector similarity search}@Overridepublic<TextendsRetrievable>List<SimilarityResult<T>>textSearch(TextSimilaritySearchRequestrequest,Class<T>clazz){// Implement full-text search}@OverridepublicStringgetLuceneSyntaxNotes(){return"Full Lucene syntax supported";}}
```

For ingestion support, extend ChunkingContentElementRepository to handle document storage and chunking.

3.12.10. Complete Example

See the [rag-demo](#) project for a complete working example including:

- Lucene-based RAG store configuration
- Document ingestion via Tika
- Chatbot with RAG-powered responses
- Jinja prompt templates for system prompts
- Spring Shell commands for interactive testing

3.13. Building Chatbots

Chatbots are an important application of Gen AI, although far from the only use, especially in enterprise. Unlike many other frameworks, Embabel does not maintain a conversation thread to do its core work. This is a good thing as it means that context compression is not required for most tasks. If you want to build a chatbot you should use the Conversation interface explicitly, and expose a Chatbot bean, typically backed by action methods that handle UserMessage events.

3.13.1. Core Concepts

Long-Lived AgentProcess

An Embabel chatbot is backed by a **long-lived AgentProcess** that pauses between user messages. This design has important implications:

- The same AgentProcess can respond to events besides user input
- The blackboard maintains state across the entire session
- Actions can be triggered by user messages, system events, or other objects added to the blackboard
- It’s a **working context** rather than just a chat session

When a user sends a message, it’s added to the blackboard as a UserMessage. The AgentProcess then runs, selects an appropriate action to handle it, and pauses again waiting for the next event.

Utility AI for Chatbots

Utility AI is often the best approach for chatbots. Instead of defining a fixed flow, you define multiple actions with costs, and the planner selects the highest-value action to respond to each message.

This allows:

- Multiple response strategies (e.g., RAG search, direct answer, clarification request)
- Dynamic behavior based on context
- Easy extensibility by adding new action methods

Goals in Chatbots

Typically, chatbot agents **do not need a goal**. The agent process simply waits for user messages and responds to them indefinitely.

However, you can define a goal if you want to ensure the conversation terminates and the AgentProcess completes rather than waiting forever. This is useful for:

- Transactional conversations (e.g., completing a booking)
- Wizard-style flows with a defined endpoint
- Conversations that should end after collecting specific information

3.13.2. Key Interfaces

Chatbot

The Chatbot interface manages multiple chat sessions:

JavaKotlin

```
publicinterface Chatbot {
    ChatSession createSession(
        User user,
        OutputChannel outputChannel,
        String contextId,
        String conversationId
    );
    ChatSession findSession(
        String conversationId
    );
}
```

Context IDs and Session State

The contextId parameter allows you to **pre-populate the session’s blackboard** with objects from a named context. This is useful when:

- **Users have multiple contexts** - A user might have different projects, accounts, or workspaces. Each context can maintain its own state that persists across sessions.
- **Resuming prior state** - When a user returns, you can restore their previous session state (e.g., user preferences, in-progress work, conversation history from a previous session).
- **Pre-loading domain objects** - You can populate the blackboard with objects that should always be present, such as the current user’s profile, active subscription, or relevant configuration.

JavaKotlin

```
// Create a session with a specific context
ChatSession session = chatbot.createSession(
    user,
    outputChannel,
    "project-alpha",
    // Context ID - loads saved state for this project
    null
);
// Or create an anonymous session without context
ChatSession anonymousSession = chatbot.createSession(
    null,
    outputChannel,
    null,
    null
);
```

The context mechanism works with `AgentPlatform’s context storage:

1. When createSession is called with a contextId, the platform looks up any saved objects for that context
2. Those objects are added to the new session’s blackboard
3. As the session runs, changes to the blackboard can be persisted back to the context
4. The next time a session is created with that contextId, the updated state is restored

This enables **stateful conversations across sessions** without requiring the chatbot to manually track and restore state.

ChatSession

Each session represents an ongoing conversation:

JavaKotlin

```
publicinterface ChatSession {
    OutputChannel getOutputChannel();
    User getUser();
    Conversation getConversation();
    String getProcessId();
    void onUserMessage(
        UserMessage userMessage
    );
    boolean isFinished();
}
```

Conversation

The Conversation interface holds the message history and tracks assets:

JavaKotlin

```
publicinterface Conversation extends StableIdentified, AssetView {
    List<Message> getMessages();
    AssetTracker getAssetTracker();
    List<Asset> getAssets();
    // Combined view of all assets
    Message addMessage(
        Message message
    );
    UserMessage lastMessageIfBeFromUser();
}
```

Message types include:

- UserMessage - messages from the user (supports multimodal content)
- AssistantMessage - responses from the chatbot (can include assets)
- SystemMessage - system-level instructions

3.13.3. Asset Tracking

Chatbots can track **assets**—structured outputs like generated documents, search results, or user-created content—at two levels:

Conversation-Level Assets

The Conversation has an AssetTracker for explicitly tracking assets throughout the session:

JavaKotlin

```
// Add an asset to the conversation tracker
conversation.getAssetTracker().addAsset(myAsset);
// Get all tracked assets
List<Asset> trackedAssets = conversation.getAssetTracker().getAssets();
```

Use conversation-level tracking when:

- Assets are created by tools or external processes
- Assets should persist across multiple messages
- You want explicit control over what’s tracked

Message-Level Assets

AssistantMessage implements AssetView and can include assets directly:

JavaKotlin

```
AssistantMessagemessage=newAssistantMessage("Here's the report you requested",null,// namenull,//
awaitableList.of(reportAsset,summaryAsset)// assets);conversation.addMessage(message);
```

Use message-level assets when:

- Assets are directly tied to a specific response
- You want assets to appear alongside the message in the UI
- The asset represents output from that specific interaction

Combined Asset View

The Conversation.assets property provides a **merged view** of all assets:

```
JavaKotlin
// Gets assets from BOTH the tracker AND all messagesList<Asset>allAssets=conversation.getAssets();
```

The merge follows these rules:

1. **Tracker assets appear first** (explicit tracking takes priority)
2. **Message assets follow** in chronological order
3. **Duplicates are removed** by ID (tracker version wins)

This allows flexible asset management:

```
JavaKotlin
@Action(canRerun=true,trigger=UserMessage.class)voidrespond(Conversationconversation,ActionContextcontext){// Create an
asset from the responseAssetresultAsset=createResultAsset(result);// Option 1: Add to message (appears with this
response)varmessage=newAssistantMessage("Here's your
analysis",null,null,List.of(resultAsset));conversation.addMessage(message);// Option 2: Add to tracker (explicitly
tracked)conversation.getAssetTracker().addAsset(resultAsset);// Either way, it's visible via conversation.getAssets()}
```

Using Assets as Tools

Assets can be exposed to the LLM as tools via their LlmReference:

```
JavaKotlin
// Get references from recent assetsList<LlmReference>refs=conversation.mostRecent(5).references();// Use in a
promptvarresponse=context.ai().withReferences(refs)// Assets become available as tools.respond(conversation.getMessages());
```

This enables scenarios like:

- Editing previously generated content
- Combining multiple assets
- Querying structured data from earlier in the conversation

3.13.4. Building a Chatbot

Step 1: Create Action Methods

Define action methods in an @EmbabelComponent that respond to user messages using the trigger parameter:

```
JavaKotlin
@EmbabelComponentpublicclassChatActions{privatefinalToolishRagtoolishRag;privatefinalRagbotPropertiesproperties;public
cChatActions(SearchOperationssearchOperations,RagbotPropertiesproperties){this.toolishRag=newToolishRag("sources","Sourc
es for answering user
questions",searchOperations);this.properties=properties;}@Action(canRerun=true,trigger=UserMessage.class)voidrespond(Conve
rsationconversation,ActionContextcontext){varassistantMessage=context.ai().withLlm(properties.chatLlm()).withReference(toolish
Rag).rendering("ragbot").respondWithSystemPrompt(conversation,Map.of("properties",properties));context.sendMessage(conversat
ion.addMessage(assistantMessage));}}
```

trigger = UserMessage.class - action is invoked when a UserMessage is the last object added to the blackboard

canRerun = true - action can be executed multiple times (for each user message)

Conversation parameter is automatically injected from the blackboard

context.sendMessage() sends the response to the output channel

Step 2: Configure the Chatbot Bean

Use AgentProcessChatbot.utilityFromPlatform() to create a utility-based chatbot that discovers all available actions:

```
JavaKotlin
@ConfigurationclassChatConfiguration{@BeanChatbotchatbot(AgentPlatformagentPlatform){returnAgentProcessChatbot.utili
tyFromPlatform(agentPlatform);}}
```

Creates a chatbot using Utility AI planning to select the best action

Discovers all @Action methods from @EmbabelComponent classes on the platform

For debugging, you can pass a custom Verbosity configuration:

```
JavaKotlin
@BeanChatbotchatbot(AgentPlatformagentPlatform){returnAgentProcessChatbot.utilityFromPlatform(agentPlatform,newInMe
moryConversationFactory(),newVerbosity().showPrompts());}
```

Conversation factory (required when specifying verbosity)

Verbosity configuration for debugging prompts

Be sure that the AgentPlatform has loaded all its actions before creating a new session on your AgentProcessChatbot. Otherwise the actions needed to respond to chat may not be available in the session.

3.13.5. Conversation Storage

By default, chatbots use **in-memory conversations** that are lost when the session ends. For production applications, you typically want to **persist conversations** to a backing store.

Storage Types

Embabel supports two conversation storage types via ConversationStoreType:

Type	Description
IN_MEMORY	Conversations stored in memory only. Fast and simple, suitable for testing and ephemeral sessions.
STORED	Conversations persisted to a backing store (e.g., Neo4j). Requires embabel-chat-store dependency.

Configuring Persistent Storage

To use persistent conversations, inject ConversationFactoryProvider and pass the appropriate factory when creating the chatbot:

```
JavaKotlin
@ConfigurationclassChatConfiguration{@BeanChatbotchatbot(AgentPlatformagentPlatform,ConversationFactoryProvidercon
versationFactoryProvider){ConversationFactoryfactory=conversationFactoryProvider.getFactory(ConversationStoreType.STORED
);returnnewAgentProcessChatbot(agentPlatform,user->createAgent(agentPlatform),factory,// ... other configuration);}}
```

Inject the ConversationFactoryProvider via Spring DI

Get the factory for the desired storage type

Pass the factory to the chatbot - storage is configured once at creation time

Storage type is configured once when creating the chatbot, not per-call. This ensures consistent behavior across all sessions.

Adding embabel-chat-store

To enable persistent storage, add the embabel-chat-store dependency:

```
<dependency><groupId>com.embabel.chat</groupId><artifactId>embabel-chat-store</artifactId></dependency>
```

This provides:

- StoredConversationFactory - creates conversations that persist to Neo4j
- StoredConversation - conversation implementation with async persistence
- Message lifecycle events (MessageEvent) for UI updates
- Title generation for conversation sessions

Restoring Conversations

To restore a conversation, pass the conversationId when creating a session:

```
JavaKotlin
// Restore existing conversation or create new oneChatSessionsession=chatbot.createSession(user,outputChannel,null,contextIdconversationId);// Messages are already loaded if conversation existedList<Message>history=session.getConversation().getMessages();
```

If the conversation exists in storage, it will be loaded automatically. If not, a new conversation is created with this ID.

This allows applications to:

- Resume conversations across server restarts
- Display conversation history to returning users
- Continue multi-turn interactions from where they left off

For lower-level access, you can also use ConversationFactory.load(conversationId) directly to check if a conversation exists before creating a session.

Step 3: Use the Chatbot

Interact with the chatbot through its session interface:

```
JavaKotlin
// New session (fresh state, generated conversation ID)ChatSessionsession=chatbot.createSession(user,outputChannel,null,null);// Session with context (restores blackboard state)ChatSessionwithContext=chatbot.createSession(user,outputChannel,"user-workspace-123",null);// Restore existing conversation by IDChatSessionrestored=chatbot.createSession(user,outputChannel,null,savedConversationId);// Both context and conversation restorationChatSessionfull=chatbot.createSession(user,outputChannel,"user-workspace-123",savedConversationId);session.onUserMessage(newUserMessage("What does this document say about taxes?"));// Response is automatically sent to the outputChannel
```

Create a new session with fresh blackboard and auto-generated conversation ID

Load prior blackboard state from the "user-workspace-123" context

Restore an existing conversation with its message history

Both: load context state AND restore conversation history

Send a user message - triggers the agent to select and run an action

3.13.6. How Message Triggering Works

When you specify trigger = UserMessage.class on an action:

1. The chatbot adds the UserMessage to both the Conversation and the AgentProcess blackboard
2. The planner evaluates all actions whose trigger conditions are satisfied
3. For utility planning, the action with the highest value (lowest cost) is selected
4. The action method receives the Conversation (with the new message) via parameter injection

This trigger-based approach means:

- You can have multiple actions that respond to user messages with different costs
- The planner picks the most appropriate response strategy
- Actions can also be triggered by other event types (not just UserMessage)

3.13.7. Dynamic Cost Methods

For more sophisticated action selection, use @Cost methods:

```
JavaKotlin
@Costdoubledynamic(Blackboardbb){returnbb.getObjects().size()>5?100:10;}@Action(canRerun=true,trigger=UserMessage.class, costMethod="dynamic")voidrespond(Conversationconversation,ActionContextcontext){// ...}
```

@Cost marks this as a cost calculation method

Receives the Blackboard to inspect current state

Returns cost value - lower costs mean higher priority

costMethod links the action to the cost calculation method

3.13.8. Prompt Templates

Chatbots typically use **Jinja prompt templates** rather than inline string prompts. This isn't strictly necessary—simple chatbots can use regular string prompts built in code:

```
JavaKotlin
varassistantMessage=context.ai().withLlm(properties.chatLlm()).withSystemPrompt("You are a helpful assistant. Answer questions concisely.").respond(conversation.getMessages());
```

Simple inline prompt - fine for basic chatbots

However, production chatbots often need **longer, more complex prompts** for:

- Personality and tone (personas)
- Guardrails and safety instructions
- Domain-specific objectives
- Dynamic behavior based on configuration

For these cases, Jinja templates are the better choice:

```
JavaKotlin
varassistantMessage=context.ai().withLlm(properties.chatLlm()).withReference(toolishRag).rendering("ragbot").respondWithSystemPrompt(conversation,Map.of("properties",properties,"persona",properties.persona(),"objective",properties.objective()));
```

Loads prompts/ragbot.jinja from resources

Template bindings - accessible in Jinja as properties.persona() etc.

Templates allow:

- Separation of prompt engineering from code

- Dynamic persona and objective selection via configuration
- Reusable prompt elements (guardrails, personalization)
- Prompt iteration without code changes

Resilient Responses with respond

In a chatbot, it’s critical never to leave the user without a reply. The respond method on Rendering wraps respondWithSystemPrompt with error handling, so that an LLM or infrastructure failure still returns an AssistantMessage to the user rather than propagating an exception:

```
JavaKotlin
varassistantMessage=context.ai().rendering("ragbot").respond(conversation,model,error->{logger.error("Failed to generate
response",error);returnnewAssistantMessage("Sorry, something went wrong. Please try again.");});
```

Template Structure Example

A typical chatbot template structure from the rag-demo project:

```
prompts/
├── ragbot.jinja          # Main entry point
├── elements/
│   ├── guardrails.jinja    # Safety restrictions
│   └── personalization.jinja  # Dynamic persona/objective loader
├── personas/
│   ├── clause.jinja        # Legal expert persona
│   └── ...
└── objectives/
    └── legal.jinja         # Legal document analysis objective
```

The main template (ragbot.jinja) composes from reusable elements:

```
{%include"elements/guardrails.jinja"%}{%include"elements/personalization.jinja"%}
```

Include safety guardrails first

Then include persona and objective (which are dynamically selected)

Guardrails define safety boundaries (elements/guardrails.jinja):

```
{# Safety and content guardrails for the ragbot. #}DO NOT DISCUSS POLITICS OR CONTROVERSIAL TOPICS.
```

Personalization dynamically loads persona and objective (elements/personalization.jinja):

```
{%setpersona_template="personas/"~properties.persona()~".jinja"%}{%includepersona_template%}{%setobjective_template="obj
ectives/"~properties.objective()~".jinja"%}{%includeobjective_template%}
```

Build template path from properties.persona() (e.g., "clause" → "personas/clause.jinja")

Build template path from properties.objective() (e.g., "legal" → "objectives/legal.jinja")

A persona template (personas/clause.jinja):

```
Your name is Clause.
You are a brilliant legal chatbot who excels at interpreting
legislation and legal documents.
```

An objective template (objectives/legal.jinja):

```
You are an authoritative interpreter of legislation and legal documents.
You are renowned for thoroughness and for never missing anything.
You answer questions definitively, in a clear and concise manner.
You cite relevant sections to back up your answers.
If you don't know, say you don't know.
NEVER FABRICATE ANSWERS.
You ground your answers in literal citations from the provided sources.
Always use the available tools.
```

Instructs the LLM to use RAG tools provided via withReference()

This modular approach lets you:

- Switch personas via application.yml without code changes
- Share guardrails across multiple chatbot configurations
- Test different objectives independently

3.13.9. Advanced: State Management with @State

For complex chatbots that need to track state across messages, use @State classes. State classes are automatically managed by the agent framework:

- State objects are persisted in the blackboard
- Actions can depend on specific state being present
- State transitions drive the conversation flow

Cross-reference the @State annotation documentation for details on:

- Defining state classes
- State-dependent actions
- Nested state machines

3.13.10. Complete Example

See the [rag-demo](#) project for a complete chatbot implementation including:

- ChatActions.java - Action methods responding to user messages
- ChatConfiguration.java - Chatbot bean configuration
- RagbotShell.java - Spring Shell integration for interactive testing
- Jinja templates for persona-driven prompts
- RAG integration for document-grounded responses

To run the example:

```
./scripts/shell.sh
# In the shell:ingest ./data/document.md
chat
>What does the document say about...
```

3.14. The AgentProcess

An AgentProcess is created every time an agent is run. It has a unique id.

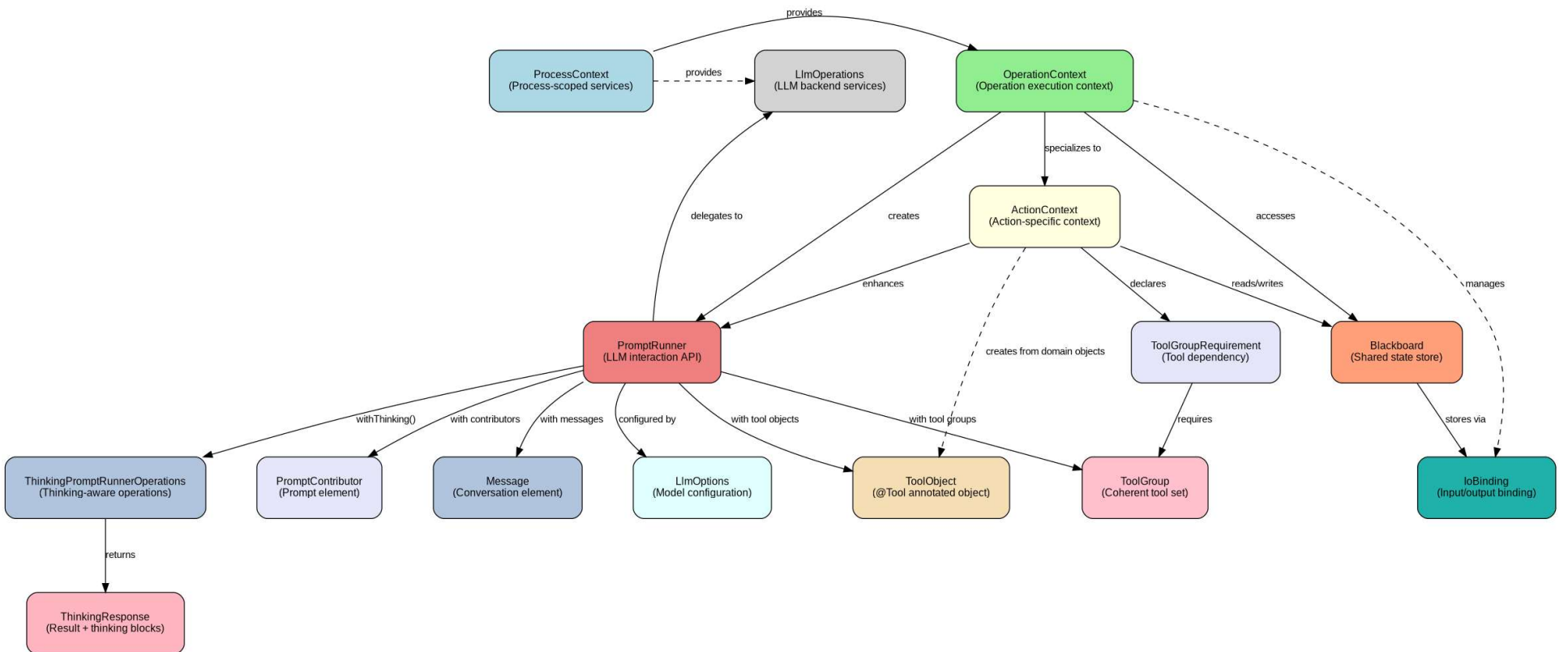
3.15. ProcessOptions

Agent processes can be configured with ProcessOptions.

ProcessOptions controls:

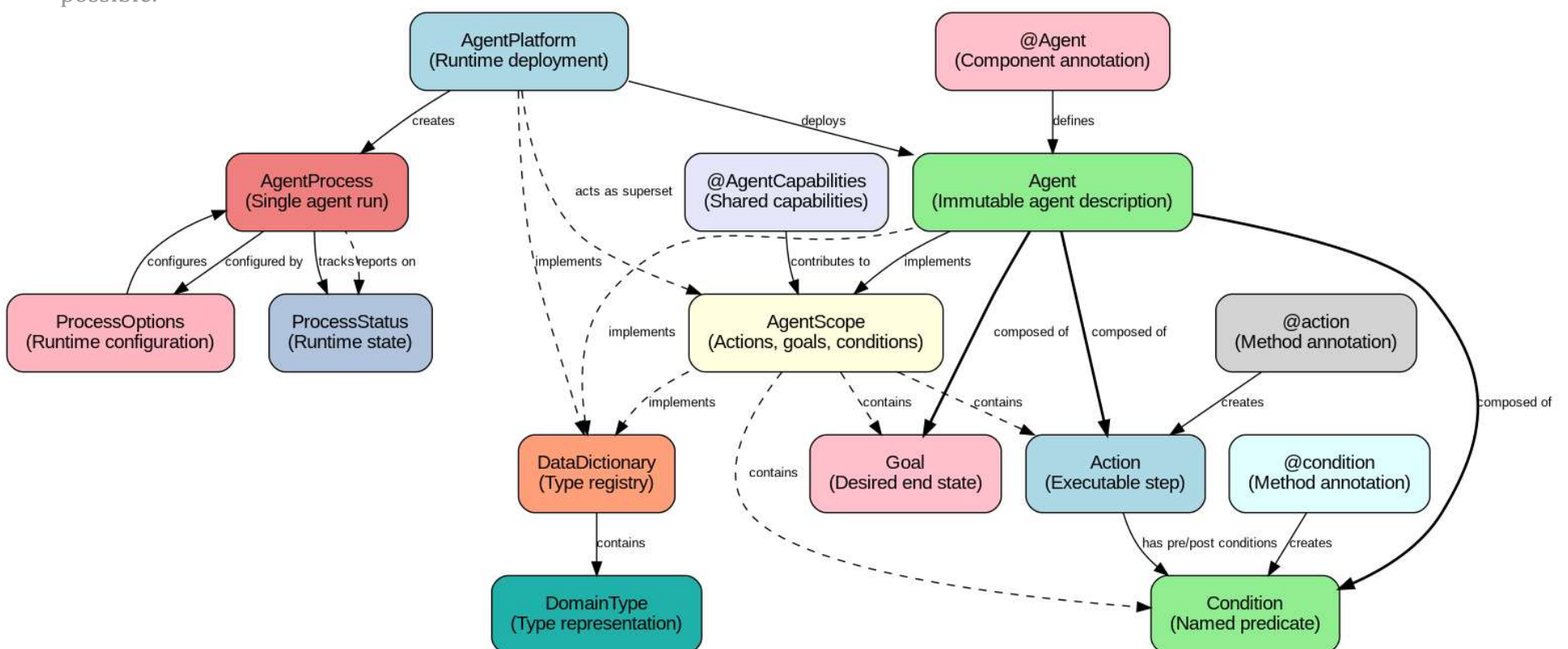
- contextId: An identifier of any existing context in which the agent is running.

- **blackboard:** The blackboard to use for the agent. Allows starting from a particular state.
- **test:** Whether the agent is running in test mode.
- **verbosity:** The verbosity level of the agent. Allows fine grained control over logging prompts, LLM returns and detailed planning information
- **control:** Control options, determining whether the agent should be terminated as a last resort. `EarlyTerminationPolicy` can be based on an absolute number of actions or a maximum budget.
- **Delays:** Both operations (actions) and tools can have delays. This is useful to avoid rate limiting.



3.16. The AgentPlatform

An AgentPlatform provides the ability to run agents in a specific environment. This is an SPI interface, so multiple implementations are possible.



3.17. Invoking Embabel Agents

While many examples show Embabel agents being invoked via `TextInput` through the Embabel shell, they can also be invoked programmatically with strong typing.

This is usually how they're used in web applications. It is also the most deterministic approach as code, rather than LLM assessment of user input, determines which agent is invoked and how.

3.17.1. Creating an AgentProcess Programmatically

You can create and execute agent processes directly using the AgentPlatform:

```

// Create an agent process with
bindingsAgentProcess=agentPlatform.createAgentProcess(myAgent,new ProcessOptions(),Map.of("input",userRequest));
// Start the process and wait for completion
Object result=agentPlatform.start(agentProcess).get();
// Or run
synchronouslyAgentProcesscompletedProcess=agentProcess.run();
MyResultType result=completedProcess.last(MyResultType.class);

```

You can create processes and populate their input map from varargs objects:

```
JavaKotlin
// Create process from objects (like in web
controllers)AgentProcessagentProcess=agentPlatform.createAgentProcessFrom(travelAgent,newProcessOptions(),travelRequest,userPreferences);
```

3.17.2. Using AgentInvocation

AgentInvocation provides a higher-level, type-safe API for invoking agents. It automatically finds the appropriate agent based on the expected result type.

Basic Usage

```
JavaKotlin
// Simple invocation with explicit result
typevar invocation=AgentInvocation.create(agentPlatform,TravelPlan.class);TravelPlanplan=invocation.invoke(travelRequest);
```

Invocation with Named Inputs

JavaKotlin

```
// Invoke with a map of named inputs
Map<String, Object>inputs=Map.of("request",travelRequest,"preferences",userPreferences);
TravelPlanplan=invocation.invoke(inputs);
```

Custom Process Options

Configure verbosity, budget, and other execution options:

```
varprocessOptions=newProcessOptions().withVerbosity(newVerbosity().withShowPrompts(true).withShowLlmResponses(true).withDebug(true));
varinvocation=AgentInvocation.builder(agentPlatform).options(processOptions).build(TravelPlan.class);
TravelPlanplan=invocation.invoke(travelRequest);
```

Asynchronous Invocation

For long-running operations, use async invocation:

```
CompletableFuture<TravelPlan>future=invocation.invokeAsync(travelRequest); // Handle result when complete
future.thenAccept(plan->{logger.info("Travel plan generated: {}",plan);}); // Or wait for completion
TravelPlanplan=future.get();
```

Agent Selection

AgentInvocation automatically finds agents by examining their goals:

- Searches all registered agents in the platform
- Finds agents with goals that produce the requested result type
- Uses the first matching agent found
- Throws an error if no suitable agent is available

Real-World Web Application Example

Here’s how AgentInvocation is used in the [Tripper travel planning application](#) with htmx for asynchronous UI updates:

```
@ControllerpublicclassTripPlanningController{privatefinalAgentPlatformagentPlatform;privatefinalConcurrentHashMap<String,CompletableFuture<TripPlan>>activeJobs=newConcurrentHashMap<>();privatestaticfinalLoggerlogger=LoggerFactory.getLogger(TripPlanningController.class);privatestaticfinalConcurrentHashMap<String,TripPlan>tripResultCache=newConcurrentHashMap<>();publicTripPlanningController(AgentPlatformagentPlatform){this.agentPlatform=agentPlatform;}@PostMapping("/plan-trip")publicStringplanTrip(@ModelAttributeTripRequesttripRequest,Modelmodel){ // Generate unique job ID for tracking
StringjobId=UUID.randomUUID().toString(); // Create agent invocation with custom options
varprocessOptions=newProcessOptions().withVerbosity(newVerbosity().withShowPrompts(true));
varinvocation=AgentInvocation.builder(agentPlatform).options(processOptions).build(TripPlan.class); // Start async agent execution
CompletableFuture<TripPlan>future=invocation.invokeAsync(tripRequest);
activeJobs.put(jobId,future); // Set up completion handler
future.whenComplete((result,throwable)->{if(throwable!=null){logger.error("Trip planning failed for job "+jobId,throwable);}else{logger.info("Trip planning completed for job "+jobId);}});
model.addAttribute("jobId",jobId);
model.addAttribute("tripRequest",tripRequest); // Return htmx template that will poll for results
return"trip-planning-progress";}@GetMapping("/trip-{jobId}/status/{jobId}")@ResponseBodypublicResponseEntity<Map<String,Object>>getTripStatus(@PathVariableStringjobId){CompletableFuture<TripPlan>future=activeJobs.get(jobId);if(future==null){returnResponseEntity.notFound().build();}if(future.isDone()){try{TripPlantripPlan=future.get();activeJobs.remove(jobId);returnResponseEntity.ok(Map.of("status","completed","result",tripPlan,"redirect","/trip-{jobId}/result/{jobId}"));
}catch(Exceptione){activeJobs.remove(jobId);returnResponseEntity.ok(Map.of("status","failed","error",e.getMessage()));}
}elseif(future.isCancelled()){activeJobs.remove(jobId);returnResponseEntity.ok(Map.of("status","cancelled"));
}else{returnResponseEntity.ok(Map.of("status","in_progress","message","Planning your amazing trip..."));}
}@GetMapping("/trip-{jobId}/result/{jobId}")publicStringshowTripResult(@PathVariableStringjobId,Modelmodel){ // Retrieve completed result from cache or database
TripPlantripPlan=tripResultCache.get(jobId);if(tripPlan==null){return"redirect:/error";}
model.addAttribute("tripPlan",tripPlan);return"trip-result";}@DeleteMapping("/cancel-trip/{jobId}")@ResponseBodypublicResponseEntity<Map<String,String>>cancelTrip(@PathVariableStringjobId){CompletableFuture<TripPlan>future=activeJobs.get(jobId);if(future!=null&&!future.isDone()){future.cancel(true);activeJobs.remove(jobId);returnResponseEntity.ok(Map.of("status","cancelled"));}
else{returnResponseEntity.badRequest().body(Map.of("error","Job not found or already completed"));}
}}
```

Key Patterns:

- **Async Execution:** Uses invokeAsync() to avoid blocking the web request
- **Job Tracking:** Maintains a map of active futures for status polling
- **htmx Integration:** Returns status updates that htmx can consume for UI updates
- **Error Handling:** Proper exception handling and user feedback
- **Resource Cleanup:** Removes completed jobs from memory
- **Process Options:** Configures verbosity and debugging for production use

Alternative: Direct AgentProcess Creation

For simpler use cases, you can create and start an AgentProcess directly without AgentInvocation. This approach is used in the [Tripper](#) application and works well with webhooks or form submissions where you want to:

- Start a long-running agent process
- Return immediately with a process ID
- Poll for status using the platform’s built-in controllers

```
varagentPlatform=AgentPlatform.builder().withAgents(newAgentPlatformBuilder().withAgents(newAgentPlatformBuilder().withAgent(newAgentPlatformBuilder().withGoals(newGoalBuilder().withGoalType(ResultType.TRAVEL_PLAN).withGoalDescription("Find the best travel plan for the user's request").build()).build()).build()).build();
varagentPlatform.start();
// Create the agent process with input bindings
AgentProcessagentProcess=agentPlatform.createAgentProcessFrom(agent,newProcessOptions(newVerbosity().withShowPrompts(true),Budget.DEFAULT// or custom budget),travelBrief// Vararg inputs bound to blackboard);
// Start the process asynchronously
agentPlatform.start(agentProcess); // Add process ID to model for status polling
model.addAttribute("processId",agentProcess.getId());
model.addAttribute("travelBrief",travelBrief); // Return a view that polls /api/v1/process/{processId} for status
return"processing";
```

The platform provides built-in REST endpoints for status checking:

- GET /api/v1/process/{processId} - Returns process status, result, and URLs
- DELETE /api/v1/process/{processId} - Terminates a running process
- GET /events/process/{processId} - SSE stream of process events

A simple status polling controller can check completion and redirect to results:

JavaKotlin

```
@Controllerpublic class ProcessStatusController { private final AgentPlatform agentPlatform; public ProcessStatusController(AgentPlatform agentPlatform) { this.agentPlatform = agentPlatform; } @GetMapping("/status/{processId}") public String checkStatus(@PathVariable String processId, @RequestParam String successView, @RequestParam String resultModelKey, Model model) { AgentProcess process = agentPlatform.getAgentProcess(processId); if (process == null) { throw new ResponseStatusException(HttpStatus.NOT_FOUND, "Process not found"); } switch (process.getStatus()) { case COMPLETED: model.addAttribute(resultModelKey, process.lastResult()); return successView; case FAILED: model.addAttribute("error", "Process failed: " + process.getFailureInfo()); return "error"; case TERMINATED: model.addAttribute("error", "Process was terminated"); return "error"; default: // Still running - return polling view model.addAttribute("processId", processId); return "processing"; } }
```

When to Use Each Approach:

Approach	Best For
AgentInvocation.invokeAsync()	When you need a CompletableFuture for programmatic handling, chaining, or integration with reactive frameworks
Direct AgentProcess creation	Webhooks, form submissions, or UI flows where you poll for status via REST/SSE

Webhook Integration Example

For webhook-triggered workflows (e.g., JIRA, GitHub), the direct approach works well:

JavaKotlin

```
@RestController @RequestMapping("/webhook") public class WebhookController { private final AgentPlatform agentPlatform; public WebhookController(AgentPlatform agentPlatform) { this.agentPlatform = agentPlatform; } @PostMapping("/jira/issue-created") public ResponseEntity<Map<String, String>> onJiraIssueCreated(@RequestBody JiraWebhookPayload payload) { // Find agent that handles JIRA issues Agent agent = agentPlatform.agents().stream().filter(a -> a.getName().contains("JiraIssue")).findFirst().orElseThrow(() -> new IllegalStateException("No JIRA agent configured")); // Create domain object from webhook payload JiraIssue issue = new JiraIssue(payload.getIssue().getKey(), payload.getIssue().getFields().getSummary(), payload.getIssue().getFields().getDescription()); // Create and start the agent AgentProcess process = agentPlatform.createAgentProcessFrom(agent, ProcessOptions.DEFAULT, issue); agentPlatform.start(process); // Return process ID for status tracking return ResponseEntity.accepted().body(Map.of("processId", process.getId(), "statusUrl", "/api/v1/process/" + process.getId(), "sseUrl", "/events/process/" + process.getId())); }
```

The webhook caller can then poll /api/v1/process/{processId} or subscribe to SSE events at /events/process/{processId} to track progress.

Agents can also be exposed as MCP servers and consumed from tools like Claude Desktop.

3.17.3. Dynamic Agent and Goal Selection with Autonomy

The Autonomy class provides LLM-powered dynamic selection of agents and goals based on user intent. Rather than programmatically choosing which agent to run, Autonomy uses an LLM to rank available agents or goals against the user’s input and select the best match. This is how the Embabel Shell processes natural language commands.

Execution Modes

Autonomy supports two execution modes:

Closed Mode (chooseAndRunAgent): The LLM selects the most appropriate agent based on the user’s intent. The selected agent runs in isolation using only its own actions and goals.

Open Mode (chooseAndAccomplishGoal): The LLM selects the most appropriate goal from all available goals across all agents. Embabel then assembles a dynamic agent that can use any action from any agent to achieve that goal.

Closed Mode Example

Use closed mode when you want strict agent boundaries:

JavaKotlin

```
@Service public class IntentHandler { private final Autonomy autonomy; public IntentHandler(Autonomy autonomy) { this.autonomy = autonomy; } public AgentProcess executionHandleUserIntent(String userInput) { // LLM ranks all agents and selects the best match return autonomy.chooseAndRunAgent(userIntent, ProcessOptions.DEFAULT); }
```

Open Mode Example

Use open mode when you want maximum flexibility in achieving goals:

JavaKotlin

```
@Service public class GoalHandler { private final Autonomy autonomy; private final AgentPlatform agentPlatform; public GoalHandler(Autonomy autonomy, AgentPlatform agentPlatform) { this.autonomy = autonomy; this.agentPlatform = agentPlatform; } public AgentProcess executionHandleUserIntent(String userInput) { // LLM ranks all goals and selects the best match // Then assembles an agent from available actions to achieve it return autonomy.chooseAndAccomplishGoal(ProcessOptions.DEFAULT, GoalChoiceApprover.APPROVE_ALL, agentPlatform, // AgentScope containing goals and actions Map.of("userInput", new UserInput(userIntent)), new GoalSelectionOptions()); }
```

Using Arbitrary Bindings

chooseAndAccomplishGoal accepts any bindings, not just UserInput. A BindingsFormatter extracts intent text from the bindings for goal ranking:

JavaKotlin

```
public AgentProcess execution processTask(Task task, Person person) { // Bindings can be any objects Map<String, Object> bindings = Map.of("task", task, "person", person); return autonomy.chooseAndAccomplishGoal(ProcessOptions.DEFAULT, GoalChoiceApprover.APPROVE_ALL, agentPlatform, bindings, new GoalSelectionOptions(), BindingsFormatter.DEFAULT // Extracts intent from bindings); }
```

The default BindingsFormatter extracts text using this priority:

- 1. PromptContributor.contribution() if the object implements PromptContributor
- 2. HasInfoString.infoString() if the object implements HasInfoString
- 3. toString() otherwise

You can provide a custom formatter:

JavaKotlin

```
BindingsFormatter customFormatter = bindings -> { Task task = (Task) bindings.get("task"); Person person = (Person) bindings.get("person"); return String.format("Process task '%s' for '%s'", task.getDescription(), person.getName()); }; return autonomy.chooseAndAccomplishGoal(ProcessOptions.DEFAULT, GoalChoiceApprover.APPROVE_ALL, agentPlatform, bindings, new GoalSelectionOptions(), customFormatter); }
```

Goal Choice Approval

You can require approval before executing a selected goal:

JavaKotlin

```
// Approve only high-confidence matchesGoalChoiceApproverapprover=GoalChoiceApprover.approveWithScoreOver(0.8);// Or
implement custom approval logicGoalChoiceApprovercustomApprover=request-
>{if(request.getGoal().getName().contains("dangerous")){returnnewGoalChoiceNotApproved("Dangerous goals require manual
approval");}returnGoalChoiceApproved.INSTANCE;};
```

Confidence Thresholds

Autonomy uses configurable confidence thresholds to filter matches. If no agent or goal exceeds the threshold, a NoAgentFound or NoGoalFound exception is thrown.

Configure thresholds in application.properties:

```
# Minimum confidence for agent selection (0.0 to 1.0)
embabel.agent.platform.autonomy.agent-confidence-cut-off=0.6# Minimum confidence for goal selection (0.0 to 1.0)
embabel.agent.platform.autonomy.goal-confidence-cut-off=0.6
```

Or override per-request using GoalSelectionOptions:

```
JavaKotlin
GoalSelectionOptionsoptions=newGoalSelectionOptions(0.5,goalConfidenceCutOff - override platform defaultnull,agentConfidenceCutOff - use platform defaultfalsemultiGoal - whether to select multiple goals);
```

Shell Usage

The Embabel Shell uses Autonomy for the execute (or x) command:

```
# Closed mode (default) - select best agentx "Find a horoscope for Alice who is a Scorpio"# Open mode - select best goal, use any actionsx
"Find a horoscope for Alice who is a Scorpio"-o# Show goal rankings without executingchoose-goal "Find a horoscope for Alice"
```

Handling Selection Failures

```
JavaKotlin
try{returnautonomy.chooseAndRunAgent(userIntent,ProcessOptions.DEFAULT);}catch(NoAgentFounde){// No agent matched
with sufficient confidencelogger.info("No matching agent. Rankings:
{"},e.getAgentRankings());returnfallbackResponse();}catch(NoGoalFounde){// No goal matched with sufficient confidence (open
mode)logger.info("No matching goal. Rankings: {"},e.getGoalRankings());returnfallbackResponse();}catch(GoalNotApprovede){//
Goal was rejected by the approverlogger.info("Goal not approved: {"},e.getReason());returnrequiresApprovalResponse();}
```

3.18. Using States

GOAP planning has many benefits, but can make looping hard to express. For this reason, Embabel supports the notion of **states** within a GOAP plan.

3.18.1. How States Work with GOAP

Within each state, GOAP planning works normally. Actions have preconditions based on the types they require, and effects based on the types they produce. The planner finds the optimal sequence of actions to reach the goal.

When an action returns a @State-annotated class, the framework:

1. **Hides previous state objects** - Any existing state objects are hidden from the blackboard
2. **Binds the new state object** - The returned state is added to the blackboard
3. **Re-plans from the new state** - The planner considers only actions from the new state
4. **Continues execution** - Until a goal is reached or no plan can be found

Context is preserved across state transitions - non-state objects (such as user messages, customer data, and conversation history) remain available. Only state objects are hidden, ensuring that only the current state's actions are considered by the planner.

State transitions **hide** previous state objects but do **not clear** the blackboard. Non-state objects remain available in the new state. To clear the entire blackboard (e.g., for looping), use clearBlackboard = true on the action.

3.18.2. When to Use States

States are ideal for:

- **Linear stages** where each stage naturally flows to the next
- **Branching workflows** where a decision point leads to different processing paths
- **Looping patterns** where processing may need to repeat (e.g., revise-and-review cycles)
- **Human-in-the-loop workflows** where user feedback determines the next state
- **Complex workflows** that are easier to reason about as discrete phases

States allow loopback to a whole state, which may contain one or more actions. This is more flexible than traditional GOAP, where looping requires careful management of preconditions.

3.18.3. Staying in the Current State

An action can return this to stay in the current state. This is useful for actions that respond to inputs without changing state, such as chat handlers:

```
JavaKotlin
@StaterecordChitchatState(Stringcontext){@Action(canRerun=true)ChitchatStaterespond(UserMessagemessage,Aiai){varrespo
nse=ai.generateText("Respond to: "+message.content());// ... send responsereturnthis;}}
```

canRerun = true is required - by default, actions only run once per process

Returning this keeps the same state instance active

When an action returns this:

- The state remains active with no transition
- The blackboard is preserved (no clearing)
- The action can run again on subsequent planning cycles (if canRerun = true)

Without canRerun = true, the action's hasRun flag would prevent it from executing again, even though it returned this.

3.18.4. Looping States

For looping patterns where an action may return to a previously-visited state type, use clearBlackboard = true on the looping action:

```
JavaKotlin
@StaterecordProcessingState(Stringdata,intiteration)implementsLoopOutcome{@Action(clearBlackboard=true)LoopOutcomep
rocess(){if(iteration>=3){returnnewDoneState(data);}returnnewProcessingState(data+"+",iteration+1);}}
```

clearBlackboard = true allows the action to loop back to the same state type

Terminal condition exits the loop

Returns a new instance of the same state type for another iteration

Without clearBlackboard = true, the planner would see the output type already exists on the blackboard and skip the action. Clearing the blackboard resets the context, allowing natural loops.

Only use clearBlackboard = true on actions that participate in loops. For linear state transitions, the default behavior (preserving the blackboard) is usually preferred.

3.18.5. The @State Annotation

Classes returned from actions that should trigger state transitions must be annotated with @State:

```
JavaKotlin
@StaterecordProcessingState(Stringdata){@ActionNextStateprocess(){returnnewNextState(data.toUpperCase());}}
```


Inheritance

The @State annotation is inherited through the class hierarchy. If a superclass or interface is annotated with @State, all subclasses and implementing classes are automatically considered state types. This means you don’t need to annotate every class in a hierarchy - just annotate the base type.

JavaKotlin

```
@StateinterfaceStage{}recordAssessStory(Stringcontent)implementsStage{...}recordReviseStory(Stringcontent)implementsStage{...}recordDone(Stringcontent)implementsStage{...}
```

Only the parent interface needs @State

Implementing records/data classes are automatically treated as state types

This works with:

- **Interfaces:** Classes implementing a @State interface are state types
- **Abstract classes:** Classes extending a @State abstract class are state types
- **Concrete classes:** Classes extending a @State class are state types
- **Deep hierarchies:** The annotation is inherited through multiple levels

Behavior

When an action returns a @State-annotated class (or a class that inherits @State):

- Any previous state objects are **hidden** from the blackboard (not removed, but no longer visible)
- The returned object is bound to the blackboard (as it)
- Planning considers only actions defined within the **current** state class
- Any @AchievesGoal methods in the state become potential goals

Context (non-state objects) is preserved across state transitions. This means user messages, customer data, conversation history, etc. remain available in the new state. Only state objects are hidden, providing **state scoping** - ensuring only the current state’s actions are considered.

For looping states that return to a previously-visited state type, use @Action(clearBlackboard = true) on the looping action. This clears the blackboard (including hasRun conditions) and allows the loop to continue. See [Looping States](#) for details.

3.18.6. Parent State Interface Pattern

For dynamic choice between states, define a parent interface (or sealed interface/class) that child states implement. Thanks to [inheritance](#), you only need to annotate the parent interface - all implementing classes are automatically state types:

JavaKotlin

```
@StateinterfaceStage{}recordAssessStory(Stringcontent)implementsStage{@ActionStageassess(){if(isAcceptable()){returnnewDone(content);}}else{returnnewReviseStory(content);}}recordReviseStory(Stringcontent)implementsStage{@ActionAssessStoryrevise(){returnnewAssessStory(improvedContent());}}recordDone(Stringcontent)implementsStage{@AchievesGoal(description="Processing complete")@ActionOutputcomplete(){returnnewOutput(content);}}
```

@State on the parent interface

No @State needed on implementing records/data classes - they inherit it from Stage

This pattern enables:

- **Polymorphic return types:** Actions can return any implementation of the parent interface
- **Dynamic routing:** The runtime value determines which state is entered
- **Looping:** States can return other states that eventually loop back

The framework automatically discovers all implementations of the parent interface and registers their actions as potential next steps.

3.18.7. Example: WriteAndReviewAgent

The following example demonstrates a complete write-and-review workflow with:

- State-based flow control with looping
- Human-in-the-loop feedback using WaitFor
- LLM-powered content generation and assessment
- Configurable properties passed through states

JavaKotlin

```
abstractclassPersonas{staticfinalRoleGoalBackstoryWRITER=RoleGoalBackstory.withRole("Creative Storyteller").andGoal("Write engaging and imaginative stories").andBackstory("Has a PhD in French literature; used to work in a circus");staticfinalPersonaREVIEWER=newPersona("Media Book Review","New York Times Book Reviewer","Professional and insightful","Help guide readers toward good stories");}@Agent(description="Generate a story based on user input and review it")publicclassWriteAndReviewAgent{publicrecordStory(Stringtext){}publicrecordReviewedStory(Storystory,Stringreview,Personareviewer)implementsHasContent,Timestamped{// ... content formatting methods}@StateinterfaceStage{}recordProperties(intstoryWordCount,intreviewWordCount){}privatefinalPropertiesproperties;WriteAndReviewAgent(@Value("${storyWordCount:100}")intstoryWordCount,@Value("${reviewWordCount:100}")intreviewWordCount){this.properties=newProperties(storyWordCount,reviewWordCount);}@ActionAssessStorycraftStory(UserInputuserInput,Aiai){vardraft=ai.withLlm(LlmOptions.withAutoLlm().withTemperature(.7)).withPromptContributor(Personas.WRITER).createObject(String.format("""Craft a short story in %d words or less. The story should be engaging and imaginative. Use the user's input as inspiration if possible. # User input %s """,properties.storyWordCount,userInput.getContent().trim(),Story.class));returnnewAssessStory(userInput,draft,properties);}recordHumanFeedback(Stringcomments){privaterecordAssessmentOfHumanFeedback(booleanacceptable){}@StaterecordAssessStory(UserInputuserInput,Storystory,Propertiesproperties)implementsStage{@ActionHumanFeedbackgetFeedback(){returnWaitFor.formSubmission("""Please provide feedback on the story %s """,properties.storyWordCount,userInput.getContent().trim(),Story.class);}@Action(clearBlackboard=true)Stageassess(HumanFeedbackfeedback,Aiai){varassessment=ai.withDefaultLlm().createObject("""Based on the following human feedback, determine if the story is acceptable. Return true if the story is acceptable, false otherwise. # Story %s # Human feedback %s """,properties.storyWordCount,userInput.getContent().trim(),Story.class,feedback.comments,HumanFeedback.class)}}}
```

```

        """formatted(story.text(),feedback.comments),AssessmentOfHumanFeedback.class);if(assessment.acceptable){returnnewDone(userInput,story,properties);}else{returnnewReviseStory(userInput,story,feedback,properties);}}@StaterecordReviseStory(UserInputuserInput,Storystory,HumanFeedbackhumanFeedback,Propertiesproperties)implementsStage{@Action(clearBlackboard=true)AssessStoryreviseStory(Aiai){vardraft=ai.withLlm(LlmOptions.withAutoLlm().withTemperature(.7)).withPromptContributor(Personas.WRITER).createObject(String.format("
        Revise a short story in %d words or less.
        Use the user's input as inspiration if possible.
# User input
        %s
# Previous story
        %s
# Revision instructions
        %s

        """,properties.storyWordCount,userInput.getContent(),story.text(),humanFeedback.comments).trim(),Story.class);returnnewAssessStory(userInput,draft,properties);}}@StaterecordDone(UserInputuserInput,Storystory,Propertiesproperties)implementsStage{@AchievesGoal(description="The story has been crafted and reviewed by a book
reviewer",export=@Export(remote=true,name="writeAndReviewStory"))@ActionReviewedStoryreviewStory(Aiai){varreview=ai.withAutoLlm().withPromptContributor(Personas.REVIEWER).generateText(String.format("
        You will be given a short story to review.
        Review it in %d words or less.
        Consider whether the story is engaging, imaginative, and well-written.
# Story
        %s
# User input that inspired the story
        %s

        """,properties.reviewWordCount,story.text(),userInput.getContent()).trim());returnnewReviewedStory(story,review,Personas.REVIEWER);}}

```

Personas: Reusable prompt contributors that give the LLM context about its role

Parent state interface: Allows actions to return any implementing state dynamically

Properties record: Configuration bundled together for easy passing through states

Entry action: Uses LLM to generate initial story draft

State transition: Returns AssessStory with all necessary data

HITL data type: Simple record/data class to capture human feedback

WaitFor integration: Pauses execution and waits for user to submit feedback form

Looping action: clearBlackboard = true enables returning to a previously-visited state type

Terminal branch: If acceptable, transitions to Done state

Loop branch: If not acceptable, transitions to ReviseStory with the feedback

Looping action: clearBlackboard = true enables looping back to AssessStory

Loop back: Returns new AssessStory for another round of feedback

Goal achievement: Final action that produces the reviewed story and exports it

3.18.8. Execution Flow

The execution flow for this agent:

1. **craftStory** executes with LLM, returns AssessStory → enters AssessStory state
 2. **getFeedback** calls WaitFor.formSubmission() → agent pauses, waits for user input
 3. User submits feedback → HumanFeedback added to blackboard
 4. **assess** executes with LLM to interpret feedback:
 - If acceptable: returns Done → blackboard cleared, enters Done state
 - If not acceptable: returns ReviseStory → blackboard cleared, enters ReviseStory state
 5. If in ReviseStory: **reviseStory** executes with LLM, returns AssessStory → blackboard cleared, loop back to step 2
 6. When in Done: **reviewStory** executes with LLM, returns ReviewedStory → goal achieved
- The planner handles all transitions automatically, including loops. The looping actions (assess and reviseStory) use clearBlackboard = true to enable returning to previously-visited state types.

3.18.9. Human-in-the-Loop with WaitFor

The WaitFor.formSubmission() method is key for human-in-the-loop workflows:

JavaKotlin

```

@ActionHumanFeedbackgetFeedback(){returnWaitFor.formSubmission("
    Please provide feedback on the story
    %s
    ""formatted(story.text()),HumanFeedback.class);}

```

When this action executes:

1. The agent process enters a WAITING state
2. A form is generated based on the HumanFeedback record structure
3. The user sees the prompt and fills out the form
4. Upon submission, the HumanFeedback instance is created and added to the blackboard
5. The agent resumes execution with the feedback available

This integrates naturally with the state pattern: the feedback stays within the current state until the next state transition.

3.18.10. Passing Data Through States

When using clearBlackboard = true for looping states, all necessary context must be passed through state records since the blackboard is cleared:

JavaKotlin


```
@State record AssessStory(UserInput userInput, // Original user request Story story, // Current story draft Properties properties, // Configuration) implements Stage {...}
@State record ReviseStory(UserInput userInput, Story story, HumanFeedback humanFeedback, // Additional context for revision Properties properties) implements Stage {...}
```

Use a `Properties` record/data class to bundle configuration values that need to pass through multiple states, rather than repeating individual fields. For non-looping state transitions (where `clearBlackboard` is not used), the blackboard is preserved, and data can be accessed from the blackboard directly. This is useful when states need access to shared context like user identity or conversation history.

3.18.11. State Class Requirements

State classes **must be** either **static nested classes** (Java) or **top-level classes** (Kotlin). Non-static inner classes are **not allowed** because they hold a reference to their enclosing instance, causing serialization and persistence issues. The framework will throw an `IllegalStateException` if it detects a non-static inner class annotated with `@State`.

JavaKotlin

```
// GOOD: Static nested class (Java record is implicitly static)
@State record AssessStory(UserInput userInput, Story story) implements Stage {...} // GOOD: Top-level class
@State record ProcessingState(String data) {...} // BAD: Non-static inner class - will throw
IllegalStateException
@State class AssessStory implements Stage {...} // Inner class in non-static context
```

In Java, records declared inside a class are implicitly static, making them ideal for state classes. In Kotlin, data classes declared inside a class are inner by default; use **top-level declarations** instead.

Top-level state classes are the recommended pattern for Kotlin. They can access the enclosing component via the `@Provided` annotation. See [The @Provided Annotation](#) for full documentation.

3.18.12. Key Points

- Annotate state classes with `@State` (or inherit from a `@State`-annotated type)
- `@State` is inherited through class hierarchies - annotate only the base type
- Use **static nested classes** (Java records) or **top-level classes** to avoid persistence issues
- Use a parent interface for polymorphic state returns
- State actions are automatically discovered and registered
- State scoping:** When entering a new state, previous states are hidden - only current state’s actions are available
- Context is preserved:** Non-state objects (user data, conversation, etc.) remain available across transitions
- Blackboard preserved:** State transitions hide previous states but preserve all other blackboard contents
- Staying in state:** Return this with `canRerun = true` to stay in the current state without transitioning
- For **looping states**, use `@Action(clearBlackboard = true)` to enable returning to previously-visited state types
- When using `clearBlackboard = true`, pass all necessary data through state record fields
- Goals are defined with `@AchievesGoal` on terminal state actions
- Use `WaitFor` for human-in-the-loop interactions within states
- Within a state, normal GOAP planning applies to sequence actions

3.19. Choosing a Planner

Embabel supports multiple planning strategies. Most are deterministic, but their behaviour differs—although it is always predictable.

All planning strategies are entirely typesafe in Java or Kotlin.

The planning strategies currently supported out of the box are:

Planner	Best For	Description
GOAP (default)	Business processes with defined outputs	Goal-oriented, deterministic planning. Plans a path from current state to goal using preconditions and effects.
Utility	Exploration and event-driven systems	Selects the highest-value available action at each step. Ideal when you don’t know the outcome upfront.
Supervisor	Flexible multi-step workflows	LLM-orchestrated composition. An LLM selects which actions to call based on type schemas and gathered artifacts.

As most of the documentation covers GOAP, this section discusses the alternative planners and nested workflows.

3.19.1. Utility AI

[Utility AI](#) selects the action with the highest *net value* from all available actions at each step. Unlike GOAP, which plans a path to a goal, Utility AI makes greedy decisions based on immediate value.

Utility AI excels in **exploratory scenarios** where you don’t know exactly what you want to achieve. Consider a GitHub issue triage system: when a new issue arrives, you don’t have a predetermined goal. Instead, you want to react appropriately based on the issue’s characteristics—maybe label it, maybe respond, maybe escalate. The "right" action depends on what you discover as you process it.

This makes Utility AI ideal for scenarios where:

- There is no clear end goal—you’re exploring possibilities
- Multiple actions could be valuable depending on context
- You want to respond to changing conditions as they emerge
- The best outcome isn’t known upfront

When to Use Utility AI

- Event-driven systems:** React to incoming events (issues, stars, webhooks) with the most appropriate action
- Chatbots:** Where the platform provides multiple response options and selects the best one
- Exploration:** When you want to discover what’s possible rather than achieve a specific goal

Using Utility AI with @EmbabelComponent

For Utility AI, actions are typically provided via `@EmbabelComponent` rather than `@Agent`. This allows the *platform* to select actions across multiple components based on utility, rather than constraining actions to a single agent.

Here’s an example from the Shepherd project that reacts to GitHub events:

JavaKotlin

```
@EmbabelComponent public class IssueActions {
    private final ShepherdProperties properties;
    private final CommunityDataManager communityDataManager;
    private final GitHubUpdater gitHubUpdater;
    public IssueActions(ShepherdProperties properties, CommunityDataManager communityDataManager, GitHubUpdater gitHubUpdater) {
        this.properties = properties;
        this.communityDataManager = communityDataManager;
        this.gitHubUpdater = gitHubUpdater;
    }
    @Action(outputBinding = "ghIssue") public GHIssue saveNewIssue(GHIssue ghIssue, OperationContext context) {
        var existing = communityDataManager.findIssueByGithubId(ghIssue.getId());
        if (existing == null) {
            var issueEntityStatus = communityDataManager.saveAndExpandIssue(ghIssue);
            context.add(issueEntityStatus);
            return ghIssue;
        }
        return null;
    }
    @Action(pre = {"spel:newEntity.newEntities.[#this instanceof T(com.embabel.shepherd.domain.Issue)].size() > 0"}) public IssueAssessment reactToNewIssue(GHIssue ghIssue, NewEntity<?> newEntity, Aiai) {
        return ai.withLlm(properties.getTriageLlm()).creating(IssueAssessment.class).fromTemplate("first_issue_response", Map.of("issue", ghIssue));
    }
    @Action(pre = {"spel:issueAssessment.urgency > 0.0"}) public void heavyHitterIssue(GHIssue issue, IssueAssessment issueAssessment) {
        // Take action on high-urgency issues
    }
}
```

`@EmbabelComponent` contributes actions to the platform, not a specific agent

`outputBinding` names the result for later actions to reference

Add entity status to context, making it available to subsequent actions

Returning null prevents further actions from firing for this issue

SpEL precondition: only fire if new issues were created

Use AI to assess the issue via a template

This action only fires if the assessment shows urgency > 0

The platform selects which action to run based on:

1. Which preconditions are satisfied (type availability + SpEL conditions)
2. The cost and value parameters on @Action (net value = value - cost)

Action Cost and Value

The @Action annotation supports cost and value parameters (both 0.0 to 1.0):

JavaKotlin

```
@Action(cost=0.1,value=0.8)publicOutpuhighValueAction(Inputinput){// Action implementation}
```

Cost to execute (0.0 to 1.0) - lower is cheaper

Value when executed (0.0 to 1.0) - higher is more valuable

The Utility planner calculates *net value* as value - cost and selects the action with the highest net value from all available actions.

The Nirvana Goal

Utility AI supports a special "Nirvana" goal that is never satisfied. This keeps the process running, continuously selecting the highest-value available action until no actions are available.

Extensibility

Utility AI fosters extensibility. For example, multiple groups within an organization can contribute their own @EmbabelComponent classes with actions that bring their own expertise to enhance behaviours around shared types, while retaining the ability to own and control their own extended model.

Utility and States

Utility AI can combine with the @State annotation to implement classification and routing patterns. This is particularly useful when you need to:

- **Classify input** into different categories at runtime
- **Route processing** through category-specific handlers
- **Achieve different goals** based on classification

The key pattern is:

1. An entry action classifies input and returns a @State type
2. Each @State class contains an @AchievesGoal action that produces the final output
3. The @AchievesGoal output is *not* a @State type (to prevent infinite loops)

Here’s an example of a ticket triage system that routes support tickets based on severity:

JavaKotlin

```
@Agent(description="Triage and process support tickets",planner=PlannerType.UTILITY)publicclassTicketTriageAgent{publicrecordTicket(Stringid,Stringdescription,StringcustomerId){}publicrecordResolvedTicket(Stringid,Stringresolution,StringhandledBy){}@StatepublicsealedinterfaceTicketCategorypermitsCriticalTicket,BugTicket,GeneralTicket{}@ActionpublicTicketCategorytriageTicket(Ticketticket){if(ticket.description().toLowerCase().contains("down")){returnnewCriticalTicket(ticket);}elseif(ticket.description().toLowerCase().contains("bug")){returnnewBugTicket(ticket);}else{returnnewGeneralTicket(ticket);}}@StatepublicrecordCriticalTicket(Ticketticket)implementsTicketCategory{@AchievesGoal(description="Handle critical ticket with immediate escalation")@ActionpublicResolvedTickethandleCritical(){returnnewResolvedTicket(ticket.id(),"Escalated to on-call engineer","CRITICAL_RESPONSE_TEAM");}}@StatepublicrecordBugTicket(Ticketticket)implementsTicketCategory{@AchievesGoal(description="Handle bug report")@ActionpublicResolvedTickethandleBug(){returnnewResolvedTicket(ticket.id(),"Bug logged in issue tracker","ENGINEERING_TEAM");}}@StatepublicrecordGeneralTicket(Ticketticket)implementsTicketCategory{@AchievesGoal(description="Handle general inquiry")@ActionpublicResolvedTickethandleGeneral(){returnnewResolvedTicket(ticket.id(),"Response sent with FAQ links","SUPPORT_TEAM");}}}
```

Use PlannerType.UTILITY for opportunistic action selection

Sealed interface as the state supertype

Entry action classifies and returns a @State instance

Each state has an @AchievesGoal action producing the final output

When a Ticket is processed:

1. The triageTicket action classifies it into one of the state types
2. Entering a state clears other objects from the blackboard
3. The Utility planner selects the @AchievesGoal action for that state
4. The goal is achieved when ResolvedTicket is produced

This pattern works well when:

- Classification determines the processing path
- Each category has distinct handling requirements
- The final output type is the same across all categories

UtilityInvocation: Lightweight Utility Pattern

For simple utility workflows, you don’t need to create an @Agent class. UtilityInvocation provides a fluent API to run utility-based workflows directly from @EmbabelComponent actions.

Example 1. Invoking with UtilityInvocation

JavaKotlin

```
UtilityInvocation.on(agentPlatform).withScope(AgentScopeBuilder.fromInstances(issueActions,labelActions)).run(newGHIssue(issueData));
```

Configuration Options

UtilityInvocation supports several configuration methods:

Method	Description
.withScope(AgentScopeBuilder)	Defines which actions are available
.withAgentName(String)	Sets a custom name for the created agent (defaults to platform name)

.withProcessOptions(ProcessOptions)	Configures process-level options
.terminateWhenStuck()	Adds early termination policy when no actions are available

Example 2. Setting a custom agent name

```
JavaKotlin
UtilityInvocation.on(agentPlatform).withScope(
AgentScopeBuilder.fromInstance(myActions)).withAgentName("issue-triage-agent").run(input);
```

3.19.2. Supervisor

The Supervisor planner uses an LLM to orchestrate actions dynamically. This is a popular pattern in frameworks like [LangGraph](#) and [Google ADK](#), where a supervisor LLM decides which tools to call and in what order. Unlike GOAP and Utility, the Supervisor planner is **non-deterministic**. The LLM may choose different action sequences for the same inputs. This makes it less suitable for business-critical workflows requiring reproducibility.

Type-Informed vs Type-Driven

A key design decision in supervisor architectures is how types relate to composition:

Approach	Description
Type-Driven (GOAP)	Types <i>constrain</i> composition. An action requiring MarketData can only run after an action produces MarketData. This is deterministic but rigid.
Type-Informed (Supervisor)	Types <i>inform</i> composition. The LLM sees type schemas and decides what to call based on semantic understanding. This is flexible but non-deterministic.

Embabel’s Supervisor planner takes the **type-informed** approach while maximizing the benefits of types:

- Actions return **typed outputs** that are validated
- The LLM sees **type schemas** to understand what each action produces
- Results are stored on the **typed blackboard** for later actions
- The same actions work with **any planner** (GOAP, Utility, or Supervisor)

This is a "typed supervisor" pattern—a middle ground between fully type-driven (GOAP) and untyped string-passing (typical LangGraph).

When to Use Supervisor

Supervisor is appropriate when:

- Action ordering is **context-dependent** and hard to predefine
- You want an LLM to **synthesize information** across multiple sources
- The workflow benefits from **flexible composition** rather than strict sequencing
- Non-determinism is acceptable for your use case

Supervisor is **not** recommended when:

- You need **reproducible**, auditable execution paths
- Actions have strict **dependency ordering** that must be enforced
- Latency and cost matter (each decision requires an LLM call)

Using Supervisor

To use Supervisor, annotate your agent with planner = PlannerType.SUPERVISOR and mark one action with @AchievesGoal:

```
JavaKotlin
@Agent(planner=PlannerType.SUPERVISOR,description="Market research report generator")publicclassMarketResearchAgent{publicrecordMarketDataRequest(Stringtopic){}publicrecordMarketData(Map<String,String>revenues,Map<String,Double>marketShare){}publicrecordCompetitorAnalysisRequest(List<String>companies){}publicrecordCompetitorAnalysis(Map<String,List<String>>strengths){}publicrecordReportRequest(Stringtopic,List<String>companies){}publicrecordFinalReport(Stringtitle,List<String>sections){}@Action(description="Gather market data including revenues and market share")publicMarketDatagatherMarketData(MarketDataRequestrequest,Aiai){returnai.withDefaultLlm().createObject("Generate market data for: "+request.topic(),MarketData.class);}@Action(description="Analyze competitors: strengths and positioning")publicCompetitorAnalysisanalyzeCompetitors(CompetitorAnalysisRequestrequest,Aiai){returnai.withDefaultLlm().createObject("Analyze competitors: "+String.join(", ",request.companies()),CompetitorAnalysis.class);}@AchievesGoal(description="Compile all information into a final report")@Action(description="Compile the final report")publicFinalReportcompileReport(ReportRequestrequest,Aiai){returnai.withDefaultLlm().createObject("Create a market research report for "+request.topic(),FinalReport.class);}}
```

Tool actions have descriptions visible to the supervisor LLM

The goal action is called when the supervisor has gathered enough information

The supervisor LLM sees type schemas for available actions:

```
Available actions:
- gatherMarketData(request: MarketDataRequest) -> MarketData
  Schema: { revenues: Map, marketShare: Map }
- analyzeCompetitors(request: CompetitorAnalysisRequest) -> CompetitorAnalysis
  Schema: { strengths: Map }
```

Current artifacts on blackboard:

```
- MarketData: { revenues: {"CompanyA": "$10B"}, marketShare: {...} }
Goal: FinalReport
```

The LLM decides action ordering based on this information, making informed decisions without being constrained by declared dependencies.

Interoperability

Using wrapper request types (like MarketDataRequest) enables actions to work with **any planner**:

- **GOAP**: Request types flow through the blackboard based on preconditions/effects
- **Utility**: Actions fire when their request types are available with highest net value
- **Supervisor**: The LLM constructs request objects to call actions

This means you can switch planners without changing your action code—useful for testing with deterministic planners (GOAP) and deploying with flexible planners (Supervisor).

Comparison with LangGraph

[LangGraph’s supervisor pattern](#) is a popular approach for multi-agent orchestration. Here’s how a similar workflow looks in LangGraph vs Embabel:

LangGraph (Python)

```
fromlanggraph_supervisorimportcreate_supervisorfromlanggraph.prebuiltimportcreate_react_agent# Tools return strings - no type information
```

```
defgather_market_data(topic:str)->str:"""Gather market data for a topic."""returnf"Revenue data for {topic}..."defanalyze_competitors(companies:str)->str:"""Analyze competitors."""returnf"Analysis of {companies}..."# Create agents with tools
research_agent=create_react_agent(model="openai:gpt-4o",tools=[gather_market_data,analyze_competitors],name="research_expert",)# Supervisor sees all tools, always
workflow=create_supervisor([research_agent],model=model)app=workflow.compile()# State is a dict of messages
result=app.invoke({"messages":[{"role":"user","content":"Research cloud market"}]})
```

Tools return strings—the LLM must parse and interpret results

All tools always visible—no filtering based on context

State is untyped message history

Example 3. Embabel

```
JavaKotlin
@Agent(planner=PlannerType.SUPERVISOR)publicclassMarketResearchAgent{// Tools return typed objects with schemas
@Action(description="Gather market data for a
topic")publicMarketDatagatherMarketData(MarketDataRequestrequest,Aiai){returnai.withDefaultLlm().createObject("Generate
market data for "+request.topic(),MarketData.class);}@Action(description="Analyze
competitors")publicCompetitorAnalysisanalyzeCompetitors(CompetitorAnalysisRequestrequest,Aiai){returnai.withDefaultLlm(
).createObject("Analyze
"+request.companies(),CompetitorAnalysis.class);}@AchievesGoal@ActionpublicFinalReportcompileReport(ReportRequestreq
uest,Aiai){...}}// State is a typed blackboard // Tools are filtered based on available inputs
```

Tools return typed, validated objects--MarketData, CompetitorAnalysis

Blackboard holds typed artifacts, not just message strings

Tools with satisfied inputs are prioritized via currying

Key Advantages

Embabel’s Supervisor offers several advantages over typical supervisor implementations:

Aspect	Typical Supervisor (LangGraph)	Embabel Supervisor
Output Types	Strings—LLM must parse	Typed objects—validated and structured
Tool Visibility	All tools always available	Tools filtered by blackboard state (currying)
Domain Awareness	None—tools are opaque functions	Type schemas visible to LLM
Determinism	Fully non-deterministic	Semi-deterministic: tool availability constrained by types
State	Untyped message history	Typed blackboard with named artifacts

Blackboard-Driven Tool Filtering

A key differentiator is **curried tool filtering**. When an action’s inputs are already on the blackboard, those parameters are "curried out"--the tool signature simplifies.

What is Currying?

[Currying](#) is a functional programming technique where a function with multiple parameters is transformed into a sequence of functions, each taking a single parameter.

In Embabel’s context: if an action requires (MarketDataRequest, Ai) and MarketDataRequest is already on the blackboard, we "curry out" that parameter—the tool exposed to the LLM only needs to provide any remaining parameters. This simplifies the LLM’s task and signals which tools are "ready" to run.

```
# Initial state: empty blackboard
Available tools:
- gatherMarketData(request: MarketDataRequest) -> MarketData
- analyzeCompetitors(request: CompetitorAnalysisRequest) -> CompetitorAnalysis
# After MarketData is gathered:
Available tools:
- gatherMarketData(request: MarketDataRequest) -> MarketData [READY - 0 params needed]
- analyzeCompetitors(request: CompetitorAnalysisRequest) -> CompetitorAnalysis
```

This reduces the LLM’s decision space and guides it toward logical next steps—tools with satisfied inputs appear "ready" with fewer parameters. This is more deterministic than showing all tools equally, while remaining more flexible than GOAP’s strict ordering.

Semi-Determinism

While still LLM-orchestrated, Embabel’s Supervisor is **more deterministic** than typical implementations:

- Type constraints:** Actions can only produce specific types—no arbitrary string outputs
 - Input filtering:** Tools unavailable until their input types exist
 - Schema guidance:** LLM sees what each action produces, not just descriptions
 - Validated outputs:** Results must conform to declared types
- This makes debugging easier and behaviour more predictable, while retaining the flexibility that makes supervisor patterns valuable.

When Embabel’s Approach Excels

- Domain-rich workflows:** When your domain has clear types (reports, analyses, forecasts), schemas help the LLM understand relationships
- Multi-step synthesis:** When actions build on each other’s outputs, typed blackboard tracks progress clearly
- Hybrid determinism:** When you want more predictability than pure LLM orchestration but more flexibility than GOAP

SupervisorInvocation: Lightweight Supervisor Pattern

For simple supervisor workflows, you don’t need to create an @Agent class. SupervisorInvocation provides a fluent API to run supervisor-orchestrated workflows directly from @EmbabelComponent actions.

This is ideal when:

- You have a small set of related actions in an @EmbabelComponent
- You want LLM-orchestrated composition without creating a full agent
- You’re prototyping or exploring supervisor patterns before committing to a full agent design

Example: Meal Preparation Workflow

Here’s a complete example from the [embabel-agent-examples](#) repository:

Example 4. Stages - Actions as @EmbabelComponent

```
JavaKotlin
@EmbabelComponentpublicclassStages{publicrecordCook(Stringname,intage){}publicrecordOrder(Stringdish,intquantity){}publicrecordMeal(Stringdish,intquantity,StringorderedBy,StringcookedBy){}@ActionpublicCookchooseCook(UserInputuserInput,Aiai){returnai.withAutoLlm().createObject("""
```



```
From the following user input, choose a cook.
User input: %s

"""
    .formatted(userInput),Cook.class);}}@ActionpublicOrdertakeOrder(UserInputuserInput,Aiai){returnai.withAutoLlm().createObject(
    """
    From the following user input, take a food order
    User input: %s
    """
    .formatted(userInput),Order.class);}}@Action@AchievesGoal(description="Cook the meal according to the
order")publicMealprepareMeal(Cookcook,Orderorder,UserInputuserInput,Aiai){// The model will get the orderedBy from
UserInputreturnai.withAutoLlm().createObject(
    """
    Prepare a meal based on the cook and order details and target customer
    Cook: %s, age %d
    Order: %d x %s
    User input: %s
    """
    .formatted(cook.name(),cook.age(),order.quantity(),order.dish(),userInput.getContent()),Meal.class);}}
```

Example 5. Invoking with SupervisorInvocation

```
JavaKotlin
Stagesstages=newStages();Mealmeal=SupervisorInvocation.on(agentPlatform).returning(Stages.Meal.class).withScope(AgentScopeBuilder.fromInstance(stages)).invoke(newUserInput(request));
```

Configuration Options

SupervisorInvocation supports several configuration methods:

Method	Description
.returning(Class)	Specifies the goal type to produce
.withScope(AgentScopeBuilder)	Defines which actions are available
.withAgentName(String)	Sets a custom name for the created agent (defaults to {platformName}.supervisor)
.withGoalDescription(String)	Provides a custom description for the goal
.withProcessOptions(ProcessOptions)	Configures process-level options

Example 6. Setting a custom agent name

```
JavaKotlin
SupervisorInvocation.on(agentPlatform).returning(Report.class).withScope(AgentScopeBuilder.fromInstance(actions)).withAgentName("market-research-supervisor").invoke(request);
```

The supervisor LLM sees:

- 1. Available actions with their type signatures and schemas
 - 2. Current artifacts on the blackboard (including UserInput content)
 - 3. Goal to produce a Meal
- It then orchestrates the actions—calling chooseCook and takeOrder (possibly in parallel), then prepareMeal when the dependencies are satisfied.

Key Design Points

- 1. Actions use UserInput explicitly: Each action receives UserInput and includes it in the LLM prompt, ensuring the actual user request is used.
- 2. @AchievesGoal marks the target: The prepareMeal action is marked with @AchievesGoal to indicate it produces the final output.
- 3. Type-driven dependencies: prepareMeal requires Cook and Order, which guides the supervisor’s orchestration.

SupervisorInvocation vs @Agent with planner = SUPERVISOR

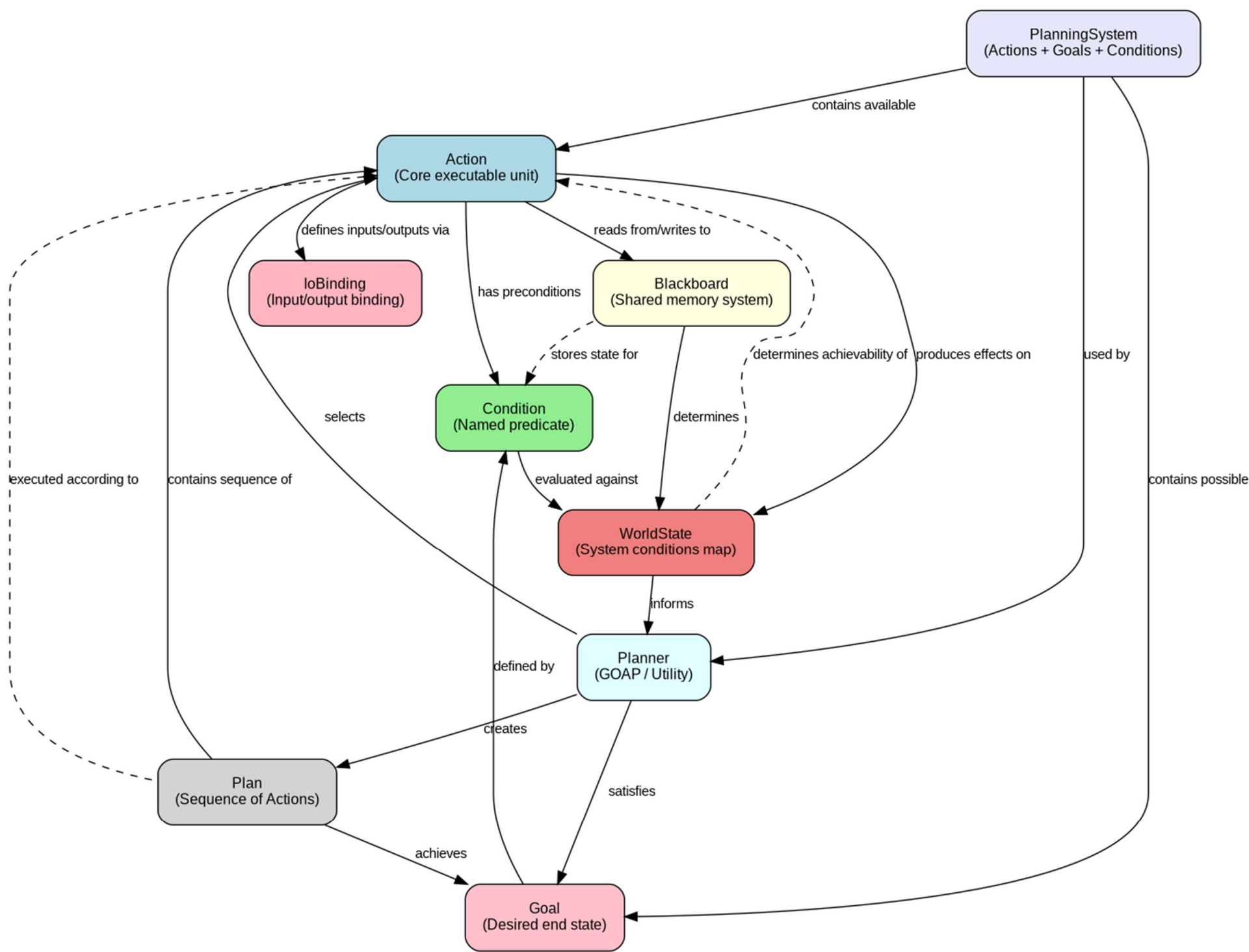
Aspect	SupervisorInvocation	@Agent(planner = SUPERVISOR)
Declaration	Fluent API, no class annotation	Annotated agent class
Action source	@EmbabelComponent or multiple components	Single @Agent class
Best for	Quick prototypes, simple workflows	Formalized, reusable agents
Goal specification	.returning(Class) fluent method	@AchievesGoal on action
Scope	Explicit via AgentScopeBuilder	Implicit from agent class

Comparison with AgenticTool

Both SupervisorInvocation and AgenticTool provide LLM-orchestrated composition, but at different levels:

Aspect	AgenticTool	SupervisorInvocation
Level	Tool (can be used within actions)	Invocation (runs a complete workflow)
Sub-components	Other Tool instances	@Action methods from @EmbabelComponent
Output	Tool.Result (text, artifact, or error)	Typed goal object (e.g., Meal)
State management	Minimal (LLM conversation only)	Full blackboard with typed artifacts
Type awareness	Tools have names and descriptions	Actions have typed inputs/outputs with schemas
Currying	None	Inputs on blackboard are curried out
Use case	Mini-orchestration within an action	Complete multi-step workflow with typed results

Use AgenticTool when you need a tool that internally orchestrates other tools. Use SupervisorInvocation when you need a complete workflow that produces a typed result with full blackboard state management.



3.20. API vs SPI

Embabel makes a clean distinction between its API and SPI. The API is the public interface that users interact with, while the SPI (Service Provider Interface) is intended for developers who want to extend or customize the behavior of Embabel, or platform providers. Application code should only depend on the API (`com.embabel.agent.api.*`) not the SPI. The SPI is subject to change and should not be used in production code.

3.21. Embabel and Spring

Embabel embraces [Spring](#). Spring was revolutionary when it arrived, and two decades on it still defines how most JVM applications are built. You may already know Spring from years of Java or Kotlin development. Or perhaps you're arriving from Python or another ecosystem. In any case it's worth noting that Embabel was spearheaded by the creator of Spring himself: the noteworthy Rod Johnson. Embabel has been assembled using the Spring core platform and then builds upon the [Spring AI](#) portfolio project. We recommend using [Spring Boot](#) for building Embabel applications. Not only does it provide a familiar environment for JVM developers, its philosophy is highly relevant for anyone aiming to craft a production-grade agentic AI application. Why? Because the foundation of the Spring framework is:

- Composability via discreet, fit-for-purpose reusable units. Dependency injection facilitates this.
- Cross-cutting abstractions — such as transaction management and security. Aspect-oriented programming (AOP) is what makes this work.

This same foundation makes it possible to craft agentic applications that are composable, testable, and built on enterprise-grade service abstractions. With ~70% of production applications deployed on the JVM, Embabel can bring AI super-powers to existing systems — extending their value rather than replacing them. In this way, Embabel applies the Spring philosophy so that agentic applications are not just clever demos, but truly production-ready systems.

3.22. Working with LLMs

Embabel supports any LLM supported by Spring AI. In practice, this is just about any LLM.

3.22.1. Choosing an LLM

Embabel encourages you to think about LLM choice for every LLM invocation. The PromptRunner interface makes this easy. Because Embabel enables you to break agentic flows up into multiple action steps, each step can use a smaller, focused prompt with fewer tools. This means it may be able to use a smaller LLM.

Considerations:

- **Consider the complexity of the return type you expect** from the LLM. This is typically a good proxy for determining required LLM quality. A small LLM is likely to struggle with a deeply nested return structure.
- **Consider the nature of the task.** LLMs have different strengths; review any available documentation. You don't necessarily need a huge, expensive model that is good at nearly everything, at the cost of your wallet and the environment.
- **Consider the sophistication of tool calling required.** Simple tool calls are fine, but complex orchestration is another indicator you'll need a strong LLM. (It may also be an indication that you should create a more sophisticated flow using Embabel GOAP.)
- **Consider trying a local LLM** running under Ollama or Docker.

Trial and error is your friend. Embabel makes it easy to switch LLMs; try the cheapest thing that could work and switch if it doesn't.

3.22.2. Advanced: Custom LLM Integration

Embabel's tool loop is framework-agnostic, allowing you to integrate any LLM provider by implementing the `LlmMessageSender` interface. This is useful when:

- You want to use an LLM provider not supported by Spring AI
- You need custom request/response handling
- You're integrating with a proprietary or internal LLM service

The LlmMessageSender Interface

The core abstraction is the LlmMessageSender functional interface:

JavaKotlin

```
@FunctionalInterfacepublic interface LlmMessageSender { LlmMessageResponse call(List<Message> messages, List<Tool> tools); }
```

The implementation makes a single LLM inference call and returns the response. Importantly, it does **not** execute tools—it only returns any tool call requests from the LLM. Tool execution is handled by Embabel’s DefaultToolLoop.

Even more advanced control over tools execution is available - tools can be executed in parallel, controlled by ParallelToolLoop. In order to activate ParallelToolLoop, please set the following parameter:

```
embabel.agent.platform.toolloop.type=parallel
```

For full list of tool loop configuration parameters please refer to ToolLoopConfiguration.

Response Types

The LlmMessageResponse contains:

- message: The LLM’s response as an Embabel Message
- textContent: Text content from the response
- usage: Optional token usage information

For responses that include tool calls, return an AssistantMessageWithToolCalls:

JavaKotlin

```
public record ToolCall(String id, // Unique identifier for the tool callString name, // Name of the tool to invokeString arguments, // JSON arguments for the tool) {}
```

Example: Custom LLM Provider

Here’s an example of implementing LlmMessageSender for a hypothetical HTTP-based LLM API:

JavaKotlin

```
public class MyCustomLlmMessageSender implements LlmMessageSender { private final HttpClient httpClient; private final String apiKey; private final String model; public MyCustomLlmMessageSender(HttpClient httpClient, String apiKey, String model) { this.httpClient = httpClient; this.apiKey = apiKey; this.model = model; } @Override public LlmMessageResponse call(List<Message> messages, List<Tool> tools) { // Convert Embabel messages to your API's format List<Map<String, Object>> apiMessages = messages.stream().map(message -> Map.<String, Object>.of("role", message.getRole().name().toLowerCase(), "content", message.getTextContent())).toList(); // Convert tool definitions to your API's format List<Map<String, Object>> apiTools = tools.stream().map(tool -> Map.<String, Object>.of("name", tool.getDefinition().getName(), "description", tool.getDefinition().getDescription(), "parameters", tool.getDefinition().getInputSchema().jsonSchema())).toList(); // Make API request (using your preferred HTTP client) MyApiResponse responseBody = httpClient.post("https://api.my-llm.com/chat").header("Authorization", "Bearer "+apiKey).body(Map.of("model", model, "messages", apiMessages, "tools", apiTools.isEmpty()?null:apiTools)).execute(MyApiResponse.class); // Check if LLM requested tool calls List<ToolCall> toolCalls = null; if (responseBody.getToolCalls() != null) { toolCalls = responseBody.getToolCalls().stream().map(call -> new ToolCall(call.getId(), call.getFunction().getName(), call.getFunction().getArguments())).toList(); } Message embabelMessage; if (toolCalls == null || toolCalls.isEmpty()) { embabelMessage = new AssistantMessage(responseBody.getContent() != null?responseBody.getContent(): ""); } else { embabelMessage = new AssistantMessageWithToolCalls(responseBody.getContent() != null?responseBody.getContent(): "", toolCalls); } Usage usage = null; if (responseBody.getUsage() != null) { usage = new Usage(responseBody.getUsage().getPromptTokens(), responseBody.getUsage().getCompletionTokens()); } return new LlmMessageResponse(embabelMessage, responseBody.getContent(), usage); }
```

Creating an LlmService

To make your custom LLM available through Embabel’s ModelProvider, implement the LlmService interface:

JavaKotlin

```
public class MyCustomLlmService implements LlmService<MyCustomLlmService> { private final String name; private final String provider; private final HttpClient httpClient; private final String apiKey; private final LocalDate knowledgeCutoffDate; private final List<PromptContributor> promptContributors; private final PricingModel pricingModel; public MyCustomLlmService(String name, String provider, HttpClient httpClient, String apiKey, LocalDate knowledgeCutoffDate, PricingModel pricingModel) { this.name = name; this.provider = provider; this.httpClient = httpClient; this.apiKey = apiKey; this.knowledgeCutoffDate = knowledgeCutoffDate; this.promptContributors = promptContributors; this.knowledgeCutoffDate != null?List.of(new KnowledgeCutoffDate(knowledgeCutoffDate)):List.of(); this.pricingModel = pricingModel; } @Override public String getName() { return name; } @Override public String getProvider() { return provider; } @Override public LocalDate getKnowledgeCutoffDate() { return knowledgeCutoffDate; } @Override public List<PromptContributor> getPromptContributors() { return promptContributors; } @Override public PricingModel getPricingModel() { return pricingModel; } @Override public LlmMessageSender createMessageSender(LlmOptions options) { return new MyCustomLlmMessageSender(httpClient, apiKey, options.getModel() != null?options.getModel():name); } @Override public MyCustomLlmService withKnowledgeCutoffDate(LocalDate date) { return new MyCustomLlmService(name, provider, httpClient, apiKey, date, pricingModel); } @Override public MyCustomLlmService withPromptContributor(PromptContributor promptContributor) { var newContributors = new ArrayList<>(promptContributors); newContributors.add(promptContributor); return new MyCustomLlmService(name, provider, httpClient, apiKey, knowledgeCutoffDate, newContributors, pricingModel); }
```

Then register it as a Spring bean:

JavaKotlin

```
@Configuration public class MyLlmConfiguration { @Bean public LlmService<?> myCustomLlm(HttpClient httpClient, @Value("${my-llm.api-key}") String apiKey) { return new MyCustomLlmService("my-custom-model", "MyProvider", httpClient, apiKey, LocalDate.of(2024, 12, 1), null); } }
```

The bean will be automatically discovered and made available through the ModelProvider.

How Model Discovery and Selection Works

When your application starts, ConfigurableModelProvider collects all LlmService beans from the Spring application context. Your custom LLM is matched by the name property you set on your LlmService implementation.

By name: Use the name from your LlmService directly. This works with @LlmCall, ai.withLlm(), and AgenticTool.withLlm():

JavaKotlin

```
// In a declarative action @LlmCall(llm="my-custom-model") String myAction(); // In an imperative action ai.withLlm("my-custom-model").create("Tell me a joke", String.class);
```

By role: Map a role name to your model name in configuration, then reference it with the # prefix:

```
embabel:models:default-llm:my-custom-model llms:best:my-custom-model cheapest:my-small-model
```

Sets the default LLM used when no explicit model is specified

Maps the best role to your custom model

Maps the cheapest role to a different model

Then reference roles with #:

JavaKotlin

```
// By role @LlmCall(llm="#best") String myAction(); // Or programmatically ai.withLlmByRole("best").create("Tell me a joke", String.class);
```

If no LLM is specified in `@LlmCall` or `withLlm()`, the default-llm from configuration is used.

Using Your Custom Implementation (Alternative)

If you need more control over the LLM operations layer itself, you can extend `ToolLoopLlmOperations`:

```
JavaKotlin
publicclassMyCustomLlmOperationsextendsToolLoopLlmOperations{privatefinalHttpClienthttpClient;privatefinalStringapiKey;
y;publicMyCustomLlmOperations(HttpClienthttpClient,StringapiKey,ModelProvidermodelProvider,ToolDecoratortoolDecorator,
Validatorvalidator){super(modelProvider,toolDecorator,validator);this.httpClient=httpClient;this.apiKey=apiKey;}@OverrideprotectedLlmMessageSendercreateMessageSender(LlmService<?>llm,LlmOptionsoptions){returnnewMyCustomLlmMessageSender
(httpClient,apiKey,options.getModel()!=null?options.getModel():"default-model");}}
```

The `ToolLoopLlmOperations` base class provides several extension points:

- `createMessageSender()`: Create the LLM communication layer
- `createOutputConverter()`: Parse LLM responses into typed objects
- `sanitizeStringOutput()`: Clean up raw text responses
- `emitCallEvent()`: Emit observability events

Key Implementation Notes

- Tool calls are not executed by your sender.** Just return the tool call requests—Embabel’s tool loop handles execution and continuation.
- Handle both tool and non-tool responses.** Return `AssistantMessage` for plain text, `AssistantMessageWithToolCalls` when the LLM wants to invoke tools.
- Include usage information when available.** This enables cost tracking and observability.
- Message types matter.** The tool loop expects specific message types:
 - `UserMessage`: User input
 - `SystemMessage`: System prompts
 - `AssistantMessage`: LLM text response
 - `AssistantMessageWithToolCalls`: LLM response with tool requests
 - `ToolResultMessage`: Result returned to LLM after tool execution

3.22.3. Advanced: Custom Embedding Service

Just as you can integrate a custom LLM, you can implement a custom embedding service that doesn’t depend on Spring AI. This is useful when:

- You want to use an embedding provider not supported by Spring AI
- You need custom pre/post-processing of embeddings
- You’re integrating with a proprietary or internal embedding API

The EmbeddingService Interface

The `EmbeddingService` interface is framework-agnostic. Unlike `SpringAiEmbeddingService`, a custom implementation does not need to wrap a Spring AI `EmbeddingModel`:

```
JavaKotlin
publicinterfaceEmbeddingService{float[]embed(Stringtext);List<float[]>embed(List<String>texts);intgetDimensions();Stringg
etName();StringgetProvider();}
```

Example: Custom Embedding Provider

Here’s an example of implementing `EmbeddingService` for an HTTP-based embedding API:

```
JavaKotlin
publicclassMyCustomEmbeddingServiceimplementsEmbeddingService{privatefinalStringname;privatefinalStringprovider;pri
vatefinalintdimensions;privatefinalHttpClienthttpClient;privatefinalStringapiKey;publicMyCustomEmbeddingService(Stringna
me,Stringprovider,intdimensions,HttpClienthttpClient,StringapiKey){this.name=name;this.provider=provider;this.dimensions=dim
ensions;this.httpClient=httpClient;this.apiKey=apiKey;}@OverridepublicStringgetName(){returnname;}@OverridepublicStringg
etProvider(){returnprovider;}@OverridepublicintgetDimensions(){returndimensions;}@Overridepublicfloat[]embed(Stringte
xt){returnembed(List.of(text)).get(0);}@OverridepublicList<float[]>embed(List<String>texts){MyEmbeddingResponserespons
e=httpClient.post("https://api.my-embeddings.com/embed").header("Authorization","Bearer"+apiKey).body(Map.of("texts",texts,"model",name)).execute(MyEmbeddingResponse.class);returnresponse.getEmbeddings();}}
```

Registering as a Spring Bean

Register your custom embedding service as a Spring bean and it will be automatically discovered:

```
JavaKotlin
@ConfigurationpublicclassMyEmbeddingConfiguration{@BeanpublicEmbeddingServicemyCustomEmbeddings(HttpClienthttp
pClient,@Value("${my-embeddings.api-key}")StringapiKey){returnnewMyCustomEmbeddingService("my-custom-
embeddings","MyProvider",384,httpClient,apiKey);}}
```

Discovery and Selection

Custom embedding services follow the same discovery and selection pattern as LLMs (see [How Model Discovery and Selection Works](#)).

By name: Use `ai.withEmbeddingService()` with the name from your implementation:

```
JavaKotlin
ai.withEmbeddingService("my-custom-embeddings").embed("Hello world");
```

By role: Map a role name to your embedding service in configuration:

```
embabel:models:default-embedding-model:my-custom-embeddingsembedding-services:cheapest:my-custom-embeddings
```

Sets the default embedding service

Maps the cheapest role to your custom embedding service

3.23. AWS Bedrock Integration

3.23.1. Add the Dependency

To use AWS Bedrock models, add the Bedrock autoconfiguration starter to your project:

MavenGradle (Kotlin)Gradle (Groovy)

```
<dependency><groupId>com.embabel.agent</groupId><artifactId>embabel-agent-bedrock-
autoconfigure</artifactId></dependency>
```

3.23.2. AWS Configuration

Configure AWS credentials and region using standard Spring AI Bedrock properties. See the [Spring AI Bedrock documentation](#) for credential configuration options.

3.23.3. Available Models

Chat Models (Claude)

Model Name	Model ID	Region	Knowledge Cutoff
us_claude_3_5_sonnet	us.anthropic.claude-3-5-sonnet-20240620-v1:0	US	2024-04-01
us_claude_3_5_sonnet_v2	us.anthropic.claude-3-5-sonnet-20241022-v2:0	US	2024-07-01
us_claude_3_5_haiku	us.anthropic.claude-3-5-haiku-20241022-v1:0	US	2024-07-01

us_claude_3_7_sonnet	us.anthropic.claude-3-7-sonnet-20250219-v1:0	US	2024-10-31
us_claude_sonnet_4	us.anthropic.claude-sonnet-4-20250514-v1:0	US	2025-03-01
us_claude_opus_4	us.anthropic.claude-opus-4-20250514-v1:0	US	2025-03-01
eu_claude_3_5_sonnet	eu.anthropic.claude-3-5-sonnet-20240620-v1:0	EU	2024-04-01
eu_claude_3_5_sonnet_v2	eu.anthropic.claude-3-5-sonnet-20241022-v2:0	EU	2024-07-01
eu_claude_3_5_haiku	eu.anthropic.claude-3-5-haiku-20241022-v1:0	EU	2024-07-01
eu_claude_3_7_sonnet	eu.anthropic.claude-3-7-sonnet-20250219-v1:0	EU	2024-10-31
eu_claude_sonnet_4	eu.anthropic.claude-sonnet-4-20250514-v1:0	EU	2025-03-01
eu_claude_opus_4	eu.anthropic.claude-opus-4-20250514-v1:0	EU	2025-03-01
apac_claude_3_5_sonnet	apac.anthropic.claude-3-5-sonnet-20240620-v1:0	APAC	2024-04-01
apac_claude_3_5_sonnet_v2	apac.anthropic.claude-3-5-sonnet-20241022-v2:0	APAC	2024-07-01
apac_claude_3_5_haiku	apac.anthropic.claude-3-5-haiku-20241022-v1:0	APAC	2024-07-01
apac_claude_3_7_sonnet	apac.anthropic.claude-3-7-sonnet-20250219-v1:0	APAC	2024-10-31
apac_claude_sonnet_4	apac.anthropic.claude-sonnet-4-20250514-v1:0	APAC	2025-03-01
apac_claude_opus_4	apac.anthropic.claude-opus-4-20250514-v1:0	APAC	2025-03-01

Embedding Models

Model Name	Model ID	Type
titan_embed_image_v1	amazon.titan-embed-image-v1	Titan
titan_embed_text_v1	amazon.titan-embed-text-v1	Titan
titan_embed_text_v2	amazon.titan-embed-text-v2:0	Titan
cohere_embed_multilingual_v3	cohere.embed-multilingual-v3	Cohere
cohere_embed_english_v3	cohere.embed-english-v3	Cohere

3.23.4. Configuration

Retry Configuration

embabel:agent:platform:models:bedrock:max-attempts:10# Default: 10backoff-millis:5000# Default: 5000backoff-multiplier:5.0# Default: 5.0backoff-max-interval:180000# Default: 180000

3.23.5. Adding New Models

To add new Bedrock models, edit the configuration file:

embabel-agent-autoconfigure/models/embabel-agent-bedrock-autoconfigure/
src/main/resources/models/bedrock-models.yml

Adding a Chat Model

models:-name:"us_claude_opus_5"model_id:"us.anthropic.claude-opus-5-20260101-v1:0"display_name:"Claude Opus 5 (US)"region:"us"knowledge_cutoff_date:"2025-10-01"pricing_model:usd_per1m_input_tokens:20.0usd_per1m_output_tokens:100.0

Adding an Embedding Model

embedding_models:-name:"titan_embed_v3"model_id:"amazon.titan-embed-v3"display_name:"Titan Embed V3"model_type:"titan"

Model type must be either titan or cohere.

3.23.6. See Also

- [Configuration Reference](#)
- [Spring AI Bedrock Integration](#)

3.24. Working with Streams

Developer can choose piping data from LLM gradually, using Embabel streaming capabilities.

In addition to streaming the raw text output from the LLM, Embabel streams can also include LLM reasoning events, so-called "thinking", and stream of objects created by the LLM. This feature is well aligned with Embabel focus on object-oriented programming model.

3.24.1. Concepts

- StreamingEvent - wraps Thinking or user Object
- StreamingPromptRunnerBuilder - runner with streaming capabilities
- Spring Reactive Programming Support for Spring AI ChatClient as underlying infrastructure
- All reactive callbacks, such as doOnNext, doOnComplete, etc. are at developer’s disposal

3.24.2. Example - Simple Thinking and Object Streaming with Callbacks

```
PromptRunnerrunner=ai.withLlm("qwen3:latest").withToolObject(Tooling.class);Stringprompt="What are exactly two the most hottest months in Florida and their respective highest temperatures";// Use StreamingPromptBuilder instead of Kotlin extensionfunctionFlux<StreamingEvent<MonthItem>>results=newStreamingPromptRunnerBuilder(runner).streaming().withPrompt(prompt).createObjectStreamWithThinking(MonthItem.class);// Subscribe with real reactive callbacks using builder patternresults.timeout(Duration.ofSeconds(150)).doOnSubscribe(subscription->{logger.info("Stream subscription started");}).doOnNext(event->{if(event.isThinking()){Stringcontent=event.getThinking();receivedEvents.add("THINKING:"+content);logger.info("Integration test received thinking: {0}",content);}elseif(event.isObject()){MonthItemobj=event.getObject();receivedEvents.add("OBJECT:"+obj.getName());logger.info("Integration test received object: {0}",obj.getName());}).doOnError(error->{errorOccurred.set(error);logger.error("Integration test stream error: {0}",error.getMessage());}).doOnComplete(()->{completionCalled.set(true);logger.info("Integration test stream completed successfully");}).blockLast(Duration.ofSeconds(6000));
```

3.24.3. Example - Simple Raw Text Streaming with Callbacks

```
PromptRunnerrunner=ai.withLlm("qwen3:latest");Stringprompt="What is the highest building in Paris?";// Use StreamingPromptBuilder instead of Kotlin extensionfunctionFlux<String>results=newStreamingPromptRunnerBuilder(runner).streaming().withPrompt(prompt).generateStream();// Subscribe with real reactive callbacks using builder patternresults.timeout(Duration.ofSeconds(150)).doOnSubscribe(subscription->{logger.info("Stream subscription started");}).doOnNext(content->{receivedTextChunks.add(content);logger.info("Integration test received text chunk: {0}",content);}).doOnError(error->{errorOccurred.set(error);logger.error("Integration test stream error:
```

```
{},error.getMessage());}).doOnComplete(()->{completionCalled.set(true);logger.info("Integration test stream completed successfully");}).blockLast(Duration.ofSeconds(6000));
```

3.25. Working with LLM Reasoning / Thinking

3.25.1. Motivation

Sometimes you need to validate an LLM’s reasoning process in addition to obtaining a structured result.

Consider this scenario: a user wants to plan a vacation and specifies that their preferred destinations are Greece and Italy, with available travel dates in June, August, or September. They ask the LLM to find flight options with affordable tickets for a one-week stay. The LLM returns a structured Flight object containing departure and return dates, destinations, and prices. Even if the output adheres to the expected schema, you may want to verify that:

- The flight dates fall within the requested months
- The destinations are actually in Greece or Italy rather than somewhere else

If the flight details fall outside the user’s criteria, access to the LLM’s reasoning process helps you understand why it made those choices.

An even more important use case arises when the LLM cannot fulfill a request—for example, when it cannot create the requested object because the user’s criteria are ambiguous or contradictory. In this case, the thinking blocks explain what went wrong, even though no result was produced.

3.25.2. Concepts

- **ThinkingBlock** — An abstraction that carries details about LLM reasoning, including the tag type, tag value, and reasoning text content.
- **ThinkingTagType** — An enum defining the types of reasoning markers: TAG (XML-style tags like <think>), PREFIX (line prefixes like //THINKING:), and NO_PREFIX (untagged reasoning text before JSON output).
- **ThinkingResponse<T>** — A response wrapper that holds both the result object and a list of ThinkingBlock instances.
- **ThinkingException** — An exception that preserves thinking blocks when object instantiation fails, enabling debugging even in error scenarios.
- **thinking()** — The core PromptRunner API method that enables thinking extraction.

3.25.3. Example: Handling Objects and Thinking Blocks

JavaKotlin

```
// Configure the PromptRunner with an LLM and optional toolsPromptRunnerrunner=ai.withLlm("claude-sonnet-4-5").withToolObject(Tooling.class);Stringprompt="""What is the hottest month in Florida and its average high temperature?Please provide a detailed analysis of your reasoning.""";// Use thinking() to enable thinking block
```

```
extractionThinkingResponse<MonthItem>response=runner.thinking().createObject(prompt,MonthItem.class);// Access the structured resultMonthItemresult=response.getResult();// Access the LLM's reasoning processList<ThinkingBlock>thinkingBlocks=response.getThinkingBlocks();// Inspect individual thinking blocksfor (ThinkingBlockblock:thinkingBlocks){System.out.println("Type: "+block.getTagType());// TAG, PREFIX, or NO_PREFIXSystem.out.println("Tag: "+block.getTagValue());// e.g., "think", "analysis"System.out.println("Content: "+block.getContent());}
```

3.25.4. Example: Handling Failures Gracefully

Use createObjectIfPossible when the LLM might not be able to produce a valid result:

JavaKotlin

```
ThinkingResponse<MonthItem>response=runner.thinking().createObjectIfPossible(prompt,MonthItem.class);MonthItemresult=response.getResult();if(result!=null){// Process the result}else{// Object creation failed—examine the reasoning to understand whyfor (ThinkingBlockblock:response.getThinkingBlocks()){logger.info("LLM reasoning: {}",block.getContent());}}
```

3.26. Working with Guardrails

Guardrails have become an essential component in agentic AI systems. They allow you to validate user inputs and LLM responses using configurable policies. For vendor-supported guardrails, see:

- [Guardrails Hub](#)
- [Amazon Bedrock Guardrails](#)

Embabel provides a framework for building custom guardrails, enabling developers to integrate validation logic of their choice.

3.26.1. Motivation

While you can validate user prompts or thinking blocks using custom validators, Embabel provides a standardized framework through the withGuardRails API. Guardrails can be implemented as POJOs or Spring beans that implement Embabel’s guardrail interfaces.

Common use cases for guardrails:

- **Input validation:** Validate user prompts with common, streaming, or thinking prompt runners
- **Response validation with thinking:** The thinking API (see [\[example simple streaming with callbacks \]](#)) provides access to LLM thinking blocks, even when the LLM cannot construct an object
- **Object response validation:** When the LLM constructs an object, you can still validate the output (the content being validated is the object’s JSON representation)
- **Streaming validation:** In streaming mode, StreamingEvent.Thinking provides direct access to LLM reasoning content via the doOnNext callback (see [Working with Streams](#))

A key benefit of this framework is access to the Blackboard object, which allows guardrail logic to consider other entities participating in the agentic workflow.

3.26.2. Concepts

- **UserInputGuardRail** and **AssistantMessageGuardRail** interfaces define guardrails for user inputs and LLM responses, respectively
- Guardrails are registered using the withGuardRails API, which can be chained
- Guardrail validation returns a ValidationResult object
- Validation errors are sorted by ValidationSeverity level and logged at the corresponding level
- A CRITICAL severity level causes a GuardRailViolationException to be thrown for user input guardrails, preventing the LLM operation from executing
- By design, createObjectIfPossible handles exceptions gracefully and completes without constructing an object; however, GuardRailViolationException is wrapped inside ThinkingResponse when using thinking mode

3.26.3. Customizing Message Combining

In multi-turn conversations, guardrails often need to validate not just a single prompt but an entire conversation history.

When doTransform or similar methods are called with multiple UserMessage objects, each UserInputGuardRail receives the full list and must combine them into a single string for validation.

The combineMessages method controls how this combination happens. Different guardrails may need different formats:

- A toxicity filter might want all messages concatenated to check the overall tone
- An audit guardrail might want each message tagged with its position in the conversation
- A PII detector might want clear separators to identify which message contains sensitive data

The default implementation joins messages with newlines:

JavaKotlin

```
defaultStringcombineMessages(List<UserMessage>userMessages){returnuserMessages.stream().map(UserMessage::getContent).collect(Collectors.joining("\n"));}
```

For example, three messages ["Hello", "How are you?", "Tell me about X"] become:

Hello

How are you?

Tell me about X

To customize this behavior, override combineMessages in your guardrail:

JavaKotlin

```
classAuditGuardRailimplementsUserInputGuardRail{@Overridepublic@NotNullStringgetName(){return"AuditGuard";}@Overridepublic@NotNullStringgetDescription(){return"Logs conversation with message markers for audit trail";}@Overridepublic@NotNullStringcombineMessages(@NotNullList<UserMessage>userMessages){// Tag each message with its position for audit loggingStringBuilderresult=newStringBuilder();for(inti=0;i<userMessages.size();i++){if(i>0){result.append("\n");}result.append("[Turn ").append(i+1).append("]:");result.append(userMessages.get(i).getContent());}returnresult.toString();}@Overridepublic@NotNullValidationResultvalidate(@NotNullStringinput,@NotNullBlackboardblackboard){// input now contains: "[Turn 1]: Hello\n[Turn 2]: How are you?\n[Turn 3]: Tell me about X"logger.info("Audit trail: {}",input);returnValidationResult.VALID;}}
```

3.26.4. Example: Blocking LLM Execution with CRITICAL Validation Errors

This example demonstrates how a guardrail with CRITICAL severity prevents LLM execution by throwing a GuardRailViolationException.

Step 1: Define the guardrails

First, define a user input guardrail that returns a CRITICAL validation error:

JavaKotlin

```
/**  
 * A guardrail that blocks execution by returning a CRITICAL validation error.
```

```
*/classCriticalUserInputGuardRailimplementsUserInputGuardRail{@Overridepublic@NotNullStringgetName(){return"CriticalUserInputGuardRail";}@Overridepublic@NotNullStringgetDescription(){return"Blocks execution when critical policy violations are detected";}@Overridepublic@NotNullValidationResultvalidate(@NotNullStringinput,@NotNullBlackboardblackboard){// Return a CRITICAL error to block LLM executionreturnnewValidationResult(true,List.of(newValidationError("policy-violation","Content violates safety policy",ValidationSeverity.CRITICAL))));}
```

Next, define an assistant message guardrail to validate LLM responses:

JavaKotlin

```
/**  
 * A guardrail that validates LLM thinking blocks.
```

```
*/classThinkingBlocksGuardRailimplementsAssistantMessageGuardRail{@Overridepublic@NotNullStringgetName(){return"ThinkingBlocksGuardRail";}@Overridepublic@NotNullStringgetDescription(){return"Validates LLM thinking blocks for compliance";}@Overridepublic@NotNullValidationResultvalidate(@NotNullThinkingResponse<?>response,@NotNullBlackboardblackboard){logger.info("Validating thinking blocks: {}",response.getThinkingBlocks());returnnewValidationResult(true,Collections.emptyList());}@Overridepublic@NotNullValidationResultvalidate(@NotNullStringinput,@NotNullBlackboardblackboard){returnnewValidationResult(true,Collections.emptyList());}}
```

Step 2: Use the guardrails with a PromptRunner

JavaKotlin

```
// Configure the PromptRunner with guardrailsPromptRunnerrunner=ai.withLlm("claude-sonnet-4-5").withToolObject(Tooling.class).withGenerateExamples(true).withGuardRails(newCriticalUserInputGuardRail(),newThinkingBlocksGuardRail());Stringprompt="""  
    What is the hottest month in Florida and provide its temperature.  
    The name should be the month name, temperature should be in Fahrenheit.  
    """;try{// Attempt to create an object with thinkingThinkingResponse<MonthItem>response=runner.thinking().createObject(prompt,MonthItem.class);}catch(GuardRailViolationExceptionex){// CRITICAL validation errors cause this exception to be thrown, preventing the LLM operation from executinglogger.error("Guardrail blocked execution: {}",ex.getMessage());}
```

3.26.5. Example: Using Guardrails for Response Analysis

When the LLM cannot construct an object (for example, when the prompt is ambiguous), guardrails can still analyze the LLM’s thinking process. This is useful for understanding why object creation failed or for extracting insights from the reasoning.

Step 1: Define a simple user input guardrail

JavaKotlin

```
/**  
 * A guardrail that logs user input with INFO-level validation messages.
```

```
*/classLoggingUserInputGuardRailimplementsUserInputGuardRail{@Overridepublic@NotNullStringgetName(){return"LoggingUserInputGuardRail";}@Overridepublic@NotNullStringgetDescription(){return"Logs user input for auditing purposes";}@Overridepublic@NotNullValidationResultvalidate(@NotNullStringinput,@NotNullBlackboardblackboard){logger.info("Processing user input: {}",input);// Return an INFO-level message (does not block execution)returnnewValidationResult(true,List.of(newValidationError("audit","Input logged",ValidationSeverity.INFO))));}
```

Step 2: Use guardrails with createObjectIfPossible

JavaKotlin

```
// Configure the PromptRunner with chained guardrailsPromptRunnerrunner=ai.withLlm("claude-sonnet-4-5").withToolObject(Tooling.class).withGuardRails(newLoggingUserInputGuardRail()).withGuardRails(newThinkingBlocksGuardRail());Stringprompt="Think about the coldest month in Alaska and its temperature. Provide your analysis.";// Use createObjectIfPossible to handle cases where object creation may failThinkingResponse<MonthItem>response=runner.thinking().createObjectIfPossible(prompt,MonthItem.class);// The LLM may not be able to construct an object if the prompt is ambiguousif(response.getResult()==null){// Analyze the thinking blocks to understand the LLM's reasoninglogger.info("Object creation not possible. Thinking blocks: {}",response.getThinkingBlocks());}
```

When the LLM cannot provide a definitive answer, you might see reasoning like:

Since I must be SURE about EVERY field and cannot make assumptions or provide approximate values,

I cannot provide the success structure with confidence.

Guardrails can automate further analysis of LLM responses, for example by using semantic text processing tools like [CoreNLP](#).

For more examples, see:

embabel-agent-autoconfigure/models/embabel-agent-anthropic-autoconfigure/
src/test/kotlin/com/embabel/agent/config/models/anthropic/LLMAnthropicThinkingIT.java

3.26.6. Relationship with Other Validation Mechanisms

The Agent API framework supports Jakarta Bean Validation (JSR-380) for domain object constraints. These constraints are injected into the schema and validated during object construction.

In addition, a planned validation framework for Agent Actions will reuse the same data structures as guardrails, including ValidationResult, ValidationError, and ContentValidator.

In summary, guardrails and bean validators are complementary but distinct:

- **Bean validation** ensures objects are well-formed and meet business constraints
- **Guardrails** ensure AI interactions are safe and compliant with policies

Both can be enabled independently and serve different aspects of the AI safety stack.

3.27. Customizing Embabel

3.27.1. Adding LLMs

You can add custom LLMs as Spring beans by implementing the LlmService interface. Embabel provides SpringAiLlmService for wrapping Spring AI ChatModel instances.

Using SpringAiLlmService

SpringAiLlmService implements the LlmService interface and provides framework-agnostic LLM operations including support for the Embabel tool loop and message sender abstraction.

JavaKotlin

```
@ConfigurationpublicclassLlmsConfig{@BeanpublicLlmService<?>myLlm(){org.springframework.ai.chat.model.ChatModelchatModel=...returnnewSpringAiLlmService("myChatModel","myChatModelProvider",chatModel).withOptionsConverter(newMyLlmOptionsConverter()).withKnowledgeCutoffDate(LocalDate.of(2025,4,1));}}
```

The name of the LLM (used for model selection).

The provider name, such as "OpenAI" or "Anthropic".

The Spring AI ChatModel instance.

Customize with an OptionsConverter implementation to convert Embabel LlmOptions to Spring AI ChatOptions.

Set the knowledge cutoff date if available.

LLM Configuration Options

SpringAiLlmService supports the following configuration:

- name (required)
- provider, such as "Mistral" (required)
- OptionsConverter to convert Embabel LlmOptions to Spring AI ChatOptions
- **knowledge cutoff date** (if available)
- any additional PromptContributor objects to be used in all LLM calls. If knowledge cutoff date is provided, add the KnowledgeCutoffDate prompt contributor.
- pricing model (if available)

A common requirement is to add an OpenAI-compatible LLM. This can be done by extending the OpenAiCompatibleModelFactory class as follows:

JavaKotlin

```
@ConfigurationpublicclassCustomOpenAiCompatibleModelsextendsOpenAiCompatibleModelFactory{publicCustomOpenAiCompatibleModels(@Value("${MY_BASE_URL:#{null}}")StringbaseUrl,@Value("${MY_API_KEY}")StringapiKey,ObservationRegistryobservationRegistry){super(baseUrl,apiKey,observationRegistry);}@BeanpublicLlmService<?>myGreatModel(){// Call superclass methodreturnopenAiCompatibleLlm("my-great-model","me",LocalDate.of(2025,1,1),newPerTokenPricingModel(0.40,1.6));}}
```

3.27.2. Adding embedding models

Embedding models can also be added as beans of the Embabel type EmbeddingService. Use the SpringAiEmbeddingService class to wrap a Spring AI EmbeddingModel.

Typically, this is done in an @Configuration class like this:

JavaKotlin

```
@ConfigurationpublicclassEmbeddingModelsConfig{@BeanpublicEmbeddingServicemyEmbeddingModel(){org.springframework.ai.embedding.EmbeddingModelembeddingModel=...returnnewSpringAiEmbeddingService("myEmbeddingModel","myEmbeddingModelProvider",embeddingModel);}}
```

3.27.3. Configuration via application.properties or application.yml

You can specify Spring configuration, your own configuration and Embabel configuration in the regular Spring configuration files.

Profile usage will work as expected.

3.27.4. Customizing logging

You can customize logging as in any Spring application.

For example, in application.properties you can set properties like:

```
logging.level.com.embabel.agent.a2a=DEBUG
```

You can also configure logging via a logback-spring.xml file if you have more sophisticated requirements.

See the [Spring Boot Logging](#) reference.

By default, many Embabel examples use personality-based logging experiences such as Star Wars. You can disable this by updating application.properties accordingly.

```
embabel.agent.logging.personality=severance
```

Remove the embabel.agent.logging.personality key to disable personality-based logging.

As all logging results from listening to events via an AgenticEventListener, you can also easily create your own customized logging.

3.28. Integrations

3.28.1. Model Context Protocol (MCP)

Publishing

Overview

Embabel Agent can expose your agents as MCP servers, making them available to external MCP clients such as Claude Desktop, VS Code extensions, or other MCP-compatible applications. The framework provides automatic publishing of agent goals as tools and prompts without requiring manual configuration.

Server Configuration

Configure MCP server functionality in your application.yml. The server type determines the execution mode:

```
spring:ai:mcp:server:type:SYNC# or ASYNC
```

Server Types

Embabel Agent supports two MCP server execution modes controlled by the spring.ai.mcp.server.type property:

SYNC Mode (Default)

- Blocking operations wrapped in reactive streams
- Simpler to develop and debug
- Suitable for most use cases
- Better error handling and logging

spring:ai:mcp:server:type:SYNC

ASYNC Mode

- True non-blocking reactive operations
- Higher throughput for concurrent requests
- More complex error handling
- Suitable for high-performance scenarios

spring:ai:mcp:server:type:ASYNC

Transport Protocol

Embabel Agent uses **SSE (Server-Sent Events) transport**, exposing your MCP server at <localhost:8080/sse>. This is compatible with Claude Desktop, MCP Inspector, Cursor, and most desktop MCP clients.

Clients requiring Streamable HTTP

Some clients (e.g., OpenWebUI) require Streamable HTTP transport instead of SSE. Use the mcpo proxy to bridge your SSE server:

uvx mcpo --port8000 --server-typesse --http://localhost:8080/sse

Then connect your client to <localhost:8000>.

Automatic Publishing

Tools

Agent goals are automatically published as MCP tools when annotated with `@Export(remote = true)`. The `PerGoalMcpToolExportCallbackPublisher` automatically discovers and exposes these goals without any additional configuration.

Prompts

Prompts are automatically generated for each goal’s starting input types through the `PerGoalStartingInputTypesPromptPublisher`. This provides ready-to-use prompt templates based on your agent definitions.

Exposing Agent Goals as Tools

Agent goals become MCP tools automatically when annotated with `@Export`:

JavaKotlin

```
@Agent(goal="Provide weather information",backstory="Weather service agent")publicclassWeatherAgent{@Goal@Export(remote=true)// Automatically becomes MCP toolpublicStringgetWeather(@Param("location")Stringlocation,@Param("units")Stringunits){return"Weather for "+location+" in "+units;}@GoalpublicStringinternalMethod(){// Not exposed to MCP (no @Export annotation)return"Internal use only";}}
```

Exposing Embabel ToolObject and LlmReference types as tools

A common requirement is to expose existing Embabel functionality via MCP. For example, an `LlmReference` might be added to a `PromptRunner` but might also be used as an external tool via MCP.

To do this, use `McpToolExport` to create a bean of type `McpToolExportCallbackPublisher`.

For example, to expose a `ToolishRag` LLM reference as an MCP tool, define a Spring configuration class as follows:

JavaKotlin

```
@ConfigurationpublicclassRagMcpTools{@BeanMcpToolExportragTools(SearchOperationssearchOperations){vartoolishRag=newToolishRag("docs","Embabel docs",searchOperations);returnMcpToolExport.fromLlmReference(toolishRag);}}
```

Your bean should be of type `McpToolExport`

Use `McpToolExport.fromLlmReference` to return the instance

Naming Strategies

When exporting tools, you can control how tool names are transformed using a naming strategy. This is useful for namespacing tools when exporting from multiple sources to avoid naming conflicts.

Using ToolObject with a naming strategy:

JavaKotlin

```
@BeanpublicMcpToolExportprefixedTools(){returnMcpToolExport.fromToolObject(newToolObject(List.of(myToolInstance),name->"myservice_"+name));}
```

All tool names will be prefixed with `myservice_`

Common naming strategies include:

- **Prefix:** `{ "namespace_${it}" }` - adds a prefix to avoid conflicts
- **Uppercase:** `{ it.uppercase() }` - converts to uppercase
- **Identity:** `StringTransformer.IDENTITY` - preserves original names (default)

LlmReference naming:

When using `fromLlmReference`, the reference’s built-in naming strategy is applied automatically. This prefixes tool names with the lowercased, normalized reference name. For example, an `LlmReference` named "MyAPI" will prefix all tools with `myapi_`.

JavaKotlin

```
// Reference named "WeatherService" will prefix tools with "weatherservice_"varreference=newMyWeatherReference();// name = "WeatherService"McpToolExport.fromLlmReference(reference);// Tool "getWeather" becomes "weatherservice_getWeather"
```

Exporting multiple sources with different prefixes:

JavaKotlin

```
@BeanpublicMcpToolExportmultiSourceTools(){returnMcpToolExport.fromToolObjects(List.of(newToolObject(List.of(weathersTools),name->"weather_"+name),newToolObject(List.of(stockTools),name->"stocks_"+name))));}
```

Filtering Tools

You can filter which tools are exported using the `filter` property on `ToolObject`:

JavaKotlin

```
@BeanpublicMcpToolExportfilteredTools(){returnMcpToolExport.fromToolObject(newToolObject(List.of(myToolInstance),StringTransformer.IDENTITY,name->name.startsWith("public_")));}
```

Only tools whose names start with `public_` will be exported

You can combine naming strategies and filters:

JavaKotlin

```
@BeanpublicMcpToolExportcombinedTools(){returnMcpToolExport.fromToolObject(newToolObject(List.of(myToolInstance),name->"api_"+name,name->!name.startsWith("internal")));}
```

The filter is applied to the original tool name before the naming strategy transforms it

Exposing Tools on Spring Components in Spring AI style

It is also possible to expose tools on Spring components as with regular Spring AI.

For example:

JavaKotlin

```
@ComponentpublicclassCalculatorTools{@McpTool(name="add",description="Add two numbers together")publicintadd(@McpToolParam(description="First number",required=true)inta,@McpToolParam(description="Second number",required=true)intb){returna+b;}@McpTool(name="multiply",description="Multiply two numbers together")publicintmultiply(@McpToolParam(description="First number",required=true)inta,@McpToolParam(description="Second number",required=true)intb){returna*b;}}
```

```
numbers")publicdoublemultiply(@McpToolParam(description="First number",required=true)doublex,@McpToolParam(description="Second number",required=true)doubley){returnx*y;}}
```

Of course, you can inject the Embabel Ai interface to help do the work of the tools if you wish, or invoke other agents from within the tool methods.

For further information, see the [Spring AI MCP Annotations Reference](#).

Server Architecture

The MCP server implementation uses several design patterns:

Template Method Pattern

- AbstractMcpServerConfiguration provides common initialization logic
- Concrete implementations (McpSyncServerConfiguration, McpAsyncServerConfiguration) handle mode-specific details

Strategy Pattern

- Server strategies abstract sync vs async operations
- Mode-specific implementations handle tool, resource, and prompt management

Publisher Pattern

- Tools, resources, and prompts are discovered through publisher interfaces
- Automatic registration and lifecycle management
- Event-driven initialization ensures proper timing

Built-in Tools

Every MCP server includes a built-in helloBanner tool that displays server information:

```
{
  "type": "banner",
  "mode": "SYNC",
  "lines": [
    "~~~~~",
    "Embabel Agent MCP SYNC Server",
    "Version: 0.3.0-SNAPSHOT",
    "Java: 21.0.2+13-LTS-58",
    "Started: 2025-01-17T14:23:47.785Z",
    "~~~~~"
  ]
}
```

Consuming

Embabel Agent can consume external MCP servers as tool sources, automatically organizing them into Tool Groups that agents can use.

Docker Tools Integration Configuration Approaches

Docker MCP Gateway (Recommended)

Uses Docker Desktop’s MCP Toolkit extension as a single gateway to multiple tools:

```
spring:ai:mcp:client:type:SYNCstdio:connections:docker-mcp:command:dockerargs:[mcp,gateway,run]
```

Individual Containers

Run each MCP server as a separate Docker container:

```
spring:ai:mcp:client:type:SYNCstdio:connections:brave-search-mcp:command:dockerargs:[run,-i,--rm,-e,BRAVE_API_KEY,mcp/brave-search]env:BRAVE_API_KEY:${BRAVE_API_KEY}
```

Available Tool Groups

Tool Groups are conditionally created based on configured MCP connections using @ConditionalOnMcpConnection:

Tool Group	Required Connections	Capabilities
Web Tools	brave-search-mcp, fetch-mcp, wikipedia-mcp, or docker-mcp	Web search, URL fetching, Wikipedia queries
Maps	google-maps-mcp or docker-mcp	Geocoding, directions, place search
Browser Automation	puppeteer-mcp or docker-mcp	Page navigation, screenshots, form interaction
GitHub	github-mcp or docker-mcp	Issues, pull requests, comments

How It Works

The @ConditionalOnMcpConnection annotation checks for configured connections at startup:

JavaKotlin

```
@Bean@ConditionalOnMcpConnection({"github-mcp","docker-mcp"})publicToolGroupgithubToolsGroup(){returnnewMcpToolGroup(CoreToolGroups.GITHUB_DESCRIPTION,"docker-github",mcpSyncClients,tool->tool.toolDefinition().name().contains("create_issue"));
```

Bean created if any listed connection is configured

Filter selects which MCP tools belong to this group

Custom Tool Groups

Define custom groups via configuration properties:

```
embabel:agent:platform:tools:includes:my-tools:description:"Custom tool collection"provider:"MyOrg"tools:-tool_name_suffix
```

3.28.2. A2A

3.28.3. Observability

Embabel Agent provides a unified observability module that automatically traces agent lifecycle, actions, LLM calls, tool invocations, and more — with zero code changes. It integrates with any OpenTelemetry-compatible backend (Zipkin, Langfuse, Jaeger, Prometheus, etc.).

Setup

Add the observability starter to your pom.xml:

```
<dependency><groupId>com.embabel.agent</groupId><artifactId>embabel-agent-starter-observability</artifactId><version>${embabel-agent.version}</version></dependency>
```

Then add an exporter dependency. For example, Zipkin:

```
<dependency><groupId>io.opentelemetry</groupId><artifactId>opentelemetry-exporter-zipkin</artifactId></dependency>
```

Or Langfuse for LLM-focused observability:

```
<dependency><groupId>com.quantpulsar</groupId><artifactId>opentelemetry-exporter-langfuse</artifactId><version>0.4.0</version></dependency>
```

You can use multiple exporters simultaneously (e.g., Langfuse for traces + Prometheus for metrics).

Configuration

Enable observability and configure your exporter in application.yml:

```
embabel:observability:enabled:trueservice-name:my-agent-appmanagement:tracing:enabled:truesampling:probability:1.0#Zipkinzipkin:tracing:endpoint:http://localhost:9411/api/v2/spans
```


For Langfuse:

management:langfuse:enabled:trueendpoint:https://cloud.langfuse.com/api/public/otel# or self-hosted URLpublic-key:pk-lf-...secret-key:sk-lf-...

What Gets Traced

All tracing is automatic once the module is on the classpath. The following events are captured as OpenTelemetry spans, organized in a parent-child hierarchy:

Agent: CustomerServiceAgent (trace root)
└─ planning:formulated [iteration=1, actions=3]
└─ Action: AnalyzeRequest
 └─ ChatModel: gpt-4 (Spring AI)
 └─ tool:searchKnowledgeBase
└─ Action: GenerateResponse
 └─ ChatModel: gpt-4 (Spring AI)
└─ goal:achieved [RequestProcessed]
└─ status: completed [duration=2340ms]

Tracing Configuration Properties

All tracing options are enabled by default and can be toggled individually:

Property	Default	Description
embabel.observability.enabled	true	Master switch for observability
embabel.observability.service-name	embabel-agent	Service name in traces
embabel.observability.trace-agent-events	true	Agent lifecycle (creation, execution, completion, failures)
embabel.observability.trace-tool-calls	true	Tool invocations with input/output
embabel.observability.trace-tool-loop	true	Tool loop execution
embabel.observability.trace-llm-calls	true	LLM calls with token usage
embabel.observability.trace-planning	true	Planning and replanning iterations
embabel.observability.trace-state-transitions	true	Workflow state changes
embabel.observability.trace-lifecycle-states	true	WAITING, PAUSED, STUCK states
embabel.observability.trace-rag	true	RAG events (request, response, pipeline)
embabel.observability.trace-ranking	true	Ranking/selection events (agent routing)
embabel.observability.trace-dynamic-agent-creation	true	Dynamic agent creation events
embabel.observability.trace-http-details	false	HTTP request/response details (bodies, headers)
embabel.observability.trace-tracked-operations	true	@Tracked annotation aspect
embabel.observability.mdc-propagation	true	Propagate agent context into SLF4J MDC
embabel.observability.metrics-enabled	true	Micrometer business metrics (counters, gauges)
embabel.observability.max-attribute-length	4000	Max span attribute length before truncation

Custom Operation Tracking with @Tracked

The @Tracked annotation lets you add observability spans to your own methods. Inputs, outputs, duration, and errors are captured automatically.

@Tracked("enrichCustomer")publicCustomerenrich(Customerinput){// Automatically creates a span with method arguments and return value}

You can specify a type and description for richer traces:

@Tracked(value="callPaymentApi",type=TrackType.EXTERNAL_CALL,description="Payment gateway call")publicPaymentResultprocessPayment(Orderorder){// ...}

Available track types:

Type	Description
CUSTOM	General-purpose (default)
PROCESSING	Data processing operation
VALIDATION	Validation or verification step
TRANSFORMATION	Data transformation
EXTERNAL_CALL	External service/API call
COMPUTATION	Computation or calculation

When called within an agent execution, @Tracked spans are automatically nested under the current action:

Agent: CustomerServiceAgent
└─ Action: ProcessOrder
 └─ @Tracked: enrichCustomer (PROCESSING)
 └─ ChatModel: gpt-4
 └─ @Tracked: callPaymentApi (EXTERNAL_CALL)
└─ status: completed

@Tracked uses Spring AOP proxies. Internal method calls within the same class are **not** intercepted. Extract tracked methods into a separate @Component bean for the annotation to work.

MDC Log Correlation

Agent context is automatically propagated into SLF4J MDC, enabling log filtering by agent run or action.

MDC keys set automatically:

MDC Key	Description	Set on	Removed on
embabel.agent.run_id	Agent process ID	Agent creation	Agent completed/failed/killed
embabel.agent.name	Agent name	Agent creation	Agent completed/failed/killed

embabel.action.name	Current action name	Action start	Action result
---------------------	---------------------	--------------	---------------

Example Logback pattern:
<pattern>%d{HH:mm:ss.SSS} [%thread] %-5level %logger{36} [runId=%X{embabel.agent.run_id} agent=%X{embabel.agent.name} action=%X{embabel.action.name}] - %msg%n</pattern>

This produces logs like:
14:23:45.123 [main] INFO c.e.MyService [runId=abc-123 agent=CustomerServiceAgent action=AnalyzeRequest] - Processing request

To disable MDC propagation:
embabel:observability:mdc-propagation:false

Supported Backends

Backend	Type	Module
Langfuse	Traces	opentelemetry-exporter-langfuse
Zipkin	Traces	opentelemetry-exporter-zipkin
OTLP (Jaeger, Tempo)	Traces	opentelemetry-exporter-otlp
Prometheus	Metrics	micrometer-registry-prometheus

For full details, see the [Observability Module Documentation](#).

3.29. Agent Skills

Agent Skills provide a standardized way to extend agent capabilities with reusable, shareable skill packages. Skills are loaded dynamically and provide instructions, resources, and tools to agents. Embabel implements the [Agent Skills Specification](#).

3.29.1. What are Agent Skills?

An Agent Skill is a directory containing a SKILL.md file with YAML frontmatter and markdown instructions. Skills can also include bundled resources:

- scripts/ - Executable scripts (Python, Bash, etc.)
- references/ - Documentation and reference materials
- assets/ - Static resources like templates and data files

Skills use a **lazy loading** pattern: only minimal metadata is included in the system prompt, with full instructions loaded when the skill is activated.

3.29.2. Using Skills with PromptRunner

The Skills class implements LlmReference, allowing it to be passed to a PromptRunner:

```
JavaKotlin
varskills=newSkills("financial-skills","Financial analysis
skills").withGitHubUrl("https://github.com/wshobson/agents/tree/main/plugins/business-
analytics/skills");varresponse=context.ai().withLlm(llm).withReference(skills).withSystemPrompt("You are a helpful financial
analyst.").respond(conversation.getMessages());
```

When skills are added as a reference, the agent can:

- See available skills in the system prompt
- Activate skills to get full instructions
- List and read skill resources

3.29.3. Loading Skills from GitHub

The simplest way to load skills is from a GitHub URL:

```
JavaKotlin
varskills=newSkills("my-skills","Skills for my agent").withGitHubUrl("https://github.com/anthropics/skills/tree/main/skills");
```

Supported URL formats:

- [github.com/owner/repo](#) - Load from repository root
- [github.com/owner/repo/tree/branch](#) - Specific branch
- [github.com/owner/repo/tree/branch/path/to/skills](#) - Specific path

For more control, use explicit parameters:

```
JavaKotlin
varskills=newSkills("my-skills","Skills for my agent").withGitHubSkills("anthropics","skills","skills","main");
```

3.29.4. Loading Skills from Local Directories

Load a single skill from a directory containing SKILL.md:

```
JavaKotlin
varskills=newSkills("my-skills","Local skills").withLocalSkill("/path/to/my-skill");
```

Load multiple skills from a parent directory:

```
JavaKotlin
varskills=newSkills("my-skills","Local skills").withLocalSkills("/path/to/skills-directory");
withLocalSkills scans immediate subdirectories only (depth 1). It does not recurse into nested directories.
```

3.29.5. Skill Directory Structure

A skill directory must contain a SKILL.md file:

```
my-skill/
├── SKILL.md      # Required - metadata and instructions
├── scripts/     # Optional - executable scripts
├── references/  # Optional - documentation
└── assets/     # Optional - static resources
```

The SKILL.md file uses YAML frontmatter:

```
---name:my-skilldescription:A skill that does something usefullicense:Apache-2.0compatibility:Requires Python 3.9+---# My Skill
InstructionsStep-by-step instructions for using this skill...
```

3.29.6. Skill Activation

Skills are activated lazily. The system prompt contains only minimal metadata (~50-100 tokens per skill). When an agent needs a skill, it calls the activate tool to load full instructions.

The Skills class exposes three LLM tools:

- activate(name) - Load full instructions for a skill
- listResources(skillName, resourceType) - List files in scripts/references/assets
- readResource(skillName, resourceType, fileName) - Read a resource file

3.29.7. Combining Skills with Other References

Skills can be combined with other LlmReference implementations:

JavaKotlin

```
varresponse=context.ai().withLlm(properties.chatLlm()).withReference(newLocalDirectory("./data/financial","Financial data files").withUsageNotes("Search to find files matching user requests.")).withReference(newSkills("analytics","Business analytics skills").withGitHubUrl("https://github.com/example/skills/tree/main/analytics")).withSystemPrompt("You are a financial analyst assistant. ").respond(conversation.getMessages());
```

3.29.8. Validation

Skills are validated when loaded:

- **Frontmatter validation** - Required fields (name, description) and field lengths
- **File reference validation** - Paths in instructions (e.g., scripts/build.sh) must exist
- **Name matching** - Skill name must match its parent directory name

To disable file reference validation:

JavaKotlin

```
varloader=newDefaultDirectorySkillDefinitionLoader(false);
```

3.29.9. Current Limitations

Script execution

Skills with scripts/ directories are loaded, but script execution is not yet supported. A warning is logged when such skills are loaded.

allowed-tools field

The allowed-tools frontmatter field is parsed but not currently enforced.

See the [Agent Skills Specification](#) for the full specification.

3.30. Testing

Like Spring, Embabel facilitates testing of user applications. The framework provides comprehensive testing support for both unit and integration testing scenarios.

Building Gen AI applications is no different from building other software. Testing is critical to delivering quality software and must be considered from the outset.

3.30.1. Unit Testing

Unit testing in Embabel enables testing individual agent actions without involving real LLM calls.

Embabel’s design means that agents are usually POJOs that can be instantiated with fake or mock objects. Actions are methods that can be called directly with test fixtures. In addition to your domain objects, you will pass a text fixture for the Embabel OperationContext, enabling you to intercept and verify LLM calls.

The framework provides FakePromptRunner and FakeOperationContext to mock LLM interactions while allowing you to verify prompts, hyperparameters, and business logic. Alternatively you can use mock objects. [Mockito](#) is the default choice for Java; [mockk](#) for Kotlin.

Testing Prompts and Hyperparameters

Here are unit tests from the [Java Agent Template](#) and [Kotlin Agent Template](#) repositories, using Embabel fake objects:

JavaKotlin

```
classWriteAndReviewAgentTest{@TestvoidtestWriteAndReviewAgent(){varcontext=FakeOperationContext.create();varpromptRunner=(FakePromptRunner)context.promptRunner();context.expectResponse(newStory("One upon a time Sir Galahad . . ."));varagent=newWriteAndReviewAgent(200,400);agent.craftStory(newUserInput("Tell me a story about a brave knight",Instant.now()),context);Stringprompt=promptRunner.getLlmInvocations().getFirst().getPrompt();assertTrue(prompt.contains("knight"),"Expected prompt to contain 'knight'");vartemp=promptRunner.getLlmInvocations().getFirst().getInteraction().getLlm().getTemperature();assertEquals(0.9,temp,0.01,"Expected temperature to be 0.9: Higher for more creative output");}@TestvoidtestReview(){varagent=newWriteAndReviewAgent(200,400);varuserInput=newUserInput("Tell me a story about a brave knight",Instant.now());varstory=newStory("Once upon a time, Sir Galahad...");varcontext=FakeOperationContext.create();context.expectResponse("A thrilling tale of bravery and adventure!");agent.reviewStory(userInput,story,context);varpromptRunner=(FakePromptRunner)context.promptRunner();Stringprompt=promptRunner.getLlmInvocations().getFirst().getPrompt();assertTrue(prompt.contains("knight"),"Expected review prompt to contain 'knight'");assertTrue(prompt.contains("review"),"Expected review prompt to contain 'review'");}}
```

Testing the Fluent API: withId() and creating()

The FakePromptRunner fully supports the fluent API patterns used in production code, enabling comprehensive unit testing of agents that use withId() for interaction tracing and creating() for structured object creation with examples.

Testing withId() for Interaction Tracing:

The withId() method sets an interaction ID for better log tracing. In tests, you can verify the interaction ID was correctly set:

JavaKotlin

```
@TestvoidshouldSetInteractionIdCorrectly(){varcontext=FakeOperationContext.create();varexpectedIntent=newUserIntent("command","Change channel names");context.expectResponse(expectedIntent);varresult=context.ai().withId("classify-intent");// Set interaction ID for tracing.creating(UserIntent.class).fromPrompt("Classify the user's intent");assertEquals(expectedIntent,result);// Verify the interaction ID was set correctlyvarinteraction=context.getLlmInvocations().getFirst().getInteraction();assertEquals("classify-intent",interaction.getId().getValue());}
```

Testing creating() with withExample():

The creating() API allows you to provide strongly-typed examples to improve LLM output quality. In tests, you can verify examples were included:

JavaKotlin

```
@TestvoidshouldIncludeExamplesInPrompt(){varcontext=FakeOperationContext.create();varexpectedPlan=newChannelEditPlan(1,"Lead Vox");context.expectResponse(expectedPlan);varresult=context.ai().withLlm(llmSelectionService.selectOptimalLlm()).withId("analyze-edit-request").creating(ChannelEditPlan.class).withExample("Rename channel 1",newChannelEditPlan(1,"Bass")).withExample("Rename channel 2",newChannelEditPlan(2,"Drums")).fromPrompt("Analyze the edit request");assertEquals(expectedPlan,result);// Verify examples were added as prompt contributorsvarpromptContributors=context.getLlmInvocations().getFirst().getInteraction().getPromptContributors();assertTrue(promptContributors.size()>=2,"Examples should be added as prompt contributors");}
```

Using CreationExample for Reusable Examples:

For cleaner code and reusability, you can use the CreationExample data class to define examples that can be shared across tests or passed as collections:

JavaKotlin

```
@TestvoidshouldUseCreationExampleDataClass(){varcontext=FakeOperationContext.create();varexpectedPlan=newChannelEditPlan(1,"Lead Vox");context.expectResponse(expectedPlan);// Create a reusable example using CreationExamplevarexample=newCreationExample<>("Rename channel example",newChannelEditPlan(2,"Rhythm"));varresult=context.ai().withDefaultLlm().creating(ChannelEditPlan.class).withExample(example)// Pass the CreationExample directly.fromPrompt("Analyze the edit request");assertEquals(expectedPlan,result);}
```

Adding Multiple Examples with withExamples():

When you have many examples to add, use `withExamples()` to pass them as a list or `vararg`. This is especially useful when examples are loaded from a file or database:

```
JavaKotlin
@TestvoidshouldAddMultipleExamplesFromList(){varcontext=FakeOperationContext.create();varexpectedPlan=newChannelEditPlan(1,"Lead Vox");context.expectResponse(expectedPlan);// Create a list of examples (could be loaded from configuration)varexamples=List.of(newCreationExample<>("Rename to Bass",newChannelEditPlan(1,"Bass")),newCreationExample<>("Rename to Drums",newChannelEditPlan(2,"Drums")),newCreationExample<>("Rename to Keys",newChannelEditPlan(3,"Keys")),newCreationExample<>("Rename to Vocals",newChannelEditPlan(4,"Vocals")));varresult=context.ai().withDefaultLlm().creating(ChannelEditPlan.class).withExamples(examples)// Pass all examples at once.fromPrompt("Analyze the request");assertEquals(expectedPlan,result);// Verify all examples were addedvarpromptContributors=context.getLlmInvocations().getFirst().getInteraction().getPromptContributors();assertTrue(promptContributors.size()>=4);}
```

You can also use `vararg` syntax for inline example lists:

```
JavaKotlin
varresult=context.ai().withDefaultLlm().creating(ChannelEditPlan.class).withExamples(newCreationExample<>("Example 1",newChannelEditPlan(1,"Bass")),newCreationExample<>("Example 2",newChannelEditPlan(2,"Drums")),newCreationExample<>("Example 3",newChannelEditPlan(3,"Keys"))).fromPrompt("Analyze the request");
```

Full Fluent API Chain Example:

Here’s a complete example showing how to test an action that uses all the fluent API features:

```
JavaKotlin
@TestvoidshouldTestCompleteFluentApiChain(){varcontext=FakeOperationContext.create();varexpectedOutput=newComplexOutput("analysis complete",42);context.expectResponse(expectedOutput);// Production code pattern with full fluent API chainvarresult=context.ai().withLlm(LlmOptions.withModel("gpt-4")).withId("complex-analysis").withSystemPrompt("You are an expert analyst").creating(ComplexOutput.class).withExample("Simple case",newComplexOutput("basic",1)).withExample("Complex case",newComplexOutput("advanced",100)).fromPrompt("Analyze the input data");assertEquals(expectedOutput,result);// Comprehensive verificationvarinvocation=context.getLlmInvocations().getFirst();assertEquals("gpt-4",invocation.getInteraction().getLlm().getModel());assertEquals("complex-analysis",invocation.getInteraction().getId().getValue());assertTrue(invocation.getInteraction().getPromptContributors().size()>=3); // system + 2 examples}
```

Key Testing Patterns Demonstrated

Testing Prompt Content:

- Use `context.getLlmInvocations().getFirst().getPrompt()` to get the actual prompt sent to the LLM
- Verify that key domain data is properly included in the prompt using `assertTrue(prompt.contains(...))`

Testing Tools:

- Access tools via `getInteraction().getTools()` to verify tools added via `withToolObject()` or `withTool()`
- Access tool group requirements via `getInteraction().getToolGroups()` to verify named tool group requirements added via `withToolGroup(ToolGroupRequirement)`

`getTools()` returns actual Tool instances (from `withToolObject()` and `withTool()`), while `getToolGroups()` returns ToolGroupRequirement objects (named requirements like "web_search" added via `withToolGroup()`). Most tests should use `getTools()`.

Testing with Spring Dependencies:

- Mock Spring-injected services like `HoroscopeService` using standard mocking frameworks - Pass mocked dependencies to agent constructor for isolated unit testing

Testing Multiple LLM Interactions

```
JavaKotlin
@TestvoidshouldHandleMultipleLlmInteractions(){// Arrangevarinput=newUserInput("Write about space exploration");varstory=newStory("The astronaut gazed at Earth...");ReviewedStoryreview=newReviewedStory("Compelling narrative with vivid imagery.");// Set up expected responses in ordercontext.expectResponse(story);context.expectResponse(review);// ActvarwrittenStory=agent.writeStory(input,context);ReviewedStoryreviewedStory=agent.reviewStory(writtenStory,context);// AssertassertEquals(story,writtenStory);assertEquals(review,reviewedStory);// Verify both LLM calls were madeList<LlmInvocation>invocations=context.getLlmInvocations();assertEquals(2,invocations.size());// Verify first call (writer)varwriterCall=invocations.get(0);assertEquals(0.8,writerCall.getInteraction().getLlm().getTemperature(),0.01);// Verify second call (reviewer)varreviewerCall=invocations.get(1);assertEquals(0.2,reviewerCall.getInteraction().getLlm().getTemperature(),0.01);}
```

You can also use Mockito or mockk directly. Consider this component, using direct injection of Ai:

```
JavaKotlin
@ComponentpublicrecordInjectedComponent(Aiai){publicrecordJoke(Stringleadup,Stringpunchline){}publicStringtellJokeAbout(Stringtopic){returnai.withDefaultLlm().generateText("Tell me a joke about "+topic);}}
```

A unit test using Mockito (Java) or mockk (Kotlin) to verify prompt and hyperparameters:

```
JavaKotlin
classInjectedComponentTest{@TestvoidtestTellJokeAbout(){varmockAi=Mockito.mock(Ai.class);varmockPromptRunner=Mockito.mock(PromptRunner.class);varprompt="Tell me a joke about frogs";// Yep, an LLM came up with this joke.varterribleJoke=""
    Why don't frogs ever pay for drinks?
    Because they always have a tadpole in their wallet!

"";when(mockAi.withDefaultLlm()).thenReturn(mockPromptRunner);when(mockPromptRunner.generateText(prompt)).thenReturn(terribleJoke);varinjectedComponent=newInjectedComponent(mockAi);varjoke=injectedComponent.tellJokeAbout("frogs");assertEquals(terribleJoke,joke);Mockito.verify(mockAi).withDefaultLlm();Mockito.verify(mockPromptRunner).generateText(prompt);}}
```

3.30.2. Integration Testing

Integration testing exercises complete agent workflows with real or mock external services while still avoiding actual LLM calls for predictability and speed.

This can ensure:

- Agents are picked up by the agent platform
- Data flow is correct within agents
- Failure scenarios are handled gracefully
- Agents interact correctly with each other and external systems
- The overall workflow behaves as expected
- LLM prompts and hyperparameters are correctly configured

Embabel integration testing is built on top of [Spring’s excellent integration testing support](#), thus allowing you to work with real databases if you wish. Spring’s [integration with Testcontainers](#) is particularly useful.

Using EmbabelMockitoIntegrationTest

Embabel provides EmbabelMockitoIntegrationTest as a base class that simplifies integration testing with convenient helper methods:

JavaKotlin

```
/**
 * Use framework superclass to test the complete workflow of writing and reviewing a story.
 * This will run under Spring Boot against an AgentPlatform instance
 * that has loaded all our agents.
 */

*/classStoryWriterIntegrationTestextendsEmbabelMockitoIntegrationTest{@TestvoidshouldExecuteCompleteWorkflow(){varinput=newUserInput("Write about artificial intelligence");varstory=newStory("AI will transform our world...");varreviewedStory=newReviewedStory(story,"Excellent exploration of AI themes.",Personas.REVIEWER);whenCreateObject(contains("Craft a short story"),Story.class).thenReturn(story);// The second call uses generateTextwhenGenerateText(contains("You will be given a short story to review")),thenReturn(reviewedStory.review());varinvocation=AgentInvocation.create(agentPlatform,ReviewedStory.class);varreviewedStoryResult=invocation.invoke(input);assertNotNull(reviewedStoryResult);assertTrue(reviewedStoryResult.getContent().contains(story.text()),"Expected story content to be present:"+reviewedStoryResult.getContent());assertEquals(reviewedStory,reviewedStoryResult,"Expected review to match:"+reviewedStoryResult);verifyCreateObjectMatching(prompt->prompt.contains("Craft a short story"),Story.class,llm->llm.getLlm().getTemperature()==0.9&&llm.getToolGroups().isEmpty());verifyGenerateTextMatching(prompt->prompt.contains("You will be given a short story to review"));verifyNoMoreInteractions();}}
```

Key Integration Testing Features

Base Class Benefits: - EmbabelMockitoIntegrationTest handles Spring Boot setup and LLM mocking automatically - Provides agentPlatform and llmOperations pre-configured - Includes helper methods for common testing patterns

Convenient Stubbing Methods: - whenCreateObject(prompt, outputClass): Mock object creation calls - whenGenerateText(prompt): Mock text generation calls - Support for both exact prompts and contains() matching

Advanced Verification: - verifyCreateObjectMatching(): Verify prompts with custom matchers - verifyGenerateTextMatching(): Verify text generation calls - verifyNoMoreInteractions(): Ensure no unexpected LLM calls

LLM Configuration Testing: - Verify temperature settings: llm.getLlm().getTemperature() == 0.9 - Check tool groups: llm.getToolGroups().isEmpty() - Validate persona and other LLM options

3.31. Embabel Architecture

Please refer to architectural diagrams in sections:

[Choosing a Planner](#)

[The AgentPlatform](#)

[The AgentProcess](#)

3.32. Troubleshooting

This section covers common issues you might encounter when developing with Embabel and provides practical solutions.

3.32.1. Common Problems and Solutions

Problem	Solution	Related Docs
Compilation Error	Check that you’re using the correct version of Embabel in your Maven or Gradle dependencies. You may be using an API from a later version (even a snapshot). Version mismatches between different Embabel modules can cause compilation issues. Ensure all com.embabel.agent artifacts use the same version, unless you’re following a specific example that does otherwise.	Configuration
Don’t Know How to Invoke Your Agent	Look at examples of processing UserInput in the documentation. Study AgentInvocation patterns to understand how to trigger your agent flows. The key is understanding how to provide the initial input that your agent expects.	Invoking Agents
Agent Flow Not Completing	This usually indicates a data flow problem. First, understand Embabel’s type-driven data flow concepts - review how input/output types create dependencies between actions. Then write an integration test to verify your flow works end-to-end. Familiarize yourself with Embabel’s GOAP planning concept.	Data Flow Concepts
LLM Prompts Look Wrong or Have Incorrect Hyperparameters	Write unit tests to capture and verify the exact prompts being sent to your LLM. This allows you to see the actual prompt content and tune temperature, model selection, and other parameters. Unit testing is the best way to debug LLM interactions.	Testing
Agent Gets Stuck in Planning	Check that all your actions have clear input/output type signatures. Missing or circular dependencies in your type flow can prevent the planner from finding a valid path to the goal. Review your @Action method signatures. Look at the log output from the planner for clues. Set your ProcessOptions.verbosity to show planning.	Type-Driven Flow
Tools Not Available to Agent	Ensure you’ve specified tools using withToolGroup() or withTools() on your PromptRunner. Tools must be explicitly added to each LLM call that needs them. Check that required tool groups are available in your environment.	Tools
Agent Runs But Produces Poor Results	Review your prompt engineering and persona configuration. Consider adjusting LLM temperature, model selection, and context provided to actions. Write tests to capture actual vs expected outputs.	Testing , LLM Configuration
You’re Struggling to Express What You Want in a Plan	Familiarize yourself with custom conditions for complex flow control. For common behavior patterns, consider using atomic actions with Embabel’s typesafe custom builders such as ScatterGatherBuilder and RepeatUntilBuilder instead of trying to express everything through individual actions.	DSL and Builders
Custom conditions not working as expected	Remember that you must declare post conditions on @Action methods that may set your custom condition, as well as pre conditions on actions that depend on them. Otherwise the planner will assume that the conditions are never set and your plan will not execute as expected.	Type-Driven Flow
Your Agent Has No Goals and Cannot Execute	Look at the @AchievesGoal annotation and ensure your terminal action is annotated with it. Every agent needs at least one action marked with @AchievesGoal to define what constitutes completion of the agent’s work.	Annotations
Your Agent Isn’t Visible to an MCP Client Like Claude Desktop	Ensure that your @AchievesGoal annotation includes @Export(remote=true). This makes your agent available for remote invocation through MCP (Model Context Protocol) clients.	Annotations , Integrations

Your Agent Can’t Use Upstream MCP Tools and You’re Seeing Errors in Logs About Possible Misconfiguration	Check that your Docker configuration is correct if using the default Docker MCP Gateway. Verify that Docker containers are running and accessible. For other MCP configurations, ensure your Spring AI MCP client configuration is correct. See the Spring AI MCP client documentation for detailed setup instructions.	Spring AI MCP Client
--	---	--------------------------------------

3.32.2. Debugging Strategies

Enable Debug Logging

Customize Embabel logging in application.yml or application.properties to see detailed agent execution. For example: `logging:level:com.embabel.agent:DEBUG`

3.32.3. Getting Help

The Embabel community is active and helpful. Join our [Discord](#) server to ask questions and share experiences.

3.33. Migrating from other frameworks

Many people start their journey with Python frameworks.

This section covers how to migrate from popular frameworks when it’s time to use a more robust and secure platform with access to existing code and services.

3.33.1. Migrating from CrewAI

CrewAI uses a collaborative multi-agent approach where agents work together on tasks. Embabel provides similar capabilities with stronger type safety and better integration with existing Java/Kotlin codebases.

Core Concept Mapping

CrewAI Concept	Embabel Equivalent	Notes
Agent Role/Goal/Backstory	RoleGoalBackstory PromptContributor	Convenience class for agent personality
Sequential Tasks	Typed data flow between actions	Type-driven execution with automatic planning
Crew (Multi-agent coordination)	Actions with shared PromptContributors	Agents can adopt personalities as needed
YAML Configuration	Standard Spring @ConfigurationProperties backed by application.yml or profile-specific configuration files	Type-safe configuration with validation

Migration Example

CrewAI Pattern:

```
research_agent=Agent(role='Research Specialist',goal='Find comprehensive information',backstory='Expert researcher with 10+ years experience')writer_agent=Agent(role='Content Writer',goal='Create engaging content',backstory='Professional writer specializing in technical content')crew=Crew(agents=[research_agent,writer_agent],tasks=[research_task,write_task],process=Process.sequential)
```

Embabel Equivalent:

```
@ConfigurationProperties("examples.book-writer")recordBookWriterConfig(LlmOptionsresearcherLlm,LlmOptionswriterLlm,RoleGoalBackstoryresearcher,RoleGoalBackstorywriter){}@Agent(description="Write a book by researching, outlining, and writing chapters")publicrecordBookWriter(BookWriterConfigconfig){@ActionResearchReportresearchTopic(BookRequestrequest,OperationContextcontext){returncontext.ai().withLlm(config.researcherLlm()).withPromptElements(config.researcher(),request).withToolGroup(CoreToolGroups.WEB).createObject("Research the topic thoroughly...",ResearchReport.class);}@ActionBookOutlinecreateOutline(BookRequestrequest,ResearchReportresearch,OperationContextcontext){returncontext.ai().withLlm(config.writerLlm()).withPromptElements(config.writer(),request,research).createObject("Create a book outline...",BookOutline.class);}@AchievesGoal(export=@Export(remote=true))@ActionBookwriteBook(BookRequestrequest,BookOutlineoutline,OperationContextcontext){// Parallel chapter writing with crew-like coordinationvarchapters=context.parallelMap(outline.chapterOutlines(),config.maxConcurrency(),chapterOutline->writeChapter(request,outline,chapterOutline,context));returnnewBook(request,outline.title(),chapters);}}
```

Key Advantages:

- **Type Safety:** Compile-time validation of data flow
- **Spring Integration:** Leverage existing enterprise infrastructure
- **Automatic Planning:** GOAP planner handles task sequencing, and is capable of more sophisticated planning
- **Tool Integration with the JVM:** Native access to existing Java/Kotlin services

3.33.2. Migrating from Pydantic AI

Pydantic AI provides a Python framework for building AI agents with type safety and validation. Embabel offers similar capabilities in the JVM ecosystem with stronger integration into enterprise environments.

Core Concept Mapping

Pydantic AI Concept	Embabel Equivalent	Notes
@system_prompt decorator	PromptContributor classes	More flexible and composable prompt management
@tool decorator	Equivalent @Tool annotated methods can be included on agent classes and domain objects	Agent class
@Agent annotated record/class	Declarative agent definition with Spring integration	RunContext
Blackboard state, accessible via OperationContext but normally not a concern for user code	SystemPrompt	Custom PromptContributor
Structured prompt contribution system	deps parameter	Spring dependency injection

Migration Example

Pydantic AI Pattern:

```
# Based on https://ai.pydantic.dev/examples/bank-support/frompydantic_aiimportAgent,RunContextfrompydantic_ai.toolsimporttool@system_promptdefsupport_prompt()->str:return"You are a support agent in our bank"@toolasyncdefget_customer_balance(customer_id:int,include_pending:bool=False)->float:# Database lookupcustomer=find_customer(customer_id)returncustomer.balance+(customer.pendingifinclude_pendingelse0)agent=Agent('openai:gpt-4-mini',system_prompt=support_prompt,tools=[get_customer_balance],)result=agent.run("What's my balance?",deps={'customer_id':123})
```


Embabel Equivalent:

```
// From embabel-agent-examples/examples-
java/src/main/java/com/embabel/example/pydantic/banksupport/SupportAgent.javarecordCustomer(Longid,Stringname,floatbalan
ce,floatpendingAmount){@Tool(description="Find the balance of a customer by
id")floatbalance(booleanincludePending){returnincludePending?balance+pendingAmount:balance;}}recordSupportInput(@JsonP
ropertyDescription("Customer ID")LongcustomerId,@JsonPropertyDescription("Query from the
customer")Stringquery){}recordSupportOutput(@JsonPropertyDescription("Advice returned to the
customer")Stringadvice,@JsonPropertyDescription("Whether to block their card or
not")booleanblockCard,@JsonPropertyDescription("Risk level of query")intrisk){}@Agent(description="Customer support
agent")recordSupportAgent(CustomerRepositorycustomerRepository){@AchievesGoal(description="Help bank customer with
their
query")@ActionSupportOutputsupportCustomer(SupportInputsupportInput,OperationContextcontext){varcustomer=customerR
epository.findById(supportInput.customerId());if(customer==null){returnnewSupportOutput("Customer not found with this
id",false,0);}returncontext.ai().withLlm(OpenAiModels.GPT_41_MINI).withToolObject(customer).createObject("""
    You are a support agent in our bank, give the
    customer support and judge the risk level of their query.
    In some cases, you may need to block their card. In this case, explain why.
    Reply using the customer's name, "%s".
    Currencies are in $.
    Their query: [%s]
    """).formatted(customer.name(),supportInput.query()),SupportOutput.class);}}}
```

Key Advantages:

- **Enterprise Integration:** Native Spring Boot integration with existing services
- **Compile-time Safety:** Strong typing catches errors at build time
- **Automatic Planning:** GOAP planner handles complex multi-step operations
- **JVM Ecosystem:** Access to mature libraries and enterprise infrastructure

3.33.3. Migrating from LangGraph

LangGraph builds agent workflows using a state machine. See the blog [Build Better Agents in Java vs Python: Embabel vs LangGraph](#) for a detailed comparison of common patterns between LangGraph and Embabel.

3.33.4. Migrating from Google ADK

tbd

3.34. API Evolution

We strive to avoid breaking changes. Because Embabel builds on Spring’s POJO support, framework code dependencies are localized and minimized. The key core surface area is the Ai and PromptRunner interfaces, which we will strive to avoid breaking. For maximum stability:

- If possible, use the latest stable version of embabel-agent rather than a snapshot. Snapshots may change frequently.
- Avoid using types under the com.embabel.agent.experimental package.
- Be aware of the status of the Embabel modules you use
- Avoid using any method or class marked with the @ApiStatus.Experimental or @ApiStatus.Internal annotations. application code should not depend on any types under the com.embabel.agent.spi package. This is intended for provision of runtime infrastructure only, and may change without notice.

4. Design Considerations

Embabel is designed to give you the ability to determine the correct balance between LLM autonomy and control from code. This section discusses the design considerations that you can use to achieve this balance.

4.1. Domain objects

A rich domain model helps build a good agentic system. Domain objects should not merely contain state, but also expose behavior. Avoid the [anemic domain model](#). Domain objects have multiple roles:

1. *Ensuring type safety and toolability.* Code can access their state; prompts will be strongly typed; and LLMs know what to return.
2. *Exposing behavior to call in code,* exactly as in any well-designed object-oriented system.
3. *Exposing tools to LLMs,* allowing them to call domain objects.

The third role is novel in the context of LLMs and Embabel. When designing your domain objects, consider which methods should be callable by LLMs and which should not. Expose methods that LLMs should be able to call using the @Tool annotation:

```
JavaKotlin
@Tool(description="Build the project using the given command in the
root")publicStringbuild(Stringcommand){BuildResultbr=ci.buildAndParse(newBuildOptions(command,true));returnbr.relevant
Output();}
```

The Spring AI @Tool annotation indicates that this method is callable by LLMs. When an @Action method issues a prompt, tool methods on all domain objects are available to the LLM. You can also add additional tool methods with the withToolObjects method on PromptRunner. Domain objects may or may not be persistent. If persistent, they will likely be stored in a familiar JVM technology such as JPA or JDBC. We advocate the use of [Spring Data](#) patterns and repositories, although you are free to use any persistence technology you like.

4.2. Tool Call Choice

When to use MCP or other tools versus method calls in agents

4.3. Mixing LLMs

It’s good practice to use multiple LLMs in your agentic system. Embabel makes it easy. One key benefit of breaking functionality into smaller actions is that you can use different LLMs for different actions, depending on their strengths and weaknesses. You can also the cheapest (greenest) possible LLM for a given task.

5. Contributing

Open source is a wonderful thing. We welcome contributions to the Embabel project. How to contribute:

- Familiarize yourself with the project by reading the documentation.
- Familiarize yourself with the [issue tracker](#) and open pull requests to ensure you’re not duplicating something.
- [Sign your commits](#)
- Always include a description with your pull requests. **PRs without descriptions will be closed.**
- Join the Embabel community on Discord at [discord.gg/t6bjkyj93q](#).

Contributions are not limited to code. You can also help by:

- Improving the documentation
- Reporting bugs
- Suggesting new features
- Engaging with the community on Discord
- Creating examples and other materials
- Talking about Embabel at meetups and conferences
- Posting about Embabel on social media

When contributing code, **do** augment your productivity using coding agents and LLMs, but avoid these pitfalls:

- **Excessive LLM comments that add no value.** Code should be self-documenting. Comments are for things that are non-obvious.
- **Bloated PR descriptions and other content.**

Nothing personal, but such contributions will automatically be rejected.

You must understand anything you contribute.

6. Resources

6.1. Rod Johnson’s Blog Posts

- [Embabel: A new Agent Platform For the JVM](#) - Introduction to the Embabel agent framework, explaining the motivation for building an agent platform specifically for the JVM ecosystem. Covers the key differentiators and benefits of the approach.
- [The Embabel Vision](#) - Rod Johnson’s vision for the future of agent frameworks and how Embabel fits into the broader AI landscape. Discusses the long-term goals and strategic direction of the project.
- [Context Engineering Needs Domain Understanding](#) - Deep dive into the DICE (Domain-Integrated Context Engineering) concept and why domain understanding is fundamental to effective context engineering in AI systems.

6.2. Examples and Tutorials

- [Creating an AI Agent in Java Using Embabel Agent Framework](#) by Baeldung - A nice introductory example, in Java.
- [Building Agents With Embabel: A Hands-On Introduction](#) by Jetro Coenradie - An excellent Java tutorial.

6.2.1. Embabel Agent Examples Repository

The [Examples Repository](#) is a comprehensive collection of example agents demonstrating different aspects of the framework:

- **Beginner Examples:** Simple horoscope agents showing basic concepts
- **Intermediate Examples:** Multi-LLM research agents with self-improvement
- **Advanced Examples:** Fact-checking agents with parallel verification and confidence scoring
- **Integration Examples:** Agents that use web tools, databases, and external APIs

Perfect starting point for learning Embabel development with hands-on examples.

6.2.2. Java Agent Template

[Template repository](#) for creating new Java-based Embabel agents. Includes:

- Pre-configured project structure
- Example WriteAndReviewAgent demonstrating multi-LLM workflows
- Build scripts and Docker configuration
- Getting started documentation

6.2.3. Kotlin Agent Template

[Template repository](#) for Kotlin-based agent development with similar features to the Java template but using idiomatic Kotlin patterns.

6.3. Sophisticated Example: Tripper Travel Planner

6.3.1. Tripper - AI-Powered Travel Planning Agent

[Tripper](#) is a production-quality example demonstrating advanced Embabel capabilities:

Features:

- Generates personalized travel itineraries using multiple AI models
- Integrates web search, mapping, and accommodation search
- Modern web interface built with htmx
- Containerized deployment with Docker
- CI/CD pipeline with GitHub Actions

Technical Highlights:

- Uses both Claude Sonnet and GPT-4.1-mini models
- Demonstrates domain-driven design principles
- Shows how to build user-facing applications with Embabel
- Practical example of deterministic planning with AI

Learning Value:

- Real-world application of Embabel concepts
- Integration patterns with external services
- Production deployment considerations
- User interface design for AI applications

6.4. Goal-Oriented Action Planning (GOAP)

- Here’s an [Introduction to GOAP](#), the planning algorithm used by Embabel. Explains the core concepts and why GOAP is effective for AI agent planning.

6.4.1. Small Language Model Agents - NVIDIA Research

- This [Research paper](#) discusses the division between "code agency" and "LLM agency" - concepts that inform Embabel’s architecture.

6.4.2. OODA Loop - Wikipedia

Here’s a [Background](#) on the Observe-Orient-Decide-Act loop that underlies Embabel’s replanning approach.

6.5. Domain-Driven Design

- Martin Fowler’s [Foundational concepts of Domain-Driven Design](#) provides a good summary of Embabel’s approach to domain modeling.

6.5.1. Domain-Driven Design: Tackling Complexity in the Heart of Software

- Eric Evans' [seminal book](#) on DDD principles. Essential reading for understanding how to model complex domains effectively.

6.5.2. DDD and Contextual Validation

- [Advanced DDD concepts](#) relevant to building sophisticated domain models for AI agents.

7. APPENDIX

8. Planning Module

8.1. Abstract

Lower level module for planning and scheduling. Used by Embabel Agent Platform.

8.2. A* GOAP Planner Algorithm Overview

The A* GOAP (Goal-Oriented Action Planning) Planner is an implementation of the A* search algorithm specifically designed for planning sequences of actions to achieve specified goals.

The algorithm efficiently finds the optimal path from an initial world state to a goal state by exploring potential action sequences and minimizing overall cost.

8.2.1. Core Algorithm Components

The A* GOAP Planner consists of several key components:

1. **A Search***: Finds optimal action sequences by exploring the state space
2. **Forward Planning**: Simulates actions from the start state toward goals
3. **Backward Planning**: Optimizes plans by working backward from goals
4. **Plan Simulation**: Verifies that plans achieve intended goals
5. **Pruning**: Removes irrelevant actions to create efficient plans
6. **Unknown Condition Handling**: Manages incomplete world state information

8.2.2. A* Search Algorithm

The A* search algorithm operates by maintaining:

- **Open List**: A priority queue of states to explore, ordered by f-score
- **Closed Set**: States already fully explored
- **g-score**: Cost accumulated so far to reach a state
- **h-score**: Heuristic estimate of remaining cost to goal
- **f-score**: Total estimated cost (g-score + h-score)

8.2.3. Process Flow

1. **Initialization**:
 - Begin with the start state in the open list
 - Set its g-score to 0 and calculate its h-score
2. **Main Loop**:
 - While the Open List is not empty:
 - Select the state with the lowest f-score from the open list
 - If this state satisfies the goal, construct and return the plan
 - Otherwise, mark the state as processed (add to closed set)
 - For each applicable action, generate the next state and add to open list if it better than existing paths
3. **Path Reconstruction**: When a goal state is found, reconstruct the path by following predecessors
 - Create a plan consisting of the sequence of actions

Reference: `link:goap/AStarGoapPlanner.kt[AStarGoapPlanner]:planToGoalFrom:`

8.2.4. Forward and Backward Planning Optimization

The planner implements a two-pass optimization strategy to eliminate unnecessary actions:

Backward Planning Optimization

This pass works backward from the goal conditions to identify only actions that contribute to achieving the goal

Reference: `link:goap/AStarGoapPlanner.kt[AStarGoapPlanner]:_backwardPlanningOptimization`

Forward Planning Optimization

This pass simulates the plan from the start state and removes actions that don't make progress toward the goal:

Reference: `link:goap/AStarGoapPlanner.kt[AStarGoapPlanner]:_forwardPlanningOptimization`

Plan Simulation

Plan simulation executes actions in sequence to verify the plan's correctness:

Reference: `function simulatePlan(startState, actions)`

8.2.5. Pruning Planning Systems

The planner can prune entire planning systems to remove irrelevant actions:

JavaKotlin

```
PlanningSystemprune(PlanningSystemplanningSystem){// Get all plans to all goalsList<Plan>allPlans=plansToGoals(planningSystem);// Keep only actions that appear in at least one planSet<Action>filteredActions=planningSystem.getActions().stream().filter(action->allPlans.stream().anyMatch(plan->plan.getActions().contains(action))).collect(Collectors.toSet());returnplanningSystem.withActions(filteredActions);}
```

Heuristic Function

The heuristic function estimates the cost to reach the goal from a given state:

8.2.6. Complete Planning Process

1. Initialize with start state, actions, and goal conditions
2. Run A* search to find an initial action sequence
3. Apply backward planning optimization to eliminate unnecessary actions
4. Apply forward planning optimization to further refine the plan
5. Verify the plan through simulation
6. Return the optimized action sequence or null if no valid plan exists

8.3. Agent Pruning Process

When pruning an agent for specific goals:

1. Identify all known conditions in the planning system
2. Set initial state based on input conditions
3. Find all possible plans to each goal
4. Keep only actions that appear in at least one plan
5. Create a new agent with the pruned action set

This comprehensive approach ensures agents contain only the actions necessary to achieve their designated goals, improving efficiency and preventing action leakage between different agents.

8.3.1. Progress Determination Logic in A* GOAP Planning

The progress determination logic in method `*forwardPlanningOptimization*` is a critical part of the forward planning optimization in the A* GOAP algorithm. This logic ensures that only actions that meaningfully progress the state toward the goal are included in the final plan.

Progress Determination Expression

JavaKotlin

```
booleanprogressMade=!nextState.equals(currentState)&&action.getEffects().entrySet().stream().anyMatch(entry->{Stringkey=entry.getKey();Objectvalue=entry.getValue();returngoal.getPreconditions().containsKey(key)&&!Objects.equals(currentState.get(key),goal.getPreconditions().get(key))&&(Objects.equals(value,goal.getPreconditions().get(key))||!nextState.containsKey(key));});
```

Detailed Explanation

The expression evaluates to true only when an action makes meaningful progress toward achieving the goal state. Let's break down each component:

1. `nextState != currentState`

- Verifies that the action actually changes the world state
 - Prevents including actions that have no effect
2. `action.effects.any { ... }`
- Examines each effect the action produces
 - Returns true if ANY effect satisfies the inner condition
3. `goal.preconditions.containsKey(key)`
- Ensures we only consider effects that relate to conditions required by the goal
 - Ignores effects that modify conditions irrelevant to our goal
4. `currentState[key] != goal.preconditions[key]`
- Checks that the current condition value differs from what the goal requires
 - Only counts progress if we’re changing a condition that needs changing
5. `(value == goal.preconditions[key] || key not in nextState)`
- This checks one of two possible ways an action can make progress:
 - `value == goal.preconditions[key]`
 - The action changes the condition to exactly match what the goal requires
 - Direct progress toward goal achievement
 - `key not in nextState`
 - The action removes the condition from the state entirely
 - This is considered progress if the condition was previously in an incorrect state
 - Allows for actions that clear obstacles or reset conditions

From <<https://docs.embabel.com/embabel-agent/guide/0.3.5-SNAPSHOT/#overview> *overview*>

Embabel: A New Agent Platform For the JVM



[Rod Johnson](#)

3 min read
.
May 22, 2025
--
5

I’m excited to share my new open source project, the [Embabel agent framework](#).
Not since I founded the [Spring Framework](#) have I been so convinced that a new project is needed. Not since I pioneered Dependency Injection and the other core Spring concepts, have I been so convinced that a new programming model is needed, and so sure of what it should be.

Why Do We Need an Agent Framework?

Aren’t LLMs smart enough to solve our problems directly? Aren’t MCP tools all we need to allow them to solve complex problems?
No. MCP is an important step forward, and of course Embabel embraces it, just as it makes it easy to work with multiple models. But there are many reasons that a higher level orchestration technology is needed, especially for business applications. Here are some of the most important:

- *Explainability*: Why were choices made in solving a problem?
- *Discoverability*: MCP skirts this important problem. How do we find the right tools at each point, and ensure that models aren’t confused in choosing between them?
- *Ability to mix models*, so that we are not reliant on God models but can use local, cheaper, private models for many tasks
- Ability to inject *guardrails* at any point in a flow
- Ability to manage flow *execution* and introduce greater resilience
- *Composability* of flows at scale. We’ll soon be seeing not just agents running on one system, but federations of agents.
- Safer *integration* with sensitive existing systems such as databases, where it is dangerous to allow even the best LLM write access.

Why the JVM?

Ok, so agents are a thing. Why the JVM? Why not just write in Python?

Python has been the dominant language for AI for the last few years. At this point, every developer needs to be fluent in Python. I'm glad that I am. However, AI is much broader than *Gen AI*. Python offers a great ecosystem for broader AI, with [TensorFlow](#), [PyTorch](#), and many other libraries.

With Gen AI we are not doing comparable low level work. We're contacting multiple LLMs via simple HTTP calls.

The critical adjacency for building business apps with LLMs is existing business logic and infrastructure. And the critical skill set is building sophisticated business applications. In both these areas, the JVM is far ahead of Python and likely to remain so.

Much of the critical business logic in the world is running on the JVM, and for good reason. Gen AI enabling it is of critical importance.

Why Embabel?

[Embabel](#) is an agent framework, initially for the JVM, that is intended not just to play catchup with Python agent frameworks, but to leapfrog them.

Some distinguishing features:

- *Embabel introduces a planning step.* The framework discovers actions and goals from your code, and can plan toward the most appropriate goal given user or other system input. This enables superior extensibility. You'll not need to rewire a state machine, for example, to add capabilities. Planning is accomplished via a [non-LLM AI algorithm](#), which provides a smart, yet deterministic and explainable approach to planning.
- *Embabel encourages building a rich domain model* in your application — typically, Kotlin data classes or Java records. This ensures that prompts are typesafe, toolable and survive refactoring. It also allows behavior to be added to domain objects, which can be exposed as LLM tools as well as used in code.

Read more on the [Embabel GitHub page](#).

Embabel is written in Kotlin, which we believe to be the best language for building JVM applications today. However, we understand that many developers and teams are happy with Java, and we're committed to ensuring that Embabel is as idiomatic from Java as from Kotlin.

This is easy not only because of Kotlin's excellent interoperability with Java, but because Embabel integrates closely with Spring.

If you're a Spring developer, you'll find building agents with Embabel to be as natural as building a Spring MVC REST interface.

It's early, but we have big plans. We want not just to build the best agent platform on the JVM, but to build the best agent platform, period.

In future blogs I'll talk more about Embabel's philosophy and show examples.

Where Next

It's an exciting time to be a developer, and an exciting time to be a JVM developer.

Please check out Embabel. AI-enable your existing applications and unlock their business value, using your existing skillset.

From <https://medium.com/@springrod/embabel-a-new-agent-platform-for-the-jvm-1c83402e0014>

The Embabel Vision



[Rod Johnson](#)

6 min read

.

In previous blogs I've [introduced](#) the [Embabel](#) agent framework, discussing how it innovates by introducing [deterministic planning](#), ensures [type safety](#), and [facilitates unit testing](#).

Today I'm going to talk about the overall vision: Where Embabel is going, and why it's important.

The agent framework space is young. On the JVM it's nascent. Embabel is an ambitious project, with the goal of being the best agent framework on any platform. Its concepts are innovative and, we believe, far in advance of the competition, on the JVM or elsewhere.

Embabel aims to help unlock the full business value of Gen AI, by giving developers the framework to make it safe and dependable.

Key objectives:

- *To bring the disruptive power of deterministic planning:* The ability to extend system capabilities without modifying existing code underpins smart, safe, agentic systems that grow with requirements.
- *To leverage the power of domain modeling* to maximize reuse, explainability, auditability, and fidelity to business requirements.
- *To enable deep, frictionless integration with existing business applications.* Because Embabel grows out of enterprise experience and integrates seamlessly with the dominant Java framework, it is uniquely placed to achieve this.
- *To enable the use of software engineering best practices to build Gen AI applications.* While the technology landscape has changed, proven practices remain essential to navigate it. No one is going to vibe code their way to unlocking the potential of Gen AI in critical systems.
- *To embrace emerging standards* such as MCP and A2A to help build the agentic web, so developers can focus on business logic rather than tracking specifications.
- *To explore the potential to create agents in natural language.* While the skills of enterprise developers create the foundation, it's now possible to empower business users in a way that has often been promised but never delivered in the past.
- *To offer inspirational examples that are useful in their own right and demonstrate the power of a good agent platform.* Watch this space!
- *To demonstrate the value of a consistent approach* that, as with Spring, is bigger than the sum of its parts.

A Little Deeper

Let's look at the most important of these objectives.

The Power of Planning

While other agent frameworks either have no planning step, require developers to specify how actions are chained together (for example, in a state machine), or rely on LLMs for planning, Embabel [uses a non-LLM AI algorithm to work out a path of actions towards a goal](#). This has benefits that will become increasingly important.

Any planning approach is superior to manual chaining of actions, because it grants the system autonomy to determine the best way to perform tasks. Embabel's approach is unique in that it is deterministic, safe and explainable.

While predictable, it remains intelligent, as the system can chain existing actions together in ways that were not explicitly programmed, and grow in power as more actions and goals are added.

Actions and goals enable an Embabel server to grow its capabilities the way that humans do: learning how to perform individual tasks that can be chained together to perform complex operations. Enjoy the best of both worlds: Embabel servers can solve problems in ways that weren't explicitly programmed, while only using steps that were designed into the system.

Domain Modeling

Embabel is also unique in its [emphasis on domain modeling](#). Domain objects work hand in hand with actions and goals, as most pre and post conditions used in planning are expressed in terms of the data flow of domain objects. As per good OO practice, domain objects can have behavior as well as state, and Embabel even provides the ability to selectively expose safe functions to LLMs as tools.

There are obvious benefits here, such as [type safety](#), [testability](#), toolability and refactoring support.

The emergent benefits may prove even more important. As with actions and goals, adding more domain objects can deliver non-linear value. The domain model becomes a data dictionary for the problem space.

We envision a future where Embabel servers sit above existing applications, growing in power as more actions and domain objects are exposed, while safely exposing and consuming functionality from the agentic web.

Integration

Integration with existing systems and infrastructure is essential to making Gen AI useful, yet has been underemphasized to date. This is not surprising given that so much of the first generation of Gen AI functionality has come out of the Python space, which is seldom intimately connected to existing business applications.

A huge part of unlocking the value of Gen AI is integration with enterprise applications, which needs to be both deep and safe.

MCP has had a transformational impact in connecting LLMs to the outside world. Tying into existing systems is even more important. MCP is not especially compelling in this case, as it introduces unnecessary friction and loss of fidelity with respect to existing domain objects and infrastructure.

By first targeting the JVM, Embabel is already taking a big step in the right direction. By building on Spring, which runs a huge proportion of the world's critical applications, and following Spring patterns, it offers frictionless integration into enormous business value.

Embracing standards

The Gen AI space is moving at breakneck speed. This is exhilarating and at times bewildering.

This pace limits the role of standardization, as innovation is essential while the industry learns. However, there are already two promising standardization initiatives.

[MCP \(Model Context Protocol\)](#) has already proven useful. While the spec has significant gaps to fill, it's progressing rapidly and making it far easier to build Gen AI applications, through providing a consistent interface to tools for LLMs to call.

More recently, [A2A \(the Agent2Agent Protocol\)](#) aims at a higher level of integration, seeking to standardize agent discovery and collaboration. A2A has yet to prove itself in practice, but it may well end up being as important.

Embabel embraces both these efforts and will react quickly to support any other worthy initiatives.

Embabel embraces standards to ensure that code built on it is able to contribute to and consume assets from the emerging agentic web. For example, developers can write Embabel agents and expose them as MCP or A2A servers with minimal effort.

Toward The PaaS for Natural Language

We're seeing two major vectors in agent platforms: Code-centric approaches like Crew AI, Google ADK and LangGraph, and graphical approaches like HyperMode, Sema4.ai and many others.

The "let's draw pretty pictures" brigade has historically underperformed their grand promises, at least in enterprise. However, this time things really might be different. It's likely that the successful products will converge.

Embabel comes from the code-centric direction, but its strong concepts can underpin higher level functionality and greater use of natural language.

A library of actions, goals and domain objects provides a strong foundation for business users to express behavior in natural language.

This approach will enable a new concept of “compilation” where the platform helps business users clarify their thoughts to eliminate ambiguity and create additional actions and goals that can be safely added to the system.

Embabel is ideally placed to push the use of natural language to define agents, given its ability to balance autonomy and control and its emphasis on a domain model that can provide a lingua franca.

Amazing examples

Any agent framework worth its salt should have impressive, non-trivial examples, and use its own capabilities can help advance its technology and community.

Our goals:

- To create agents that are realistic and useful in themselves. The world does not need more trivial “You are a helpful assistant” examples.
- To inspire the community to build amazing agents themselves — hopefully contributing to open source to create broader value.
- To demonstrate how many Gen AI verticals are indefensible based on technology alone, given a good agent framework. (Of course many businesses are defensible based on understanding of their domain and customers.)

We will [eat our own dogfood](#) as much as possible.

We will strive to automate as many things as possible using agents built on Embabel. We will not use third party products such as commercial coding agents, and we will open source all agents used for the Embabel open source project and community.

Press enter or click to view image in full size

Dog food is tasty. Our Duke as a pup

Per the flight safety advice to put your own mask on before helping others, we will prioritize agents that accelerate Embabel development — most notably, our [coding agent](#).

Next Steps

We’re looking ahead, and Embabel doesn’t yet do everything I’ve described. But it will, and it’s already very powerful.

Check out [Embabel](#). Join our growing [community](#). AI-enable your JVM applications and unlock their business value, using your existing skillset.

Hopefully this has whetted your appetite for joining — and contributing to — the journey. We think Embabel will become a core building block of the agentic web. Bring your skills to help solve the most important technical problems of our time, as we build the future of agent technology, on the JVM and beyond.

From <<https://medium.com/@springrod/the-embabel-vision-967654f13793>>

From Wikipedia, the free encyclopedia

"Ooda" redirects here. For the Japanese city, see [Ōda, Shimane](#).

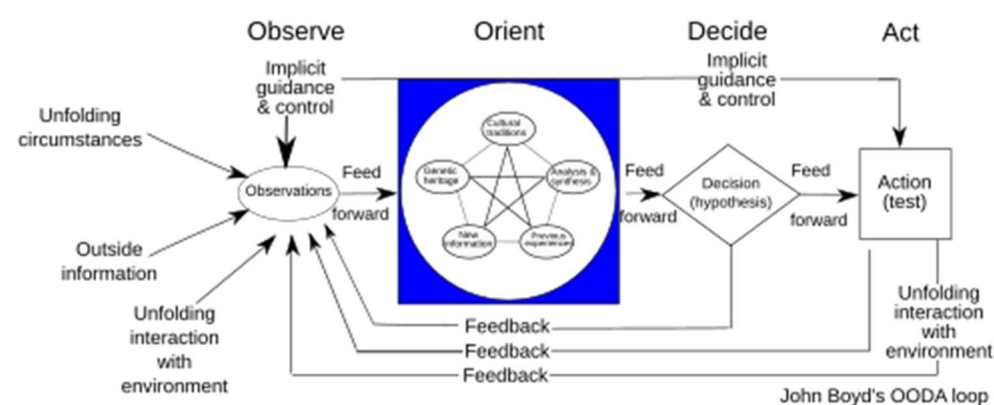


Diagram of the OODA loop

The **OODA loop (observe, orient, decide, act loop)** is a decision-making model developed by [United States Air Force](#) Colonel [John Boyd](#) in the early 1970s. He applied the concept to the [combat operations process](#), often at the operational level during military campaigns. It is often applied to understand commercial operations and learning processes.^[1] The approach explains how agility can overcome raw power in dealing with human opponents.

The OODA loop

As can be seen from the diagram, the OODA loop includes continuous collection of feedback and observations. This enables late commitment, which is an important element of agility. This is in contrast to the [PDCA](#) (plan–do–check–act) cycle which requires early commitment.

Use in law and business

The OODA loop has become an important concept in [litigation](#),^[2] business,^[3] [law enforcement](#),^[4] management education,^{[5][6]} [military strategy](#) and [cyber security](#), and [cyberwarfare](#).^[7] According to Boyd, [decision-making](#) occurs in an iterative cycle of "observe, orient, decide, act". An entity (whether an individual or an organization) that can process this cycle quickly, observing and reacting to unfolding events more rapidly and/or more effectively than an opponent, can thereby get inside the opponent's decision cycle and gain the advantage.

The OODA loop has been applied in financial analysis as a framework for managing information flow and making investment decisions under uncertainty.^[8] The framework has also been examined in innovation management research, which has explored its application at individual, team, and organizational levels.^[9]

[Jamie Dimon](#), chairman and CEO of [JPMorgan Chase](#), has said he uses the OODA loop in scenario evaluation.^{[10][11][12]}

Criticism

Some scholars are critical of the concept. Aviation historian Michael Hankins, for example, writes that "the OODA loop is vague enough that its defenders and attackers can each see what they want to see in it. For some, the OODA concept's flexibility is its strength, but for others it becomes so generalized as to lose its usefulness." He concludes that "The OODA loop is merely one way among a myriad of ways of describing intuitive processes of learning and decision making that most people experience daily. It is not incorrect, but neither is it unique or especially profound."^[13]

From <https://en.wikipedia.org/wiki/OODA_loop>

Learn agentic AI development with Spring Framework and Java or Kotlin. These examples demonstrate building intelligent agents that can plan, execute workflows, use tools, and interact with humans.

This repository uses the latest Embabel snapshots to illustrate current best practice, whereas the [Java](#) and [Kotlin](#) template repositories s use the latest milestone release for greater stability. There may be some minor API incompatilibites and not everything you see here may work in your own project created from one of those templates, unless you upgrade the `embabel-agent.version` property in the POM file, as in this repository.

Quick Start

Prerequisites

- **Java 21+**
- **API Key (at least one): [OpenAI](#) or [Anthropic](#)**
- **Maven 3.9+ (optional - project includes Maven wrapper)**

1. Clone & Build

```
git clone https://github.com/embabel/embabel-agent-examples.git
cdembabel-agent-examples
./mvnw clean install #Unix/Linux/macOSmvnw.cmd clean install #Windows
```

2. Set API Keys

```
#Required (choose one or both)exportOPENAI_API_KEY="your_openai_key"exportANTHROPIC_API_KEY="your_anthropic_key"
```

3. Run Examples

Kotlin Examples

```
cdscripts/kotlin
./shell.sh #Unix/Linux/macOS - With Docker tools (default)shell.cmd #Windows - With Docker tools (default)./shell.sh --no-docker-tools
#Unix/Linux/macOS - Basic features onlyshell.cmd --no-docker-tools #Windows - Basic features only
```

Java Examples

```
cdscripts/java
./shell.sh #Unix/Linux/macOS - With Docker tools (default)shell.cmd #Windows - With Docker tools (default)./shell.sh --no-docker-tools
#Unix/Linux/macOS - Basic features onlyshell.cmd --no-docker-tools #Windows - Basic features only
```

Creating Your Own Project

You can create your own agent repo from our [Java](#) or [Kotlin](#) GitHub template by clicking the "Use this template" button.

You can also create your own Embabel agent project locally with our quick start tool, which allows some customization:

```
uvx --from git+https://github.com/embabel/project-creator.git project-creator
```

Choose Java or Kotlin and specify your project name and package name and you'll have an agent running in under a minute, if you already have an OPENAI_API_KEY and have Maven installed.

 Spring Boot Integration Architecture

Embabel Agent Starter Guide:

embabel-agent-starter

- ✔ Application decides on startup mode (console, web application, etc)
- ✔ Agent discovery and registration
- ✔ Agent Platform beans are available via Dependency Injection mechanism for Application to use as needed
- ✔ Progress tracking and logging
- ✔ Development-friendly error handling

embabel-agent-starter-shell

- ✔ Interactive command-line interface
- ✔ Agent discovery and registration
- ✔ Human-in-the-loop capabilities
- ✔ Progress tracking and logging
- ✔ Development-friendly error handling

embabel-agent-starter-mcpserver

- ✔ MCP protocol server implementation
- ✔ Tool registration and discovery
- ✔ JSON-RPC communication via SSE (Server-Sent Events)
- ✔ Integration with MCP-compatible clients
- ✔ Security and sandboxing

Application Examples

Shell Mode with Logging Theme

```
@SpringBootApplication
@ConfigurationPropertiesScan(basePackages = ["com.embabel.example"])
class KotlinAgentShellApplicationfunmain(args:Array<String>) {
    runApplication<KotlinAgentShellApplication>(*args) {
        setDefaultProperties(
            mapOf("embabel.agent.logging.personality"to LoggingThemes.STAR_WARS)
        )
    }
}

// Java version
@SpringBootApplication
@ConfigurationPropertiesScan(basePackages= {"com.embabel.example"})
publicclassJavaAgentShellApplication{
    publicstaticvoidmain(String[] args) {
        SpringApplicationapp= newSpringApplication(JavaAgentShellApplication.class);
        app.setDefaultProperties(Map.of(
            "embabel.agent.logging.personality", LoggingThemes.STAR_WARS));
        app.run(args);
    }
}
```

Shell Mode with MCP Client Support (Docker Tools)

```
@SpringBootApplication
@ConfigurationPropertiesScan(basePackages = ["com.embabel.example"])
class KotlinAgentShellApplicationfunmain(args:Array<String>) {
    runApplication<KotlinAgentShellApplication>(*args) {
        setAdditionalProfiles(McpServers.DOCKER) //Activates application-docker-ce.ymlsetDefaultProperties(
            mapOf("embabel.agent.logging.personality"to LoggingThemes.SEVERANCE)
        )
    }
}
```

MCP Server Mode

```
@SpringBootApplication
@ConfigurationPropertiesScan(basePackages = ["com.embabel.example"])
class KotlinAgentMcpServerApplicationfunmain(args:Array<String>) {
    runApplication<KotlinAgentMcpServerApplication>(*args) {
        setAdditionalProfiles(McpServers.DOCKER, McpServers.DOCKER_DESKTOP)
    }
}
```

Configuration Guide

Logging Personality

Set via the `embabel.agent.logging.personality` property:

- `"starwars"` - May the Force be with your logs!
- `"severance"` - Welcome to Lumon Industries (default)

MCP Client Integration

Configure via Spring profiles that define Spring AI MCP client connections in `application-{profile}.yaml`:

```
#application-docker-ce.yaml
spring:
  ai:
    mcp:
      client:
        type: SYNCstdio:
          connections:
            docker-mcp:
              command: dockerargs: [mcp, gateway, run]
```

Setting up MCP Tools

Several of the examples use the Model Context Protocol (MCP) to access tools and services.

The default source is the Docker Desktop MCP server, which is installed with Docker Desktop.

To ensure tools are available and startup doesn't time out, first pull models with:

```
docker login
docker mcp gateway run
When the gateway has come up you can kill it and start the Embabel server.
```

📖 Examples by Learning Level

🔧 Beginner: InjectedComponent

Available in: Java | Concept: Just a Little AI
Demonstrates how you can inject any Spring component with an Embabel `OperationContext` and use it to call LLMs with the rich Embabel API.

```
@Component
public record InjectedComponent(Ai ai) {
    public record Joke(String leadup, String punchline) {}
}

public String tellJokeAbout(String topic) {
    return ai.withDefaultLlm()
        .generateText("Tell me a joke about "+ topic);
}

public Joke createJokeObjectAbout(String topic1, String topic2, String voice) {
    return ai.withLlm(LlmOptions.withDefaultLlm().withTemperature(.8))
        .createObject("""
            Tell me a joke about %s and %s.
            The voice of the joke should be %s.
            The joke should have a leadup and a punchline.
            """, topic1, topic2, voice), Joke.class);
}
```

🌟 Beginner: Horoscope News Agent

Available in: Java & Kotlin | Concept: Basic Agent Workflow
A fun introduction to agent development that finds personalized news based on someone's star sign.

What It Teaches:

- 📄 Action-based workflows with `@Action` annotations
- 🔍 Data extraction from user input using LLMs
- 🌐 Web tool integration for finding news stories
- 📝 Content generation with personality and context
- 🎯 Goal achievement with `@AchievesGoal`

How It Works:

1. Extract person's name from user input
2. Get their star sign (via form if needed)
3. Retrieve daily horoscope
4. Search web for relevant news stories
5. Create amusing writeup combining horoscope + news

Try It:

Start the agent shell, then type:

```
x "Find horoscope news for Alice who is a Gemini"
x is short for execute, which triggers the agent to run its workflow.
```

Code Comparison:

- Kotlin: `examples-kotlin/src/main/kotlin/com/embabel/example/horoscope/StarNewsFinder.kt`
- Java: `examples-java/src/main/java/com/embabel/example/horoscope/StarNewsFinder.java`

Key Patterns:


```
@Agent(description = "Find news based on a person's star sign")
class StarNewsFinder {
    @Action
    fun extractPerson(userInput: UserInput, context: OperationContext): Person? @Action(toolGroups = [CoreToolGroups.WEB])
    fun findNewsStories(person: StarPerson, horoscope: Horoscope, context: OperationContext): RelevantNewsStories @AchievesGoal(description = "Create an amusing writeup")
    @Action
    fun starNewsWriteup(/*params */): Writeup }
```

 Expert: Multi-LLM Research Agent

Available in: Java, Kotlin | Concept: Self-Improving AI Workflows
A sophisticated research agent using multiple AI models with self-critique capabilities.

What It Teaches:

-  Multi-model consensus (GPT-4 + Claude working together)
-  Self-improvement loops with critique and retry
-  Configuration-driven behavior with Spring Boot properties
-  Parallel processing of research tasks
-  Quality control through automated review

Architecture:

```
@ConfigurationProperties(prefix = "embabel.examples.researcher")
data class ResearcherProperties {
    val maxWordCount: Int = 300,
    val claudeModelName: String = AnthropicModels.CLAUDE_35_HAIKU,
    val openAiModelName: String = OpenAiModels.GPT_41_MINI }
```

Self-Improvement Pattern:

```
@Action(outputBinding = "gpt4Report")
fun researchWithGpt4(/*params */): SingleLlmReport @Action(outputBinding = "claudeReport")
fun researchWithClaude(/*params */): SingleLlmReport @Action(outputBinding = "mergedReport")
fun mergeReports(gpt4: SingleLlmReport, claude: SingleLlmReport): ResearchReport @Action
fun critiqueReport(report: ResearchReport): Critique @AchievesGoal(description = "Completes research with quality assurance")
fun acceptReport(report: ResearchReport, critique: Critique): ResearchReport
```

Try It:

"Research the latest developments in renewable energy adoption"
Location: examples-kotlin/src/main/kotlin/com/embabel/example/researcher/

☒ Expert: Fact-Checking Agent (DSL Style)

Available in: Kotlin | Concept: Functional Agent Construction
A fact-verification agent built using Embabel's functional DSL approach instead of annotations.

What It Teaches:

-  Functional DSL construction for agents
-  Parallel fact verification across multiple claims
-  Confidence scoring and source trust evaluation
-  Web research integration for verification
-  Functional programming patterns in agent design

DSL Construction:

```
fun factCheckerAgent(llms: List<LlmOptions>, properties: FactCheckerProperties) = agent(name = "FactChecker", description = "Check content for factual accuracy") {
    {
        flow {
            aggregate<UserInput, FactualAssertions, RationalizedFactualAssertions> {
                transforms = llms.map { llm -> { context -> /*extract assertions with this LLM */ } },
                merge = { list, context -> /*rationalize overlapping claims */ }
            }
        }
        transformation<RationalizedFactualAssertions, FactCheck> {
            /*parallel fact-checking */
        }
    }
}
```

Domain Model:

```
data class FactualAssertion {
    val claim: String,
    val reasoning: String
}
data class AssertionCheck {
    val assertion: FactualAssertion,
    val isFactual: Boolean,
    val confidence: Double,
    val sources: List<String>
}
```

Try It:

"Check these facts: The Earth is flat. Paris is the capital of France."
Location: examples-kotlin/src/main/kotlin/com/embabel/example/factchecker/

Spring Framework Integration

- Auto-Configuration: Starters handle all configuration automatically
- Profile-Based Configuration: MCP clients configured via Spring profiles
- Maven Profiles: enable-shell, enable-shell-mcp-client, enable-agent-mcp-server
- Dependency Injection: Constructor-based injection with agents as Spring beans
- Configuration Properties: Type-safe configuration with @ConfigurationProperties
- Conditional Beans: Environment-specific components with @ConditionalOnBean
- Repository Pattern: Spring Data integration for domain entities

Modern Spring Boot Patterns

- Auto-Configuration: Starters handle all configuration automatically
- Profile-Based Configuration: MCP clients configured via Spring profiles
- Profile-Based Execution: Maven profiles control which application class runs
- Auto-Configuration Classes: Understanding Spring Boot's auto-configuration
- Conditional Configuration: Mode-specific bean loading
- Theme-Based Customization: Dynamic behavior based on properties

Modern Kotlin Features

- Data Classes: Rich domain models with computed properties
- Type Aliases: Domain-specific types (typealias OneThroughTen = Int)
- Extension Functions: Enhanced functionality for existing types
- Delegation: Clean composition patterns
- DSL Construction: Functional agent building
- Coroutines: Parallel execution with structured concurrency

Agent Design Patterns

- Workflow Orchestration: Multi-step processes with @Action chains
- Blackboard Pattern: Shared workspace for data between actions
- Human-in-the-Loop: User confirmations and form submissions
- Self-Improvement: Critique and retry loops for quality
- Multi-Model Consensus: Combining results from different LLMs
- Condition-Based Flow: Workflow control with @Condition
- Progress Tracking: Event publishing for monitoring

Additional Examples

Some of our examples are projects in their own right, and are therefore in separate repositories.

See:

- [Coding Agent](#): An open source coding agent
- [Flicker](#): A movie recommendation engine that takes into account the user's tastes and what's available to them in their country on the streaming services they subscribe to. Uses external APIs and PostgreSQL via JPA. Illustrates a complex workflow where recommendations are generated until enough available movies have been found.
- [Decker](#): An agent to build presentations using Embabel
- [Tripper](#): Travel planning agent. Uses mapping APIs to find routes and places of interest, and generates a travel itinerary. Performs research on points of interest in parallel.

 Running Specific Examples

Interactive Shell Mode (Default)

```
cdscripts/kotlin &&./shell.sh      #With Docker tools (default)cdscripts/kotlin &&shell.cmd      #With Docker tools (Windows)#orcdscripts/java &&./shell.sh
#With Docker tools (default)cdscripts/java &&shell.cmd      #With Docker tools (Windows)
```

Uses Maven profile: enable-shell-mcp-client

Shell Without Docker Tools

```
cdscripts/kotlin &&./shell.sh --no-docker-tools  #Basic features onlycdscripts/kotlin &&shell.cmd --no-docker-tools  #Basic features
(Windows)#orcdscripts/java &&./shell.sh --no-docker-tools  #Basic features onlycdscripts/java &&shell.cmd --no-docker-tools  #Basic features (Windows)
```

Uses Maven profile: enable-shell

With Observability (Zipkin Tracing)

Enable distributed tracing with Zipkin by adding the --observability flag:

```
cdscripts/kotlin &&./shell.sh --observability  #Enable observabilitycdscripts/kotlin &&shell.cmd --observability  #Enable observability
(Windows)#orcdscripts/java &&./shell.sh --observability  #Enable observabilitycdscripts/java &&shell.cmd --observability  #Enable observability
(Windows)
```

Make sure to run docker compose up in the project root to start Zipkin trace collector:

```
docker compose up
You should be able to access Zipkin Console: http://127.0.0.1:9411/zipkin/
```

MCP Server Mode

```
cdscripts/kotlin &&./mcp_server.sh
cdscripts/kotlin &&mcp_server.cmd      #Windows#orcdscripts/java &&./mcp_server.sh
cdscripts/java &&mcp_server.cmd      #Windows
```

Uses Maven profile: enable-agent-mcp-server

You can use the Embabel agent platform as an MCP server from a UI like Claude Desktop. The Embabel MCP server is available over SSE.

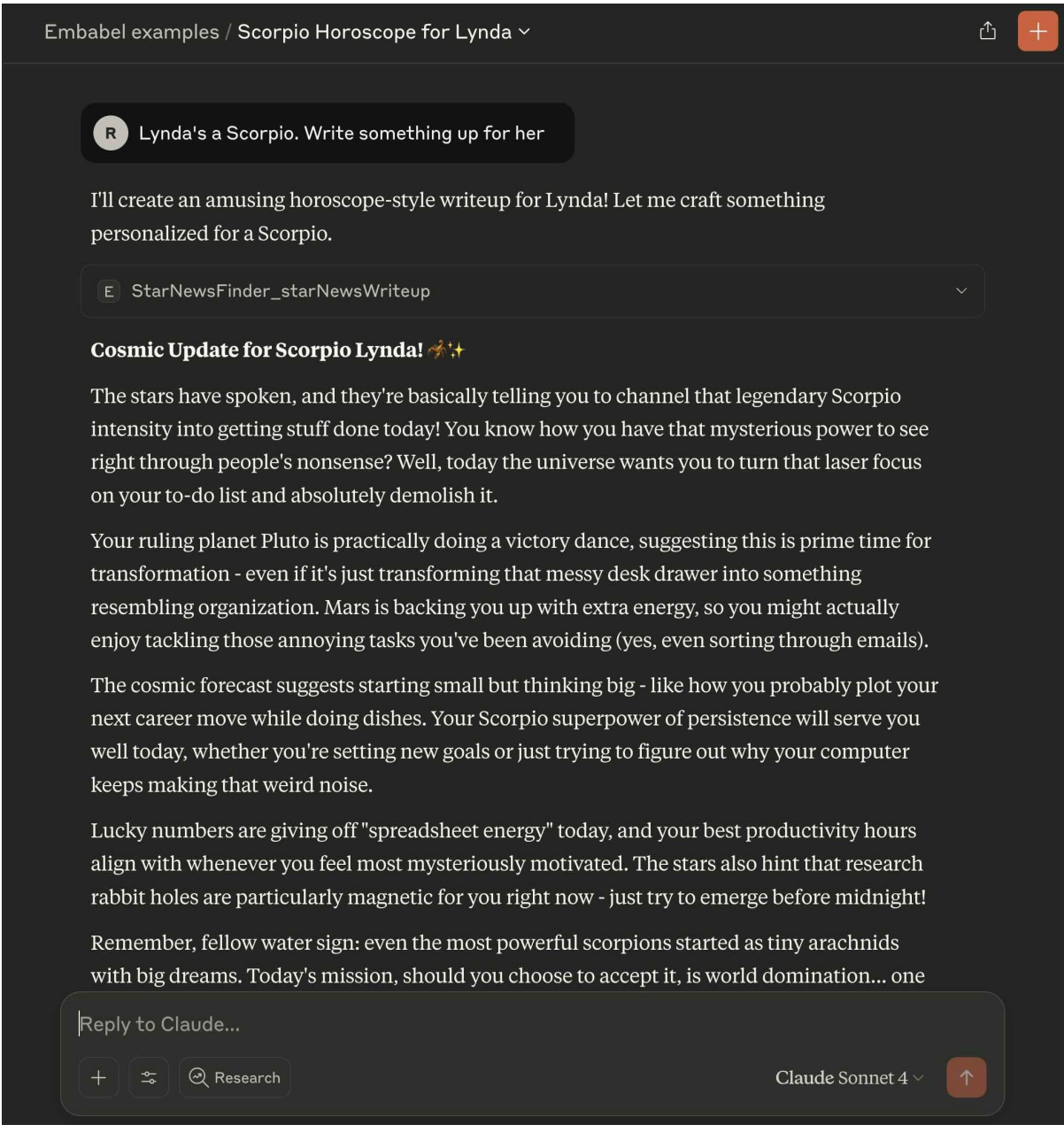
Configure Claude Desktop as follows in your claude_desktop_config.yml:

```
{
  "mcpServers": {
    "embabel-examples": {
      "command": "npx",
      "args": [
        "-y",
        "mcp-remote",
        "http://localhost:8080/sse"
      ]
    }
  }
}
```

See [MCP Quickstart for Claude Desktop Users](#) for how to configure Claude Desktop.

Create a project in Claude Desktop to work with Embabel examples. This will enable you to add a custom system prompt.

The Embabel server will expose each goal as an MCP tool, enabling Claude Desktop to invoke them like this:



The [MCP Inspector](#) is a helpful tool for interacting with your Embabel SSE server, manually invoking tools and checking the exposed prompts and resources.

Start the MCP Inspector with:

`npx @modelcontextprotocol/inspector`

Manual Execution

```
#Kotlin shell mode with MCP client (default)cdexamples-kotlin
mvn -P enable-shell-mcp-client spring-boot:run
#Kotlin shell mode without MCP clientcdexamples-kotlin
mvn -P enable-shell spring-boot:run
#Kotlin MCP server modecdexamples-kotlin
mvn -P enable-agent-mcp-server spring-boot:run
#Java equivalents use the same patterncdexamples-java
mvn -P enable-shell-mcp-client spring-boot:run
```

Testing

```
#Run all tests./mvnw test#Unix/Linux/macOSmvnw.cmd test#Windows#Module-specific testscdexamples-kotlin &&./mvnw testcdexamples-java &&./mvnw test
```


MCP (Model Context Protocol) is an open protocol that enables AI assistants and applications to securely connect to data sources and tools. Embabel supports MCP in two ways:

1. MCP Server Mode: Your agents become tools that can be called by MCP clients
2. MCP Client Support: Your agents can call external MCP servers (like Docker Desktop)

MCP Server Mode

Run your agents as an MCP server that exposes tools over Server-Sent Events (SSE):

```
#Start Kotlin agents as MCP servercdscripts/kotlin &&./mcp_server.sh
#Start Java agents as MCP server cdscripts/java &&./mcp_server.sh
Your agents become available as tools:
```

- StarNewsFinder - find_horoscope_news
- Researcher - research_topic
- FactChecker - check_facts

MCP Client Support

Docker tools are enabled by default. To disable:

```
#Disable Docker MCP integrationcdscripts/kotlin &&./shell.sh --no-docker-tools
cdscripts/java &&./shell.sh --no-docker-tools
With Docker tools enabled, your agents can:
```

- Execute commands in Docker containers
- Access containerized services
- Integrate with other MCP-compatible tools

Benefits of MCP

-  Tool Interoperability - Agents can use and be used as tools
-  Domain Expertise - Specialized agents for specific tasks
-  Tool Composition - Combine multiple tools in workflows
-  Secure Access - MCP handles authentication and sandboxing
-  Scalable Architecture - Add new tools without changing code

 Creating Your Own Agent Application

Basic Shell Application

```
@SpringBootApplication
classMyAgentApplicationfunmain(args:Array<String>) {
    runApplication<MyAgentApplication>(*args)
}
```

Add the shell starter to your pom.xml:

```
<dependency>
  <groupId>com.embabel.agent</groupId>
  <artifactId>embabel-agent-starter-shell</artifactId>
</dependency>
```

Shell with Theme and MCP Client

```
@SpringBootApplication
@ConfigurationPropertiesScan(basePackages =["com.example"])
classMyThemedAgentApplicationfunmain(args:Array<String>) {
    runApplication<MyThemedAgentApplication>(*args) {
        setAdditionalProfiles("docker-ce") //Enable MCP client via profilesetDefaultProperties(
            mapOf("embabel.agent.logging.personality"to LoggingThemes.STAR_WARS)
        )
    }
}
```

MCP Server Application

```
@SpringBootApplication
classMyMcpServerApplicationfunmain(args:Array<String>) {
    runApplication<MyMcpServerApplication>(*args)
}
```

Add the MCP server starter to your pom.xml:

```
<dependency>
  <groupId>com.embabel.agent</groupId>
  <artifactId>embabel-agent-starter-mcpserver</artifactId>
</dependency>
```

 Getting Started Recommendations

New to Agents?

1. Start with Horoscope News Agent (Java or Kotlin)
2. Compare the Java vs Kotlin implementations

- 3. Experiment with different prompts and see how the agent plans different workflows
- 4. Try different logging themes to make development more fun!

Spring Developer?

- 1. Look at the configuration classes and repository integration
- 2. Study the domain model design and service composition
- 3. Explore the different application modes and Maven profiles
- 4. See how profiles and properties configure MCP clients

Kotlin Enthusiast?

- 1. Progress to Researcher for multi-model patterns
- 2. Explore Fact Checker for functional DSL approaches

AI/ML Developer?

- 1. Study prompt engineering techniques in any example
- 2. Examine the Researcher for multi-model consensus patterns
- 3. Look at Fact Checker for confidence scoring and source evaluation
- 4. Explore MCP integration for tool composition

🔔 Common Issues & Solutions

Problem	Solution
"No API keys found"	Set OPENAI_API_KEY or ANTHROPIC_API_KEY
Wrong examples load	Use correct script: kotlin/shell.sh vs java/shell.sh
Build failures	Run ./mvnw clean install (Unix/macOS) or mvnw.cmd clean install (Windows) from project root
Application class not found	Check Maven profile matches application class
MCP client fails to connect	Check port availability and Docker Desktop status. See instructions on pulling models above.

Look at the log output in the event of failure as it may contain hints as to the solution.

📁 Project Structure

```
embabel-agent-examples/
├── examples-kotlin/ # 🗂️ Kotlin implementations
│   ├── src/main/kotlin/com/embabel/example/
│   │   ├── KotlinAgentShellApplication.kt # Shell + MCP client (default)
│   │   ├── KotlinAgentSimpleShellApplication.kt # Shell without MCP
│   │   └── KotlinAgentMcpServerApplication.kt # MCP server mode
│   ├── horoscope/ # 🌟 Beginner: Star news agent
│   ├── pom.xml # Maven profiles for each mode
│   └── README.md # 📖 Kotlin-specific documentation
├── examples-java/ # ☕ Java implementations
│   ├── src/main/java/com/embabel/example/
│   │   ├── JavaAgentShellApplication.java # Shell with themes
│   │   ├── JavaAgentSimpleShellApplication.java # Shell without MCP
│   │   └── JavaMcpServerApplication.java # MCP server mode
│   ├── horoscope/ # 🌟 Beginner: Star news agent
│   ├── README.md # 📖 Java-specific documentation
├── examples-common/ # 🔗 Shared services & utilities
├── scripts/ # 📜 Quick-start scripts
│   ├── kotlin/
│   │   ├── shell.sh # Launch shell (--no-docker-tools to disable)
│   │   └── mcp_server.sh # Launch MCP server
│   ├── java/
│   │   ├── shell.sh # Launch shell (--no-docker-tools to disable)
│   │   └── mcp_server.sh # Launch MCP server
│   ├── support/ # Shared script utilities
│   ├── README.md # 📖 Scripts documentation
└── pom.xml # Parent Maven configuration
```

📄 License

Licensed under the Apache License 2.0. See [LICENSE](#) for details.

🎉 Happy coding with Spring Framework and agentic AI!